

Reinforcement Learning Basics

Table of Contents

Finite Markov Decision Processes	2
Temporal-Difference Learning	25
On-policy Prediction with Approximation	45
Eligibility Traces	85
A (Long) Peek into Reinforcement Learning	117
DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills	138
END	154

Chapter 3

Finite Markov Decision Processes

In this chapter we introduce the formal problem of finite Markov decision processes, or finite MDPs, which we try to solve in the rest of the book. This problem involves evaluative feedback, as in bandits, but also an associative aspect—choosing different actions in different situations. MDPs are a classical formalization of sequential decision making, where actions influence not just immediate rewards, but also subsequent situations, or states, and through those future rewards. Thus MDPs involve delayed reward and the need to tradeoff immediate and delayed reward. Whereas in bandit problems we estimated the value $q_*(a)$ of each action a , in MDPs we estimate the value $q_*(s, a)$ of each action a in each state s , or we estimate the value $v_*(s)$ of each state given optimal action selections. These state-dependent quantities are essential to accurately assigning credit for long-term consequences to individual action selections.

MDPs are a mathematically idealized form of the reinforcement learning problem for which precise theoretical statements can be made. We introduce key elements of the problem’s mathematical structure, such as returns, value functions, and Bellman equations. We try to convey the wide range of applications that can be formulated as finite MDPs. As in all of artificial intelligence, there is a tension between breadth of applicability and mathematical tractability. In this chapter we introduce this tension and discuss some of the trade-offs and challenges that it implies. Some ways in which reinforcement learning can be taken beyond MDPs are treated in Chapter 17.

3.1 The Agent–Environment Interface

MDPs are meant to be a straightforward framing of the problem of learning from interaction to achieve a goal. The learner and decision maker is called the *agent*. The thing it interacts with, comprising everything outside the agent, is called the *environment*. These interact continually, the agent selecting actions and the environment responding to

these actions and presenting new situations to the agent.¹ The environment also gives rise to rewards, special numerical values that the agent seeks to maximize over time through its choice of actions.

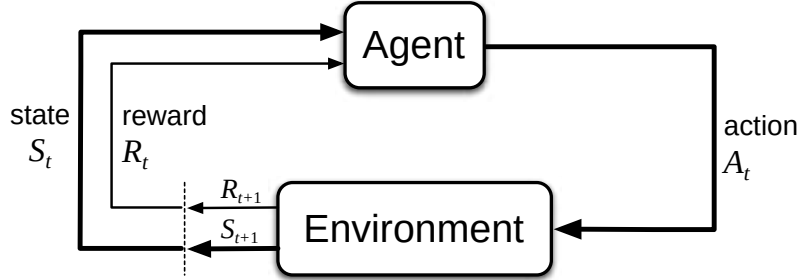


Figure 3.1: The agent–environment interaction in a Markov decision process.

More specifically, the agent and environment interact at each of a sequence of discrete time steps, $t = 0, 1, 2, 3, \dots$ ² At each time step t , the agent receives some representation of the environment’s *state*, $S_t \in \mathcal{S}$, and on that basis selects an *action*, $A_t \in \mathcal{A}(s)$.³ One time step later, in part as a consequence of its action, the agent receives a numerical *reward*, $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$, and finds itself in a new state, S_{t+1} .⁴ The MDP and agent together thereby give rise to a sequence or *trajectory* that begins like this:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots \quad (3.1)$$

In a *finite* MDP, the sets of states, actions, and rewards ($\mathcal{S}$, $\mathcal{A}$, and $\mathcal{R}$) all have a finite number of elements. In this case, the random variables R_t and S_t have well defined discrete probability distributions dependent only on the preceding state and action. That is, for particular values of these random variables, $s' \in \mathcal{S}$ and $r \in \mathcal{R}$, there is a probability of those values occurring at time t , given particular values of the preceding state and action:

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}, \quad (3.2)$$

for all $s', s \in \mathcal{S}$, $r \in \mathcal{R}$, and $a \in \mathcal{A}(s)$. The function p defines the *dynamics* of the MDP. The dot over the equals sign in the equation reminds us that it is a definition (in this case of the function p) rather than a fact that follows from previous definitions. The dynamics function $p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is an ordinary deterministic function of four arguments. The ‘|’ in the middle of it comes from the notation for conditional probability,

¹We use the terms *agent*, *environment*, and *action* instead of the engineers’ terms *controller*, *controlled system* (or *plant*), and *control signal* because they are meaningful to a wider audience.

²We restrict attention to discrete time to keep things as simple as possible, even though many of the ideas can be extended to the continuous-time case (e.g., see Bertsekas and Tsitsiklis, 1996; Doya, 1996).

³To simplify notation, we sometimes assume the special case in which the action set is the same in all states and write it simply as $\mathcal{A}$.

⁴We use R_{t+1} instead of R_t to denote the reward due to A_t because it emphasizes that the next reward and next state, R_{t+1} and S_{t+1} , are jointly determined. Unfortunately, both conventions are widely used in the literature.

but here it just reminds us that p specifies a probability distribution for each choice of s and a , that is, that

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s). \quad (3.3)$$

In a *Markov* decision process, the probabilities given by p completely characterize the environment's dynamics. That is, the probability of each possible value for S_t and R_t depends only on the immediately preceding state and action, S_{t-1} and A_{t-1} , and, given them, not at all on earlier states and actions. This is best viewed a restriction not on the decision process, but on the *state*. The state must include information about all aspects of the past agent–environment interaction that make a difference for the future. If it does, then the state is said to have the *Markov property*. We will assume the Markov property throughout this book, though starting in Part II we will consider approximation methods that do not rely on it, and in Chapter 17 we consider how a Markov state can be learned and constructed from non-Markov observations.

From the four-argument dynamics function, p , one can compute anything else one might want to know about the environment, such as the *state-transition probabilities* (which we denote, with a slight abuse of notation, as a three-argument function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$),

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a). \quad (3.4)$$

We can also compute the expected rewards for state–action pairs as a two-argument function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$:

$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a), \quad (3.5)$$

and the expected rewards for state–action–next-state triples as a three-argument function $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$,

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)}. \quad (3.6)$$

In this book, we usually use the four-argument p function (3.2), but each of these other notations are also occasionally convenient.

The MDP framework is abstract and flexible and can be applied to many different problems in many different ways. For example, the time steps need not refer to fixed intervals of real time; they can refer to arbitrary successive stages of decision making and acting. The actions can be low-level controls, such as the voltages applied to the motors of a robot arm, or high-level decisions, such as whether or not to have lunch or to go to graduate school. Similarly, the states can take a wide variety of forms. They can be completely determined by low-level sensations, such as direct sensor readings, or they can be more high-level and abstract, such as symbolic descriptions of objects in a room. Some of what makes up a state could be based on memory of past sensations or

even be entirely mental or subjective. For example, an agent could be in the state of not being sure where an object is, or of having just been surprised in some clearly defined sense. Similarly, some actions might be totally mental or computational. For example, some actions might control what an agent chooses to think about, or where it focuses its attention. In general, actions can be any decisions we want to learn how to make, and the states can be anything we can know that might be useful in making them.

In particular, the boundary between agent and environment is typically not the same as the physical boundary of a robot's or animal's body. Usually, the boundary is drawn closer to the agent than that. For example, the motors and mechanical linkages of a robot and its sensing hardware should usually be considered parts of the environment rather than parts of the agent. Similarly, if we apply the MDP framework to a person or animal, the muscles, skeleton, and sensory organs should be considered part of the environment. Rewards, too, presumably are computed inside the physical bodies of natural and artificial learning systems, but are considered external to the agent.

The general rule we follow is that anything that cannot be changed arbitrarily by the agent is considered to be outside of it and thus part of its environment. We do not assume that everything in the environment is unknown to the agent. For example, the agent often knows quite a bit about how its rewards are computed as a function of its actions and the states in which they are taken. But we always consider the reward computation to be external to the agent because it defines the task facing the agent and thus must be beyond its ability to change arbitrarily. In fact, in some cases the agent may know *everything* about how its environment works and still face a difficult reinforcement learning task, just as we may know exactly how a puzzle like Rubik's cube works, but still be unable to solve it. The agent–environment boundary represents the limit of the agent's *absolute control*, not of its knowledge.

The agent–environment boundary can be located at different places for different purposes. In a complicated robot, many different agents may be operating at once, each with its own boundary. For example, one agent may make high-level decisions which form part of the states faced by a lower-level agent that implements the high-level decisions. In practice, the agent–environment boundary is determined once one has selected particular states, actions, and rewards, and thus has identified a specific decision making task of interest.

The MDP framework is a considerable abstraction of the problem of goal-directed learning from interaction. It proposes that whatever the details of the sensory, memory, and control apparatus, and whatever objective one is trying to achieve, any problem of learning goal-directed behavior can be reduced to three signals passing back and forth between an agent and its environment: one signal to represent the choices made by the agent (the actions), one signal to represent the basis on which the choices are made (the states), and one signal to define the agent's goal (the rewards). This framework may not be sufficient to represent all decision-learning problems usefully, but it has proved to be widely useful and applicable.

Of course, the particular states and actions vary greatly from task to task, and how they are represented can strongly affect performance. In reinforcement learning, as in other kinds of learning, such representational choices are at present more art than science.

In this book we offer some advice and examples regarding good ways of representing states and actions, but our primary focus is on general principles for learning how to behave once the representations have been selected.

Example 3.1: Bioreactor Suppose reinforcement learning is being applied to determine moment-by-moment temperatures and stirring rates for a bioreactor (a large vat of nutrients and bacteria used to produce useful chemicals). The actions in such an application might be target temperatures and target stirring rates that are passed to lower-level control systems that, in turn, directly activate heating elements and motors to attain the targets. The states are likely to be thermocouple and other sensory readings, perhaps filtered and delayed, plus symbolic inputs representing the ingredients in the vat and the target chemical. The rewards might be moment-by-moment measures of the rate at which the useful chemical is produced by the bioreactor. Notice that here each state is a list, or vector, of sensor readings and symbolic inputs, and each action is a vector consisting of a target temperature and a stirring rate. It is typical of reinforcement learning tasks to have states and actions with such structured representations. Rewards, on the other hand, are always single numbers. ■

Example 3.2: Pick-and-Place Robot Consider using reinforcement learning to control the motion of a robot arm in a repetitive pick-and-place task. If we want to learn movements that are fast and smooth, the learning agent will have to control the motors directly and have low-latency information about the current positions and velocities of the mechanical linkages. The actions in this case might be the voltages applied to each motor at each joint, and the states might be the latest readings of joint angles and velocities. The reward might be +1 for each object successfully picked up and placed. To encourage smooth movements, on each time step a small, negative reward can be given as a function of the moment-to-moment “jerkiness” of the motion. ■

Exercise 3.1 Devise three example tasks of your own that fit into the MDP framework, identifying for each its states, actions, and rewards. Make the three examples as *different* from each other as possible. The framework is abstract and flexible and can be applied in many different ways. Stretch its limits in some way in at least one of your examples. □

Exercise 3.2 Is the MDP framework adequate to usefully represent *all* goal-directed learning tasks? Can you think of any clear exceptions? □

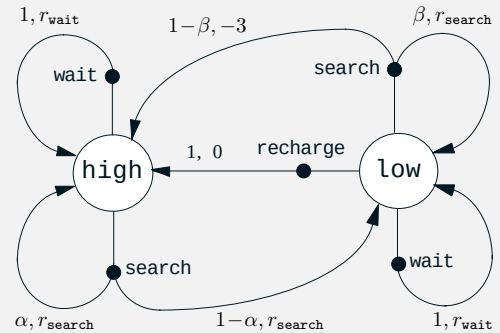
Exercise 3.3 Consider the problem of driving. You could define the actions in terms of the accelerator, steering wheel, and brake, that is, where your body meets the machine. Or you could define them farther out—say, where the rubber meets the road, considering your actions to be tire torques. Or you could define them farther in—say, where your brain meets your body, the actions being muscle twitches to control your limbs. Or you could go to a really high level and say that your actions are your choices of *where* to drive. What is the right level, the right place to draw the line between agent and environment? On what basis is one location of the line to be preferred over another? Is there any fundamental reason for preferring one location over another, or is it a free choice? □

Example 3.3 Recycling Robot

A mobile robot has the job of collecting empty soda cans in an office environment. It has sensors for detecting cans, and an arm and gripper that can pick them up and place them in an onboard bin; it runs on a rechargeable battery. The robot's control system has components for interpreting sensory information, for navigating, and for controlling the arm and gripper. High-level decisions about how to search for cans are made by a reinforcement learning agent based on the current charge level of the battery. To make a simple example, we assume that only two charge levels can be distinguished, comprising a small state set $\mathcal{S} = \{\text{high}, \text{low}\}$. In each state, the agent can decide whether to (1) actively **search** for a can for a certain period of time, (2) remain stationary and **wait** for someone to bring it a can, or (3) head back to its home base to **recharge** its battery. When the energy level is **high**, recharging would always be foolish, so we do not include it in the action set for this state. The action sets are then $\mathcal{A}(\text{high}) = \{\text{search}, \text{wait}\}$ and $\mathcal{A}(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$.

The rewards are zero most of the time, but become positive when the robot secures an empty can, or large and negative if the battery runs all the way down. The best way to find cans is to actively search for them, but this runs down the robot's battery, whereas waiting does not. Whenever the robot is searching, the possibility exists that its battery will become depleted. In this case the robot must shut down and wait to be rescued (producing a low reward). If the energy level is **high**, then a period of active search can always be completed without risk of depleting the battery. A period of searching that begins with a **high** energy level leaves the energy level **high** with probability α and reduces it to **low** with probability $1 - \alpha$. On the other hand, a period of searching undertaken when the energy level is **low** leaves it **low** with probability β and depletes the battery with probability $1 - \beta$. In the latter case, the robot must be rescued, and the battery is then recharged back to **high**. Each can collected by the robot counts as a unit reward, whereas a reward of -3 results whenever the robot has to be rescued. Let r_{search} and r_{wait} , with $r_{\text{search}} > r_{\text{wait}}$, respectively denote the expected number of cans the robot will collect (and hence the expected reward) while searching and while waiting. Finally, suppose that no cans can be collected during a run home for recharging, and that no cans can be collected on a step in which the battery is depleted. This system is then a finite MDP, and we can write down the transition probabilities and the expected rewards, with dynamics as indicated in the table on the left:

s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	-
low	wait	high	0	-
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	-



Note that there is a row in the table for each possible combination of current state, s , action, $a \in \mathcal{A}(s)$, and next state, s' . Some transitions have zero probability of occurring, so no expected reward is specified for them. Shown on the right is another useful way of

summarizing the dynamics of a finite MDP, as a *transition graph*. There are two kinds of nodes: *state nodes* and *action nodes*. There is a state node for each possible state (a large open circle labeled by the name of the state), and an action node for each state–action pair (a small solid circle labeled by the name of the action and connected by a line to the state node). Starting in state s and taking action a moves you along the line from state node s to action node (s, a) . Then the environment responds with a transition to the next state’s node via one of the arrows leaving action node (s, a) . Each arrow corresponds to a triple (s, s', a) , where s' is the next state, and we label the arrow with the transition probability, $p(s'|s, a)$, and the expected reward for that transition, $r(s, a, s')$. Note that the transition probabilities labeling the arrows leaving an action node always sum to 1.

Exercise 3.4 Give a table analogous to that in Example 3.3, but for $p(s', r|s, a)$. It should have columns for s , a , s' , r , and $p(s', r|s, a)$, and a row for every 4-tuple for which $p(s', r|s, a) > 0$. $\square$

3.2 Goals and Rewards

In reinforcement learning, the purpose or goal of the agent is formalized in terms of a special signal, called the *reward*, passing from the environment to the agent. At each time step, the reward is a simple number, $R_t \in \mathbb{R}$. Informally, the agent’s goal is to maximize the total amount of reward it receives. This means maximizing not immediate reward, but cumulative reward in the long run. We can clearly state this informal idea as the *reward hypothesis*:

That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).

The use of a reward signal to formalize the idea of a goal is one of the most distinctive features of reinforcement learning.

Although formulating goals in terms of reward signals might at first appear limiting, in practice it has proved to be flexible and widely applicable. The best way to see this is to consider examples of how it has been, or could be, used. For example, to make a robot learn to walk, researchers have provided reward on each time step proportional to the robot’s forward motion. In making a robot learn how to escape from a maze, the reward is often -1 for every time step that passes prior to escape; this encourages the agent to escape as quickly as possible. To make a robot learn to find and collect empty soda cans for recycling, one might give it a reward of zero most of the time, and then a reward of $+1$ for each can collected. One might also want to give the robot negative rewards when it bumps into things or when somebody yells at it. For an agent to learn to play checkers or chess, the natural rewards are $+1$ for winning, -1 for losing, and 0 for drawing and for all nonterminal positions.

You can see what is happening in all of these examples. The agent always learns to maximize its reward. If we want it to do something for us, we must provide rewards to it in such a way that in maximizing them the agent will also achieve our goals. It

is thus critical that the rewards we set up truly indicate what we want accomplished. In particular, the reward signal is not the place to impart to the agent prior knowledge about *how* to achieve what we want it to do.⁵ For example, a chess-playing agent should be rewarded only for actually winning, not for achieving subgoals such as taking its opponent's pieces or gaining control of the center of the board. If achieving these sorts of subgoals were rewarded, then the agent might find a way to achieve them without achieving the real goal. For example, it might find a way to take the opponent's pieces even at the cost of losing the game. The reward signal is your way of communicating to the robot *what* you want it to achieve, not *how* you want it achieved.⁶

3.3 Returns and Episodes

So far we have discussed the objective of learning informally. We have said that the agent's goal is to maximize the cumulative reward it receives in the long run. How might this be defined formally? If the sequence of rewards received after time step t is denoted $R_{t+1}, R_{t+2}, R_{t+3}, \dots$, then what precise aspect of this sequence do we wish to maximize? In general, we seek to maximize the *expected return*, where the return, denoted G_t , is defined as some specific function of the reward sequence. In the simplest case the return is the sum of the rewards:

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T, \quad (3.7)$$

where T is a final time step. This approach makes sense in applications in which there is a natural notion of final time step, that is, when the agent–environment interaction breaks naturally into subsequences, which we call *episodes*,⁷ such as plays of a game, trips through a maze, or any sort of repeated interaction. Each episode ends in a special state called the *terminal state*, followed by a reset to a standard starting state or to a sample from a standard distribution of starting states. Even if you think of episodes as ending in different ways, such as winning and losing a game, the next episode begins independently of how the previous one ended. Thus the episodes can all be considered to end in the same terminal state, with different rewards for the different outcomes. Tasks with episodes of this kind are called *episodic tasks*. In episodic tasks we sometimes need to distinguish the set of all nonterminal states, denoted $\mathcal{S}$, from the set of all states plus the terminal state, denoted $\mathcal{S}^+$. The time of termination, T , is a random variable that normally varies from episode to episode.

On the other hand, in many cases the agent–environment interaction does not break naturally into identifiable episodes, but goes on continually without limit. For example, this would be the natural way to formulate an on-going process-control task, or an application to a robot with a long life span. We call these *continuing tasks*. The return formulation (3.7) is problematic for continuing tasks because the final time step would

⁵Better places for imparting this kind of prior knowledge are the initial policy or initial value function, or in influences on these.

⁶Section 17.4 delves further into the issue of designing effective reward signals.

⁷Episodes are sometimes called “trials” in the literature.

be $T = \infty$, and the return, which is what we are trying to maximize, could itself easily be infinite. (For example, suppose the agent receives a reward of +1 at each time step.) Thus, in this book we usually use a definition of return that is slightly more complex conceptually but much simpler mathematically.

The additional concept that we need is that of *discounting*. According to this approach, the agent tries to select actions so that the sum of the discounted rewards it receives over the future is maximized. In particular, it chooses A_t to maximize the expected *discounted return*:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad (3.8)$$

where γ is a parameter, $0 \leq \gamma \leq 1$, called the *discount rate*.

The discount rate determines the present value of future rewards: a reward received k time steps in the future is worth only γ^{k-1} times what it would be worth if it were received immediately. If $\gamma < 1$, the infinite sum in (3.8) has a finite value as long as the reward sequence $\{R_k\}$ is bounded. If $\gamma = 0$, the agent is “myopic” in being concerned only with maximizing immediate rewards: its objective in this case is to learn how to choose A_t so as to maximize only R_{t+1} . If each of the agent’s actions happened to influence only the immediate reward, not future rewards as well, then a myopic agent could maximize (3.8) by separately maximizing each immediate reward. But in general, acting to maximize immediate reward can reduce access to future rewards so that the return is reduced. As γ approaches 1, the return objective takes future rewards into account more strongly; the agent becomes more farsighted.

Returns at successive time steps are related to each other in a way that is important for the theory and algorithms of reinforcement learning:

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \quad (3.9)$$

Note that this works for all time steps $t < T$, even if termination occurs at $t + 1$, if we define $G_T = 0$. This often makes it easy to compute returns from reward sequences.

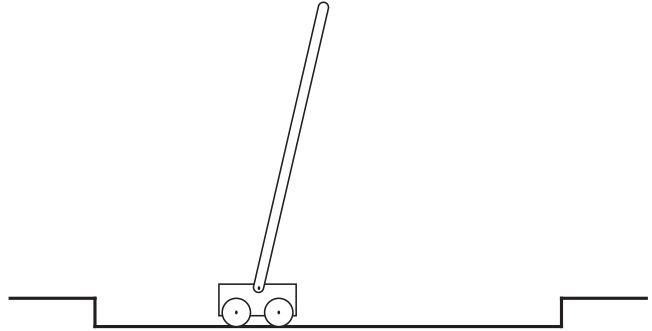
Note that although the return (3.8) is a sum of an infinite number of terms, it is still finite if the reward is nonzero and constant—if $\gamma < 1$. For example, if the reward is a constant +1, then the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}. \quad (3.10)$$

Exercise 3.5 The equations in Section 3.1 are for the continuing case and need to be modified (very slightly) to apply to episodic tasks. Show that you know the modifications needed by giving the modified version of (3.3). $\square$

Example 3.4: Pole-Balancing

The objective in this task is to apply forces to a cart moving along a track so as to keep a pole hinged to the cart from falling over: A failure is said to occur if the pole falls past a given angle from vertical or if the cart runs off the track. The pole is reset to vertical after each failure. This task could be



treated as episodic, where the natural episodes are the repeated attempts to balance the pole. The reward in this case could be +1 for every time step on which failure did not occur, so that the return at each time would be the number of steps until failure. In this case, successful balancing forever would mean a return of infinity. Alternatively, we could treat pole-balancing as a continuing task, using discounting. In this case the reward would be -1 on each failure and zero at all other times. The return at each time would then be related to $-\gamma^K$, where K is the number of time steps before failure. In either case, the return is maximized by keeping the pole balanced for as long as possible. ■

Exercise 3.6 Suppose you treated pole-balancing as an episodic task but also used discounting, with all rewards zero except for -1 upon failure. What then would the return be at each time? How does this return differ from that in the discounted, continuing formulation of this task? □

Exercise 3.7 Imagine that you are designing a robot to run a maze. You decide to give it a reward of $+1$ for escaping from the maze and a reward of zero at all other times. The task seems to break down naturally into episodes—the successive runs through the maze—so you decide to treat it as an episodic task, where the goal is to maximize expected total reward (3.7). After running the learning agent for a while, you find that it is showing no improvement in escaping from the maze. What is going wrong? Have you effectively communicated to the agent what you want it to achieve? □

Exercise 3.8 Suppose $\gamma = 0.5$ and the following sequence of rewards is received $R_1 = -1$, $R_2 = 2$, $R_3 = 6$, $R_4 = 3$, and $R_5 = 2$, with $T = 5$. What are G_0 , G_1 , ..., G_5 ? Hint: Work backwards. □

Exercise 3.9 Suppose $\gamma = 0.9$ and the reward sequence is $R_1 = 2$ followed by an infinite sequence of 7s. What are G_1 and G_0 ? □

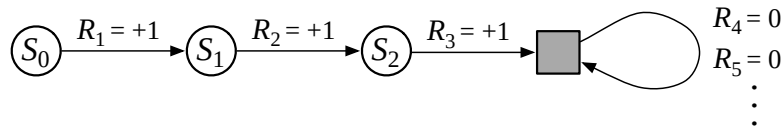
Exercise 3.10 Prove the second equality in (3.10). □

3.4 Unified Notation for Episodic and Continuing Tasks

In the preceding section we described two kinds of reinforcement learning tasks, one in which the agent–environment interaction naturally breaks down into a sequence of separate episodes (episodic tasks), and one in which it does not (continuing tasks). The former case is mathematically easier because each action affects only the finite number of rewards subsequently received during the episode. In this book we consider sometimes one kind of problem and sometimes the other, but often both. It is therefore useful to establish one notation that enables us to talk precisely about both cases simultaneously.

To be precise about episodic tasks requires some additional notation. Rather than one long sequence of time steps, we need to consider a series of episodes, each of which consists of a finite sequence of time steps. We number the time steps of each episode starting anew from zero. Therefore, we have to refer not just to S_t , the state representation at time t , but to $S_{t,i}$, the state representation at time t of episode i (and similarly for $A_{t,i}$, $R_{t,i}$, $\pi_{t,i}$, T_i , etc.). However, it turns out that when we discuss episodic tasks we almost never have to distinguish between different episodes. We are almost always considering a particular single episode, or stating something that is true for all episodes. Accordingly, in practice we almost always abuse notation slightly by dropping the explicit reference to episode number. That is, we write S_t to refer to $S_{t,i}$, and so on.

We need one other convention to obtain a single notation that covers both episodic and continuing tasks. We have defined the return as a sum over a finite number of terms in one case (3.7) and as a sum over an infinite number of terms in the other (3.8). These two can be unified by considering episode termination to be the entering of a special *absorbing state* that transitions only to itself and that generates only rewards of zero. For example, consider the state transition diagram:



Here the solid square represents the special absorbing state corresponding to the end of an episode. Starting from S_0 , we get the reward sequence $+1, +1, +1, 0, 0, 0, \dots$. Summing these, we get the same return whether we sum over the first T rewards (here $T = 3$) or over the full infinite sequence. This remains true even if we introduce discounting. Thus, we can define the return, in general, according to (3.8), using the convention of omitting episode numbers when they are not needed, and including the possibility that $\gamma = 1$ if the sum remains defined (e.g., because all episodes terminate). Alternatively, we can write

$$G_t \doteq \sum_{k=t+1}^T \gamma^{k-t-1} R_k, \quad (3.11)$$

including the possibility that $T = \infty$ or $\gamma = 1$ (but not both). We use these conventions throughout the rest of the book to simplify notation and to express the close parallels between episodic and continuing tasks. (Later, in Chapter 10, we will introduce a formulation that is both continuing and undiscounted.)

3.5 Policies and Value Functions

Almost all reinforcement learning algorithms involve estimating *value functions*—functions of states (or of state–action pairs) that estimate *how good* it is for the agent to be in a given state (or how good it is to perform a given action in a given state). The notion of “how good” here is defined in terms of future rewards that can be expected, or, to be precise, in terms of expected return. Of course the rewards the agent can expect to receive in the future depend on what actions it will take. Accordingly, value functions are defined with respect to particular ways of acting, called policies.

Formally, a *policy* is a mapping from states to probabilities of selecting each possible action. If the agent is following policy π at time t , then $\pi(a|s)$ is the probability that $A_t = a$ if $S_t = s$. Like p , π is an ordinary function; the “|” in the middle of $\pi(a|s)$ merely reminds that it defines a probability distribution over $a \in \mathcal{A}(s)$ for each $s \in \mathcal{S}$. Reinforcement learning methods specify how the agent’s policy is changed as a result of its experience.

Exercise 3.11 If the current state is S_t , and actions are selected according to stochastic policy π , then what is the expectation of R_{t+1} in terms of π and the four-argument function p (3.2)? $\square$

The *value function* of a state s under a policy π , denoted $v_\pi(s)$, is the expected return when starting in s and following π thereafter. For MDPs, we can define v_π formally by

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \text{ for all } s \in \mathcal{S}, \quad (3.12)$$

where $\mathbb{E}_\pi[\cdot]$ denotes the expected value of a random variable given that the agent follows policy π , and t is any time step. Note that the value of the terminal state, if any, is always zero. We call the function v_π the *state-value function for policy π* .

Similarly, we define the value of taking action a in state s under a policy π , denoted $q_\pi(s, a)$, as the expected return starting from s , taking the action a , and thereafter following policy π :

$$q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]. \quad (3.13)$$

We call q_π the *action-value function for policy π* .

Exercise 3.12 Give an equation for v_π in terms of q_π and π . $\square$

Exercise 3.13 Give an equation for q_π in terms of v_π and the four-argument p . $\square$

The value functions v_π and q_π can be estimated from experience. For example, if an agent follows policy π and maintains an average, for each state encountered, of the actual returns that have followed that state, then the average will converge to the state’s value, $v_\pi(s)$, as the number of times that state is encountered approaches infinity. If separate averages are kept for each action taken in each state, then these averages will similarly converge to the action values, $q_\pi(s, a)$. We call estimation methods of this kind *Monte Carlo methods* because they involve averaging over many random samples of actual returns.

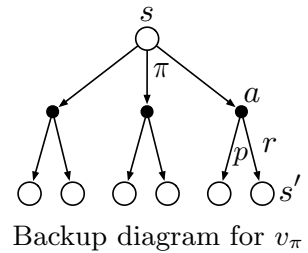
These kinds of methods are presented in Chapter 5. Of course, if there are very many states, then it may not be practical to keep separate averages for each state individually. Instead, the agent would have to maintain v_π and q_π as parameterized functions (with fewer parameters than states) and adjust the parameters to better match the observed returns. This can also produce accurate estimates, although much depends on the nature of the parameterized function approximator. These possibilities are discussed in Part II of the book.

A fundamental property of value functions used throughout reinforcement learning and dynamic programming is that they satisfy recursive relationships similar to that which we have already established for the return (3.9). For any policy π and any state s , the following consistency condition holds between the value of s and the value of its possible successor states:

$$\begin{aligned}
 v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
 &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] && \text{(by (3.9))} \\
 &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r|s, a) \left[r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s'] \right] \\
 &= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) \left[r + \gamma v_\pi(s') \right], \quad \text{for all } s \in \mathcal{S},
 \end{aligned} \tag{3.14}$$

where it is implicit that the actions, a , are taken from the set $\mathcal{A}(s)$, that the next states, s' , are taken from the set $\mathcal{S}$ (or from $\mathcal{S}^+$ in the case of an episodic problem), and that the rewards, r , are taken from the set $\mathcal{R}$. Note also how in the last equation we have merged the two sums, one over all the values of s' and the other over all the values of r , into one sum over all the possible values of both. We use this kind of merged sum often to simplify formulas. Note how the final expression can be read easily as an expected value. It is really a sum over all values of the three variables, a , s' , and r . For each triple, we compute its probability, $\pi(a|s)p(s', r|s, a)$, weight the quantity in brackets by that probability, then sum over all possibilities to get an expected value.

Equation (3.14) is the *Bellman equation for v_π* . It expresses a relationship between the value of a state and the values of its successor states. Think of looking ahead from a state to its possible successor states, as suggested by the diagram to the right. Each open circle represents a state and each solid circle represents a state–action pair. Starting from state s , the root node at the top, the agent could take any of some set of actions—three are shown in the diagram—based on its policy π . From



each of these, the environment could respond with one of several next states, s' (two are shown in the figure), along with a reward, r , depending on its dynamics given by the function p . The Bellman equation (3.14) averages over all the possibilities, weighting each by its probability of occurring. It states that the value of the start state must equal the (discounted) value of the expected next state, plus the reward expected along the way.

The value function v_π is the unique solution to its Bellman equation. We show in subsequent chapters how this Bellman equation forms the basis of a number of ways to

compute, approximate, and learn v_π . We call diagrams like that above *backup diagrams* because they diagram relationships that form the basis of the update or *backup* operations that are at the heart of reinforcement learning methods. These operations transfer value information *back* to a state (or a state–action pair) from its successor states (or state–action pairs). We use backup diagrams throughout the book to provide graphical summaries of the algorithms we discuss. (Note that, unlike transition graphs, the state nodes of backup diagrams do not necessarily represent distinct states; for example, a state might be its own successor.)

Example 3.5: Gridworld Figure 3.2 (left) shows a rectangular gridworld representation of a simple finite MDP. The cells of the grid correspond to the states of the environment. At each cell, four actions are possible: **north**, **south**, **east**, and **west**, which deterministically cause the agent to move one cell in the respective direction on the grid. Actions that would take the agent off the grid leave its location unchanged, but also result in a reward of -1 . Other actions result in a reward of 0, except those that move the agent out of the special states A and B. From state A, all four actions yield a reward of $+10$ and take the agent to A' . From state B, all actions yield a reward of $+5$ and take the agent to B' .

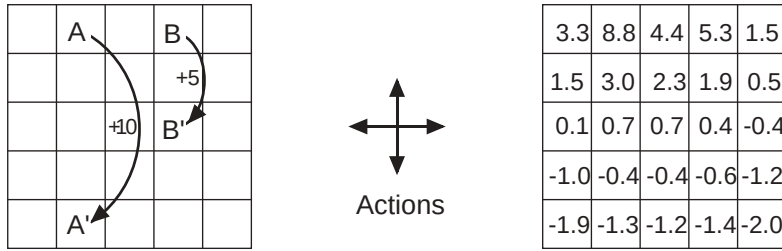


Figure 3.2: Gridworld example: exceptional reward dynamics (left) and state-value function for the equiprobable random policy (right).

Suppose the agent selects all four actions with equal probability in all states. Figure 3.2 (right) shows the value function, v_π , for this policy, for the discounted reward case with $\gamma = 0.9$. This value function was computed by solving the system of linear equations (3.14). Notice the negative values near the lower edge; these are the result of the high probability of hitting the edge of the grid there under the random policy. State A is the best state to be in under this policy, but its expected return is less than 10, its immediate reward, because from A the agent is taken to A' , from which it is likely to run into the edge of the grid. State B, on the other hand, is valued more than 5, its immediate reward, because from B the agent is taken to B' , which has a positive value. From B' the expected penalty (negative reward) for possibly running into an edge is more than compensated for by the expected gain for possibly stumbling onto A or B. ■

Exercise 3.14 The Bellman equation (3.14) must hold for each state for the value function v_π shown in Figure 3.2 (right) of Example 3.5. Show numerically that this equation holds for the center state, valued at $+0.7$, with respect to its four neighboring states, valued at $+2.3$, $+0.4$, -0.4 , and $+0.7$. (These numbers are accurate only to one decimal place.) □

Exercise 3.15 In the gridworld example, rewards are positive for goals, negative for running into the edge of the world, and zero the rest of the time. Are the signs of these

rewards important, or only the intervals between them? Prove, using (3.8), that adding a constant c to all the rewards adds a constant, v_c , to the values of all states, and thus does not affect the relative values of any states under any policies. What is v_c in terms of c and γ ? $\square$

Exercise 3.16 Now consider adding a constant c to all the rewards in an episodic task, such as maze running. Would this have any effect, or would it leave the task unchanged as in the continuing task above? Why or why not? Give an example. $\square$

Example 3.6: Golf To formulate playing a hole of golf as a reinforcement learning task, we count a penalty (negative reward) of -1 for each stroke until we hit the ball into the hole. The state is the location of the ball. The value of a state is the negative of the number of strokes to the hole from that location. Our actions are how we aim and swing at the ball, of course, and which club we select. Let us take the former as given and consider just the choice of club, which we assume is either a putter or a driver. The upper part of Figure 3.3 shows a possible state-value function, $v_{\text{putt}}(s)$, for the policy that always uses the putter. The terminal state *in-the-hole* has a value of 0. From anywhere on the green we assume we can make a putt; these states have value -1 . Off the green we cannot reach the hole by putting, and the value is greater. If we can reach the green from a state by putting, then that state must have value one less than the green's value, that is, -2 . For simplicity, let us assume we can putt very precisely and deterministically, but with a limited range. This gives us the sharp contour line labeled -2 in the figure; all locations between that line and the green require exactly two strokes to complete the hole. Similarly, any location within putting range of the -2 contour line must have a value of -3 , and so on to get all the contour lines shown in the figure. Putting doesn't get us out of sand traps, so they have a value of $-\infty$. Overall, it takes us six strokes to get from the tee to the hole by putting.

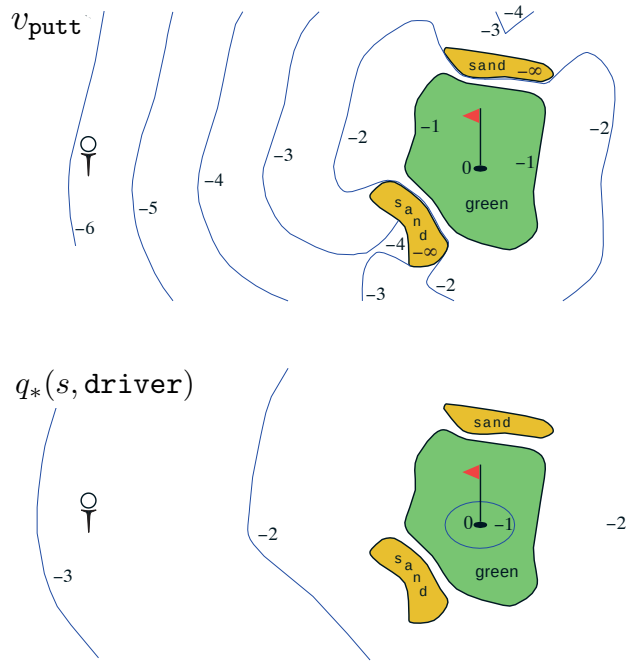
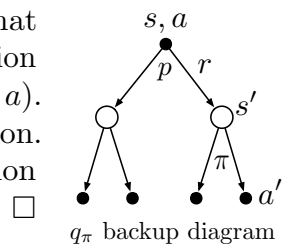
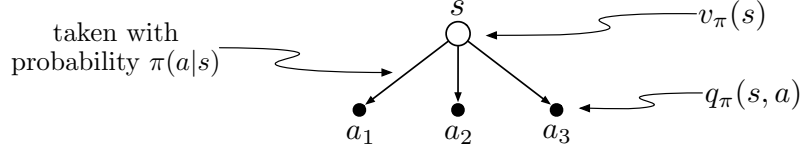


Figure 3.3: A golf example: the state-value function for putting (upper) and the optimal action-value function for using the driver (lower). $\blacksquare$

Exercise 3.17 What is the Bellman equation for action values, that is, for q_π ? It must give the action value $q_\pi(s, a)$ in terms of the action values, $q_\pi(s', a')$, of possible successors to the state-action pair (s, a) . Hint: the backup diagram to the right corresponds to this equation. Show the sequence of equations analogous to (3.14), but for action values. $\square$

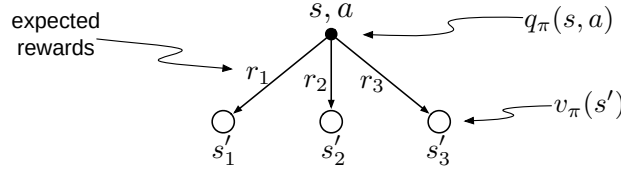


Exercise 3.18 The value of a state depends on the values of the actions possible in that state and on how likely each action is to be taken under the current policy. We can think of this in terms of a small backup diagram rooted at the state and considering each possible action:



Give the equation corresponding to this intuition and diagram for the value at the root node, $v_\pi(s)$, in terms of the value at the expected leaf node, $q_\pi(s, a)$, given $S_t = s$. This equation should include an expectation conditioned on following the policy, π . Then give a second equation in which the expected value is written out explicitly in terms of $\pi(a|s)$ such that no expected value notation appears in the equation. $\square$

Exercise 3.19 The value of an action, $q_\pi(s, a)$, depends on the expected next reward and the expected sum of the remaining rewards. Again we can think of this in terms of a small backup diagram, this one rooted at an action (state–action pair) and branching to the possible next states:



Give the equation corresponding to this intuition and diagram for the action value, $q_\pi(s, a)$, in terms of the expected next reward, R_{t+1} , and the expected next state value, $v_\pi(S_{t+1})$, given that $S_t = s$ and $A_t = a$. This equation should include an expectation but *not* one conditioned on following the policy. Then give a second equation, writing out the expected value explicitly in terms of $p(s', r|s, a)$ defined by (3.2), such that no expected value notation appears in the equation. $\square$

3.6 Optimal Policies and Optimal Value Functions

Solving a reinforcement learning task means, roughly, finding a policy that achieves a lot of reward over the long run. For finite MDPs, we can precisely define an optimal policy in the following way. Value functions define a partial ordering over policies. A policy π is defined to be better than or equal to a policy π' if its expected return is greater than or equal to that of π' for all states. In other words, $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. There is always at least one policy that is better than or equal to all other policies. This is an *optimal policy*. Although there may be more than one, we denote all the optimal policies by π_* . They share the same state-value function, called the *optimal state-value function*, denoted v_* , and defined as

$$v_*(s) \doteq \max_{\pi} v_\pi(s), \quad (3.15)$$

for all $s \in \mathcal{S}$.

Optimal policies also share the same *optimal action-value function*, denoted q_* , and defined as

$$q_*(s, a) \doteq \max_{\pi} q_{\pi}(s, a), \quad (3.16)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$. For the state–action pair (s, a) , this function gives the expected return for taking action a in state s and thereafter following an optimal policy. Thus, we can write q_* in terms of v_* as follows:

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]. \quad (3.17)$$

Example 3.7: Optimal Value Functions for Golf The lower part of Figure 3.3 shows the contours of a possible optimal action-value function $q_*(s, \text{driver})$. These are the values of each state if we first play a stroke with the driver and afterward select either the driver or the putter, whichever is better. The driver enables us to hit the ball farther, but with less accuracy. We can reach the hole in one shot using the driver only if we are already very close; thus the -1 contour for $q_*(s, \text{driver})$ covers only a small portion of the green. If we have two strokes, however, then we can reach the hole from much farther away, as shown by the -2 contour. In this case we don't have to drive all the way to within the small -1 contour, but only to anywhere on the green; from there we can use the putter. The optimal action-value function gives the values after committing to a particular *first* action, in this case, to the driver, but afterward using whichever actions are best. The -3 contour is still farther out and includes the starting tee. From the tee, the best sequence of actions is two drives and one putt, sinking the ball in three strokes. ■

Because v_* is the value function for a policy, it must satisfy the self-consistency condition given by the Bellman equation for state values (3.14). Because it is the optimal value function, however, v_* 's consistency condition can be written in a special form without reference to any specific policy. This is the Bellman equation for v_* , or the *Bellman optimality equation*. Intuitively, the Bellman optimality equation expresses the fact that the value of a state under an optimal policy must equal the expected return for the best action from that state:

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \quad (\text{by (3.9)}) \end{aligned}$$

$$= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \quad (3.18)$$

$$= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \quad (3.19)$$

The last two equations are two forms of the Bellman optimality equation for v_* . The Bellman optimality equation for q_* is

$$\begin{aligned} q_*(s, a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right] \\ &= \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a')\right]. \end{aligned} \quad (3.20)$$

The backup diagrams in the figure below show graphically the spans of future states and actions considered in the Bellman optimality equations for v_* and q_* . These are the same as the backup diagrams for v_π and q_π presented earlier except that arcs have been added at the agent's choice points to represent that the maximum over that choice is taken rather than the expected value given some policy. The backup diagram on the left graphically represents the Bellman optimality equation (3.19) and the backup diagram on the right graphically represents (3.20).

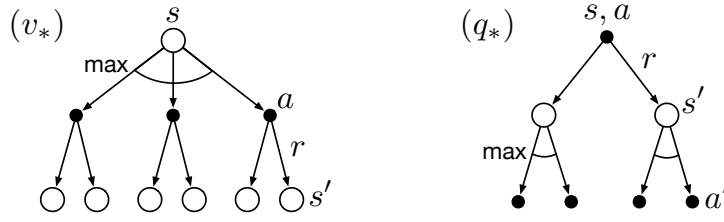


Figure 3.4: Backup diagrams for v_* and q_*

For finite MDPs, the Bellman optimality equation for v_π (3.19) has a unique solution independent of the policy. The Bellman optimality equation is actually a system of equations, one for each state, so if there are n states, then there are n equations in n unknowns. If the dynamics p of the environment are known, then in principle one can solve this system of equations for v_* using any one of a variety of methods for solving systems of nonlinear equations. One can solve a related set of equations for q_* .

Once one has v_* , it is relatively easy to determine an optimal policy. For each state s , there will be one or more actions at which the maximum is obtained in the Bellman optimality equation. Any policy that assigns nonzero probability only to these actions is an optimal policy. You can think of this as a one-step search. If you have the optimal value function, v_* , then the actions that appear best after a one-step search will be optimal actions. Another way of saying this is that any policy that is *greedy* with respect to the optimal evaluation function v_* is an optimal policy. The term greedy is used in computer science to describe any search or decision procedure that selects alternatives based only on local or immediate considerations, without considering the possibility that such a selection may prevent future access to even better alternatives. Consequently, it describes policies that select actions based only on their short-term consequences. The beauty of v_* is that if one uses it to evaluate the short-term consequences of actions—specifically, the one-step consequences—then a greedy policy is actually optimal in the long-term sense in which we are interested because v_* already takes into account the reward consequences of all possible future behavior. By means of v_* , the optimal expected long-term return is turned into a quantity that is locally and immediately available for each state. Hence, a one-step-ahead search yields the long-term optimal actions.

Having q_* makes choosing optimal actions even easier. With q_* , the agent does not even have to do a one-step-ahead search: for any state s , it can simply find any action that maximizes $q_*(s, a)$. The action-value function effectively caches the results of all one-step-ahead searches. It provides the optimal expected long-term return as a value that is locally and immediately available for each state–action pair. Hence, at the cost of

representing a function of state–action pairs, instead of just of states, the optimal action–value function allows optimal actions to be selected without having to know anything about possible successor states and their values, that is, without having to know anything about the environment’s dynamics.

Example 3.8: Solving the Gridworld Suppose we solve the Bellman equation for v_* for the simple grid task introduced in Example 3.5 and shown again in Figure 3.5 (left). Recall that state A is followed by a reward of +10 and transition to state A', while state B is followed by a reward of +5 and transition to state B'. Figure 3.5 (middle) shows the optimal value function, and Figure 3.5 (right) shows the corresponding optimal policies. Where there are multiple arrows in a cell, all of the corresponding actions are optimal.

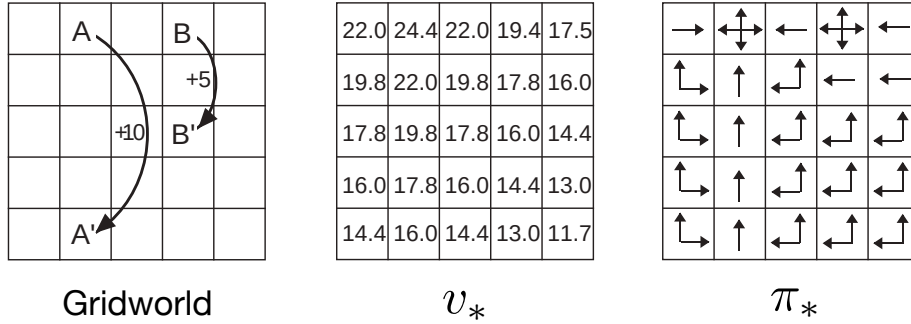


Figure 3.5: Optimal solutions to the gridworld example. ■

Example 3.9: Bellman Optimality Equations for the Recycling Robot Using (3.19), we can explicitly give the Bellman optimality equation for the recycling robot example. To make things more compact, we abbreviate the states **high** and **low**, and the actions **search**, **wait**, and **recharge** respectively by **h**, **l**, **s**, **w**, and **re**. Because there are only two states, the Bellman optimality equation consists of two equations. The equation for $v_*(\mathbf{h})$ can be written as follows:

$$\begin{aligned}
 v_*(\mathbf{h}) &= \max \left\{ \begin{array}{l} p(\mathbf{h}|\mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l}|\mathbf{h}, \mathbf{s})[r(\mathbf{h}, \mathbf{s}, \mathbf{l}) + \gamma v_*(\mathbf{l})], \\ p(\mathbf{h}|\mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{h}) + \gamma v_*(\mathbf{h})] + p(\mathbf{l}|\mathbf{h}, \mathbf{w})[r(\mathbf{h}, \mathbf{w}, \mathbf{l}) + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} \alpha[r_{\mathbf{s}} + \gamma v_*(\mathbf{h})] + (1 - \alpha)[r_{\mathbf{s}} + \gamma v_*(\mathbf{l})], \\ 1[r_{\mathbf{w}} + \gamma v_*(\mathbf{h})] + 0[r_{\mathbf{w}} + \gamma v_*(\mathbf{l})] \end{array} \right\} \\
 &= \max \left\{ \begin{array}{l} r_{\mathbf{s}} + \gamma[\alpha v_*(\mathbf{h}) + (1 - \alpha)v_*(\mathbf{l})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{h}) \end{array} \right\}.
 \end{aligned}$$

Following the same procedure for $v_*(\mathbf{l})$ yields the equation

$$v_*(\mathbf{l}) = \max \left\{ \begin{array}{l} \beta r_{\mathbf{s}} - 3(1 - \beta) + \gamma[(1 - \beta)v_*(\mathbf{h}) + \beta v_*(\mathbf{l})], \\ r_{\mathbf{w}} + \gamma v_*(\mathbf{l}), \\ \gamma v_*(\mathbf{h}) \end{array} \right\}.$$

For any choice of $r_{\mathbf{s}}$, $r_{\mathbf{w}}$, α , β , and γ , with $0 \leq \gamma < 1$, $0 \leq \alpha, \beta \leq 1$, there is exactly one pair of numbers, $v_*(\mathbf{h})$ and $v_*(\mathbf{l})$, that simultaneously satisfy these two nonlinear equations. ■

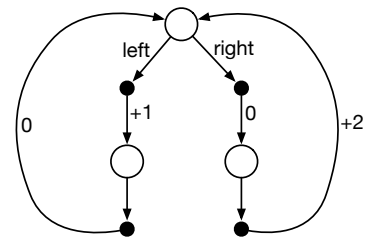
Explicitly solving the Bellman optimality equation provides one route to finding an optimal policy, and thus to solving the reinforcement learning problem. However, this solution is rarely directly useful. It is akin to an exhaustive search, looking ahead at all possibilities, computing their probabilities of occurrence and their desirabilities in terms of expected rewards. This solution relies on at least three assumptions that are rarely true in practice: (1) we accurately know the dynamics of the environment; (2) we have enough computational resources to complete the computation of the solution; and (3) the Markov property. For the kinds of tasks in which we are interested, one is generally not able to implement this solution exactly because various combinations of these assumptions are violated. For example, although the first and third assumptions present no problems for the game of backgammon, the second is a major impediment. Because the game has about 10^{20} states, it would take thousands of years on today's fastest computers to solve the Bellman equation for v_* , and the same is true for finding q_* . In reinforcement learning one typically has to settle for approximate solutions.

Many different decision-making methods can be viewed as ways of approximately solving the Bellman optimality equation. For example, heuristic search methods can be viewed as expanding the right-hand side of (3.19) several times, up to some depth, forming a “tree” of possibilities, and then using a heuristic evaluation function to approximate v_* at the “leaf” nodes. (Heuristic search methods such as A* are almost always based on the episodic case.) The methods of dynamic programming can be related even more closely to the Bellman optimality equation. Many reinforcement learning methods can be clearly understood as approximately solving the Bellman optimality equation, using actual experienced transitions in place of knowledge of the expected transitions. We consider a variety of such methods in the following chapters.

Exercise 3.20 Draw or describe the optimal state-value function for the golf example. $\square$

Exercise 3.21 Draw or describe the contours of the optimal action-value function for putting, $q_*(s, \text{putter})$, for the golf example. $\square$

Exercise 3.22 Consider the continuing MDP shown on to the right. The only decision to be made is that in the top state, where two actions are available, left and right. The numbers show the rewards that are received deterministically after each action. There are exactly two deterministic policies, π_{left} and π_{right} . What policy is optimal if $\gamma = 0$? If $\gamma = 0.9$? If $\gamma = 0.5$? $\square$



Exercise 3.23 Give the Bellman equation for q_* for the recycling robot. $\square$

Exercise 3.24 Figure 3.5 gives the optimal value of the best state of the gridworld as 24.4, to one decimal place. Use your knowledge of the optimal policy and (3.8) to express this value symbolically, and then to compute it to three decimal places. $\square$

Exercise 3.25 Give an equation for v_* in terms of q_* . $\square$

Exercise 3.26 Give an equation for q_* in terms of v_* and the four-argument p . $\square$

Exercise 3.27 Give an equation for π_* in terms of q_* . □

Exercise 3.28 Give an equation for π_* in terms of v_* and the four-argument p . □

Exercise 3.29 Rewrite the four Bellman equations for the four value functions (v_π , v_* , q_π , and q_*) in terms of the three argument function p (3.4) and the two-argument function r (3.5). □

3.7 Optimality and Approximation

We have defined optimal value functions and optimal policies. Clearly, an agent that learns an optimal policy has done very well, but in practice this rarely happens. For the kinds of tasks in which we are interested, optimal policies can be generated only with extreme computational cost. A well-defined notion of optimality organizes the approach to learning we describe in this book and provides a way to understand the theoretical properties of various learning algorithms, but it is an ideal that agents can only approximate to varying degrees. As we discussed above, even if we have a complete and accurate model of the environment's dynamics, it is usually not possible to simply compute an optimal policy by solving the Bellman optimality equation. For example, board games such as chess are a tiny fraction of human experience, yet large, custom-designed computers still cannot compute the optimal moves. A critical aspect of the problem facing the agent is always the computational power available to it, in particular, the amount of computation it can perform in a single time step.

The memory available is also an important constraint. A large amount of memory is often required to build up approximations of value functions, policies, and models. In tasks with small, finite state sets, it is possible to form these approximations using arrays or tables with one entry for each state (or state-action pair). This we call the *tabular* case, and the corresponding methods we call tabular methods. In many cases of practical interest, however, there are far more states than could possibly be entries in a table. In these cases the functions must be approximated, using some sort of more compact parameterized function representation.

Our framing of the reinforcement learning problem forces us to settle for approximations. However, it also presents us with some unique opportunities for achieving useful approximations. For example, in approximating optimal behavior, there may be many states that the agent faces with such a low probability that selecting suboptimal actions for them has little impact on the amount of reward the agent receives. Tesauro's backgammon player, for example, plays with exceptional skill even though it might make very bad decisions on board configurations that never occur in games against experts. In fact, it is possible that TD-Gammon makes bad decisions for a large fraction of the game's state set. The online nature of reinforcement learning makes it possible to approximate optimal policies in ways that put more effort into learning to make good decisions for frequently encountered states, at the expense of less effort for infrequently encountered states. This is one key property that distinguishes reinforcement learning from other approaches to approximately solving MDPs.

3.8 Summary

Let us summarize the elements of the reinforcement learning problem that we have presented in this chapter. Reinforcement learning is about learning from interaction how to behave in order to achieve a goal. The reinforcement learning *agent* and its *environment* interact over a sequence of discrete time steps. The specification of their interface defines a particular task: the *actions* are the choices made by the agent; the *states* are the basis for making the choices; and the *rewards* are the basis for evaluating the choices. Everything inside the agent is completely known and controllable by the agent; everything outside is incompletely controllable but may or may not be completely known. A *policy* is a stochastic rule by which the agent selects actions as a function of states. The agent's objective is to maximize the amount of reward it receives over time.

When the reinforcement learning setup described above is formulated with well defined transition probabilities it constitutes a *Markov decision process* (MDP). A *finite MDP* is an MDP with finite state, action, and (as we formulate it here) reward sets. Much of the current theory of reinforcement learning is restricted to finite MDPs, but the methods and ideas apply more generally.

The *return* is the function of future rewards that the agent seeks to maximize (in expected value). It has several different definitions depending upon the nature of the task and whether one wishes to *discount* delayed reward. The undiscounted formulation is appropriate for *episodic tasks*, in which the agent–environment interaction breaks naturally into *episodes*; the discounted formulation is appropriate for *continuing tasks*, in which the interaction does not naturally break into episodes but continues without limit. We try to define the returns for the two kinds of tasks such that one set of equations can apply to both the episodic and continuing cases.

A policy's *value functions* assign to each state, or state–action pair, the expected return from that state, or state–action pair, given that the agent uses the policy. The *optimal value functions* assign to each state, or state–action pair, the largest expected return achievable by any policy. A policy whose value functions are optimal is an *optimal policy*. Whereas the optimal value functions for states and state–action pairs are unique for a given MDP, there can be many optimal policies. Any policy that is *greedy* with respect to the optimal value functions must be an optimal policy. The *Bellman optimality equations* are special consistency conditions that the optimal value functions must satisfy and that can, in principle, be solved for the optimal value functions, from which an optimal policy can be determined with relative ease.

A reinforcement learning problem can be posed in a variety of different ways depending on assumptions about the level of knowledge initially available to the agent. In problems of *complete knowledge*, the agent has a complete and accurate model of the environment's dynamics. If the environment is an MDP, then such a model consists of the complete four-argument dynamics function p (3.2). In problems of *incomplete knowledge*, a complete and perfect model of the environment is not available.

Even if the agent has a complete and accurate environment model, the agent is typically unable to perform enough computation per time step to fully use it. The memory available is also an important constraint. Memory may be required to build

up accurate approximations of value functions, policies, and models. In most cases of practical interest there are far more states than could possibly be entries in a table, and approximations must be made.

A well-defined notion of optimality organizes the approach to learning we describe in this book and provides a way to understand the theoretical properties of various learning algorithms, but it is an ideal that reinforcement learning agents can only approximate to varying degrees. In reinforcement learning we are very much concerned with cases in which optimal solutions cannot be found but must be approximated in some way.

Bibliographical and Historical Remarks

The reinforcement learning problem is deeply indebted to the idea of Markov decision processes (MDPs) from the field of optimal control. These historical influences and other major influences from psychology are described in the brief history given in Chapter 1. Reinforcement learning adds to MDPs a focus on approximation and incomplete information for realistically large problems. MDPs and the reinforcement learning problem are only weakly linked to traditional learning and decision-making problems in artificial intelligence. However, artificial intelligence is now vigorously exploring MDP formulations for planning and decision making from a variety of perspectives. MDPs are more general than previous formulations used in artificial intelligence in that they permit more general kinds of goals and uncertainty.

The theory of MDPs is treated by, for example, Bertsekas (2005), White (1969), Whittle (1982, 1983), and Puterman (1994). A particularly compact treatment of the finite case is given by Ross (1983). MDPs are also studied under the heading of stochastic optimal control, where *adaptive* optimal control methods are most closely related to reinforcement learning (e.g., Kumar, 1985; Kumar and Varaiya, 1986).

The theory of MDPs evolved from efforts to understand the problem of making sequences of decisions under uncertainty, where each decision can depend on the previous decisions and their outcomes. It is sometimes called the theory of multistage decision processes, or sequential decision processes, and has roots in the statistical literature on sequential sampling beginning with the papers by Thompson (1933, 1934) and Robbins (1952) that we cited in Chapter 2 in connection with bandit problems (which are prototypical MDPs if formulated as multiple-situation problems).

The earliest instance of which we are aware in which reinforcement learning was discussed using the MDP formalism is Andrae's (1969b) description of a unified view of learning machines. Witten and Corbin (1973) experimented with a reinforcement learning system later analyzed by Witten (1977, 1976a) using the MDP formalism. Although he did not explicitly mention MDPs, Werbos (1977) suggested approximate solution methods for stochastic optimal control problems that are related to modern reinforcement learning methods (see also Werbos, 1982, 1987, 1988, 1989, 1992). Although Werbos's ideas were not widely recognized at the time, they were prescient in emphasizing the importance of approximately solving optimal control problems in a variety of domains, including artificial intelligence. The most influential integration of reinforcement learning and MDPs is due to Watkins (1989).

Chapter 6

Temporal-Difference Learning

If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be *temporal-difference* (TD) learning. TD learning is a combination of Monte Carlo ideas and dynamic programming (DP) ideas. Like Monte Carlo methods, TD methods can learn directly from raw experience without a model of the environment's dynamics. Like DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome (they bootstrap). The relationship between TD, DP, and Monte Carlo methods is a recurring theme in the theory of reinforcement learning; this chapter is the beginning of our exploration of it. Before we are done, we will see that these ideas and methods blend into each other and can be combined in many ways. In particular, in Chapter 7 we introduce n -step algorithms, which provide a bridge from TD to Monte Carlo methods, and in Chapter 12 we introduce the TD(λ) algorithm, which seamlessly unifies them.

As usual, we start by focusing on the policy evaluation or *prediction* problem, the problem of estimating the value function v_π for a given policy π . For the *control* problem (finding an optimal policy), DP, TD, and Monte Carlo methods all use some variation of generalized policy iteration (GPI). The differences in the methods are primarily differences in their approaches to the prediction problem.

6.1 TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following a policy π , both methods update their estimate V of v_π for the nonterminal states S_t occurring in that experience. Roughly speaking, Monte Carlo methods wait until the return following the visit is known, then use that return as a target for $V(S_t)$. A simple every-visit Monte Carlo method suitable for nonstationary environments is

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)], \quad (6.1)$$

where G_t is the actual return following time t , and α is a constant step-size parameter (c.f., Equation 2.4). Let us call this method *constant- α MC*. Whereas Monte Carlo methods must wait until the end of the episode to determine the increment to $V(S_t)$ (only then is G_t known), TD methods need to wait only until the next time step. At time $t + 1$ they immediately form a target and make a useful update using the observed reward R_{t+1} and the estimate $V(S_{t+1})$. The simplest TD method makes the update

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (6.2)$$

immediately on transition to S_{t+1} and receiving R_{t+1} . In effect, the target for the Monte Carlo update is G_t , whereas the target for the TD update is $R_{t+1} + \gamma V(S_{t+1})$. This TD method is called *TD(0)*, or *one-step TD*, because it is a special case of the $TD(\lambda)$ and n -step TD methods developed in Chapter 12 and Chapter 7. The box below specifies TD(0) completely in procedural form.

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
 Algorithm parameter: step size $\alpha \in (0, 1]$
 Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$
 Loop for each episode:
 Initialize S
 Loop for each step of episode:
 $A \leftarrow$ action given by π for S
 Take action A , observe R, S'
 $V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$
 $S \leftarrow S'$
 until S is terminal

Because TD(0) bases its update in part on an existing estimate, we say that it is a *bootstrapping* method, like DP. We know from Chapter 3 that

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] \quad (6.3)$$

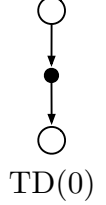
$$= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \quad (\text{from (3.9)})$$

$$= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]. \quad (6.4)$$

Roughly speaking, Monte Carlo methods use an estimate of (6.3) as a target, whereas DP methods use an estimate of (6.4) as a target. The Monte Carlo target is an estimate because the expected value in (6.3) is not known; a sample return is used in place of the real expected return. The DP target is an estimate not because of the expected values, which are assumed to be completely provided by a model of the environment, but because $v_\pi(S_{t+1})$ is not known and the current estimate, $V(S_{t+1})$, is used instead. The TD target is an estimate for both reasons: it samples the expected values in (6.4) *and* it uses the current estimate V instead of the true v_π . Thus, TD methods combine the sampling of

Monte Carlo with the bootstrapping of DP. As we shall see, with care and imagination this can take us a long way toward obtaining the advantages of both Monte Carlo and DP methods.

Shown to the right is the backup diagram for tabular TD(0). The value estimate for the state node at the top of the backup diagram is updated on the basis of the one sample transition from it to the immediately following state. We refer to TD and Monte Carlo updates as *sample updates* because they involve looking ahead to a sample successor state (or state–action pair), using the value of the successor and the reward along the way to compute a backed-up value, and then updating the value of the original state (or state–action pair) accordingly. *Sample* updates differ from the *expected* updates of DP methods in that they are based on a single sample successor rather than on a complete distribution of all possible successors.



Finally, note that the quantity in brackets in the TD(0) update is a sort of error, measuring the difference between the estimated value of S_t and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This quantity, called the *TD error*, arises in various forms throughout reinforcement learning:

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \quad (6.5)$$

Notice that the TD error at each time is the error in the estimate *made at that time*. Because the TD error depends on the next state and next reward, it is not actually available until one time step later. That is, δ_t is the error in $V(S_t)$, available at time $t + 1$. Also note that if the array V does not change during the episode (as it does not in Monte Carlo methods), then the Monte Carlo error can be written as a sum of TD errors:

$$\begin{aligned} G_t - V(S_t) &= R_{t+1} + \gamma G_{t+1} - V(S_t) + \gamma V(S_{t+1}) - \gamma V(S_{t+1}) && \text{(from (3.9))} \\ &= \delta_t + \gamma(G_{t+1} - V(S_{t+1})) \\ &= \delta_t + \gamma\delta_{t+1} + \gamma^2(G_{t+2} - V(S_{t+2})) \\ &= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}(G_T - V(S_T)) \\ &= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}(0 - 0) \\ &= \sum_{k=t}^{T-1} \gamma^{k-t} \delta_k. && (6.6) \end{aligned}$$

This identity is not exact if V is updated during the episode (as it is in TD(0)), but if the step size is small then it may still hold approximately. Generalizations of this identity play an important role in the theory and algorithms of temporal-difference learning.

Exercise 6.1 If V changes during the episode, then (6.6) only holds approximately; what would the difference be between the two sides? Let V_t denote the array of state values used at time t in the TD error (6.5) and in the TD update (6.2). Redo the derivation above to determine the additional amount that must be added to the sum of TD errors in order to equal the Monte Carlo error. $\square$

Example 6.1: Driving Home Each day as you drive home from work, you try to predict how long it will take to get home. When you leave your office, you note the time, the day of week, the weather, and anything else that might be relevant. Say on this Friday you are leaving at exactly 6 o'clock, and you estimate that it will take 30 minutes to get home. As you reach your car it is 6:05, and you notice it is starting to rain. Traffic is often slower in the rain, so you reestimate that it will take 35 minutes from then, or a total of 40 minutes. Fifteen minutes later you have completed the highway portion of your journey in good time. As you exit onto a secondary road you cut your estimate of total travel time to 35 minutes. Unfortunately, at this point you get stuck behind a slow truck, and the road is too narrow to pass. You end up having to follow the truck until you turn onto the side street where you live at 6:40. Three minutes later you are home. The sequence of states, times, and predictions is thus as follows:

<i>State</i>	<i>Elapsed Time</i> (minutes)	<i>Predicted</i> <i>Time to Go</i>	<i>Predicted</i> <i>Total Time</i>
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

The rewards in this example are the elapsed times on each leg of the journey.¹ We are not discounting ($\gamma = 1$), and thus the return for each state is the actual time to go from that state. The value of each state is the *expected* time to go. The second column of numbers gives the current estimated value for each state encountered.

A simple way to view the operation of Monte Carlo methods is to plot the predicted total time (the last column) over the sequence, as in Figure 6.1 (left). The red arrows show the changes in predictions recommended by the constant- α MC method (6.1), for $\alpha = 1$. These are exactly the errors between the estimated value (predicted time to go) in each state and the actual return (actual time to go). For example, when you exited the highway you thought it would take only 15 minutes more to get home, but in fact it took 23 minutes. Equation 6.1 applies at this point and determines an increment in the estimate of time to go after exiting the highway. The error, $G_t - V(S_t)$, at this time is eight minutes. Suppose the step-size parameter, α , is $1/2$. Then the predicted time to go after exiting the highway would be revised upward by four minutes as a result of this experience. This is probably too large a change in this case; the truck was probably just an unlucky break. In any event, the change can only be made offline, that is, after you have reached home. Only at this point do you know any of the actual returns.

Is it necessary to wait until the final outcome is known before learning can begin? Suppose on another day you again estimate when leaving your office that it will take 30 minutes to drive home, but then you become stuck in a massive traffic jam. Twenty-five minutes after leaving the office you are still bumper-to-bumper on the highway. You now

¹If this were a control problem with the objective of minimizing travel time, then we would of course make the rewards the *negative* of the elapsed time. But because we are concerned here only with prediction (policy evaluation), we can keep things simple by using positive numbers.

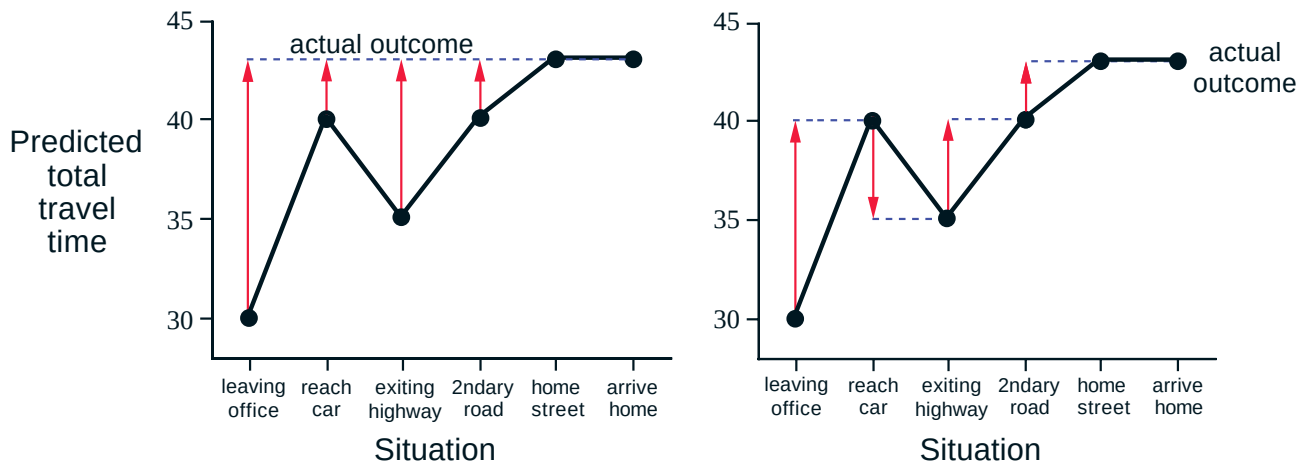


Figure 6.1: Changes recommended in the driving home example by Monte Carlo methods (left) and TD methods (right).

estimate that it will take another 25 minutes to get home, for a total of 50 minutes. As you wait in traffic, you already know that your initial estimate of 30 minutes was too optimistic. Must you wait until you get home before increasing your estimate for the initial state? According to the Monte Carlo approach you must, because you don't yet know the true return.

According to a TD approach, on the other hand, you would learn immediately, shifting your initial estimate from 30 minutes toward 50. In fact, each estimate would be shifted toward the estimate that immediately follows it. Returning to our first day of driving, Figure 6.1 (right) shows the changes in the predictions recommended by the TD rule (6.2) (these are the changes made by the rule if $\alpha = 1$). Each error is proportional to the change over time of the prediction, that is, to the *temporal differences* in predictions.

Besides giving you something to do while waiting in traffic, there are several computational reasons why it is advantageous to learn based on your current predictions rather than waiting until termination when you know the actual return. We briefly discuss some of these in the next section. ■

Exercise 6.2 This is an exercise to help develop your intuition about why TD methods are often more efficient than Monte Carlo methods. Consider the driving home example and how it is addressed by TD and Monte Carlo methods. Can you imagine a scenario in which a TD update would be better on average than a Monte Carlo update? Give an example scenario—a description of past experience and a current state—in which you would expect the TD update to be better. Here's a hint: Suppose you have lots of experience driving home from work. Then you move to a new building and a new parking lot (but you still enter the highway at the same place). Now you are starting to learn predictions for the new building. Can you see why TD updates are likely to be much better, at least initially, in this case? Might the same sort of thing happen in the original scenario? □

6.2 Advantages of TD Prediction Methods

TD methods update their estimates based in part on other estimates. They learn a guess from a guess—they *bootstrap*. Is this a good thing to do? What advantages do TD methods have over Monte Carlo and DP methods? Developing and answering such questions will take the rest of this book and more. In this section we briefly anticipate some of the answers.

Obviously, TD methods have an advantage over DP methods in that they do not require a model of the environment, of its reward and next-state probability distributions.

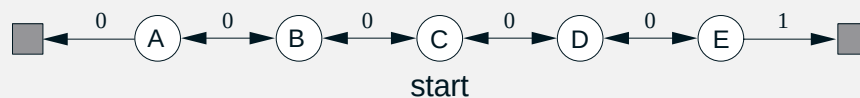
The next most obvious advantage of TD methods over Monte Carlo methods is that they are naturally implemented in an online, fully incremental fashion. With Monte Carlo methods one must wait until the end of an episode, because only then is the return known, whereas with TD methods one need wait only one time step. Surprisingly often this turns out to be a critical consideration. Some applications have very long episodes, so that delaying all learning until the end of the episode is too slow. Other applications are continuing tasks and have no episodes at all. Finally, as we noted in the previous chapter, some Monte Carlo methods must ignore or discount episodes on which experimental actions are taken, which can greatly slow learning. TD methods are much less susceptible to these problems because they learn from each transition regardless of what subsequent actions are taken.

But are TD methods sound? Certainly it is convenient to learn one guess from the next, without waiting for an actual outcome, but can we still guarantee convergence to the correct answer? Happily, the answer is yes. For any fixed policy π , TD(0) has been proved to converge to v_π , in the mean for a constant step-size parameter if it is sufficiently small, and with probability 1 if the step-size parameter decreases according to the usual stochastic approximation conditions (2.7). Most convergence proofs apply only to the table-based case of the algorithm presented above (6.2), but some also apply to the case of general linear function approximation. These results are discussed in a more general setting in Chapter 9.

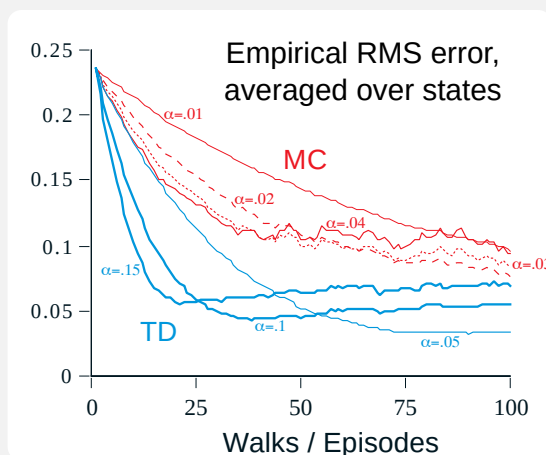
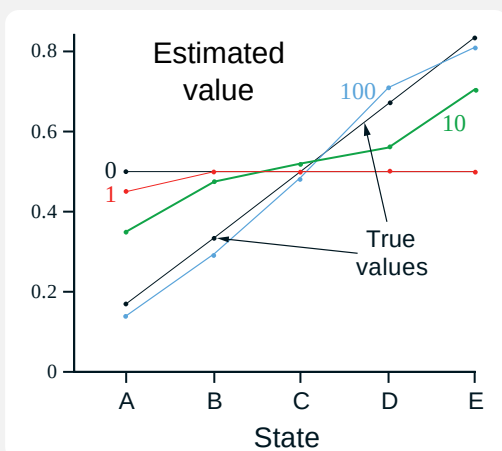
If both TD and Monte Carlo methods converge asymptotically to the correct predictions, then a natural next question is “Which gets there first?” In other words, which method learns faster? Which makes the more efficient use of limited data? At the current time this is an open question in the sense that no one has been able to prove mathematically that one method converges faster than the other. In fact, it is not even clear what is the most appropriate formal way to phrase this question! In practice, however, TD methods have usually been found to converge faster than constant- α MC methods on stochastic tasks, as illustrated in Example 6.2.

Example 6.2 Random Walk

In this example we empirically compare the prediction abilities of TD(0) and constant- α MC when applied to the following Markov reward process:



A *Markov reward process*, or MRP, is a Markov decision process without actions. We will often use MRPs when focusing on the prediction problem, in which there is no need to distinguish the dynamics due to the environment from those due to the agent. In this MRP, all episodes start in the center state, C, then proceed either left or right by one state on each step, with equal probability. Episodes terminate either on the extreme left or the extreme right. When an episode terminates on the right, a reward of +1 occurs; all other rewards are zero. For example, a typical episode might consist of the following state-and-reward sequence: C, 0, B, 0, C, 0, D, 0, E, 1. Because this task is undiscounted, the true value of each state is the probability of terminating on the right if starting from that state. Thus, the true value of the center state is $v_{\pi}(C) = 0.5$. The true values of all the states, A through E, are $\frac{1}{6}$, $\frac{2}{6}$, $\frac{3}{6}$, $\frac{4}{6}$, and $\frac{5}{6}$.



The left graph above shows the values learned after various numbers of episodes on a single run of TD(0). The estimates after 100 episodes are about as close as they ever come to the true values—with a constant step-size parameter ($\alpha = 0.1$ in this example), the values fluctuate indefinitely in response to the outcomes of the most recent episodes. The right graph shows learning curves for the two methods for various values of α . The performance measure shown is the root mean-squared (RMS) error between the value function learned and the true value function, averaged over the five states, then averaged over 100 runs. In all cases the approximate value function was initialized to the intermediate value $V(s) = 0.5$, for all s . The TD method was consistently better than the MC method on this task.

Exercise 6.3 From the results shown in the left graph of the random walk example it appears that the first episode results in a change in only $V(A)$. What does this tell you about what happened on the first episode? Why was only the estimate for this one state changed? By exactly how much was it changed? $\square$

Exercise 6.4 The specific results shown in the right graph of the random walk example are dependent on the value of the step-size parameter, α . Do you think the conclusions about which algorithm is better would be affected if a wider range of α values were used? Is there a different, fixed value of α at which either algorithm would have performed significantly better than shown? Why or why not? $\square$

**Exercise 6.5* In the right graph of the random walk example, the RMS error of the TD method seems to go down and then up again, particularly at high α 's. What could have caused this? Do you think this always occurs, or might it be a function of how the approximate value function was initialized? $\square$

Exercise 6.6 In Example 6.2 we stated that the true values for the random walk example are $\frac{1}{6}$, $\frac{2}{6}$, $\frac{3}{6}$, $\frac{4}{6}$, and $\frac{5}{6}$, for states A through E. Describe at least two different ways that these could have been computed. Which would you guess we actually used? Why? $\square$

6.3 Optimality of TD(0)

Suppose there is available only a finite amount of experience, say 10 episodes or 100 time steps. In this case, a common approach with incremental learning methods is to present the experience repeatedly until the method converges upon an answer. Given an approximate value function, V , the increments specified by (6.1) or (6.2) are computed for every time step t at which a nonterminal state is visited, but the value function is changed only once, by the sum of all the increments. Then all the available experience is processed again with the new value function to produce a new overall increment, and so on, until the value function converges. We call this *batch updating* because updates are made only after processing each complete *batch* of training data.

Under batch updating, TD(0) converges deterministically to a single answer independent of the step-size parameter, α , as long as α is chosen to be sufficiently small. The constant- α MC method also converges deterministically under the same conditions, but to a different answer. Understanding these two answers will help us understand the difference between the two methods. Under normal updating the methods do not move all the way to their respective batch answers, but in some sense they take steps in these directions. Before trying to understand the two answers in general, for all possible tasks, we first look at a few examples.

Example 6.3: Random walk under batch updating Batch-updating versions of TD(0) and constant- α MC were applied as follows to the random walk prediction example (Example 6.2). After each new episode, all episodes seen so far were treated as a batch. They were repeatedly presented to the algorithm, either TD(0) or constant- α MC, with α sufficiently small that the value function converged. The resulting value function was then compared with v_π , and the average root mean-squared error across the five states (and across 100 independent repetitions of the whole experiment) was plotted to obtain

the learning curves shown in Figure 6.2. Note that the batch TD method was consistently better than the batch Monte Carlo method.

Under batch training, constant- α MC converges to values, $V(s)$, that are sample averages of the actual returns experienced after visiting each state s . These are optimal estimates in the sense that they minimize the mean-squared error from the actual returns in the training set. In this sense it is surprising that the batch TD method was able to perform better according to the root mean-squared error measure shown in the figure to the right. How is it that batch TD was able to perform better than this optimal method? The answer is that the Monte Carlo method is optimal only in a limited way, and that TD is optimal in a way that is more relevant to predicting returns. ■

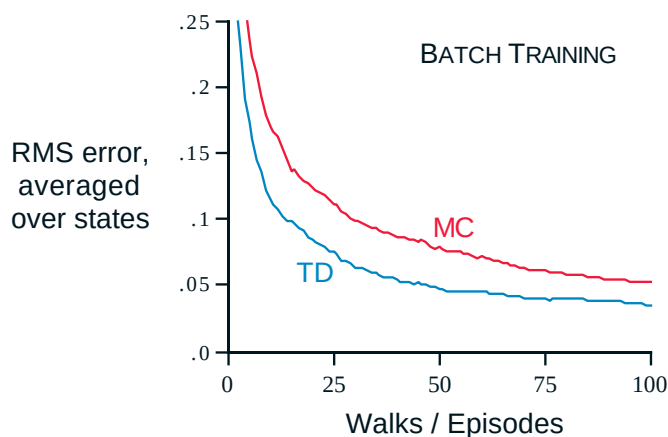


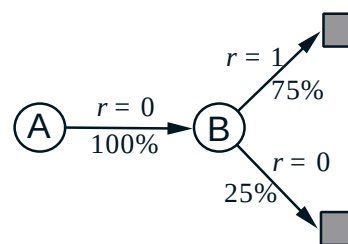
Figure 6.2: Performance of TD(0) and constant- α MC under batch training on the random walk task.

Example 6.4: You are the Predictor Place yourself now in the role of the predictor of returns for an unknown Markov reward process. Suppose you observe the following eight episodes:

A, 0, B, 0	B, 1
B, 1	B, 1
B, 1	B, 1
B, 1	B, 0

This means that the first episode started in state A, transitioned to B with a reward of 0, and then terminated from B with a reward of 0. The other seven episodes were even shorter, starting from B and terminating immediately. Given this batch of data, what would you say are the optimal predictions, the best values for the estimates $V(A)$ and $V(B)$? Everyone would probably agree that the optimal value for $V(B)$ is $\frac{3}{4}$, because six out of the eight times in state B the process terminated immediately with a return of 1, and the other two times in B the process terminated immediately with a return of 0.

But what is the optimal value for the estimate $V(A)$ given this data? Here there are two reasonable answers. One is to observe that 100% of the times the process was in state A it traversed immediately to B (with a reward of 0); and because we have already decided that B has value $\frac{3}{4}$, therefore A must have value $\frac{3}{4}$ as well. One way of viewing this answer is that it is based on first modeling the Markov process, in this case as shown to the right, and then computing the correct estimates given the model, which indeed in this case gives $V(A) = \frac{3}{4}$. This is



also the answer that batch TD(0) gives.

The other reasonable answer is simply to observe that we have seen A once and the return that followed it was 0; we therefore estimate $V(A)$ as 0. This is the answer that batch Monte Carlo methods give. Notice that it is also the answer that gives minimum squared error on the training data. In fact, it gives zero error on the data. But still we expect the first answer to be better. If the process is Markov, we expect that the first answer will produce lower error on *future* data, even though the Monte Carlo answer is better on the existing data. ■

Example 6.4 illustrates a general difference between the estimates found by batch TD(0) and batch Monte Carlo methods. Batch Monte Carlo methods always find the estimates that minimize mean-squared error on the training set, whereas batch TD(0) always finds the estimates that would be exactly correct for the maximum-likelihood model of the Markov process. In general, the *maximum-likelihood estimate* of a parameter is the parameter value whose probability of generating the data is greatest. In this case, the maximum-likelihood estimate is the model of the Markov process formed in the obvious way from the observed episodes: the estimated transition probability from i to j is the fraction of observed transitions from i that went to j , and the associated expected reward is the average of the rewards observed on those transitions. Given this model, we can compute the estimate of the value function that would be exactly correct if the model were exactly correct. This is called the *certainty-equivalence estimate* because it is equivalent to assuming that the estimate of the underlying process was known with certainty rather than being approximated. In general, batch TD(0) converges to the certainty-equivalence estimate.

This helps explain why TD methods converge more quickly than Monte Carlo methods. In batch form, TD(0) is faster than Monte Carlo methods because it computes the true certainty-equivalence estimate. This explains the advantage of TD(0) shown in the batch results on the random walk task (Figure 6.2). The relationship to the certainty-equivalence estimate may also explain in part the speed advantage of nonbatch TD(0) (e.g., Example 6.2, page 125, right graph). Although the nonbatch methods do not achieve either the certainty-equivalence or the minimum squared-error estimates, they can be understood as moving roughly in these directions. Nonbatch TD(0) may be faster than constant- α MC because it is moving toward a better estimate, even though it is not getting all the way there. At the current time nothing more definite can be said about the relative efficiency of online TD and Monte Carlo methods.

Finally, it is worth noting that although the certainty-equivalence estimate is in some sense an optimal solution, it is almost never feasible to compute it directly. If $n = |\mathcal{S}|$ is the number of states, then just forming the maximum-likelihood estimate of the process may require on the order of n^2 memory, and computing the corresponding value function requires on the order of n^3 computational steps if done conventionally. In these terms it is indeed striking that TD methods can approximate the same solution using memory no more than order n and repeated computations over the training set. On tasks with large state spaces, TD methods may be the only feasible way of approximating the certainty-equivalence solution.

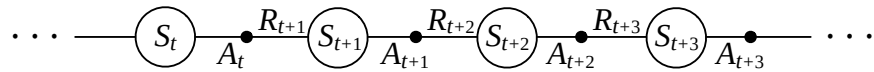
**Exercise 6.7* Design an off-policy version of the TD(0) update that can be used with

arbitrary target policy π and covering behavior policy b , using at each step t the importance sampling ratio $\rho_{t:t}$ (5.3). $\square$

6.4 Sarsa: On-policy TD Control

We turn now to the use of TD prediction methods for the control problem. As usual, we follow the pattern of generalized policy iteration (GPI), only this time using TD methods for the evaluation or prediction part. As with Monte Carlo methods, we face the need to trade off exploration and exploitation, and again approaches fall into two main classes: on-policy and off-policy. In this section we present an on-policy TD control method.

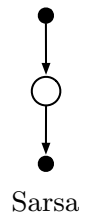
The first step is to learn an action-value function rather than a state-value function. In particular, for an on-policy method we must estimate $q_\pi(s, a)$ for the current behavior policy π and for all states s and actions a . This can be done using essentially the same TD method described above for learning v_π . Recall that an episode consists of an alternating sequence of states and state-action pairs:



In the previous section we considered transitions from state to state and learned the values of states. Now we consider transitions from state-action pair to state-action pair, and learn the values of state-action pairs. Formally these cases are identical: they are both Markov chains with a reward process. The theorems assuring the convergence of state values under TD(0) also apply to the corresponding algorithm for action values:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]. \quad (6.7)$$

This update is done after every transition from a nonterminal state S_t . If S_{t+1} is terminal, then $Q(S_{t+1}, A_{t+1})$ is defined as zero. This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action pair to the next. This quintuple gives rise to the name *Sarsa* for the algorithm. The backup diagram for Sarsa is as shown to the right.



It is straightforward to design an on-policy control algorithm based on the Sarsa prediction method. As in all on-policy methods, we continually estimate q_π for the behavior policy π , and at the same time change π toward greediness with respect to q_π . The general form of the Sarsa control algorithm is given in the box on the next page.

The convergence properties of the Sarsa algorithm depend on the nature of the policy's dependence on Q . For example, one could use ε -greedy or ε -soft policies. Sarsa converges with probability 1 to an optimal policy and action-value function as long as all state-action pairs are visited an infinite number of times and the policy converges in the limit to the greedy policy (which can be arranged, for example, with ε -greedy policies by setting $\varepsilon = 1/t$).

Exercise 6.8 Show that an action-value version of (6.6) holds for the action-value form of the TD error $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$, again assuming that the values don't change from step to step. $\square$

usual four. How much better can you do with the extra actions? Can you do even better by including a ninth action that causes no movement at all other than that caused by the wind? $\square$

Exercise 6.10: Stochastic Wind (programming) Re-solve the windy gridworld task with King's moves, assuming that the effect of the wind, if there is any, is stochastic, sometimes varying by 1 from the mean values given for each column. That is, a third of the time you move exactly according to these values, as in the previous exercise, but also a third of the time you move one cell above that, and another third of the time you move one cell below that. For example, if you are one cell to the right of the goal and you move `left`, then one-third of the time you move one cell above the goal, one-third of the time you move two cells above the goal, and one-third of the time you move to the goal. $\square$

6.5 Q-learning: Off-policy TD Control

One of the early breakthroughs in reinforcement learning was the development of an off-policy TD control algorithm known as *Q-learning* (Watkins, 1989), defined by

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]. \quad (6.8)$$

In this case, the learned action-value function, Q , directly approximates q_* , the optimal action-value function, independent of the policy being followed. This dramatically simplifies the analysis of the algorithm and enabled early convergence proofs. The policy still has an effect in that it determines which state-action pairs are visited and updated. However, all that is required for correct convergence is that all pairs continue to be updated. As we observed in Chapter 5, this is a minimal requirement in the sense that any method guaranteed to find optimal behavior in the general case must require it. Under this assumption and a variant of the usual stochastic approximation conditions on the sequence of step-size parameters, Q has been shown to converge with probability 1 to q_* . The Q-learning algorithm is shown below in procedural form.

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R , S'

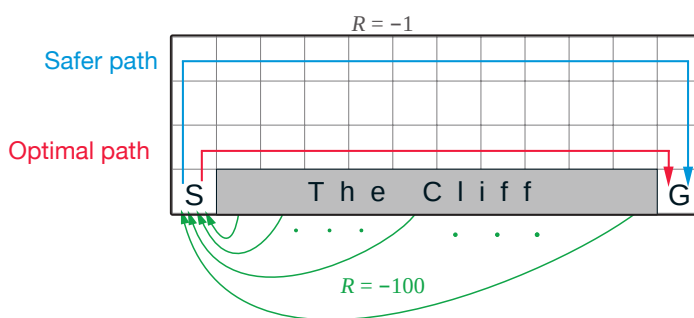
$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

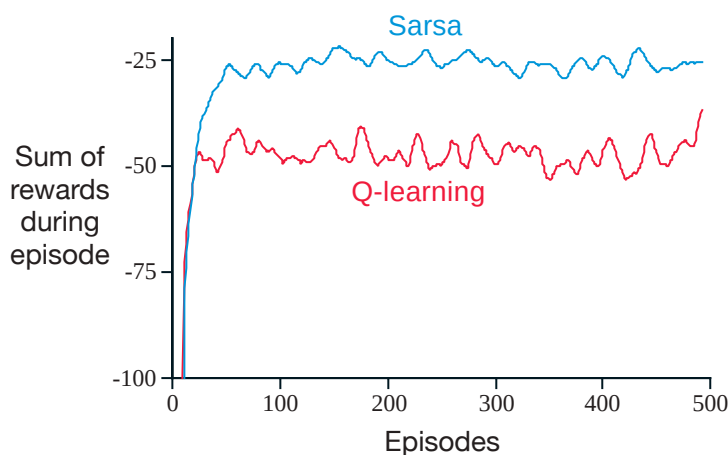
 until S is terminal

What is the backup diagram for Q-learning? The rule (6.8) updates a state-action pair, so the top node, the root of the update, must be a small, filled action node. The update is also *from* action nodes, maximizing over all those actions possible in the next state. Thus the bottom nodes of the backup diagram should be all these action nodes. Finally, remember that we indicate taking the maximum of these “next action” nodes with an arc across them (Figure 3.4-right). Can you guess now what the diagram is? If so, please do make a guess before turning to the answer in Figure 6.4 on page 134.

Example 6.6: Cliff Walking This gridworld example compares Sarsa and Q-learning, highlighting the difference between on-policy (Sarsa) and off-policy (Q-learning) methods. Consider the gridworld shown to the right. This is a standard undiscounted, episodic task, with start and goal states, and the usual actions causing movement up, down, right, and left. Reward is -1 on all transitions except those into the region marked “The Cliff.” Stepping into this region incurs a reward of -100 and sends the agent instantly back to the start.



The graph to the right shows the performance of the Sarsa and Q-learning methods with ϵ -greedy action selection, $\epsilon = 0.1$. After an initial transient, Q-learning learns values for the optimal policy, that which travels right along the edge of the cliff. Unfortunately, this results in its occasionally falling off the cliff because of the ϵ -greedy action selection. Sarsa, on the other hand, takes the action selection into account and learns the longer but safer path through the upper part of the grid. Although Q-learning actually learns the values of the optimal policy, its online performance is worse than that of Sarsa, which learns the roundabout policy. Of course, if ϵ were gradually reduced, then both methods would asymptotically converge to the optimal policy. ■



Exercise 6.11 Why is Q-learning considered an *off-policy* control method? ☐

Exercise 6.12 Suppose action selection is greedy. Is Q-learning then exactly the same algorithm as Sarsa? Will they make exactly the same action selections and weight updates? ☐

6.6 Expected Sarsa

Consider the learning algorithm that is just like Q-learning except that instead of the maximum over next state–action pairs it uses the expected value, taking into account how likely each action is under the current policy. That is, consider the algorithm with the update rule

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t) \right] \\ &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right], \end{aligned} \quad (6.9)$$

but that otherwise follows the schema of Q-learning. Given the next state, S_{t+1} , this algorithm moves *deterministically* in the same direction as Sarsa moves *in expectation*, and accordingly it is called *Expected Sarsa*. Its backup diagram is shown on the right in Figure 6.4.

Expected Sarsa is more complex computationally than Sarsa but, in return, it eliminates the variance due to the random selection of A_{t+1} . Given the same amount of experience we might expect it to perform slightly better than Sarsa, and indeed it generally does. Figure 6.3 shows summary results on the cliff-walking task with Expected Sarsa compared to Sarsa and Q-learning. Expected Sarsa retains the significant advantage of Sarsa over Q-learning on this problem. In addition, Expected Sarsa shows a significant improvement

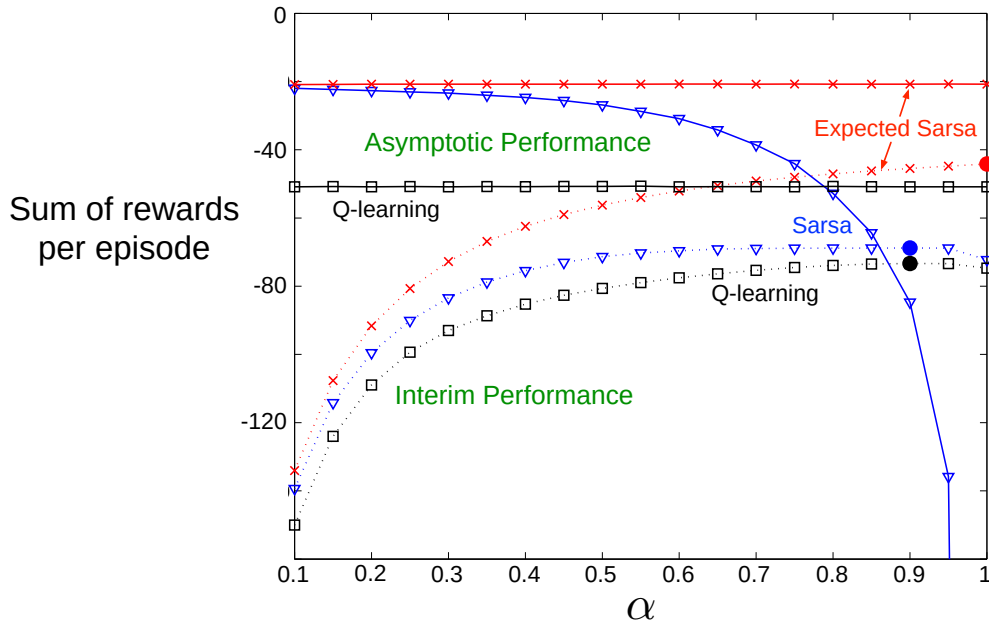


Figure 6.3: Interim and asymptotic performance of TD control methods on the cliff-walking task as a function of α . All algorithms used an ε -greedy policy with $\varepsilon = 0.1$. Asymptotic performance is an average over 100,000 episodes whereas interim performance is an average over the first 100 episodes. These data are averages of over 50,000 and 10 runs for the interim and asymptotic cases respectively. The solid circles mark the best interim performance of each method. Adapted from van Seijen et al. (2009).



Figure 6.4: The backup diagrams for Q-learning and Expected Sarsa.

over Sarsa over a wide range of values for the step-size parameter α . In cliff walking the state transitions are all deterministic and all randomness comes from the policy. In such cases, Expected Sarsa can safely set $\alpha = 1$ without suffering any degradation of asymptotic performance, whereas Sarsa can only perform well in the long run at a small value of α , at which short-term performance is poor. In this and other examples there is a consistent empirical advantage of Expected Sarsa over Sarsa.

In these cliff walking results Expected Sarsa was used on-policy, but in general it might use a policy different from the target policy π to generate behavior, in which case it becomes an off-policy algorithm. For example, suppose π is the greedy policy while behavior is more exploratory; then Expected Sarsa is exactly Q-learning. In this sense Expected Sarsa subsumes and generalizes Q-learning while reliably improving over Sarsa. Except for the small additional computational cost, Expected Sarsa may completely dominate both of the other more-well-known TD control algorithms.

6.7 Maximization Bias and Double Learning

All the control algorithms that we have discussed so far involve maximization in the construction of their target policies. For example, in Q-learning the target policy is the greedy policy given the current action values, which is defined with a max, and in Sarsa the policy is often ϵ -greedy, which also involves a maximization operation. In these algorithms, a maximum over estimated values is used implicitly as an estimate of the maximum value, which can lead to a significant positive bias. To see why, consider a single state s where there are many actions a whose true values, $q(s, a)$, are all zero but whose estimated values, $Q(s, a)$, are uncertain and thus distributed some above and some below zero. The maximum of the true values is zero, but the maximum of the estimates is positive, a positive bias. We call this *maximization bias*.

Example 6.7: Maximization Bias Example The small MDP shown inset in Figure 6.5 provides a simple example of how maximization bias can harm the performance of TD control algorithms. The MDP has two non-terminal states A and B. Episodes always start in A with a choice between two actions, left and right. The right action transitions immediately to the terminal state with a reward and return of zero. The left action transitions to B, also with a reward of zero, from which there are many possible actions all of which cause immediate termination with a reward drawn from a normal distribution with mean -0.1 and variance 1.0 . Thus, the expected return for any trajectory starting with left is -0.1 , and thus taking left in state A is always a

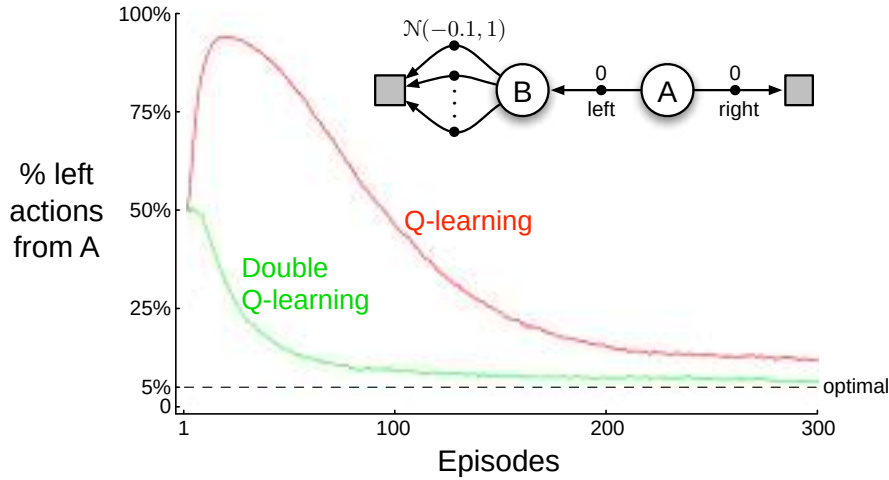


Figure 6.5: Comparison of Q-learning and Double Q-learning on a simple episodic MDP (shown inset). Q-learning initially learns to take the **left** action much more often than the **right** action, and always takes it significantly more often than the 5% minimum probability enforced by ε -greedy action selection with $\varepsilon = 0.1$. In contrast, Double Q-learning is essentially unaffected by maximization bias. These data are averaged over 10,000 runs. The initial action-value estimates were zero. Any ties in ε -greedy action selection were broken randomly.

mistake. Nevertheless, our control methods may favor **left** because of maximization bias making **B** appear to have a positive value. Figure 6.5 shows that Q-learning with ε -greedy action selection initially learns to strongly favor the **left** action on this example. Even at asymptote, Q-learning takes the **left** action about 5% more often than is optimal at our parameter settings ($\varepsilon = 0.1$, $\alpha = 0.1$, and $\gamma = 1$). ■

Are there algorithms that avoid maximization bias? To start, consider a bandit case in which we have noisy estimates of the value of each of many actions, obtained as sample averages of the rewards received on all the plays with each action. As we discussed above, there will be a positive maximization bias if we use the maximum of the estimates as an estimate of the maximum of the true values. One way to view the problem is that it is due to using the same samples (plays) both to determine the maximizing action and to estimate its value. Suppose we divided the plays in two sets and used them to learn two independent estimates, call them $Q_1(a)$ and $Q_2(a)$, each an estimate of the true value $q(a)$, for all $a \in \mathcal{A}$. We could then use one estimate, say Q_1 , to determine the maximizing action $A^* = \arg\max_a Q_1(a)$, and the other, Q_2 , to provide the estimate of its value, $Q_2(A^*) = Q_2(\arg\max_a Q_1(a))$. This estimate will then be unbiased in the sense that $\mathbb{E}[Q_2(A^*)] = q(A^*)$. We can also repeat the process with the role of the two estimates reversed to yield a second unbiased estimate $Q_1(\arg\max_a Q_2(a))$. This is the idea of *double learning*. Note that although we learn two estimates, only one estimate is updated on each play; double learning doubles the memory requirements, but does not increase the amount of computation per step.

The idea of double learning extends naturally to algorithms for full MDPs. For example, the double learning algorithm analogous to Q-learning, called Double Q-learning, divides the time steps in two, perhaps by flipping a coin on each step. If the coin comes up heads,

the update is

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1(S_t, A_t) \right]. \quad (6.10)$$

If the coin comes up tails, then the same update is done with Q_1 and Q_2 switched, so that Q_2 is updated. The two approximate value functions are treated completely symmetrically. The behavior policy can use both action-value estimates. For example, an ε -greedy policy for Double Q-learning could be based on the average (or sum) of the two action-value estimates. A complete algorithm for Double Q-learning is given in the box below. This is the algorithm used to produce the results in Figure 6.5. In that example, double learning seems to eliminate the harm caused by maximization bias. Of course there are also double versions of Sarsa and Expected Sarsa.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\text{terminal}, \cdot) = 0$
Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose A from S using the policy ε -greedy in $Q_1 + Q_2$
 Take action A , observe R, S'
 With 0.5 probability:
 $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha \left(R + \gamma Q_2(S', \arg \max_a Q_1(S', a)) - Q_1(S, A) \right)$
 else:
 $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha \left(R + \gamma Q_1(S', \arg \max_a Q_2(S', a)) - Q_2(S, A) \right)$
 $S \leftarrow S'$
 until S is terminal

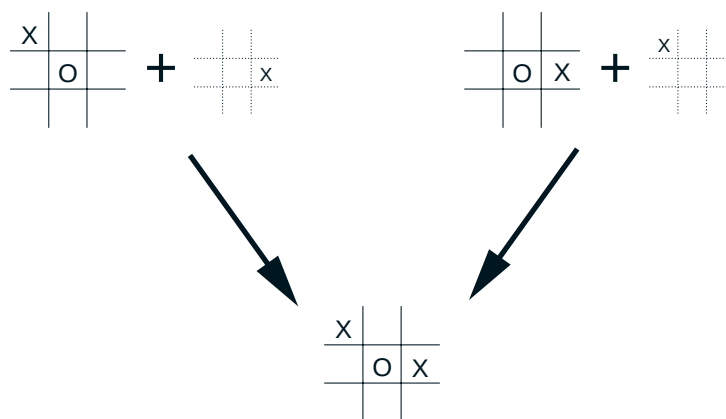
**Exercise 6.13* What are the update equations for Double Expected Sarsa with an ε -greedy target policy? $\square$

6.8 Games, Afterstates, and Other Special Cases

In this book we try to present a uniform approach to a wide class of tasks, but of course there are always exceptional tasks that are better treated in a specialized way. For example, our general approach involves learning an *action*-value function, but in Chapter 1 we presented a TD method for learning to play tic-tac-toe that learned something much more like a *state*-value function. If we look closely at that example, it becomes apparent that the function learned there is neither an action-value function nor a state-value function in the usual sense. A conventional state-value function evaluates states in which the agent has the option of selecting an action, but the state-value function used in

tic-tac-toe evaluates board positions *after* the agent has made its move. Let us call these *afterstates*, and value functions over these, *afterstate value functions*. Afterstates are useful when we have knowledge of an initial part of the environment's dynamics but not necessarily of the full dynamics. For example, in games we typically know the immediate effects of our moves. We know for each possible chess move what the resulting position will be, but not how our opponent will reply. Afterstate value functions are a natural way to take advantage of this kind of knowledge and thereby produce a more efficient learning method.

The reason it is more efficient to design algorithms in terms of afterstates is apparent from the tic-tac-toe example. A conventional action-value function would map from positions *and* moves to an estimate of the value. But many position-move pairs produce the same resulting position, as in the example below:



In such cases the position-move pairs are different but produce the same “afterposition,” and thus must have the same value. A conventional action-value function would have to separately assess both pairs, whereas an afterstate value function would immediately assess both equally. Any learning about the position-move pair on the left would immediately transfer to the pair on the right.

Afterstates arise in many tasks, not just games. For example, in queuing tasks there are actions such as assigning customers to servers, rejecting customers, or discarding information. In such cases the actions are in fact defined in terms of their immediate effects, which are completely known.

It is impossible to describe all the possible kinds of specialized problems and corresponding specialized learning algorithms. However, the principles developed in this book should apply widely. For example, afterstate methods are still aptly described in terms of generalized policy iteration, with a policy and (afterstate) value function interacting in essentially the same way. In many cases one will still face the choice between on-policy and off-policy methods for managing the need for persistent exploration.

Exercise 6.14 Describe how the task of Jack’s Car Rental (Example 4.2) could be reformulated in terms of afterstates. Why, in terms of this specific task, would such a reformulation be likely to speed convergence? $\square$

6.9 Summary

In this chapter we introduced a new kind of learning method, temporal-difference (TD) learning, and showed how it can be applied to the reinforcement learning problem. As usual, we divided the overall problem into a prediction problem and a control problem. TD methods are alternatives to Monte Carlo methods for solving the prediction problem. In both cases, the extension to the control problem is via the idea of generalized policy iteration (GPI) that we abstracted from dynamic programming. This is the idea that approximate policy and value functions should interact in such a way that they both move toward their optimal values.

One of the two processes making up GPI drives the value function to accurately predict returns for the current policy; this is the prediction problem. The other process drives the policy to improve locally (e.g., to be ε -greedy) with respect to the current value function. When the first process is based on experience, a complication arises concerning maintaining sufficient exploration. We can classify TD control methods according to whether they deal with this complication by using an on-policy or off-policy approach. Sarsa is an on-policy method, and Q-learning is an off-policy method. Expected Sarsa is also an off-policy method as we present it here. There is a third way in which TD methods can be extended to control which we did not include in this chapter, called actor-critic methods. These methods are covered in full in Chapter 13.

The methods presented in this chapter are today the most widely used reinforcement learning methods. This is probably due to their great simplicity: they can be applied online, with a minimal amount of computation, to experience generated from interaction with an environment; they can be expressed nearly completely by single equations that can be implemented with small computer programs. In the next few chapters we extend these algorithms, making them slightly more complicated and significantly more powerful. All the new algorithms will retain the essence of those introduced here: they will be able to process experience online, with relatively little computation, and they will be driven by TD errors. The special cases of TD methods introduced in the present chapter should rightly be called *one-step*, *tabular*, *model-free* TD methods. In the next two chapters we extend them to *n*-step forms (a link to Monte Carlo methods) and forms that include a model of the environment (a link to planning and dynamic programming). Then, in the second part of the book we extend them to various forms of function approximation rather than tables (a link to deep learning and artificial neural networks).

Finally, in this chapter we have discussed TD methods entirely within the context of reinforcement learning problems, but TD methods are actually more general than this. They are general methods for learning to make long-term predictions about dynamical systems. For example, TD methods may be relevant to predicting financial data, life spans, election outcomes, weather patterns, animal behavior, demands on power stations, or customer purchases. It was only when TD methods were analyzed as pure prediction methods, independent of their use in reinforcement learning, that their theoretical properties first came to be well understood. Even so, these other potential applications of TD learning methods have not yet been extensively explored.

Chapter 9

On-policy Prediction with Approximation

In this chapter, we begin our study of function approximation in reinforcement learning by considering its use in estimating the state-value function from on-policy data, that is, in approximating v_π from experience generated using a known policy π . The novelty in this chapter is that the approximate value function is represented not as a table but as a parameterized functional form with weight vector $\mathbf{w} \in \mathbb{R}^d$. We will write $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ for the approximate value of state s given weight vector $\mathbf{w}$. For example, $\hat{v}$ might be a linear function in features of the state, with $\mathbf{w}$ the vector of feature weights. More generally, $\hat{v}$ might be the function computed by a multi-layer artificial neural network, with $\mathbf{w}$ the vector of connection weights in all the layers. By adjusting the weights, any of a wide range of different functions can be implemented by the network. Or $\hat{v}$ might be the function computed by a decision tree, where $\mathbf{w}$ is all the numbers defining the split points and leaf values of the tree. Typically, the number of weights (the dimensionality of $\mathbf{w}$) is much less than the number of states ($d \ll |\mathcal{S}|$), and changing one weight changes the estimated value of many states. Consequently, when a single state is updated, the change generalizes from that state to affect the values of many other states. Such *generalization* makes the learning potentially more powerful but also potentially more difficult to manage and understand.

Perhaps surprisingly, extending reinforcement learning to function approximation also makes it applicable to partially observable problems, in which the full state is not available to the agent. If the parameterized function form for $\hat{v}$ does not allow the estimated value to depend on certain aspects of the state, then it is just as if those aspects are unobservable. In fact, all the theoretical results for methods using function approximation presented in this part of the book apply equally well to cases of partial observability. What function approximation can't do, however, is augment the state representation with memories of past observations. Some such possible further extensions are discussed briefly in Section 17.3.

9.1 Value-function Approximation

All of the prediction methods covered in this book have been described as updates to an estimated value function that shift its value at particular states toward a “backed-up value,” or *update target*, for that state. Let us refer to an individual update by the notation $s \mapsto u$, where s is the state updated and u is the update target that s ’s estimated value is shifted toward. For example, the Monte Carlo update for value prediction is $S_t \mapsto G_t$, the TD(0) update is $S_t \mapsto R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t)$, and the n -step TD update is $S_t \mapsto G_{t:t+n}$. In the DP (dynamic programming) policy-evaluation update, $s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t = s]$, an arbitrary state s is updated, whereas in the other cases the state encountered in actual experience, S_t , is updated.

It is natural to interpret each update as specifying an example of the desired input–output behavior of the value function. In a sense, the update $s \mapsto u$ means that the estimated value for state s should be more like the update target u . Up to now, the actual update has been trivial: the table entry for s ’s estimated value has simply been shifted a fraction of the way toward u , and the estimated values of all other states were left unchanged. Now we permit arbitrarily complex and sophisticated methods to implement the update, and updating at s generalizes so that the estimated values of many other states are changed as well. Machine learning methods that learn to mimic input–output examples in this way are called *supervised learning* methods, and when the outputs are numbers, like u , the process is often called *function approximation*. Function approximation methods expect to receive examples of the desired input–output behavior of the function they are trying to approximate. We use these methods for value prediction simply by passing to them the $s \mapsto g$ of each update as a training example. We then interpret the approximate function they produce as an estimated value function.

Viewing each update as a conventional training example in this way enables us to use any of a wide range of existing function approximation methods for value prediction. In principle, we can use any method for supervised learning from examples, including artificial neural networks, decision trees, and various kinds of multivariate regression. However, not all function approximation methods are equally well suited for use in reinforcement learning. The most sophisticated artificial neural network and statistical methods all assume a static training set over which multiple passes are made. In reinforcement learning, however, it is important that learning be able to occur online, while the agent interacts with its environment or with a model of its environment. To do this requires methods that are able to learn efficiently from incrementally acquired data. In addition, reinforcement learning generally requires function approximation methods able to handle nonstationary target functions (target functions that change over time). For example, in control methods based on GPI (generalized policy iteration) we often seek to learn q_π while π changes. Even if the policy remains the same, the target values of training examples are nonstationary if they are generated by bootstrapping methods (DP and TD learning). Methods that cannot easily handle such nonstationarity are less suitable for reinforcement learning.

9.2 The Prediction Objective ($\overline{VE}$)

Up to now we have not specified an explicit objective for prediction. In the tabular case a continuous measure of prediction quality was not necessary because the learned value function could come to equal the true value function exactly. Moreover, the learned values at each state were decoupled—an update at one state affected no other. But with genuine approximation, an update at one state affects many others, and it is not possible to get the values of all states exactly correct. By assumption we have far more states than weights, so making one state’s estimate more accurate invariably means making others’ less accurate. We are obligated then to say which states we care most about. We must specify a state distribution $\mu(s) \geq 0$, $\sum_s \mu(s) = 1$, representing how much we care about the error in each state s . By the error in a state s we mean the square of the difference between the approximate value $\hat{v}(s, \mathbf{w})$ and the true value $v_\pi(s)$. Weighting this over the state space by μ , we obtain a natural objective function, the *Mean Squared Value Error*, denoted $\overline{VE}$:

$$\overline{VE}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2. \quad (9.1)$$

The square root of this measure, the root $\overline{VE}$, gives a rough measure of how much the approximate values differ from the true values and is often used in plots. Often $\mu(s)$ is chosen to be the fraction of time spent in s . Under on-policy training this is called the *on-policy distribution*; we focus entirely on this case in this chapter. In continuing tasks, the on-policy distribution is the stationary distribution under π .

The on-policy distribution in episodic tasks

In an episodic task, the on-policy distribution is a little different in that it depends on how the initial states of episodes are chosen. Let $h(s)$ denote the probability that an episode begins in each state s , and let $\eta(s)$ denote the number of time steps spent, on average, in state s in a single episode. Time is spent in a state s if episodes start in s , or if transitions are made into s from a preceding state $\bar{s}$ in which time is spent:

$$\eta(s) = h(s) + \sum_{\bar{s}} \eta(\bar{s}) \sum_a \pi(a|\bar{s}) p(s|\bar{s}, a), \quad \text{for all } s \in \mathcal{S}. \quad (9.2)$$

This system of equations can be solved for the expected number of visits $\eta(s)$. The on-policy distribution is then the fraction of time spent in each state normalized to sum to one:

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}, \quad \text{for all } s \in \mathcal{S}. \quad (9.3)$$

This is the natural choice without discounting. If there is discounting ($\gamma < 1$) it should be treated as a form of termination, which can be done simply by including a factor of γ in the second term of (9.2).

The two cases, continuing and episodic, behave similarly, but with approximation they must be treated separately in formal analyses, as we will see repeatedly in this part of the book. This completes the specification of the learning objective.

But it is not completely clear that the $\overline{VE}$ is the right performance objective for reinforcement learning. Remember that our ultimate purpose—the reason we are learning a value function—is to find a better policy. The best value function for this purpose is not necessarily the best for minimizing $\overline{VE}$. Nevertheless, it is not yet clear what a more useful alternative goal for value prediction might be. For now, we will focus on $\overline{VE}$.

An ideal goal in terms of $\overline{VE}$ would be to find a *global optimum*, a weight vector $\mathbf{w}^*$ for which $\overline{VE}(\mathbf{w}^*) \leq \overline{VE}(\mathbf{w})$ for all possible $\mathbf{w}$. Reaching this goal is sometimes possible for simple function approximators such as linear ones, but is rarely possible for complex function approximators such as artificial neural networks and decision trees. Short of this, complex function approximators may seek to converge instead to a *local optimum*, a weight vector $\mathbf{w}^*$ for which $\overline{VE}(\mathbf{w}^*) \leq \overline{VE}(\mathbf{w})$ for all $\mathbf{w}$ in some neighborhood of $\mathbf{w}^*$. Although this guarantee is only slightly reassuring, it is typically the best that can be said for nonlinear function approximators, and often it is enough. Still, for many cases of interest in reinforcement learning there is no guarantee of convergence to an optimum, or even to within a bounded distance of an optimum. Some methods may in fact diverge, with their $\overline{VE}$ approaching infinity in the limit.

In the last two sections we outlined a framework for combining a wide range of reinforcement learning methods for value prediction with a wide range of function approximation methods, using the updates of the former to generate training examples for the latter. We also described a $\overline{VE}$ performance measure which these methods may aspire to minimize. The range of possible function approximation methods is far too large to cover all, and anyway too little is known about most of them to make a reliable evaluation or recommendation. Of necessity, we consider only a few possibilities. In the rest of this chapter we focus on function approximation methods based on gradient principles, and on linear gradient-descent methods in particular. We focus on these methods in part because we consider them to be particularly promising and because they reveal key theoretical issues, but also because they are simple and our space is limited.

9.3 Stochastic-gradient and Semi-gradient Methods

We now develop in detail one class of learning methods for function approximation in value prediction, those based on stochastic gradient descent (SGD). SGD methods are among the most widely used of all function approximation methods and are particularly well suited to online reinforcement learning.

In gradient-descent methods, the weight vector is a column vector with a fixed number of real valued components, $\mathbf{w} \doteq (w_1, w_2, \dots, w_d)^\top$,¹ and the approximate value function $\hat{v}(s, \mathbf{w})$ is a differentiable function of $\mathbf{w}$ for all $s \in \mathcal{S}$. We will be updating $\mathbf{w}$ at each of a series of discrete time steps, $t = 0, 1, 2, 3, \dots$, so we will need a notation $\mathbf{w}_t$ for the

¹The $^\top$ denotes transpose, needed here to turn the horizontal row vector in the text into a vertical column vector; in this book vectors are generally taken to be column vectors unless explicitly written out horizontally or transposed.

weight vector at each step. For now, let us assume that, on each step, we observe a new example $S_t \mapsto v_\pi(S_t)$ consisting of a (possibly randomly selected) state S_t and its true value under the policy. These states might be successive states from an interaction with the environment, but for now we do not assume so. Even though we are given the exact, correct values, $v_\pi(S_t)$ for each S_t , there is still a difficult problem because our function approximator has limited resources and thus limited resolution. In particular, there is generally no $\mathbf{w}$ that gets all the states, or even all the examples, exactly correct. In addition, we must generalize to all the other states that have not appeared in examples.

We assume that states appear in examples with the same distribution, μ , over which we are trying to minimize the $\overline{\text{VE}}$ as given by (9.1). A good strategy in this case is to try to minimize error on the observed examples. *Stochastic gradient-descent* (SGD) methods do this by adjusting the weight vector after each example by a small amount in the direction that would most reduce the error on that example:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t - \frac{1}{2} \alpha \nabla \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right]^2 \quad (9.4)$$

$$= \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t), \quad (9.5)$$

where α is a positive step-size parameter, and $\nabla f(\mathbf{w})$, for any scalar expression $f(\mathbf{w})$ that is a function of a vector (here $\mathbf{w}$), denotes the column vector of partial derivatives of the expression with respect to the components of the vector:

$$\nabla f(\mathbf{w}) \doteq \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial f(\mathbf{w})}{\partial w_d} \right)^\top. \quad (9.6)$$

This derivative vector is the *gradient* of f with respect to $\mathbf{w}$. SGD methods are “gradient descent” methods because the overall step in $\mathbf{w}_t$ is proportional to the negative gradient of the example’s squared error (9.4). This is the direction in which the error falls most rapidly. Gradient descent methods are called “stochastic” when the update is done, as here, on only a single example, which might have been selected stochastically. Over many examples, making small steps, the overall effect is to minimize an average performance measure such as the $\overline{\text{VE}}$.

It may not be immediately apparent why SGD takes only a small step in the direction of the gradient. Could we not move all the way in this direction and completely eliminate the error on the example? In many cases this could be done, but usually it is not desirable. Remember that we do not seek or expect to find a value function that has zero error for all states, but only an approximation that balances the errors in different states. If we completely corrected each example in one step, then we would not find such a balance. In fact, the convergence results for SGD methods assume that α decreases over time. If it decreases in such a way as to satisfy the standard stochastic approximation conditions (2.7), then the SGD method (9.5) is guaranteed to converge to a local optimum.

We turn now to the case in which the target output, here denoted $U_t \in \mathbb{R}$, of the t th training example, $S_t \mapsto U_t$, is not the true value, $v_\pi(S_t)$, but some, possibly random, approximation to it. For example, U_t might be a noise-corrupted version of $v_\pi(S_t)$, or it might be one of the bootstrapping targets using $\hat{v}$ mentioned in the previous section. In

these cases we cannot perform the exact update (9.5) because $v_\pi(S_t)$ is unknown, but we can approximate it by substituting U_t in place of $v_\pi(S_t)$. This yields the following general SGD method for state-value prediction:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [U_t - \hat{v}(S_t, \mathbf{w}_t)] \nabla \hat{v}(S_t, \mathbf{w}_t). \quad (9.7)$$

If U_t is an *unbiased* estimate, that is, if $\mathbb{E}[U_t | S_t = s] = v_\pi(S_t)$, for each t , then $\mathbf{w}_t$ is guaranteed to converge to a local optimum under the usual stochastic approximation conditions (2.7) for decreasing α .

For example, suppose the states in the examples are the states generated by interaction (or simulated interaction) with the environment using policy π . Because the true value of a state is the expected value of the return following it, the Monte Carlo target $U_t \doteq G_t$ is by definition an unbiased estimate of $v_\pi(S_t)$. With this choice, the general SGD method (9.7) converges to a locally optimal approximation to $v_\pi(S_t)$. Thus, the gradient-descent version of Monte Carlo state-value prediction is guaranteed to find a locally optimal solution. Pseudocode for a complete algorithm is shown in the box below.

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
 Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$
 Algorithm parameter: step size $\alpha > 0$
 Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop forever (for each episode):
 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π
 Loop for each step of episode, $t = 0, 1, \dots, T - 1$:
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$

One does not obtain the same guarantees if a bootstrapping estimate of $v_\pi(S_t)$ is used as the target U_t in (9.7). Bootstrapping targets such as n -step returns $G_{t:t+n}$ or the DP target $\sum_{a,s',r} \pi(a|S_t) p(s', r | S_t, a) [r + \gamma \hat{v}(s', \mathbf{w}_t)]$ all depend on the current value of the weight vector $\mathbf{w}_t$, which implies that they will be biased and that they will not produce a true gradient-descent method. One way to look at this is that the key step from (9.4) to (9.5) relies on the target being independent of $\mathbf{w}_t$. This step would not be valid if a bootstrapping estimate were used in place of $v_\pi(S_t)$. Bootstrapping methods are not in fact instances of true gradient descent (Barnard, 1993). They take into account the effect of changing the weight vector $\mathbf{w}_t$ on the estimate, but ignore its effect on the target. They include only a part of the gradient and, accordingly, we call them *semi-gradient methods*.

Although semi-gradient (bootstrapping) methods do not converge as robustly as gradient methods, they do converge reliably in important cases such as the linear case discussed in the next section. Moreover, they offer important advantages that make them often clearly preferred. One reason for this is that they typically enable significantly faster learning, as we have seen in Chapters 6 and 7. Another is that they enable learning to

be continual and online, without waiting for the end of an episode. This enables them to be used on continuing problems and provides computational advantages. A prototypical semi-gradient method is semi-gradient TD(0), which uses $U_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ as its target. Complete pseudocode for this method is given in the box below.

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
 Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
 Algorithm parameter: step size $\alpha > 0$
 Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
 Initialize S
 Loop for each step of episode:
 Choose $A \sim \pi(\cdot|S)$
 Take action A , observe R, S'
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$
 $S \leftarrow S'$
 until S is terminal

State aggregation is a simple form of generalizing function approximation in which states are grouped together, with one estimated value (one component of the weight vector $\mathbf{w}$) for each group. The value of a state is estimated as its group's component, and when the state is updated, that component alone is updated. State aggregation is a special case of SGD (9.7) in which the gradient, $\nabla \hat{v}(S_t, \mathbf{w}_t)$, is 1 for S_t 's group's component and 0 for the other components.

Example 9.1: State Aggregation on the 1000-state Random Walk Consider a 1000-state version of the random walk task (Examples 6.2 and 7.1 on pages 125 and 144). The states are numbered from 1 to 1000, left to right, and all episodes begin near the center, in state 500. State transitions are from the current state to one of the 100 neighboring states to its left, or to one of the 100 neighboring states to its right, all with equal probability. Of course, if the current state is near an edge, then there may be fewer than 100 neighbors on that side of it. In this case, all the probability that would have gone into those missing neighbors goes into the probability of terminating on that side (thus, state 1 has a 0.5 chance of terminating on the left, and state 950 has a 0.25 chance of terminating on the right). As usual, termination on the left produces a reward of -1 , and termination on the right produces a reward of $+1$. All other transitions have a reward of zero. We use this task as a running example throughout this section.

Figure 9.1 shows the true value function v_π for this task. It is nearly a straight line, but curving slightly toward the horizontal for the last 100 states at each end. Also shown is the final approximate value function learned by the gradient Monte-Carlo algorithm with state aggregation after 100,000 episodes with a step size of $\alpha = 2 \times 10^{-5}$. For the state aggregation, the 1000 states were partitioned into 10 groups of 100 states each (i.e., states 1–100 were one group, states 101–200 were another, and so on). The staircase effect

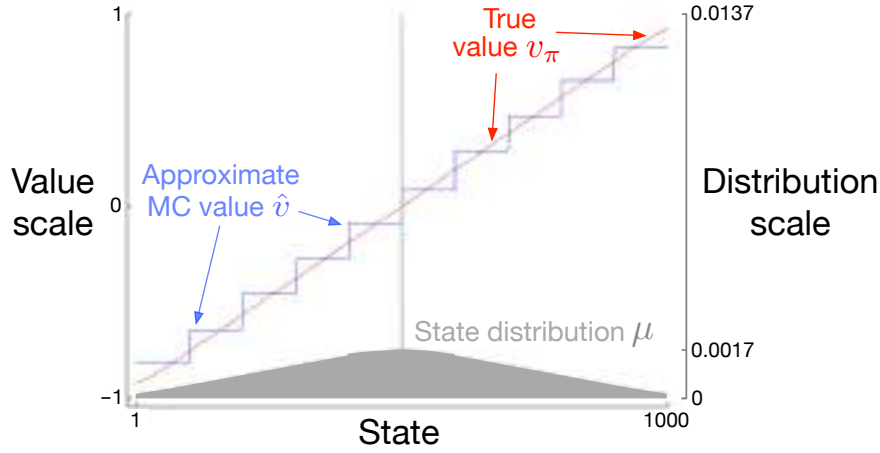


Figure 9.1: Function approximation by state aggregation on the 1000-state random walk task, using the gradient Monte Carlo algorithm (page 202).

shown in the figure is typical of state aggregation; within each group, the approximate value is constant, and it changes abruptly from one group to the next. These approximate values are close to the global minimum of the $\overline{VE}$ (9.1).

Some of the details of the approximate values are best appreciated by reference to the state distribution μ for this task, shown in the lower portion of the figure with a right-side scale. State 500, in the center, is the first state of every episode, but is rarely visited again. On average, about 1.37% of the time steps are spent in the start state. The states reachable in one step from the start state are the second most visited, with about 0.17% of the time steps being spent in each of them. From there μ falls off almost linearly, reaching about 0.0147% at the extreme states 1 and 1000. The most visible effect of the distribution is on the leftmost groups, whose values are clearly shifted higher than the unweighted average of the true values of states within the group, and on the rightmost groups, whose values are clearly shifted lower. This is due to the states in these areas having the greatest asymmetry in their weightings by μ . For example, in the leftmost group, state 100 is weighted more than 3 times more strongly than state 1. Thus the estimate for the group is biased toward the true value of state 100, which is higher than the true value of state 1. ■

9.4 Linear Methods

One of the most important special cases of function approximation is that in which the approximate function, $\hat{v}(\cdot, \mathbf{w})$, is a linear function of the weight vector, $\mathbf{w}$. Corresponding to every state s , there is a real-valued vector $\mathbf{x}(s) \doteq (x_1(s), x_2(s), \dots, x_d(s))^\top$, with the same number of components as $\mathbf{w}$. Linear methods approximate state-value function by

the inner product between $\mathbf{w}$ and $\mathbf{x}(s)$:

$$\hat{v}(s, \mathbf{w}) \doteq \mathbf{w}^\top \mathbf{x}(s) \doteq \sum_{i=1}^d w_i x_i(s). \quad (9.8)$$

In this case the approximate value function is said to be *linear in the weights*, or simply *linear*.

The vector $\mathbf{x}(s)$ is called a *feature vector* representing state s . Each component $x_i(s)$ of $\mathbf{x}(s)$ is the value of a function $x_i : \mathcal{S} \rightarrow \mathbb{R}$. We think of a *feature* as the entirety of one of these functions, and we call its value for a state s a *feature of s* . For linear methods, features are *basis functions* because they form a linear basis for the set of approximate functions. Constructing d -dimensional feature vectors to represent states is the same as selecting a set of d basis functions. Features may be defined in many different ways; we cover a few possibilities in the next sections.

It is natural to use SGD updates with linear function approximation. The gradient of the approximate value function with respect to $\mathbf{w}$ in this case is

$$\nabla \hat{v}(s, \mathbf{w}) = \mathbf{x}(s).$$

Thus, in the linear case the general SGD update (9.7) reduces to a particularly simple form:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[U_t - \hat{v}(S_t, \mathbf{w}_t) \right] \mathbf{x}(S_t).$$

Because it is so simple, the linear SGD case is one of the most favorable for mathematical analysis. Almost all useful convergence results for learning systems of all kinds are for linear (or simpler) function approximation methods.

In particular, in the linear case there is only one optimum (or, in degenerate cases, one set of equally good optima), and thus any method that is guaranteed to converge to or near a local optimum is automatically guaranteed to converge to or near the global optimum. For example, the gradient Monte Carlo algorithm presented in the previous section converges to the global optimum of the $\overline{\text{VE}}$ under linear function approximation if α is reduced over time according to the usual conditions.

The semi-gradient TD(0) algorithm presented in the previous section also converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary. The weight vector converged to is also not the global optimum, but rather a point near the local optimum. It is useful to consider this important case in more detail, specifically for the continuing case. The update at each time t is

$$\begin{aligned} \mathbf{w}_{t+1} &\doteq \mathbf{w}_t + \alpha \left(R_{t+1} + \gamma \mathbf{w}_t^\top \mathbf{x}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t \right) \mathbf{x}_t \\ &= \mathbf{w}_t + \alpha \left(R_{t+1} \mathbf{x}_t - \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \mathbf{w}_t \right), \end{aligned} \quad (9.9)$$

where here we have used the notational shorthand $\mathbf{x}_t = \mathbf{x}(S_t)$. Once the system has reached steady state, for any given $\mathbf{w}_t$, the expected next weight vector can be written

$$\mathbb{E}[\mathbf{w}_{t+1} | \mathbf{w}_t] = \mathbf{w}_t + \alpha (\mathbf{b} - \mathbf{A} \mathbf{w}_t), \quad (9.10)$$

where

$$\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d \quad \text{and} \quad \mathbf{A} \doteq \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^\top\right] \in \mathbb{R}^d \times \mathbb{R}^d \quad (9.11)$$

From (9.10) it is clear that, if the system converges, it must converge to the weight vector $\mathbf{w}_{\text{TD}}$ at which

$$\begin{aligned} \mathbf{b} - \mathbf{A}\mathbf{w}_{\text{TD}} &= \mathbf{0} \\ \Rightarrow \quad \mathbf{b} &= \mathbf{A}\mathbf{w}_{\text{TD}} \\ \Rightarrow \quad \mathbf{w}_{\text{TD}} &\doteq \mathbf{A}^{-1}\mathbf{b}. \end{aligned} \quad (9.12)$$

This quantity is called the *TD fixed point*. In fact linear semi-gradient TD(0) converges to this point. Some of the theory proving its convergence, and the existence of the inverse above, is given in the box.

Proof of Convergence of Linear TD(0)

What properties assure convergence of the linear TD(0) algorithm (9.9)? Some insight can be gained by rewriting (9.10) as

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b}. \quad (9.13)$$

Note that the matrix $\mathbf{A}$ multiplies the weight vector $\mathbf{w}_t$ and not $\mathbf{b}$; only $\mathbf{A}$ is important to convergence. To develop intuition, consider the special case in which $\mathbf{A}$ is a diagonal matrix. If any of the diagonal elements are negative, then the corresponding diagonal element of $\mathbf{I} - \alpha\mathbf{A}$ will be greater than one, and the corresponding component of $\mathbf{w}_t$ will be amplified, which will lead to divergence if continued. On the other hand, if the diagonal elements of $\mathbf{A}$ are all positive, then α can be chosen smaller than one over the largest of them, such that $\mathbf{I} - \alpha\mathbf{A}$ is diagonal with all diagonal elements between 0 and 1. In this case the first term of the update tends to shrink $\mathbf{w}_t$, and stability is assured. In general, $\mathbf{w}_t$ will be reduced toward zero whenever $\mathbf{A}$ is *positive definite*, meaning $y^\top \mathbf{A} y > 0$ for any real vector $y \neq 0$. Positive definiteness also ensures that the inverse $\mathbf{A}^{-1}$ exists.

For linear TD(0), in the continuing case with $\gamma < 1$, the $\mathbf{A}$ matrix (9.11) can be written

$$\begin{aligned} \mathbf{A} &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{r,s'} p(r,s'|s,a) \mathbf{x}(s) (\mathbf{x}(s) - \gamma\mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \sum_{s'} p(s'|s) \mathbf{x}(s) (\mathbf{x}(s) - \gamma\mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \mathbf{x}(s) \left(\mathbf{x}(s) - \gamma \sum_{s'} p(s'|s) \mathbf{x}(s') \right)^\top \\ &= \mathbf{X}^\top \mathbf{D} (\mathbf{I} - \gamma\mathbf{P}) \mathbf{X}, \end{aligned}$$

where $\mu(s)$ is the stationary distribution under π , $p(s'|s)$ is the probability of

transition from s to s' under policy π , $\mathbf{P}$ is the $|\mathcal{S}| \times |\mathcal{S}|$ matrix of these probabilities, $\mathbf{D}$ is the $|\mathcal{S}| \times |\mathcal{S}|$ diagonal matrix with the $\mu(s)$ on its diagonal, and $\mathbf{X}$ is the $|\mathcal{S}| \times d$ matrix with $\mathbf{x}(s)$ as its rows. From here it is clear that the inner matrix $\mathbf{D}(\mathbf{I} - \gamma\mathbf{P})$ is key to determining the positive definiteness of $\mathbf{A}$.

For a key matrix of this form, positive definiteness is assured if all of its columns sum to a nonnegative number. This was shown by Sutton (1988, p. 27) based on two previously established theorems. One theorem says that any matrix $\mathbf{M}$ is positive definite if and only if the symmetric matrix $\mathbf{S} = \mathbf{M} + \mathbf{M}^\top$ is positive definite (Sutton 1988, appendix). The second theorem says that any symmetric real matrix $\mathbf{S}$ is positive definite if all of its diagonal entries are positive and greater than the sum of the absolute values of the corresponding off-diagonal entries (Varga 1962, p. 23). For our key matrix, $\mathbf{D}(\mathbf{I} - \gamma\mathbf{P})$, the diagonal entries are positive and the off-diagonal entries are negative, so all we have to show is that each row sum plus the corresponding column sum is positive. The row sums are all positive because $\mathbf{P}$ is a stochastic matrix and $\gamma < 1$. Thus it only remains to show that the column sums are nonnegative. Note that the row vector of the column sums of any matrix $\mathbf{M}$ can be written as $\mathbf{1}^\top \mathbf{M}$, where $\mathbf{1}$ is the column vector with all components equal to 1. Let $\boldsymbol{\mu}$ denote the $|\mathcal{S}|$ -vector of the $\mu(s)$, where $\boldsymbol{\mu} = \mathbf{P}^\top \boldsymbol{\mu}$ by virtue of $\boldsymbol{\mu}$ being the stationary distribution. The column sums of our key matrix, then, are:

$$\begin{aligned} \mathbf{1}^\top \mathbf{D}(\mathbf{I} - \gamma\mathbf{P}) &= \boldsymbol{\mu}^\top (\mathbf{I} - \gamma\mathbf{P}) \\ &= \boldsymbol{\mu}^\top - \gamma \boldsymbol{\mu}^\top \mathbf{P} \\ &= \boldsymbol{\mu}^\top - \gamma \boldsymbol{\mu}^\top \quad (\text{because } \boldsymbol{\mu} \text{ is the stationary distribution}) \\ &= (1 - \gamma) \boldsymbol{\mu}^\top, \end{aligned}$$

all components of which are positive. Thus, the key matrix and its $\mathbf{A}$ matrix are positive definite, and on-policy TD(0) is stable. (Additional conditions and a schedule for reducing α over time are needed to prove convergence with probability one.)

At the TD fixed point, it has also been proven (in the continuing case) that the $\overline{\text{VE}}$ is within a bounded expansion of the lowest possible error:

$$\overline{\text{VE}}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1 - \gamma} \min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w}). \quad (9.14)$$

That is, the asymptotic error of the TD method is no more than $\frac{1}{1 - \gamma}$ times the smallest possible error, that attained in the limit by the Monte Carlo method. Because γ is often near one, this expansion factor can be quite large, so there is substantial potential loss in asymptotic performance with the TD method. On the other hand, recall that the TD methods are often of vastly reduced variance compared to Monte Carlo methods, and thus faster, as we saw in Chapters 6 and 7. Which method will be best depends on the nature of the approximation and problem, and on how long learning continues.

A bound analogous to (9.14) applies to other on-policy bootstrapping methods as well. For example, linear semi-gradient DP (Eq. 9.7 with $U_t \doteq \sum_a \pi(a|S_t) \sum_{s',r} p(s',r|S_t,a)[r + \gamma \hat{v}(s',\mathbf{w}_t)]$) with updates according to the on-policy distribution will also converge to the TD fixed point. One-step semi-gradient *action-value* methods, such as semi-gradient Sarsa(0) covered in the next chapter converge to an analogous fixed point and an analogous bound. For episodic tasks, there is a slightly different but related bound (see Bertsekas and Tsitsiklis, 1996). There are also a few technical conditions on the rewards, features, and decrease in the step-size parameter, which we have omitted here. The full details can be found in the original paper (Tsitsiklis and Van Roy, 1997).

Critical to these convergence results is that states are updated according to the on-policy distribution. For other update distributions, bootstrapping methods using function approximation may actually diverge to infinity. Examples of this and a discussion of possible solution methods are given in Chapter 11.

Example 9.2: Bootstrapping on the 1000-state Random Walk State aggregation is a special case of linear function approximation, so let's return to the 1000-state random walk to illustrate some of the observations made in this chapter. The left panel of Figure 9.2 shows the final value function learned by the semi-gradient TD(0) algorithm (page 203) using the same state aggregation as in Example 9.1. We see that the near-asymptotic TD approximation is indeed farther from the true values than the Monte Carlo approximation shown in Figure 9.1.

Nevertheless, TD methods retain large potential advantages in learning rate, and generalize Monte Carlo methods, as we investigated fully with n -step TD methods in Chapter 7. The right panel of Figure 9.2 shows results with an n -step semi-gradient TD method using state aggregation on the 1000-state random walk that are strikingly similar to those we obtained earlier with tabular methods and the 19-state random walk (Figure 7.2). To obtain such quantitatively similar results we switched the state aggregation to 20 groups of 50 states each. The 20 groups were then quantitatively close

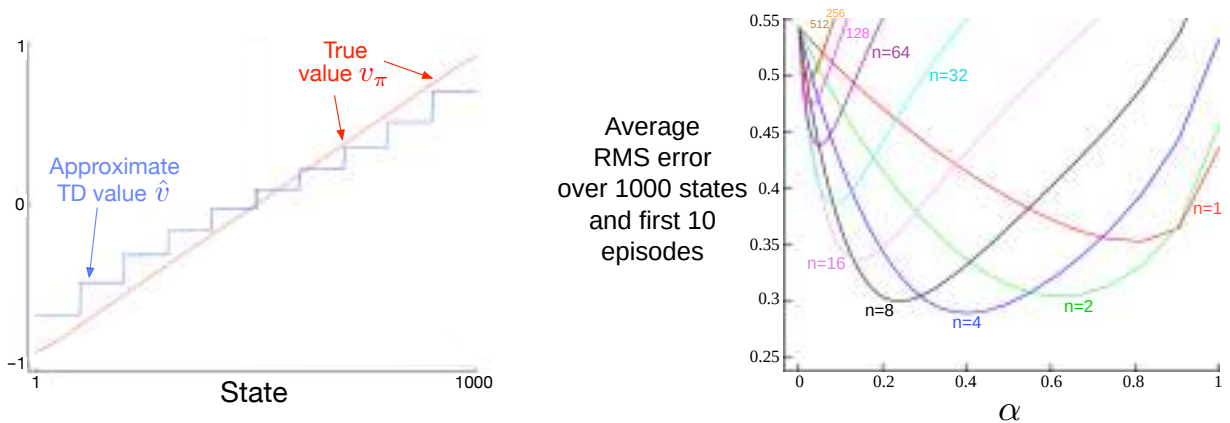


Figure 9.2: Bootstrapping with state aggregation on the 1000-state random walk task. *Left:* Asymptotic values of semi-gradient TD are worse than the asymptotic Monte Carlo values in Figure 9.1. *Right:* Performance of n -step methods with state-aggregation are strikingly similar to those with tabular representations (cf. Figure 7.2). These data are averages over 100 runs.

to the 19 states of the tabular problem. In particular, recall that state transitions were up to 100 states to the left or right. A typical transition would then be of 50 states to the right or left, which is quantitatively analogous to the single-state state transitions of the 19-state tabular system. To complete the match, we use here the same performance measure—an unweighted average of the RMS error over all states and over the first 10 episodes—rather than a $\overline{VE}$ objective as is otherwise more appropriate when using function approximation. ■

The semi-gradient n -step TD algorithm used in the example above is the natural extension of the tabular n -step TD algorithm presented in Chapter 7 to semi-gradient function approximation. Pseudocode is given in the box below.

n -step semi-gradient TD for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
 Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
 Algorithm parameters: step size $\alpha > 0$, a positive integer n
 Initialize value-function weights $\mathbf{w}$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
 All store and access operations (S_t and R_t) can take their index mod $n + 1$

Loop for each episode:
 Initialize and store $S_0 \neq \text{terminal}$
 $T \leftarrow \infty$
 Loop for $t = 0, 1, 2, \dots$:
 If $t < T$, then:
 Take an action according to $\pi(\cdot | S_t)$
 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 If S_{t+1} is terminal, then $T \leftarrow t + 1$
 $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
 If $\tau \geq 0$:
 $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n \hat{v}(S_{\tau+n}, \mathbf{w})$ ($G_{\tau:\tau+n}$)
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha [G - \hat{v}(S_\tau, \mathbf{w})] \nabla \hat{v}(S_\tau, \mathbf{w})$
 Until $\tau = T - 1$

The key equation of this algorithm, analogous to (7.2), is

$$\mathbf{w}_{t+n} \doteq \mathbf{w}_{t+n-1} + \alpha [G_{t:t+n} - \hat{v}(S_t, \mathbf{w}_{t+n-1})] \nabla \hat{v}(S_t, \mathbf{w}_{t+n-1}), \quad 0 \leq t < T, \quad (9.15)$$

where the n -step return is generalized from (7.1) to

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_{t+n-1}), \quad 0 \leq t \leq T - n. \quad (9.16)$$

Exercise 9.1 Show that tabular methods such as presented in Part I of this book are a special case of linear function approximation. What would the feature vectors be? □

9.5 Feature Construction for Linear Methods

Linear methods are interesting because of their convergence guarantees, but also because in practice they can be very efficient in terms of both data and computation. Whether or not this is so depends critically on how the states are represented in terms of features, which we investigate in this large section. Choosing features appropriate to the task is an important way of adding prior domain knowledge to reinforcement learning systems. Intuitively, the features should correspond to the aspects of the state space along which generalization may be appropriate. If we are valuing geometric objects, for example, we might want to have features for each possible shape, color, size, or function. If we are valuing states of a mobile robot, then we might want to have features for locations, degrees of remaining battery power, recent sonar readings, and so on.

A limitation of the linear form is that it cannot take into account any interactions between features, such as the presence of feature i being good only in the absence of feature j . For example, in the pole-balancing task (Example 3.4) high angular velocity can be either good or bad depending on the angle. If the angle is high, then high angular velocity means an imminent danger of falling—a bad state—whereas if the angle is low, then high angular velocity means the pole is righting itself—a good state. A linear value function could not represent this if its features coded separately for the angle and the angular velocity. It needs instead, or in addition, features for combinations of these two underlying state dimensions. In the following subsections we consider a variety of general ways of doing this.

9.5.1 Polynomials

The states of many problems are initially expressed as numbers, such as positions and velocities in the pole-balancing task (Example 3.4), the number of cars in each lot in the Jack’s car rental problem (Example 4.2), or the gambler’s capital in the gambler problem (Example 4.3). In these types of problems, function approximation for reinforcement learning has much in common with the familiar tasks of interpolation and regression. Various families of features commonly used for interpolation and regression can also be used in reinforcement learning. Polynomials make up one of the simplest families of features used for interpolation and regression. While the basic polynomial features we discuss here do not work as well as other types of features in reinforcement learning, they serve as a good introduction because they are simple and familiar.

As an example, suppose a reinforcement learning problem has states with two numerical dimensions. For a single representative state s , let its two numbers be $s_1 \in \mathbb{R}$ and $s_2 \in \mathbb{R}$. You might choose to represent s simply by its two state dimensions, so that $\mathbf{x}(s) = (s_1, s_2)^\top$, but then you would not be able to take into account any interactions between these dimensions. In addition, if both s_1 and s_2 were zero, then the approximate value would have to also be zero. Both limitations can be overcome by instead representing s by the four-dimensional feature vector $\mathbf{x}(s) = (1, s_1, s_2, s_1 s_2)^\top$. The initial 1 feature allows the representation of affine functions in the original state numbers, and the final product feature, $s_1 s_2$, enables interactions to be taken into account. Or you might choose to use higher-dimensional feature vectors like $\mathbf{x}(s) = (1, s_1, s_2, s_1 s_2, s_1^2, s_2^2, s_1 s_2^2, s_1^2 s_2, s_1^2 s_2^2)^\top$ to

take more complex interactions into account. Such feature vectors enable approximations as arbitrary quadratic functions of the state numbers—even though the approximation is still linear in the weights that have to be learned. Generalizing this example from two to k numbers, we can represent highly-complex interactions among a problem's state dimensions:

Suppose each state s corresponds to k numbers, $s_1, s_2, \dots, s_k$, with each $s_i \in \mathbb{R}$. For this k -dimensional state space, each order- n polynomial-basis feature x_i can be written as

$$x_i(s) = \prod_{j=1}^k s_j^{c_{i,j}}, \quad (9.17)$$

where each $c_{i,j}$ is an integer in the set $\{0, 1, \dots, n\}$ for an integer $n \geq 0$. These features make up the order- n polynomial basis for dimension k , which contains $(n+1)^k$ different features.

Higher-order polynomial bases allow for more accurate approximations of more complicated functions. But because the number of features in an order- n polynomial basis grows exponentially with the dimension k of the natural state space (if $n > 0$), it is generally necessary to select a subset of them for function approximation. This can be done using prior beliefs about the nature of the function to be approximated, and some automated selection methods developed for polynomial regression can be adapted to deal with the incremental and nonstationary nature of reinforcement learning.

Exercise 9.2 Why does (9.17) define $(n+1)^k$ distinct features for dimension k ? □

Exercise 9.3 What n and $c_{i,j}$ produce the feature vectors $\mathbf{x}(s) = (1, s_1, s_2, s_1 s_2, s_1^2, s_2^2, s_1 s_2^2, s_1^2 s_2, s_1^2 s_2^2)^\top$? □

9.5.2 Fourier Basis

Another linear function approximation method is based on the time-honored Fourier series, which expresses periodic functions as weighted sums of sine and cosine basis functions (features) of different frequencies. (A function f is periodic if $f(x) = f(x + \tau)$ for all x and some period τ .) The Fourier series and the more general Fourier transform are widely used in applied sciences in part because if a function to be approximated is known, then the basis function weights are given by simple formulae and, further, with enough basis functions essentially any function can be approximated as accurately as desired. In reinforcement learning, where the functions to be approximated are unknown, Fourier basis functions are of interest because they are easy to use and can perform well in a range of reinforcement learning problems.

First consider the one-dimensional case. The usual Fourier series representation of a function of one dimension having period τ represents the function as a linear combination of sine and cosine functions that are each periodic with periods that evenly divide τ (in other words, whose frequencies are integer multiples of a fundamental frequency $1/\tau$). But if you are interested in approximating an aperiodic function defined over a bounded interval, then you can use these Fourier basis features with τ set to the length the interval.

The function of interest is then just one period of the periodic linear combination of the sine and cosine features.

Furthermore, if you set τ to twice the length of the interval of interest and restrict attention to the approximation over the half interval $[0, \tau/2]$, then you can use just the cosine features. This is possible because you can represent any *even* function, that is, any function that is symmetric about the origin, with just the cosine basis. So any function over the half-period $[0, \tau/2]$ can be approximated as closely as desired with enough cosine features. (Saying “any function” is not exactly correct because the function has to be mathematically well-behaved, but we skip this technicality here.) Alternatively, it is possible to use just sine features, linear combinations of which are always *odd* functions, that is functions that are anti-symmetric about the origin. But it is generally better to keep just the cosine features because “half-even” functions tend to be easier to approximate than “half-odd” functions because the latter are often discontinuous at the origin. Of course, this does not rule out using both sine and cosine features to approximate over the interval $[0, \tau/2]$, which might be advantageous in some circumstances.

Following this logic and letting $\tau = 2$ so that the features are defined over the half- τ interval $[0, 1]$, the one-dimensional order- n Fourier cosine basis consists of the $n + 1$ features

$$x_i(s) = \cos(i\pi s), \quad s \in [0, 1],$$

for $i = 0, \dots, n$. Figure 9.3 shows one-dimensional Fourier cosine features x_i , for $i = 1, 2, 3, 4$; x_0 is a constant function.

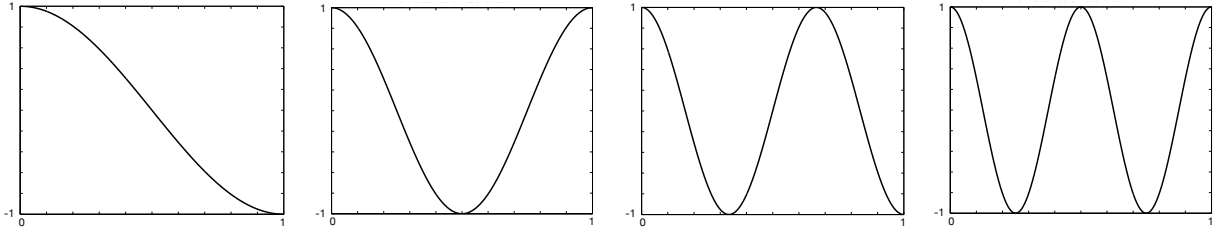


Figure 9.3: One-dimensional Fourier cosine-basis features x_i , $i = 1, 2, 3, 4$, for approximating functions over the interval $[0, 1]$. After Konidaris et al. (2011).

This same reasoning applies to the Fourier cosine series approximation in the multi-dimensional case as described in the box below.

Suppose each state s corresponds to a vector of k numbers, $\mathbf{s} = (s_1, s_2, \dots, s_k)^\top$, with each $s_i \in [0, 1]$. The i th feature in the order- n Fourier cosine basis can then be written

$$x_i(s) = \cos(\pi \mathbf{s}^\top \mathbf{c}^i), \quad (9.18)$$

where $\mathbf{c}^i = (c_1^i, \dots, c_k^i)^\top$, with $c_j^i \in \{0, \dots, n\}$ for $j = 1, \dots, k$ and $i = 0, \dots, (n+1)^k$. This defines a feature for each of the $(n+1)^k$ possible integer vectors $\mathbf{c}^i$. The inner

product $\mathbf{s}^\top \mathbf{c}^i$ has the effect of assigning an integer in $\{0, \dots, n\}$ to each dimension of $\mathbf{s}$. As in the one-dimensional case, this integer determines the feature's frequency along that dimension. The features can of course be shifted and scaled to suit the bounded state space of a particular application.

As an example, consider the $k = 2$ case in which $\mathbf{s} = (s_1, s_2)^\top$, where each $\mathbf{c}^i = (c_1^i, c_2^i)^\top$. Figure 9.4 shows a selection of six Fourier cosine features, each labeled by the vector $\mathbf{c}^i$ that defines it (s_1 is the horizontal axis and $\mathbf{c}^i$ is shown as a row vector with the index i omitted). Any zero in $\mathbf{c}$ means the feature is constant along that state dimension. So if $\mathbf{c} = (0, 0)^\top$, the feature is constant over both dimensions; if $\mathbf{c} = (c_1, 0)^\top$ the feature is constant over the second dimension and varies over the first with frequency depending on c_1 ; and similarly, for $\mathbf{c} = (0, c_2)^\top$. When $\mathbf{c} = (c_1, c_2)^\top$ with neither $c_j = 0$, the feature varies along both dimensions and represents an interaction between the two state variables. The values of c_1 and c_2 determine the frequency along each dimension, and their ratio gives the direction of the interaction.

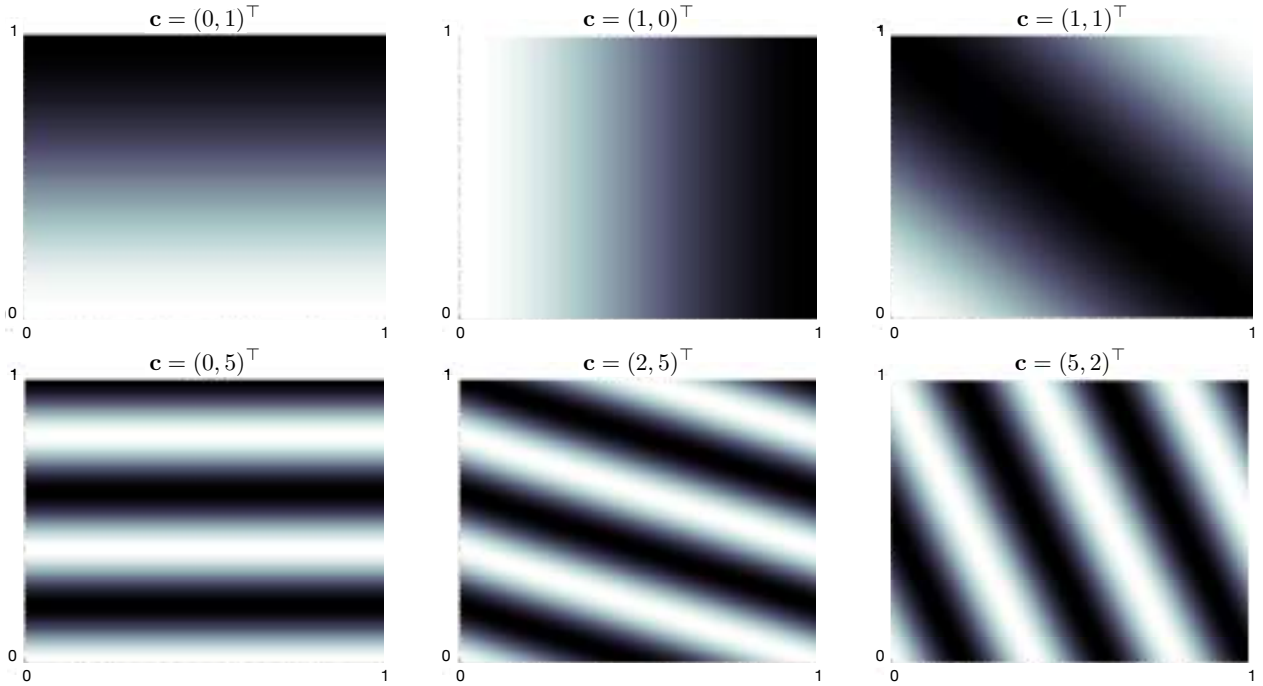


Figure 9.4: A selection of six two-dimensional Fourier cosine features, each labeled by the vector $\mathbf{c}^i$ that defines it (s_1 is the horizontal axis, and $\mathbf{c}^i$ is shown with the index i omitted). After Konidaris et al. (2011).

When using Fourier cosine features with a learning algorithm such as (9.7), semi-gradient TD(0), or semi-gradient Sarsa, it may be helpful to use a different step-size parameter for each feature. If α is the basic step-size parameter, then Konidaris, Osentoski, and Thomas (2011) suggest setting the step-size parameter for feature x_i to $\alpha_i = \alpha / \sqrt{(c_1^i)^2 + \dots + (c_k^i)^2}$ (except when each $c_j^i = 0$, in which case $\alpha_i = \alpha$).

Fourier cosine features with Sarsa can produce good performance compared to several

other collections of basis functions, including polynomial and radial basis functions. Not surprisingly, however, Fourier features have trouble with discontinuities because it is difficult to avoid “ringing” around points of discontinuity unless very high frequency basis functions are included.

The number of features in the order- n Fourier basis grows exponentially with the dimension of the state space, but if that dimension is small enough (e.g., $k \leq 5$), then one can select n so that all of the order- n Fourier features can be used. This makes the selection of features more-or-less automatic. For high dimension state spaces, however, it is necessary to select a subset of these features. This can be done using prior beliefs about the nature of the function to be approximated, and some automated selection methods can be adapted to deal with the incremental and nonstationary nature of reinforcement learning. An advantage of Fourier basis features in this regard is that it is easy to select features by setting the $\mathbf{c}^i$ vectors to account for suspected interactions among the state variables and by limiting the values in the $\mathbf{c}^j$ vectors so that the approximation can filter out high frequency components considered to be noise. On the other hand, because Fourier features are non-zero over the entire state space (with the few zeros excepted), they represent global properties of states, which can make it difficult to find good ways to represent local properties.

Figure 9.5 shows learning curves comparing the Fourier and polynomial bases on the 1000-state random walk example. In general, we do not recommend using polynomials for online learning.²

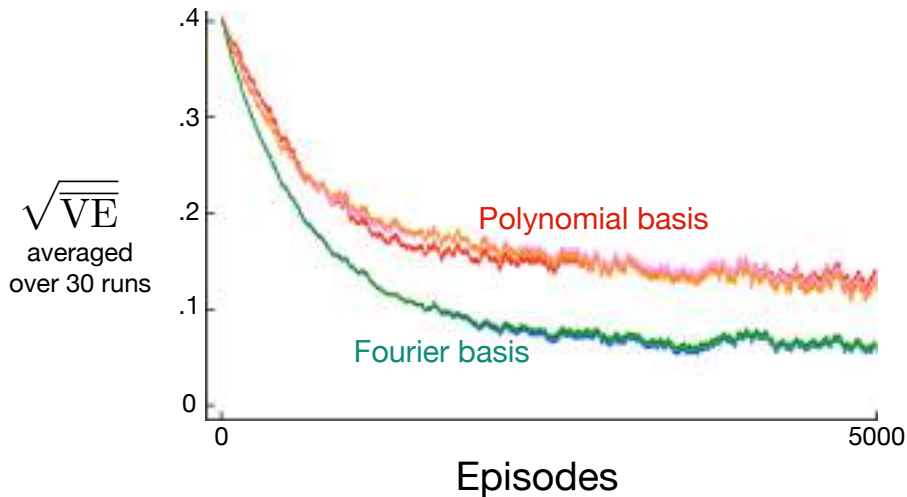


Figure 9.5: Fourier basis vs polynomials on the 1000-state random walk. Shown are learning curves for the gradient Monte Carlo method with Fourier and polynomial bases of order 5, 10, and 20. The step-size parameters were roughly optimized for each case: $\alpha = 0.0001$ for the polynomial basis and $\alpha = 0.00005$ for the Fourier basis. The performance measure (y-axis) is the root mean squared value error (9.1).

²There are families of polynomials more complicated than those we have discussed, for example, different families of orthogonal polynomials, and these might work better, but at present there is little experience with them in reinforcement learning.

9.5.3 Coarse Coding

Consider a task in which the natural representation of the state set is a continuous two-dimensional space. One kind of representation for this case is made up of features corresponding to *circles* in state space, as shown to the right. If the state is inside a circle, then the corresponding feature has the value 1 and is said to be *present*; otherwise the feature is 0 and is said to be *absent*. This kind of 1–0-valued feature is called a *binary feature*. Given a state, which binary features are present indicate within which circles the state lies, and thus coarsely code for its location. Representing a state with features that overlap in this way (although they need not be circles or binary) is known as *coarse coding*.

Assuming linear gradient-descent function approximation, consider the effect of the size and density of the circles. Corresponding to each circle is a single weight (a component of $\mathbf{w}$) that is affected by learning. If we train at one state, a point in the space, then the weights of all circles intersecting that state will be affected. Thus, by (9.8), the approximate value function will be affected at all states within the union of the circles, with a greater effect the more circles a point has “in common” with the state, as shown in Figure 9.6. If the circles are small, then the generalization will be over a short distance, as in Figure 9.7 (left), whereas if they are large, it will be over a large distance, as in Figure 9.7 (middle). Moreover,

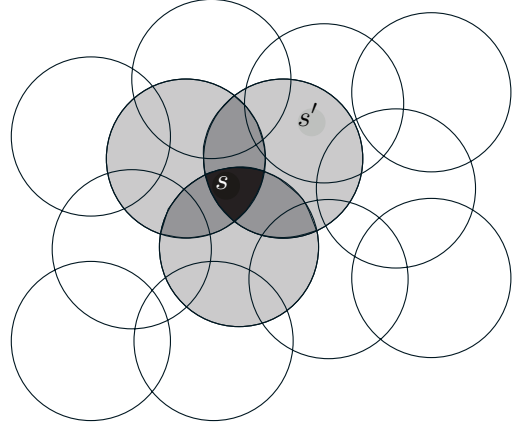


Figure 9.6: Coarse coding. Generalization from state s to state s' depends on the number of their features whose receptive fields (in this case, circles) overlap. These states have one feature in common, so there will be slight generalization between them.

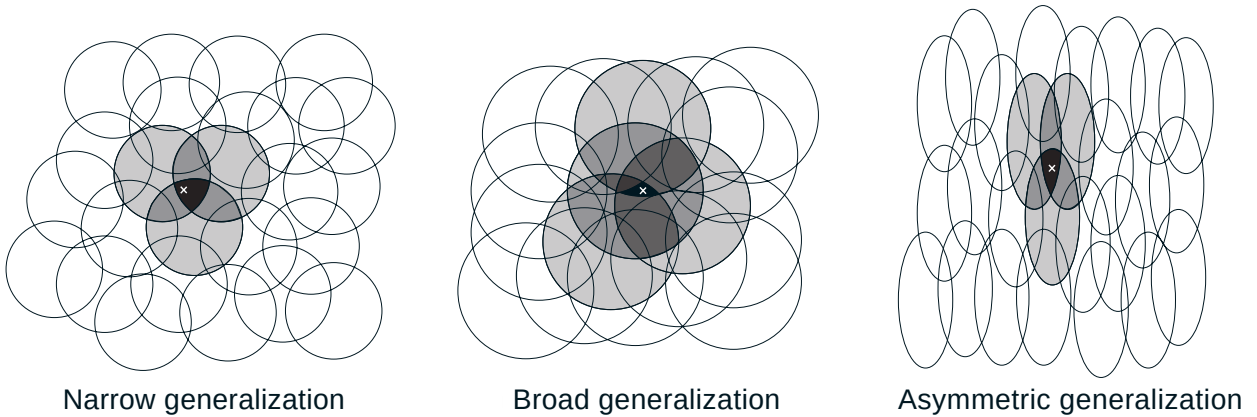


Figure 9.7: Generalization in linear function approximation methods is determined by the sizes and shapes of the features’ receptive fields. All three of these cases have roughly the same number and density of features.

the shape of the features will determine the nature of the generalization. For example, if they are not strictly circular, but are elongated in one direction, then generalization will be similarly affected, as in Figure 9.7 (right).

Features with large receptive fields give broad generalization, but might also seem to limit the learned function to a coarse approximation, unable to make discriminations much finer than the width of the receptive fields. Happily, this is not the case. Initial generalization from one point to another is indeed controlled by the size and shape of the receptive fields, but acuity, the finest discrimination ultimately possible, is controlled more by the total number of features.

Example 9.3: Coarseness of Coarse Coding This example illustrates the effect on learning of the size of the receptive fields in coarse coding. Linear function approximation based on coarse coding and (9.7) was used to learn a one-dimensional square-wave function (shown at the top of Figure 9.8). The values of this function were used as the targets, U_t . With just one dimension, the receptive fields were intervals rather than circles. Learning was repeated with three different sizes of the intervals: narrow, medium, and broad, as shown at the bottom of the figure. All three cases had the same density of features, about 50 over the extent of the function being learned. Training examples were generated uniformly at random over this extent. The step-size parameter was $\alpha = \frac{0.2}{n}$, where n is the number of features that were present at one time. Figure 9.8 shows the functions learned in all three cases over the course of learning. Note that the width of the features had a strong effect early in learning. With broad features, the generalization tended to be broad; with narrow features, only the close neighbors of each trained point were changed, causing the function learned to be more bumpy. However, the final function learned was affected only slightly by the width of the features. Receptive field shape tends to have a strong effect on generalization but little effect on asymptotic solution quality.

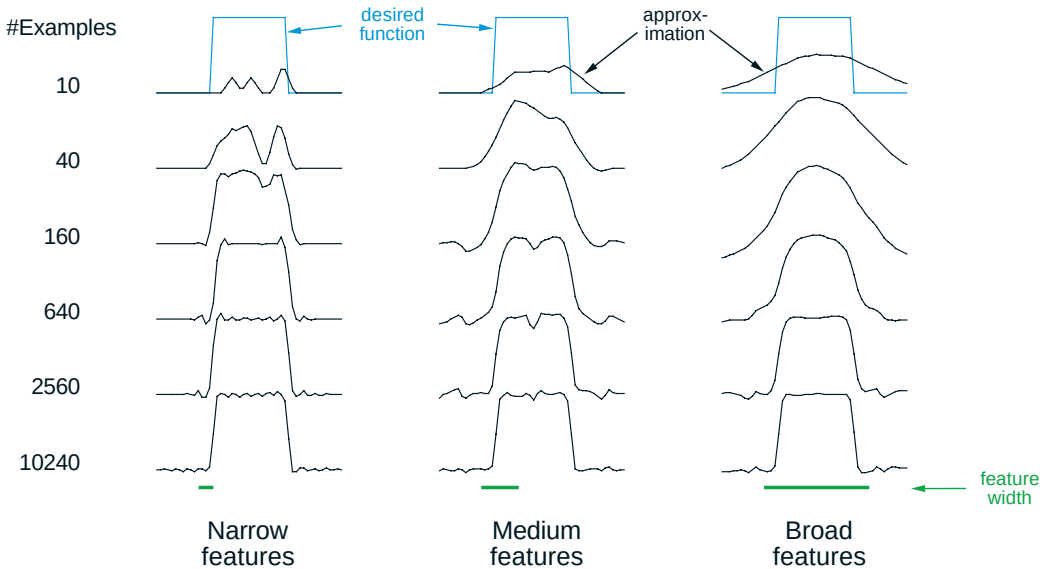


Figure 9.8: Example of feature width's strong effect on initial generalization (first row) and weak effect on asymptotic accuracy (last row). ■

9.5.4 Tile Coding

Tile coding is a form of coarse coding for multi-dimensional continuous spaces that is flexible and computationally efficient. It may be the most practical feature representation for modern sequential digital computers.

In tile coding the receptive fields of the features are grouped into partitions of the state space. Each such partition is called a *tiling*, and each element of the partition is called a *tile*. For example, the simplest tiling of a two-dimensional state space is a uniform grid such as that shown on the left side of Figure 9.9. The tiles or receptive field here are squares rather than the circles in Figure 9.6. If just this single tiling were used, then the state indicated by the white spot would be represented by the single feature whose tile it falls within; generalization would be complete to all states within the same tile and nonexistent to states outside it. With just one tiling, we would not have coarse coding but just a case of state aggregation.

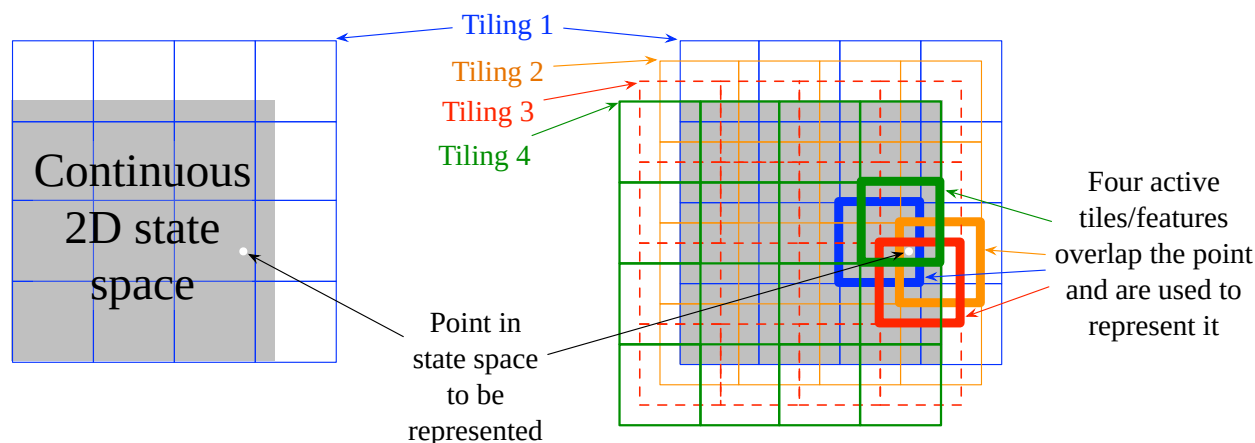


Figure 9.9: Multiple, overlapping grid-tilings on a limited two-dimensional space. These tilings are offset from one another by a uniform amount in each dimension.

To get the strengths of coarse coding requires overlapping receptive fields, and by definition the tiles of a partition do not overlap. To get true coarse coding with tile coding, multiple tilings are used, each offset by a fraction of a tile width. A simple case with four tilings is shown on the right side of Figure 9.9. Every state, such as that indicated by the white spot, falls in exactly one tile in each of the four tilings. These four tiles correspond to four features that become active when the state occurs. Specifically, the feature vector $\mathbf{x}(s)$ has one component for each tile in each tiling. In this example there are $4 \times 4 \times 4 = 64$ components, all of which will be 0 except for the four corresponding to the tiles that s falls within. Figure 9.10 shows the advantage of multiple offset tilings (coarse coding) over a single tiling on the 1000-state random walk example.

An immediate practical advantage of tile coding is that, because it works with partitions, the overall number of features that are active at one time is the same for any state. Exactly one feature is present in each tiling, so the total number of features present is always the same as the number of tilings. This allows the step-size parameter, α , to be set in an easy, intuitive way. For example, choosing $\alpha = \frac{1}{n}$, where n is the number

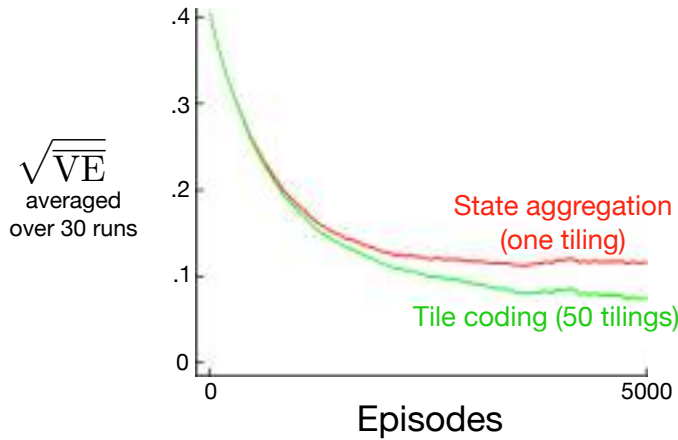


Figure 9.10: Why we use coarse coding. Shown are learning curves on the 1000-state random walk example for the gradient Monte Carlo algorithm with a single tiling and with multiple tilings. The space of 1000 states was treated as a single continuous dimension, covered with tiles each 200 states wide. The multiple tilings were offset from each other by 4 states. The step-size parameter was set so that the initial learning rate in the two cases was the same, $\alpha = 0.0001$ for the single tiling and $\alpha = 0.0001/50$ for the 50 tilings.

of tilings, results in exact one-trial learning. If the example $s \mapsto v$ is trained on, then whatever the prior estimate, $\hat{v}(s, \mathbf{w}_t)$, the new estimate will be $\hat{v}(s, \mathbf{w}_{t+1}) = v$. Usually one wishes to change more slowly than this, to allow for generalization and stochastic variation in target outputs. For example, one might choose $\alpha = \frac{1}{10n}$, in which case the estimate for the trained state would move one-tenth of the way to the target in one update, and neighboring states will be moved less, proportional to the number of tiles they have in common.

Tile coding also gains computational advantages from its use of binary feature vectors. Because each component is either 0 or 1, the weighted sum making up the approximate value function (9.8) is almost trivial to compute. Rather than performing d multiplications and additions, one simply computes the indices of the $n \ll d$ active features and then adds up the n corresponding components of the weight vector.

Generalization occurs to states other than the one trained if those states fall within any of the same tiles, proportional to the number of tiles in common. Even the choice of how to offset the tilings from each other affects generalization. If they are offset uniformly in each dimension, as they were in Figure 9.9, then different states can generalize in qualitatively different ways, as shown in the upper half of Figure 9.11. Each of the eight subfigures show the pattern of generalization from a trained state to nearby points. In this example there are eight tilings, thus 64 subregions within a tile that generalize distinctly, but all according to one of these eight patterns. Note how uniform offsets result in a strong effect along the diagonal in many patterns. These artifacts can be avoided if the tilings are offset asymmetrically, as shown in the lower half of the figure. These lower generalization patterns are better because they are all well centered on the trained state with no obvious asymmetries.

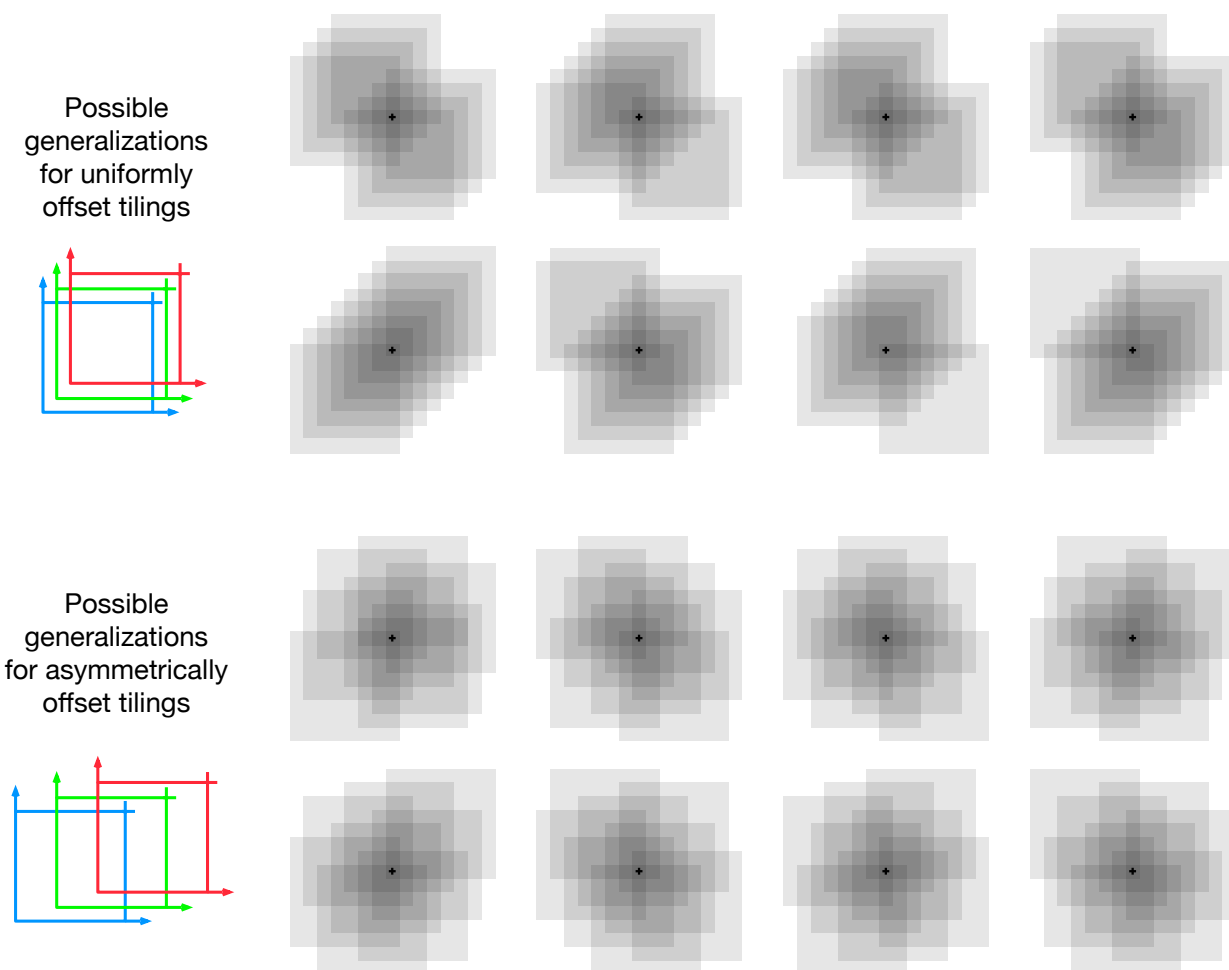


Figure 9.11: Why tile asymmetrical offsets are preferred in tile coding. Shown is the strength of generalization from a trained state, indicated by the small black plus, to nearby states, for the case of eight tilings. If the tilings are uniformly offset (above), then there are diagonal artifacts and substantial variations in the generalization, whereas with asymmetrically offset tilings the generalization is more spherical and homogeneous.

Tilings in all cases are offset from each other by a fraction of a tile width in each dimension. If w denotes the tile width and n the number of tilings, then $\frac{w}{n}$ is a fundamental unit. Within small squares $\frac{w}{n}$ on a side, all states activate the same tiles, have the same feature representation, and the same approximated value. If a state is moved by $\frac{w}{n}$ in any cartesian direction, the feature representation changes by one component/tile. Uniformly offset tilings are offset from each other by exactly this unit distance. For a two-dimensional space, we say that each tiling is offset by the displacement vector $(1, 1)$, meaning that it is offset from the previous tiling by $\frac{w}{n}$ times this vector. In these terms, the asymmetrically offset tilings shown in the lower part of Figure 9.11 are offset by a displacement vector of $(1, 3)$.

Extensive studies have been made of the effect of different displacement vectors on the generalization of tile coding (Parks and Miltzer, 1991; An, 1991; An, Miller and Parks,

1991; Miller, An, Glanz and Carter, 1990), assessing their homogeneity and tendency toward diagonal artifacts like those seen for the $(1, 1)$ displacement vectors. Based on this work, Miller and Glanz (1996) recommend using displacement vectors consisting of the first odd integers. In particular, for a continuous space of dimension k , a good choice is to use the first odd integers $(1, 3, 5, 7, \dots, 2k - 1)$, with n (the number of tilings) set to an integer power of 2 greater than or equal to $4k$. This is what we have done to produce the tilings in the lower half of Figure 9.11, in which $k = 2$, $n = 2^3 \geq 4k$, and the displacement vector is $(1, 3)$. In a three-dimensional case, the first four tilings would be offset in total from a base position by $(0, 0, 0)$, $(1, 3, 5)$, $(2, 6, 10)$, and $(3, 9, 15)$. Open-source software that can efficiently make tilings like this for any k is readily available.

In choosing a tiling strategy, one has to pick the number of the tilings and the shape of the tiles. The number of tilings, along with the size of the tiles, determines the resolution or fineness of the asymptotic approximation, as in general coarse coding and illustrated in Figure 9.8. The shape of the tiles will determine the nature of generalization as in Figure 9.7. Square tiles will generalize roughly equally in each dimension as indicated in Figure 9.11 (lower). Tiles that are elongated along one dimension, such as the stripe tilings in Figure 9.12 (middle), will promote generalization along that dimension. The tilings in Figure 9.12 (middle) are also denser and thinner on the left, promoting discrimination along the horizontal dimension at lower values along that dimension. The diagonal stripe tiling in Figure 9.12 (right) will promote generalization along one diagonal. In higher dimensions, axis-aligned stripes correspond to ignoring some of the dimensions in some of the tilings, that is, to hyperplanar slices. Irregular tilings such as shown in Figure 9.12 (left) are also possible, though rare in practice and beyond the standard software.

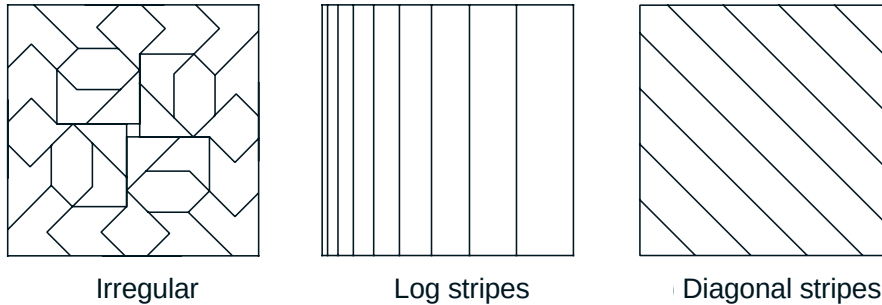
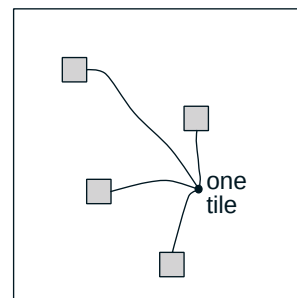


Figure 9.12: Tilings need not be grids. They can be arbitrarily shaped and non-uniform, while still in many cases being computationally efficient to compute.

In practice, it is often desirable to use different shaped tiles in different tilings. For example, one might use some vertical stripe tilings and some horizontal stripe tilings. This would encourage generalization along either dimension. However, with stripe tilings alone it is not possible to learn that a particular conjunction of horizontal and vertical coordinates has a distinctive value (whatever is learned for it will bleed into states with the same horizontal and vertical coordinates). For this one needs the conjunctive rectangular tiles such as originally shown in Figure 9.9. With multiple tilings—some horizontal, some vertical, and some conjunctive—one can get everything: a preference for generalizing along each dimension, yet the ability to learn specific values for conjunctions (see Sutton,

1996 for examples). The choice of tilings determines generalization, and until this choice can be effectively automated, it is important that tile coding enables the choice to be made flexibly and in a way that makes sense to people.

Another useful trick for reducing memory requirements is *hashing*—a consistent pseudo-random collapsing of a large tiling into a much smaller set of tiles. Hashing produces tiles consisting of noncontiguous, disjoint regions randomly spread throughout the state space, but that still form an exhaustive partition. For example, one tile might consist of the four subtiles shown to the right. Through hashing, memory requirements are often reduced by large factors with little loss of performance. This is possible because high resolution is needed in only a small fraction of the state space. Hashing frees us from the curse of dimensionality in the sense that memory requirements need not be exponential in the number of dimensions, but need merely match the real demands of the task. Open-source implementations of tile coding commonly include efficient hashing.



Exercise 9.4 Suppose we believe that one of two state dimensions is more likely to have an effect on the value function than is the other, that generalization should be primarily across this dimension rather than along it. What kind of tilings could be used to take advantage of this prior knowledge? $\square$

9.5.5 Radial Basis Functions

Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features. Rather than each feature being either 0 or 1, it can be anything in the interval $[0, 1]$, reflecting various *degrees* to which the feature is present. A typical RBF feature, x_i , has a Gaussian (bell-shaped) response $x_i(s)$ dependent only on the distance between the state, s , and the feature's prototypical or center state, c_i , and relative to the feature's width, σ_i :

$$x_i(s) \doteq \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right).$$

The norm or distance metric of course can be chosen in whatever way seems most appropriate to the states and task at hand. The figure below shows a one-dimensional example with a Euclidean distance metric.

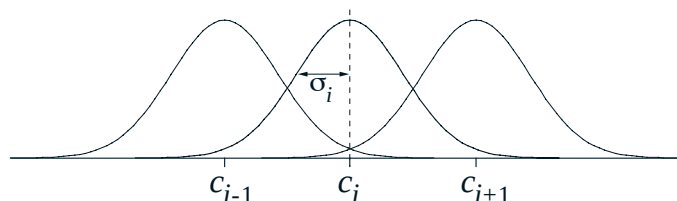


Figure 9.13: One-dimensional radial basis functions.

The primary advantage of RBFs over binary features is that they produce approximate functions that vary smoothly and are differentiable. Although this is appealing, in most cases it has no practical significance. Nevertheless, extensive studies have been made of graded response functions such as RBFs in the context of tile coding (An, 1991; Miller et al., 1991; An et al., 1991; Lane, Handelman and Gelfand, 1992). All of these methods require substantial additional computational complexity (over tile coding) and often reduce performance when there are more than two state dimensions. In high dimensions the edges of tiles are much more important, and it has proven difficult to obtain well controlled graded tile activations near the edges.

An *RBF network* is a linear function approximator using RBFs for its features. Learning is defined by equations (9.7) and (9.8), exactly as in other linear function approximators. In addition, some learning methods for RBF networks change the centers and widths of the features as well, bringing them into the realm of nonlinear function approximators. Nonlinear methods may be able to fit target functions much more precisely. The downside to RBF networks, and to nonlinear RBF networks especially, is greater computational complexity and, often, more manual tuning before learning is robust and efficient.

9.6 Selecting Step-Size Parameters Manually

Most SGD methods require the designer to select an appropriate step-size parameter α . Ideally this selection would be automated, and in some cases it has been, but for most cases it is still common practice to set it manually. To do this, and to better understand the algorithms, it is useful to develop some intuitive sense of the role of the step-size parameter. Can we say in general how it should be set?

Theoretical considerations are unfortunately of little help. The theory of stochastic approximation gives us conditions (2.7) on a slowly decreasing step-size sequence that are sufficient to guarantee convergence, but these tend to result in learning that is too slow. The classical choice $\alpha_t = 1/t$, which produces sample averages in tabular MC methods, is not appropriate for TD methods, for nonstationary problems, or for any method using function approximation. For linear methods, there are recursive least-squares methods that set an optimal *matrix* step size, and these methods can be extended to temporal-difference learning as in the LSTD method described in Section 9.8, but these require $O(d^2)$ step-size parameters, or d times more parameters than we are learning. For this reason we rule them out for use on large problems where function approximation is most needed.

To get some intuitive feel for how to set the step-size parameter manually, it is best to go back momentarily to the tabular case. There we can understand that a step size of $\alpha = 1$ will result in a complete elimination of the sample error after one target (see (2.4) with a step size of one). As discussed on page 201, we usually want to learn slower than this. In the tabular case, a step size of $\alpha = \frac{1}{10}$ would take about 10 experiences to converge approximately to their mean target, and if we wanted to learn in 100 experiences we would use $\alpha = \frac{1}{100}$. In general, if $\alpha = \frac{1}{\tau}$, then the tabular estimate for a state will approach the mean of its targets, with the most recent targets having the greatest effect, after about τ experiences with the state.

With general function approximation there is not such a clear notion of *number* of experiences with a state, as each state may be similar to and dissimilar from all the others to various degrees. However, there is a similar rule that gives similar behavior in the case of linear function approximation. Suppose you wanted to learn in about τ experiences with substantially the same feature vector. A good rule of thumb for setting the step-size parameter of linear SGD methods is then

$$\alpha \doteq (\tau \mathbb{E}[\mathbf{x}^\top \mathbf{x}])^{-1}, \quad (9.19)$$

where $\mathbf{x}$ is a random feature vector chosen from the same distribution as input vectors will be in the SGD. This method works best if the feature vectors do not vary greatly in length; ideally $\mathbf{x}^\top \mathbf{x}$ is a constant.

Exercise 9.5 Suppose you are using tile coding to transform a seven-dimensional continuous state space into binary feature vectors to estimate a state value function $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$. You believe that the dimensions do not interact strongly, so you decide to use eight tilings of each dimension separately (stripe tilings), for $7 \times 8 = 56$ tilings. In addition, in case there are some pairwise interactions between the dimensions, you also take all $\binom{7}{2} = 21$ pairs of dimensions and tile each pair conjunctively with rectangular tiles. You make two tilings for each pair of dimensions, making a grand total of $21 \times 2 + 56 = 98$ tilings. Given these feature vectors, you suspect that you still have to average out some noise, so you decide that you want learning to be gradual, taking about 10 presentations with the same feature vector before learning nears its asymptote. What step-size parameter α should you use? Why? $\square$

9.7 Nonlinear Function Approximation: Artificial Neural Networks

Artificial neural networks (ANNs) are widely used for nonlinear function approximation. An ANN is a network of interconnected units that have some of the properties of neurons, the main components of nervous systems. ANNs have a long history, with the latest advances in training deeply-layered ANNs (deep learning) being responsible for some of the most impressive abilities of machine learning systems, including reinforcement learning systems. In Chapter 16 we describe several impressive examples of reinforcement learning systems that use ANN function approximation.

Figure 9.14 shows a generic feedforward ANN, meaning that there are no loops in the network, that is, there are no paths within the network by which a unit's output can influence its input. The network in the figure has an output layer consisting of two output units, an input layer with four input units, and two "hidden layers": layers that are neither input nor output layers. A real-valued weight is associated with each link. A weight roughly corresponds to the efficacy of a synaptic connection in a real neural network (see Section 15.1). If an ANN has at least one loop in its connections, it is a recurrent rather than a feedforward ANN. Although both feedforward and recurrent ANNs have been used in reinforcement learning, here we look only at the simpler feedforward case.

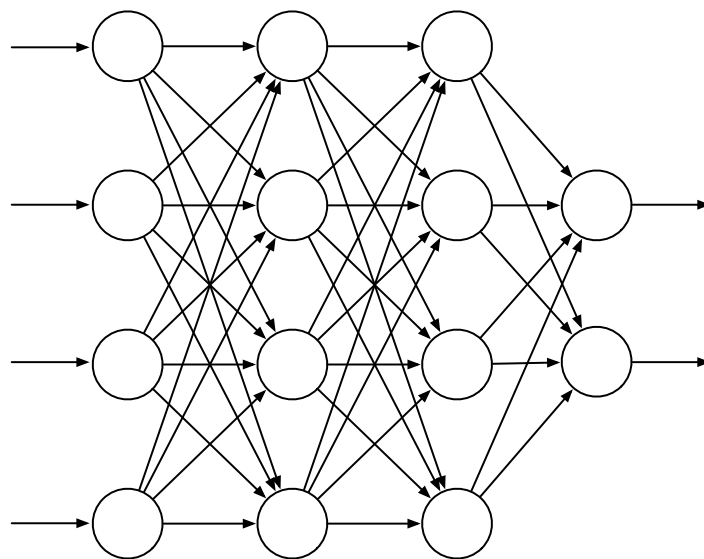


Figure 9.14: A generic feedforward ANN with four input units, two output units, and two hidden layers.

The units (the circles in Figure 9.14) are typically semi-linear units, meaning that they compute a weighted sum of their input signals and then apply to the result a nonlinear function, called the *activation function*, to produce the unit’s output, or activation. Different activation functions are used, but they are typically S-shaped, or sigmoid, functions such as the logistic function $f(x) = 1/(1 + e^{-x})$, though sometimes the rectifier nonlinearity $f(x) = \max(0, x)$ is used. A step function like $f(x) = 1$ if $x \geq \theta$, and 0 otherwise, results in a binary unit with threshold θ . The units in a network’s input layer are somewhat different in having their activations set to externally-supplied values that are the inputs to the function the network is approximating.

The activation of each output unit of a feedforward ANN is a nonlinear function of the activation patterns over the network’s input units. The functions are parameterized by the network’s connection weights. An ANN with no hidden layers can represent only a very small fraction of the possible input-output functions. However an ANN with a single hidden layer containing a large enough finite number of sigmoid units can approximate any continuous function on a compact region of the network’s input space to any degree of accuracy (Cybenko, 1989). This is also true for other nonlinear activation functions that satisfy mild conditions, but nonlinearity is essential: if all the units in a multi-layer feedforward ANN have linear activation functions, the entire network is equivalent to a network with no hidden layers (because linear functions of linear functions are themselves linear).

Despite this “universal approximation” property of one-hidden-layer ANNs, both experience and theory show that approximating the complex functions needed for many artificial intelligence tasks is made easier—indeed may require—abstractions that are hierarchical compositions of many layers of lower-level abstractions, that is, abstractions produced by deep architectures such as ANNs with many hidden layers. (See Bengio, 2009, for a thorough review.) The successive layers of a deep ANN compute increasingly

abstract representations of the network’s “raw” input, with each unit providing a feature contributing to a hierarchical representation of the overall input-output function of the network.

Training the hidden layers of an ANN is therefore a way to automatically create features appropriate for a given problem so that hierarchical representations can be produced without relying exclusively on hand-crafted features. This has been an enduring challenge for artificial intelligence and explains why learning algorithms for ANNs with hidden layers have received so much attention over the years. ANNs typically learn by a stochastic gradient method (Section 9.3). Each weight is adjusted in a direction aimed at improving the network’s overall performance as measured by an objective function to be either minimized or maximized. In the most common supervised learning case, the objective function is the expected error, or loss, over a set of labeled training examples. In reinforcement learning, ANNs can use TD errors to learn value functions, or they can aim to maximize expected reward as in a gradient bandit (Section 2.8) or a policy-gradient algorithm (Chapter 13). In all of these cases it is necessary to estimate how a change in each connection weight would influence the network’s overall performance, in other words, to estimate the partial derivative of an objective function with respect to each weight, given the current values of all the network’s weights. The gradient is the vector of these partial derivatives.

The most successful way to do this for ANNs with hidden layers (provided the units have differentiable activation functions) is the backpropagation algorithm, which consists of alternating forward and backward passes through the network. Each forward pass computes the activation of each unit given the current activations of the network’s input units. After each forward pass, a backward pass efficiently computes a partial derivative for each weight. (As in other stochastic gradient learning algorithms, the vector of these partial derivatives is an estimate of the true gradient.) In Section 15.10 we discuss methods for training ANNs with hidden layers that use reinforcement learning principles instead of backpropagation. These methods are less efficient than the backpropagation algorithm, but they may be closer to how real neural networks learn.

The backpropagation algorithm can produce good results for shallow networks having 1 or 2 hidden layers, but it may not work well for deeper ANNs. In fact, training a network with $k + 1$ hidden layers can actually result in poorer performance than training a network with k hidden layers, even though the deeper network can represent all the functions that the shallower network can (Bengio, 2009). Explaining results like these is not easy, but several factors are important. First, the large number of weights in a typical deep ANN makes it difficult to avoid the problem of overfitting, that is, the problem of failing to generalize correctly to cases on which the network has not been trained. Second, backpropagation does not work well for deep ANNs because the partial derivatives computed by its backward passes either decay rapidly toward the input side of the network, making learning by deep layers extremely slow, or the partial derivatives grow rapidly toward the input side of the network, making learning unstable. Methods for dealing with these problems are largely responsible for many impressive recent results achieved by systems that use deep ANNs.

Overfitting is a problem for any function approximation method that adjusts functions

with many degrees of freedom on the basis of limited training data. It is less of a problem for online reinforcement learning that does not rely on limited training sets, but generalizing effectively is still an important issue. Overfitting is a problem for ANNs in general, but especially so for deep ANNs because they tend to have very large numbers of weights. Many methods have been developed for reducing overfitting. These include stopping training when performance begins to decrease on validation data different from the training data (cross validation), modifying the objective function to discourage complexity of the approximation (regularization), and introducing dependencies among the weights to reduce the number of degrees of freedom (e.g., weight sharing).

A particularly effective method for reducing overfitting by deep ANNs is the dropout method introduced by Srivastava, Hinton, Krizhevsky, Sutskever, and Salakhutdinov (2014). During training, units are randomly removed from the network (dropped out) along with their connections. This can be thought of as training a large number of “thinned” networks. Combining the results of these thinned networks at test time is a way to improve generalization performance. The dropout method efficiently approximates this combination by multiplying each outgoing weight of a unit by the probability that that unit was retained during training. Srivastava et al. found that this method significantly improves generalization performance. It encourages individual hidden units to learn features that work well with random collections of other features. This increases the versatility of the features formed by the hidden units so that the network does not overly specialize to rarely-occurring cases.

Hinton, Osindero, and Teh (2006) took a major step toward solving the problem of training the deep layers of a deep ANN in their work with deep belief networks, layered networks closely related to the deep ANNs discussed here. In their method, the deepest layers are trained one at a time using an unsupervised learning algorithm. Without relying on the overall objective function, unsupervised learning can extract features that capture statistical regularities of the input stream. The deepest layer is trained first, then with input provided by this trained layer, the next deepest layer is trained, and so on, until the weights in all, or many, of the network’s layers are set to values that now act as initial values for supervised learning. The network is then fine-tuned by backpropagation with respect to the overall objective function. Studies show that this approach generally works much better than backpropagation with weights initialized with random values. The better performance of networks trained with weights initialized this way could be due to many factors, but one idea is that this method places the network in a region of weight space from which a gradient-based algorithm can make good progress.

Batch normalization (Ioffe and Szegedy, 2015) is another technique that makes it easier to train deep ANNs. It has long been known that ANN learning is easier if the network input is normalized, for example, by adjusting each input variable to have zero mean and unit variance. Batch normalization for training deep ANNs normalizes the output of deep layers before they feed into the following layer. Ioffe and Szegedy (2015) used statistics from subsets, or “mini-batches,” of training examples to normalize these between-layer signals to improve the learning rate of deep ANNs.

Another technique useful for training deep ANNs is *deep residual learning* (He, Zhang, Ren, and Sun, 2016). Sometimes it is easier to learn how a function differs from the

identity function than to learn the function itself. Then adding this difference, or residual function, to the input produces the desired function. In deep ANNs, a block of layers can be made to learn a residual function simply by adding shortcut, or skip, connections around the block. These connections add the input to the block to its output, and no additional weights are needed. He et al. (2016) evaluated this method using deep convolutional networks with skip connections around every pair of adjacent layers, finding substantial improvement over networks without the skip connections on benchmark image classification tasks. Both batch normalization and deep residual learning were used in the reinforcement learning application to the game of Go that we describe in Chapter 16.

A type of deep ANN that has proven to be very successful in applications, including impressive reinforcement learning applications (Chapter 16), is the *deep convolutional network*. This type of network is specialized for processing high-dimensional data arranged in spatial arrays, such as images. It was inspired by how early visual processing works in the brain (LeCun, Bottou, Bengio and Haffner, 1998). Because of its special architecture, a deep convolutional network can be trained by backpropagation without resorting to methods like those described above to train the deep layers.

Figure 9.15 illustrates the architecture of a deep convolutional network. This instance, from LeCun et al. (1998), was designed to recognize hand-written characters. It consists of alternating convolutional and subsampling layers, followed by several fully connected final layers. Each convolutional layer produces a number of feature maps. A feature map is a pattern of activity over an array of units, where each unit performs the same operation on data in its receptive field, which is the part of the data it “sees” from the preceding layer (or from the external input in the case of the first convolutional layer). The units of a feature map are identical to one another except that their receptive fields, which are all the same size and shape, are shifted to different locations on the arrays of incoming data. Units in the same feature map share the same weights. This means that a feature map detects the same feature no matter where it is located in the input array. In the network in Figure 9.15, for example, the first convolutional layer produces 6 feature maps, each consisting of 28×28 units. Each unit in each feature map has a 5×5 receptive field, and these receptive fields overlap (in this case by four columns and four

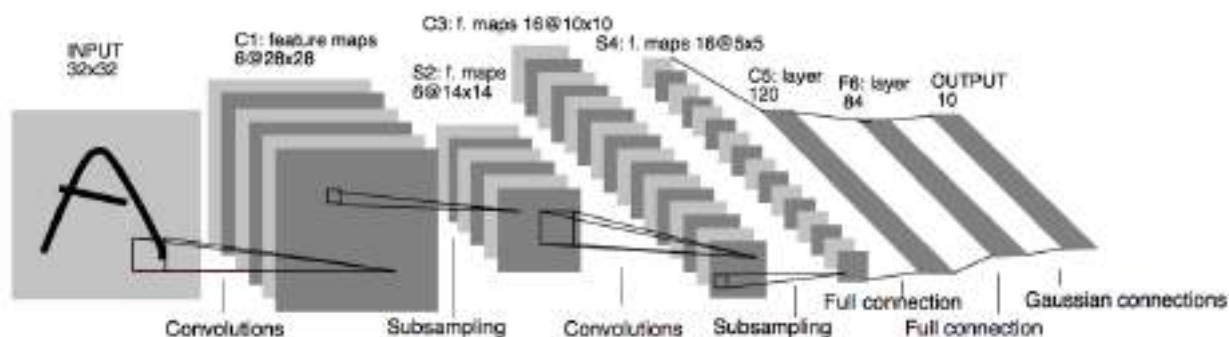


Figure 9.15: Deep Convolutional Network. Republished with permission of Proceedings of the IEEE, from Gradient-based learning applied to document recognition, LeCun, Bottou, Bengio, and Haffner, volume 86, 1998; permission conveyed through Copyright Clearance Center, Inc.

rows). Consequently, each of the 6 feature maps is specified by just 25 adjustable weights.

The subsampling layers of a deep convolutional network reduce the spatial resolution of the feature maps. Each feature map in a subsampling layer consists of units that average over a receptive field of units in the feature maps of the preceding convolutional layer. For example, each unit in each of the 6 feature maps in the first subsampling layer of the network of Figure 9.15 averages over a 2×2 non-overlapping receptive field over one of the feature maps produced by the first convolutional layer, resulting in six 14×14 feature maps. Subsampling layers reduce the network's sensitivity to the spatial locations of the features detected, that is, they help make the network's responses spatially invariant. This is useful because a feature detected at one place in an image is likely to be useful at other places as well.

Advances in the design and training of ANNs—of which we have only mentioned a few—all contribute to reinforcement learning. Although current reinforcement learning theory is mostly limited to methods using tabular or linear function approximation methods, the impressive performances of notable reinforcement learning applications owe much of their success to nonlinear function approximation by multi-layer ANNs. We discuss several of these applications in Chapter 16.

9.8 Least-Squares TD

All the methods we have discussed so far in this chapter have required computation per time step proportional to the number of parameters. With more computation, however, one can do better. In this section we present a method for linear function approximation that is arguably the best that can be done for this case.

As we established in Section 9.4 TD(0) with linear function approximation converges asymptotically (for appropriately decreasing step sizes) to the TD fixed point:

$$\mathbf{w}_{\text{TD}} = \mathbf{A}^{-1}\mathbf{b},$$

where

$$\mathbf{A} \doteq \mathbb{E}[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^\top] \quad \text{and} \quad \mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t].$$

Why, one might ask, must we compute this solution iteratively? This is wasteful of data! Could one not do better by computing estimates of $\mathbf{A}$ and $\mathbf{b}$, and then directly computing the TD fixed point? The *Least-Squares TD* algorithm, commonly known as *LSTD*, does exactly this. It forms the natural estimates

$$\hat{\mathbf{A}}_t \doteq \sum_{k=0}^{t-1} \mathbf{x}_k(\mathbf{x}_k - \gamma\mathbf{x}_{k+1})^\top + \varepsilon\mathbf{I} \quad \text{and} \quad \hat{\mathbf{b}}_t \doteq \sum_{k=0}^{t-1} R_{k+1}\mathbf{x}_k, \quad (9.20)$$

where $\mathbf{I}$ is the identity matrix, and $\varepsilon\mathbf{I}$, for some small $\varepsilon > 0$, ensures that $\hat{\mathbf{A}}_t$ is always invertible. It might seem that these estimates should both be divided by t , and indeed they should; as defined here, these are really estimates of t times $\mathbf{A}$ and t times $\mathbf{b}$.

However, the extra t factors cancel out when LSTD uses these estimates to estimate the TD fixed point as

$$\mathbf{w}_t \doteq \hat{\mathbf{A}}_t^{-1} \hat{\mathbf{b}}_t. \quad (9.21)$$

This algorithm is the most data efficient form of linear TD(0), but it is also more expensive computationally. Recall that semi-gradient TD(0) requires memory and per-step computation that is only $O(d)$.

How complex is LSTD? As it is written above the complexity seems to increase with t , but the two approximations in (9.20) could be implemented incrementally using the techniques we have covered earlier (e.g., in Chapter 2) so that they can be done in constant time per step. Even so, the update for $\hat{\mathbf{A}}_t$ would involve an outer product (a column vector times a row vector) and thus would be a matrix update; its computational complexity would be $O(d^2)$, and of course the memory required to hold the $\hat{\mathbf{A}}_t$ matrix would be $O(d^2)$.

A potentially greater problem is that our final computation (9.21) uses the inverse of $\hat{\mathbf{A}}_t$, and the computational complexity of a general inverse computation is $O(d^3)$. Fortunately, an inverse of a matrix of our special form—a sum of outer products—can also be updated incrementally with only $O(d^2)$ computations, as

$$\begin{aligned} \hat{\mathbf{A}}_t^{-1} &= \left(\hat{\mathbf{A}}_{t-1} + \mathbf{x}_t(\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right)^{-1} && \text{(from (9.20))} \\ &= \hat{\mathbf{A}}_{t-1}^{-1} - \frac{\hat{\mathbf{A}}_{t-1}^{-1} \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \hat{\mathbf{A}}_{t-1}^{-1}}{1 + (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \hat{\mathbf{A}}_{t-1}^{-1} \mathbf{x}_t}, && (9.22) \end{aligned}$$

for $t > 0$, with $\hat{\mathbf{A}}_0 \doteq \varepsilon \mathbf{I}$. Although the identity (9.22), known as *the Sherman-Morrison formula*, is superficially complicated, it involves only vector-matrix and vector-vector multiplications and thus is only $O(d^2)$. Thus we can store the inverse matrix $\hat{\mathbf{A}}_t^{-1}$, maintain it with (9.22), and then use it in (9.21), all with only $O(d^2)$ memory and per-step computation. The complete algorithm is given in the box on the next page.

Of course, $O(d^2)$ is still significantly more expensive than the $O(d)$ of semi-gradient TD. Whether the greater data efficiency of LSTD is worth this computational expense depends on how large d is, how important it is to learn quickly, and the expense of other parts of the system. The fact that LSTD requires no step-size parameter is sometimes also touted, but the advantage of this is probably overstated. LSTD does not require a step size, but it does require ε ; if ε is chosen too small the sequence of inverses can vary wildly, and if ε is chosen too large then learning is slowed. In addition, LSTD's lack of a step-size parameter means that it never forgets. This is sometimes desirable, but it is problematic if the target policy π changes as it does in reinforcement learning and GPI. In control applications, LSTD typically has to be combined with some other mechanism to induce forgetting, mooted any initial advantage of not requiring a step-size parameter.

LSTD for estimating $\hat{v} = \mathbf{w}^\top \mathbf{x}(\cdot) \approx v_\pi$ ($O(d^2)$ version)Input: feature representation $\mathbf{x} : \mathcal{S}^+ \rightarrow \mathbb{R}^d$ such that $\mathbf{x}(\text{terminal}) = \mathbf{0}$ Algorithm parameter: small $\varepsilon > 0$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \varepsilon^{-1} \mathbf{I}$$

A $d \times d$ matrix

$$\widehat{\mathbf{b}} \leftarrow \mathbf{0}$$

A d -dimensional vector

Loop for each episode:

 Initialize S ; $\mathbf{x} \leftarrow \mathbf{x}(S)$

Loop for each step of episode:

 Choose and take action $A \sim \pi(\cdot|S)$, observe R, S' ; $\mathbf{x}' \leftarrow \mathbf{x}(S')$

$$\mathbf{v} \leftarrow \widehat{\mathbf{A}}^{-1} (\mathbf{x} - \gamma \mathbf{x}')$$

$$\widehat{\mathbf{A}}^{-1} \leftarrow \widehat{\mathbf{A}}^{-1} - (\widehat{\mathbf{A}}^{-1} \mathbf{x}) \mathbf{v}^\top / (1 + \mathbf{v}^\top \mathbf{x})$$

$$\widehat{\mathbf{b}} \leftarrow \widehat{\mathbf{b}} + R \mathbf{x}$$

$$\mathbf{w} \leftarrow \widehat{\mathbf{A}}^{-1} \widehat{\mathbf{b}}$$

$$S \leftarrow S'; \mathbf{x} \leftarrow \mathbf{x}'$$

 until S' is terminal

9.9 Memory-based Function Approximation

So far we have discussed the *parametric* approach to approximating value functions. In this approach, a learning algorithm adjusts the parameters of a functional form intended to approximate the value function over a problem's entire state space. Each update, $s \mapsto g$, is a training example used by the learning algorithm to change the parameters with the aim of reducing the approximation error. After the update, the training example can be discarded (although it might be saved to be used again). When an approximate value of a state (which we will call the *query state*) is needed, the function is simply evaluated at that state using the latest parameters produced by the learning algorithm.

Memory-based function approximation methods are very different. They simply save training examples in memory as they arrive (or at least save a subset of the examples) without updating any parameters. Then, whenever a query state's value estimate is needed, a set of examples is retrieved from memory and used to compute a value estimate for the query state. This approach is sometimes called *lazy learning* because processing training examples is postponed until the system is queried to provide an output.

Memory-based function approximation methods are prime examples of *nonparametric* methods. Unlike parametric methods, the approximating function's form is not limited to a fixed parameterized class of functions, such as linear functions or polynomials, but is instead determined by the training examples themselves, together with some means for combining them to output estimated values for query states. As more training examples accumulate in memory, one expects nonparametric methods to produce increasingly accurate approximations of any target function.

There are many different memory-based methods depending on how the stored training examples are selected and how they are used to respond to a query. Here, we focus on *local-learning* methods that approximate a value function only locally in the neighborhood of the current query state. These methods retrieve a set of training examples from memory whose states are judged to be the most relevant to the query state, where relevance usually depends on the distance between states: the closer a training example's state is to the query state, the more relevant it is considered to be, where distance can be defined in many different ways. After the query state is given a value, the local approximation is discarded.

The simplest example of the memory-based approach is the *nearest neighbor* method, which simply finds the example in memory whose state is closest to the query state and returns that example's value as the approximate value of the query state. In other words, if the query state is s , and $s' \mapsto g$ is the example in memory in which s' is the closest state to s , then g is returned as the approximate value of s . Slightly more complicated are *weighted average* methods that retrieve a set of nearest neighbor examples and return a weighted average of their target values, where the weights generally decrease with increasing distance between their states and the query state. *Locally weighted regression* is similar, but it fits a surface to the values of a set of nearest states by means of a parametric approximation method that minimizes a weighted error measure like (9.1), where the weights depend on distances from the query state. The value returned is the evaluation of the locally-fitted surface at the query state, after which the local approximation surface is discarded.

Being nonparametric, memory-based methods have the advantage over parametric methods of not limiting approximations to pre-specified functional forms. This allows accuracy to improve as more data accumulates. Memory-based *local* approximation methods have other properties that make them well suited for reinforcement learning. Because trajectory sampling is of such importance in reinforcement learning, as discussed in Section 8.6, memory-based local methods can focus function approximation on local neighborhoods of states (or state-action pairs) visited in real or simulated trajectories. There may be no need for global approximation because many areas of the state space will never (or almost never) be reached. In addition, memory-based methods allow an agent's experience to have a relatively immediate affect on value estimates in the neighborhood of the current state, in contrast with a parametric method's need to incrementally adjust parameters of a global approximation.

Avoiding global approximation is also a way to address the curse of dimensionality. For example, for a state space with k dimensions, a tabular method storing a global approximation requires memory exponential in k . On the other hand, in storing examples for a memory-based method, each example requires memory proportional to k , and the memory required to store, say, n examples is linear in n . Nothing is exponential in k or n . Of course, the critical remaining issue is whether a memory-based method can answer queries quickly enough to be useful to an agent. A related concern is how speed degrades as the size of the memory grows. Finding nearest neighbors in a large database can take too long to be practical in many applications.

Proponents of memory-based methods have developed ways to accelerate the nearest neighbor search. Using parallel computers or special purpose hardware is one approach; another is the use of special multi-dimensional data structures to store the training data. One data structure studied for this application is the k -d tree (short for k -dimensional tree), which recursively splits a k -dimensional space into regions arranged as nodes of a binary tree. Depending on the amount of data and how it is distributed over the state space, nearest-neighbor search using k -d trees can quickly eliminate large regions of the space in the search for neighbors, making the searches feasible in some problems where naive searches would take too long.

Locally weighted regression additionally requires fast ways to do the local regression computations which have to be repeated to answer each query. Researchers have developed many ways to address these problems, including methods for forgetting entries in order to keep the size of the database within bounds. The Bibliographic and Historical Comments section at the end of this chapter points to some of the relevant literature, including a selection of papers describing applications of memory-based learning to reinforcement learning.

9.10 Kernel-based Function Approximation

Memory-based methods such as the weighted average and locally weighted regression methods described above depend on assigning weights to examples $s' \mapsto g$ in the database depending on the distance between s' and a query states s . The function that assigns these weights is called a *kernel function*, or simply a *kernel*. In the weighted average and locally weighted regressions methods, for example, a kernel function $k : \mathbb{R} \rightarrow \mathbb{R}$ assigns weights to distances between states. More generally, weights do not have to depend on distances; they can depend on some other measure of similarity between states. In this case, $k : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}$, so that $k(s, s')$ is the weight given to data about s' in its influence on answering queries about s .

Viewed slightly differently, $k(s, s')$ is a measure of the strength of generalization from s' to s . Kernel functions numerically express how *relevant* knowledge about any state is to any other state. As an example, the strengths of generalization for tile coding shown in Figure 9.11 correspond to different kernel functions resulting from uniform and asymmetrical tile offsets. Although tile coding does not explicitly use a kernel function in its operation, it generalizes according to one. In fact, as we discuss more below, the strength of generalization resulting from linear parametric function approximation can always be described by a kernel function.

Kernel regression is the memory-based method that computes a kernel weighted average of the targets of *all* examples stored in memory, assigning the result to the query state. If $\mathcal{D}$ is the set of stored examples, and $g(s')$ denotes the target for state s' in a stored example, then kernel regression approximates the target function, in this case a value function depending on $\mathcal{D}$, as

$$\hat{v}(s, \mathcal{D}) = \sum_{s' \in \mathcal{D}} k(s, s') g(s'). \quad (9.23)$$

The weighted average method described above is a special case in which $k(s, s')$ is non-zero only when s and s' are close to one another so that the sum need not be computed over all of $\mathcal{D}$.

A common kernel is the Gaussian radial basis function (RBF) used in RBF function approximation as described in Section 9.5.5. In the method described there, RBFs are features whose centers and widths are either fixed from the start, with centers presumably concentrated in areas where many examples are expected to fall, or are adjusted in some way during learning. Barring methods that adjust centers and widths, this is a linear parametric method whose parameters are the weights of each RBF, which are typically learned by stochastic gradient, or semi-gradient, descent. The form of the approximation is a linear combination of the pre-determined RBFs. Kernel regression with an RBF kernel differs from this in two ways. First, it is memory-based: the RBFs are centered on the states of the stored examples. Second, it is nonparametric: there are no parameters to learn; the response to a query is given by (9.23).

Of course, many issues have to be addressed for practical implementation of kernel regression, issues that are beyond the scope of our brief discussion. However, it turns out that any linear parametric regression method like those we described in Section 9.4, with states represented by feature vectors $\mathbf{x}(s) = (x_1(s), x_2(s), \dots, x_d(s))^\top$, can be recast as kernel regression where $k(s, s')$ is the inner product of the feature vector representations of s and s' ; that is

$$k(s, s') = \mathbf{x}(s)^\top \mathbf{x}(s'). \quad (9.24)$$

Kernel regression with this kernel function produces the same approximation that a linear parametric method would if it used these feature vectors and learned with the same training data.

We skip the mathematical justification for this, which can be found in any modern machine learning text, such as Bishop (2006), and simply point out an important implication. Instead of constructing features for linear parametric function approximators, one can instead construct kernel functions directly without referring at all to feature vectors. Not all kernel functions can be expressed as inner products of feature vectors as in (9.24), but a kernel function that can be expressed like this can offer significant advantages over the equivalent parametric method. For many sets of feature vectors, (9.24) has a compact functional form that can be evaluated without any computation taking place in the d -dimensional feature space. In these cases, kernel regression is much less complex than directly using a linear parametric method with states represented by these feature vectors. This is the so-called “kernel trick” that allows effectively working in the high-dimension of an expansive feature space while actually working only with the set of stored training examples. The kernel trick is the basis of many machine learning methods, and researchers have shown how it can sometimes benefit reinforcement learning.

9.11 Looking Deeper at On-policy Learning: Interest and Emphasis

The algorithms we have considered so far in this chapter have treated all the states encountered equally, as if they were all equally important. In some cases, however, we are more interested in some states than others. In discounted episodic problems, for example, we may be more interested in accurately valuing early states in the episode than in later states where discounting may have made the rewards much less important to the value of the start state. Or, if an action-value function is being learned, it may be less important to accurately value poor actions whose value is much less than the greedy action. Function approximation resources are always limited, and if they were used in a more targeted way, then performance could be improved.

One reason we have treated all states encountered equally is that then we are updating according to the on-policy distribution, for which stronger theoretical results are available for semi-gradient methods. Recall that the on-policy distribution was defined as the distribution of states encountered in an MDP while following the target policy. Now we will generalize this concept significantly. Rather than having one on-policy distribution for the MDP, we will have many. All of them will have in common that they are a distribution of states encountered in trajectories while following the target policy, but they will vary in how the trajectories are, in a sense, initiated.

We now introduce some new concepts. First we introduce a non-negative scalar measure, a random variable I_t called *interest*, indicating the degree to which we are interested in accurately valuing the state (or state-action pair) at time t . If we don't care at all about the state, then the interest should be zero; if we fully care, it might be one, though it is formally allowed to take any non-negative value. The interest can be set in any causal way; for example, it may depend on the trajectory up to time t or the learned parameters at time t . The distribution μ in the $\overline{\text{VE}}$ (9.1) is then defined as the distribution of states encountered while following the target policy, weighted by the interest. Second, we introduce another non-negative scalar random variable, the *emphasis* M_t . This scalar multiplies the learning update and thus emphasizes or de-emphasizes the learning done at time t . The general n -step learning rule, replacing (9.15), is

$$\mathbf{w}_{t+n} \doteq \mathbf{w}_{t+n-1} + \alpha M_t [G_{t:t+n} - \hat{v}(S_t, \mathbf{w}_{t+n-1})] \nabla \hat{v}(S_t, \mathbf{w}_{t+n-1}), \quad 0 \leq t < T, \quad (9.25)$$

with the n -step return given by (9.16) and the emphasis determined recursively from the interest by:

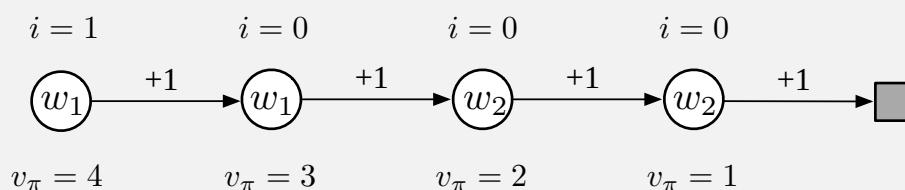
$$M_t = I_t + \gamma^n M_{t-n}, \quad 0 \leq t < T, \quad (9.26)$$

with $M_t \doteq 0$, for all $t < 0$. These equations are taken to include the Monte Carlo case, for which $G_{t:t+n} = G_t$, all the updates are made at end of the episode, $n = T - t$, and $M_t = I_t$.

Example 9.4 illustrates how interest and emphasis can result in more accurate value estimates.

Example 9.4: Interest and Emphasis

To see the potential benefits of using interest and emphasis, consider the four-state Markov reward process shown below:



Episodes start in the leftmost state, then transition one state to the right, with a reward of $+1$, on each step until the terminal state is reached. The true value of the first state is thus 4, of the second state 3, and so on as shown below each state. These are the true values; the estimated values can only approximate these because they are constrained by the parameterization. There are two components to the parameter vector $\mathbf{w} = (w_1, w_2)^\top$, and the parameterization is as written inside each state. The estimated values of the first two states are given by w_1 alone and thus must be the same even though their true values are different. Similarly, the estimated values of the third and fourth states are given by w_2 alone and must be the same even though their true values are different. Suppose that we are interested in accurately valuing only the leftmost state; we assign it an interest of 1 while all the other states are assigned an interest of 0, as indicated above the states.

First consider applying gradient Monte Carlo algorithms to this problem. The algorithms presented earlier in this chapter that do not take into account interest and emphasis (in (9.7) and the box on page 202) will converge (for decreasing step sizes) to the parameter vector $\mathbf{w}_\infty = (3.5, 1.5)$, which gives the first state—the only one we are interested in—a value of 3.5 (i.e., intermediate between the true values of the first and second states). The methods presented in this section that do use interest and emphasis, on the other hand, will learn the value of the first state exactly correctly; w_1 will converge to 4 while w_2 will never be updated because the emphasis is zero in all states save the leftmost.

Now consider applying two-step semi-gradient TD methods. The methods from earlier in this chapter without interest and emphasis (in (9.15) and (9.16) and the box on page 209) will again converge to $\mathbf{w}_\infty = (3.5, 1.5)$, while the methods with interest and emphasis converge to $\mathbf{w}_\infty = (4, 2)$. The latter produces the exactly correct values for the first state and for the third state (which the first state bootstraps from) while never making any updates corresponding to the second or fourth states.

9.12 Summary

Reinforcement learning systems must be capable of *generalization* if they are to be applicable to artificial intelligence or to large engineering applications. To achieve this, any of a broad range of existing methods for *supervised-learning function approximation* can be used simply by treating each update as a training example.

Perhaps the most suitable supervised learning methods are those using *parameterized function approximation*, in which the policy is parameterized by a weight vector $\mathbf{w}$. Although the weight vector has many components, the state space is much larger still, and we must settle for an approximate solution. We defined the *mean squared value error*, $\overline{\text{VE}}(\mathbf{w})$, as a measure of the error in the values $v_{\pi_{\mathbf{w}}}(s)$ for a weight vector $\mathbf{w}$ under the *on-policy distribution*, μ . The $\overline{\text{VE}}$ gives us a clear way to rank different value-function approximations in the on-policy case.

To find a good weight vector, the most popular methods are variations of *stochastic gradient descent* (SGD). In this chapter we have focused on the *on-policy* case with a *fixed policy*, also known as policy evaluation or prediction; a natural learning algorithm for this case is *n-step semi-gradient TD*, which includes gradient Monte Carlo and semi-gradient TD(0) algorithms as the special cases when $n = \infty$ and $n = 1$ respectively. Semi-gradient TD methods are not true gradient methods. In such bootstrapping methods (including DP), the weight vector appears in the update target, yet this is not taken into account in computing the gradient—thus they are *semi-gradient* methods. As such, they cannot rely on classical SGD results.

Nevertheless, good results can be obtained for semi-gradient methods in the special case of *linear* function approximation, in which the value estimates are sums of features times corresponding weights. The linear case is the most well understood theoretically and works well in practice when provided with appropriate features. Choosing the features is one of the most important ways of adding prior domain knowledge to reinforcement learning systems. They can be chosen as polynomials, but this case generalizes poorly in the online learning setting typically considered in reinforcement learning. Better is to choose features according to the Fourier basis, or according to some form of coarse coding with sparse overlapping receptive fields. Tile coding is a form of coarse coding that is particularly computationally efficient and flexible. Radial basis functions are useful for one- or two-dimensional tasks in which a smoothly varying response is important. LSTD is the most data-efficient linear TD prediction method, but requires computation proportional to the square of the number of weights, whereas all the other methods are of complexity linear in the number of weights. Nonlinear methods include artificial neural networks trained by backpropagation and variations of SGD; these methods have become very popular in recent years under the name *deep reinforcement learning*.

Linear semi-gradient *n*-step TD is guaranteed to converge under standard conditions, for all n , to a $\overline{\text{VE}}$ that is within a bound of the optimal error (achieved asymptotically by Monte Carlo methods). This bound is always tighter for higher n and approaches zero as $n \rightarrow \infty$. However, in practice very high n results in very slow learning, and some degree of bootstrapping ($n < \infty$) is usually preferable, just as we saw in comparisons of tabular *n*-step methods in Chapter 7 and in comparisons of tabular TD and Monte Carlo methods in Chapter 6.

Chapter 12

Eligibility Traces

Eligibility traces are one of the basic mechanisms of reinforcement learning. For example, in the popular $TD(\lambda)$ algorithm, the λ refers to the use of an eligibility trace. Almost any temporal-difference (TD) method, such as Q-learning or Sarsa, can be combined with eligibility traces to obtain a more general method that may learn more efficiently.

Eligibility traces unify and generalize TD and Monte Carlo methods. When TD methods are augmented with eligibility traces, they produce a family of methods spanning a spectrum that has Monte Carlo methods at one end ($\lambda=1$) and one-step TD methods at the other ($\lambda=0$). In between are intermediate methods that are often better than either extreme method. Eligibility traces also provide a way of implementing Monte Carlo methods online and on continuing problems without episodes.

Of course, we have already seen one way of unifying TD and Monte Carlo methods: the n -step TD methods of Chapter 7. What eligibility traces offer beyond these is an elegant algorithmic mechanism with significant computational advantages. The mechanism is a short-term memory vector, the *eligibility trace* $\mathbf{z}_t \in \mathbb{R}^d$, that parallels the long-term weight vector $\mathbf{w}_t \in \mathbb{R}^d$. The rough idea is that when a component of $\mathbf{w}_t$ participates in producing an estimated value, then the corresponding component of $\mathbf{z}_t$ is bumped up and then begins to fade away. Learning will then occur in that component of $\mathbf{w}_t$ if a nonzero TD error occurs before the trace falls back to zero. The trace-decay parameter $\lambda \in [0, 1]$ determines the rate at which the trace falls.

The primary computational advantage of eligibility traces over n -step methods is that only a single trace vector is required rather than a store of the last n feature vectors. Learning also occurs continually and uniformly in time rather than being delayed and then catching up at the end of the episode. In addition learning can occur and affect behavior immediately after a state is encountered rather than being delayed n steps.

Eligibility traces illustrate that a learning algorithm can sometimes be implemented in a different way to obtain computational advantages. Many algorithms are most naturally formulated and understood as an update of a state's value based on events that follow that state over multiple future time steps. For example, Monte Carlo methods (Chapter 5) update a state based on all the future rewards, and n -step TD methods (Chapter 7)

update based on the next n rewards and state n steps in the future. Such formulations, based on looking forward from the updated state, are called *forward views*. Forward views are always somewhat complex to implement because the update depends on later things that are not available at the time. However, as we show in this chapter it is often possible to achieve nearly the same updates—and sometimes *exactly* the same updates—with an algorithm that uses the current TD error, looking backward to recently visited states using an eligibility trace. These alternate ways of looking at and implementing learning algorithms are called *backward views*. Backward views, transformations between forward views and backward views, and equivalences between them, date back to the introduction of temporal difference learning but have become much more powerful and sophisticated since 2014. Here we present the basics of the modern view.

As usual, first we fully develop the ideas for state values and prediction, then extend them to action values and control. We develop them first for the on-policy case then extend them to off-policy learning. Our treatment pays special attention to the case of linear function approximation, for which the results with eligibility traces are stronger. All these results apply also to the tabular and state aggregation cases because these are special cases of linear function approximation.

12.1 The λ -return

In Chapter 7 we defined an n -step return as the sum of the first n rewards plus the estimated value of the state reached in n steps, each appropriately discounted (7.1). The general form of that equation, for any parameterized function approximator, is

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_{t+n-1}), \quad 0 \leq t \leq T-n, \quad (12.1)$$

where $\hat{v}(s, \mathbf{w})$ is the approximate value of state s given weight vector $\mathbf{w}$ (Chapter 9), and T is the time of episode termination, if any. We noted in Chapter 7 that each n -step return, for $n \geq 1$, is a valid update target for a tabular learning update, just as it is for an approximate SGD learning update such as (9.7).

Now we note that a valid update can be done not just toward any n -step return, but toward any *average* of n -step returns for different ns . For example, an update can be done toward a target that is half of a two-step return and half of a four-step return: $\frac{1}{2}G_{t:t+2} + \frac{1}{2}G_{t:t+4}$. Any set of n -step returns can be averaged in this way, even an infinite set, as long as the weights on the component returns are positive and sum to 1. The composite return possesses an error reduction property similar to that of individual n -step returns (7.3) and thus can be used to construct updates with guaranteed convergence properties. Averaging produces a substantial new range of algorithms. For example, one could average one-step and infinite-step returns to obtain another way of interrelating TD and Monte Carlo methods. In principle, one could even average experience-based updates with DP updates to get a simple combination of experience-based and model-based methods (cf. Chapter 8).

An update that averages simpler component updates is called a *compound update*. The backup diagram for a compound update consists of the backup diagrams for each of the component updates with a horizontal line above them and the weighting fractions below.

For example, the compound update for the case mentioned at the start of this section, mixing half of a two-step return and half of a four-step return, has the diagram shown to the right. A compound update can only be done when the longest of its component updates is complete. The update at the right, for example, could only be done at time $t+4$ for the estimate formed at time t . In general one would like to limit the length of the longest component update because of the corresponding delay in the updates.

The TD(λ) algorithm can be understood as one particular way of averaging n -step updates. This average contains all the n -step updates, each weighted proportionally to λ^{n-1} (where $\lambda \in [0, 1]$), and is normalized by a factor of $1 - \lambda$ to ensure that the weights sum to 1 (Figure 12.1). The resulting update is toward a return, called the λ -return, defined in its state-based form by

$$G_t^\lambda \doteq (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}. \quad (12.2)$$

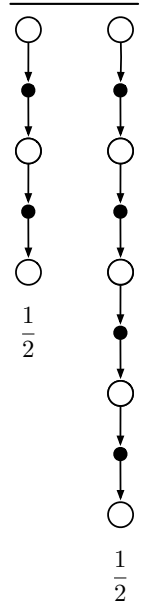


Figure 12.2 further illustrates the weighting on the sequence of n -step returns in the λ -return. The one-step return is given the largest weight, $1 - \lambda$; the two-step return is given the next largest weight, $(1 - \lambda)\lambda$; the three-step return is given the weight $(1 - \lambda)\lambda^2$; and so on. The weight fades by λ with each additional step. After a terminal state has been reached, all subsequent n -step returns are equal to the conventional return, G_t . If

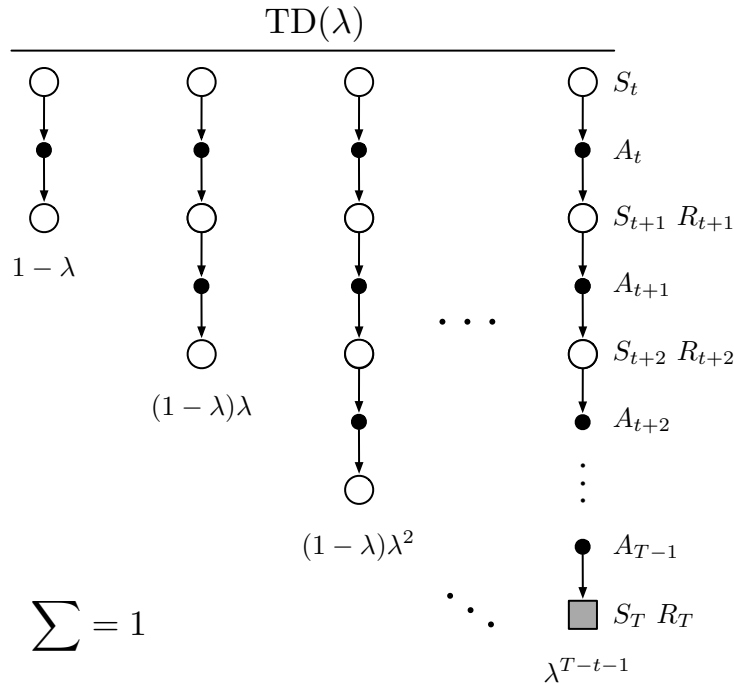


Figure 12.1: The backup diagram for TD(λ). If $\lambda = 0$, then the overall update reduces to its first component, the one-step TD update, whereas if $\lambda = 1$, then the overall update reduces to its last component, the Monte Carlo update.

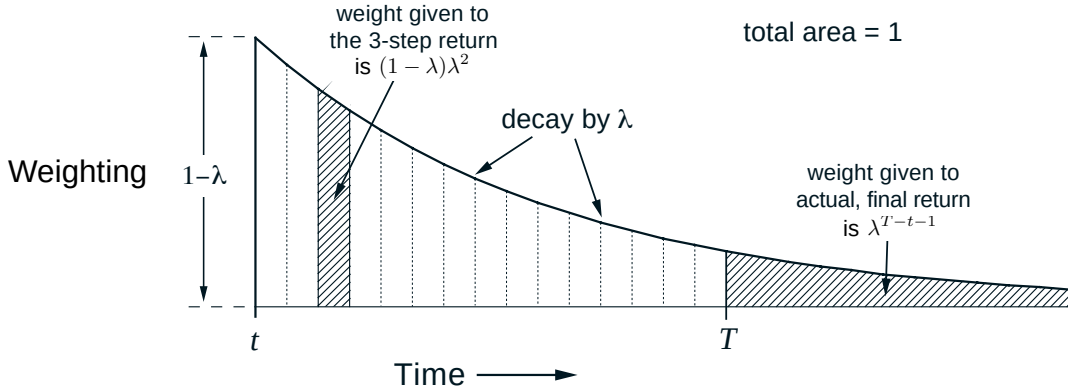


Figure 12.2: Weighting given in the λ -return to each of the n -step returns.

we want, we can separate these post-termination terms from the main sum, yielding

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_T, \quad (12.3)$$

as indicated in the figures. This equation makes it clearer what happens when $\lambda = 1$. In this case the main sum goes to zero, and the remaining term reduces to the conventional return. Thus, for $\lambda = 1$, updating according to the λ -return is a Monte Carlo algorithm. On the other hand, if $\lambda = 0$, then the λ -return reduces to $G_{t:t+1}$, the one-step return. Thus, for $\lambda = 0$, updating according to the λ -return is a one-step TD method.

Exercise 12.1 Just as the return can be written recursively in terms of the first reward and itself one-step later (3.9), so can the λ -return. Derive the analogous recursive relationship from (12.2) and (12.1). $\square$

Exercise 12.2 The parameter λ characterizes how fast the exponential weighting in Figure 12.2 falls off, and thus how far into the future the λ -return algorithm looks in determining its update. But a rate factor such as λ is sometimes an awkward way of characterizing the speed of the decay. For some purposes it is better to specify a time constant, or half-life. What is the equation relating λ and the half-life, τ_λ , the time by which the weighting sequence will have fallen to half of its initial value? $\square$

We are now ready to define our first learning algorithm based on the λ -return: the *offline λ -return algorithm*. As an offline algorithm, it makes no changes to the weight vector during the episode. Then, at the end of the episode, a whole sequence of offline updates are made according to our usual semi-gradient rule, using the λ -return as the target:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[G_t^\lambda - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t), \quad t = 0, \dots, T-1. \quad (12.4)$$

The λ -return gives us an alternative way of moving smoothly between Monte Carlo and one-step TD methods that can be compared with the n -step bootstrapping way developed in Chapter 7. There we assessed effectiveness on a 19-state random walk task (Example 7.1, page 144). Figure 12.3 shows the performance of the offline λ -return algorithm on this task alongside that of the n -step methods (repeated from Figure 7.2). The experiment was just as described earlier except that for the λ -return algorithm we varied λ instead of n . The performance measure used is the estimated root-mean-squared error between the correct and estimated values of each state measured at the end of the episode, averaged over the first 10 episodes and the 19 states. Note that overall performance of the offline λ -return algorithms is comparable to that of the n -step algorithms. In both cases we get best performance with an intermediate value of the bootstrapping parameter, n for n -step methods and λ for the offline λ -return algorithm.

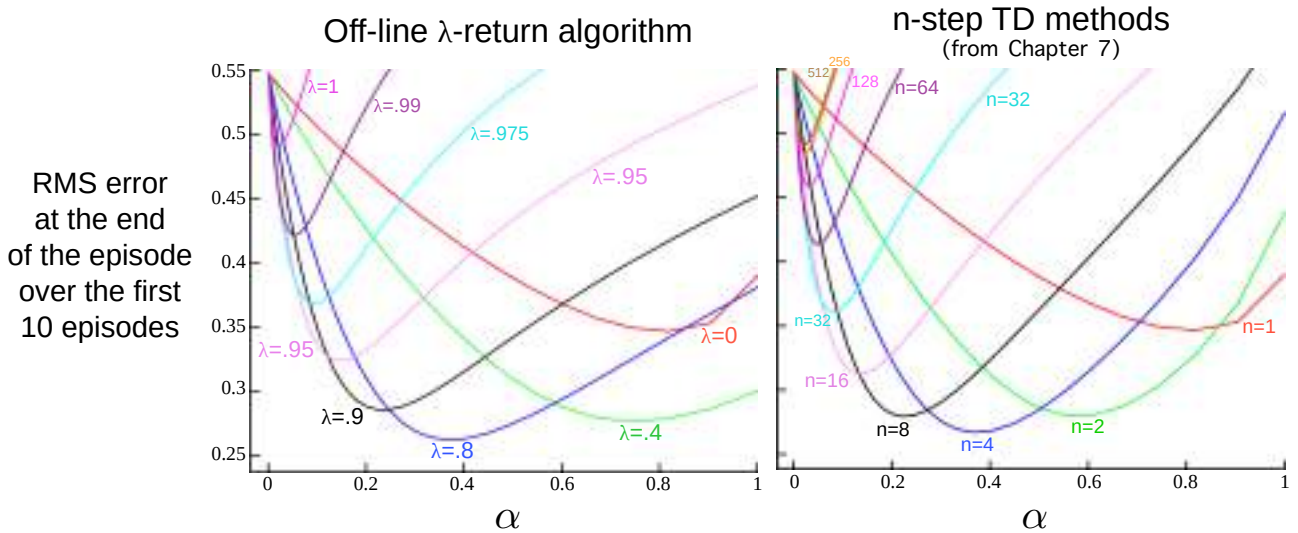


Figure 12.3: 19-state Random walk results (Example 7.1): Performance of the offline λ -return algorithm alongside that of the n -step TD methods. In both case, intermediate values of the bootstrapping parameter (λ or n) performed best. The results with the offline λ -return algorithm are slightly better at the best values of α and λ , and at high α .

The approach that we have been taking so far is what we call the theoretical, or *forward*, view of a learning algorithm. For each state visited, we look forward in time to all the future rewards and decide how best to combine them. We might imagine ourselves riding the stream of states, looking forward from each state to determine its update, as suggested by Figure 12.4. After looking forward from and updating one state, we move on to the next and never have to work with the preceding state again. Future states, on the other hand, are viewed and processed repeatedly, once from each vantage point preceding them.

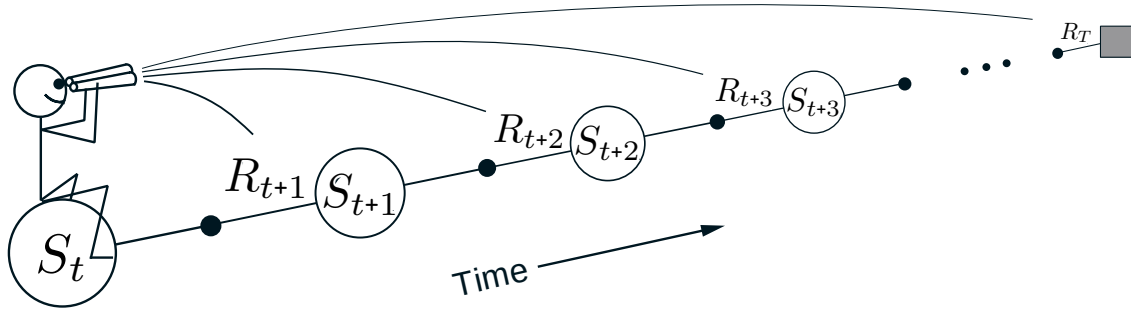


Figure 12.4: The forward view. We decide how to update each state by looking forward to future rewards and states.

12.2 TD(λ)

TD(λ) is one of the oldest and most widely used algorithms in reinforcement learning. It was the first algorithm for which a formal relationship was shown between a more theoretical forward view and a more computationally-congenial backward view using eligibility traces. Here we will show empirically that it approximates the offline λ -return algorithm presented in the previous section.

TD(λ) improves over the offline λ -return algorithm in three ways. First it updates the weight vector on every step of an episode rather than only at the end, and thus its estimates may be better sooner. Second, its computations are equally distributed in time rather than all at the end of the episode. And third, it can be applied to continuing problems rather than just to episodic problems. In this section we present the semi-gradient version of TD(λ) with function approximation.

With function approximation, the eligibility trace is a vector $\mathbf{z}_t \in \mathbb{R}^d$ with the same number of components as the weight vector $\mathbf{w}_t$. Whereas the weight vector is a long-term memory, accumulating over the lifetime of the system, the eligibility trace is a short-term memory, typically lasting less time than the length of an episode. Eligibility traces assist in the learning process; their only consequence is that they affect the weight vector, and then the weight vector determines the estimated value.

In TD(λ), the eligibility trace vector is initialized to zero at the beginning of the episode, is incremented on each time step by the value gradient, and then fades away by $\gamma\lambda$:

$$\begin{aligned} \mathbf{z}_{-1} &\doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \gamma\lambda\mathbf{z}_{t-1} + \nabla\hat{v}(S_t, \mathbf{w}_t), \quad 0 \leq t \leq T, \end{aligned} \tag{12.5}$$

where γ is the discount rate and λ is the parameter introduced in the previous section, which we henceforth call the trace-decay parameter. The eligibility trace keeps track of which components of the weight vector have contributed, positively or negatively, to recent state valuations, where “recent” is defined in terms of $\gamma\lambda$. (Recall that in linear function approximation, $\nabla\hat{v}(S_t, \mathbf{w}_t)$ is just the feature vector, $\mathbf{x}_t$, in which case the eligibility trace vector is just a sum of past, fading, input vectors.) The trace is said to indicate the eligibility of each component of the weight vector for undergoing learning

changes should a reinforcing event occur. The reinforcing events we are concerned with are the moment-by-moment one-step TD errors. The TD error for state-value prediction is

$$\delta_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t). \quad (12.6)$$

In TD(λ), the weight vector is updated on each step proportional to the scalar TD error and the vector eligibility trace:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t. \quad (12.7)$$

Semi-gradient TD(λ) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameters: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$

Initialize value-function weights $\mathbf{w}$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

$\mathbf{z} \leftarrow \mathbf{0}$

(a d -dimensional vector)

 Loop for each step of episode:

 | Choose $A \sim \pi(\cdot | S)$

 | Take action A , observe R, S'

 | $\mathbf{z} \leftarrow \gamma \lambda \mathbf{z} + \nabla \hat{v}(S, \mathbf{w})$

 | $\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

 | $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \mathbf{z}$

 | $S \leftarrow S'$

 until S' is terminal

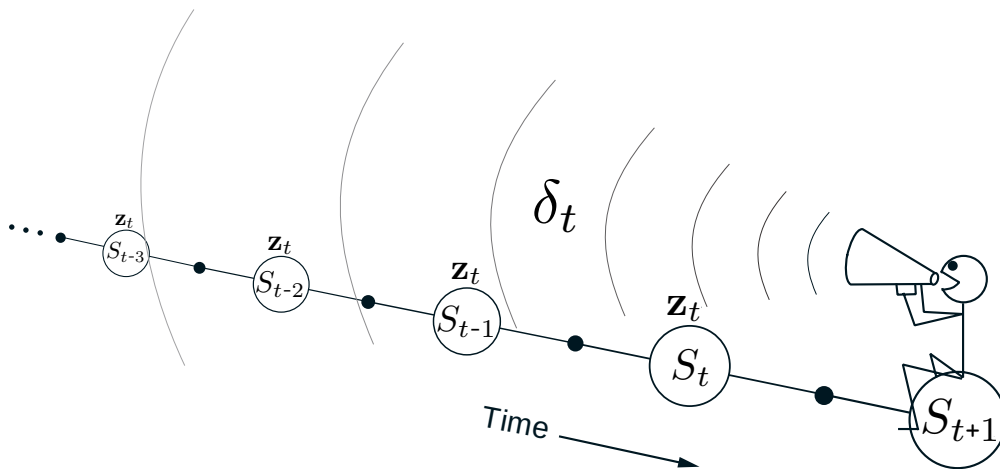


Figure 12.5: The backward or mechanistic view of TD(λ). Each update depends on the current TD error combined with the current eligibility traces of past events.

$TD(\lambda)$ is oriented backward in time. At each moment we look at the current TD error and assign it backward to each prior state according to how much that state contributed to the current eligibility trace at that time. We might imagine ourselves riding along the stream of states, computing TD errors, and shouting them back to the previously visited states, as suggested by Figure 12.5. Where the TD error and traces come together, we get the update given by (12.7), changing the values of those past states for when they occur again in the future.

To better understand the backward view of $TD(\lambda)$, consider what happens at various values of λ . If $\lambda = 0$, then by (12.5) the trace at t is exactly the value gradient corresponding to S_t . Thus the $TD(\lambda)$ update (12.7) reduces to the one-step semi-gradient TD update treated in Chapter 9 (and, in the tabular case, to the simple TD rule (6.2)). This is why that algorithm was called $TD(0)$. In terms of Figure 12.5, $TD(0)$ is the case in which only the one state preceding the current one is changed by the TD error. For larger values of λ , but still $\lambda < 1$, more of the preceding states are changed, but each more temporally distant state is changed less because the corresponding eligibility trace is smaller, as suggested by the figure. We say that the earlier states are given less *credit* for the TD error.

If $\lambda = 1$, then the credit given to earlier states falls only by γ per step. This turns out to be just the right thing to do to achieve Monte Carlo behavior. For example, remember that the TD error, δ_t , includes an undiscounted term of R_{t+1} . In passing this back k steps it needs to be discounted, like any reward in a return, by γ^k , which is just what the falling eligibility trace achieves. If $\lambda = 1$ and $\gamma = 1$, then the eligibility traces do not decay at all with time. In this case the method behaves like a Monte Carlo method for an undiscounted, episodic task. If $\lambda = 1$, the algorithm is also known as $TD(1)$.

$TD(1)$ is a way of implementing Monte Carlo algorithms that is more general than those presented earlier and that significantly increases their range of applicability. Whereas the earlier Monte Carlo methods were limited to episodic tasks, $TD(1)$ can be applied to discounted continuing tasks as well. Moreover, $TD(1)$ can be performed incrementally and online. One disadvantage of Monte Carlo methods is that they learn nothing from an episode until it is over. For example, if a Monte Carlo control method takes an action that produces a very poor reward but does not end the episode, then the agent's tendency to repeat the action will be undiminished during the episode. Online $TD(1)$, on the other hand, learns in an n -step TD way from the incomplete ongoing episode, where the n steps are all the way up to the current step. If something unusually good or bad happens during an episode, control methods based on $TD(1)$ can learn immediately and alter their behavior on that same episode.

It is revealing to revisit the 19-state random walk example (Example 7.1) to see how well $TD(\lambda)$ does in approximating the offline λ -return algorithm. The results for both algorithms are shown in Figure 12.6. For each λ value, if α is selected optimally for it (or smaller), then the two algorithms perform virtually identically. If α is chosen larger than is optimal, however, then the λ -return algorithm is only a little worse whereas $TD(\lambda)$ is much worse and may even be unstable. This is not catastrophic for $TD(\lambda)$ on this problem, as these higher parameter values are not what one would want to use anyway, but for other problems it can be a significant weakness.

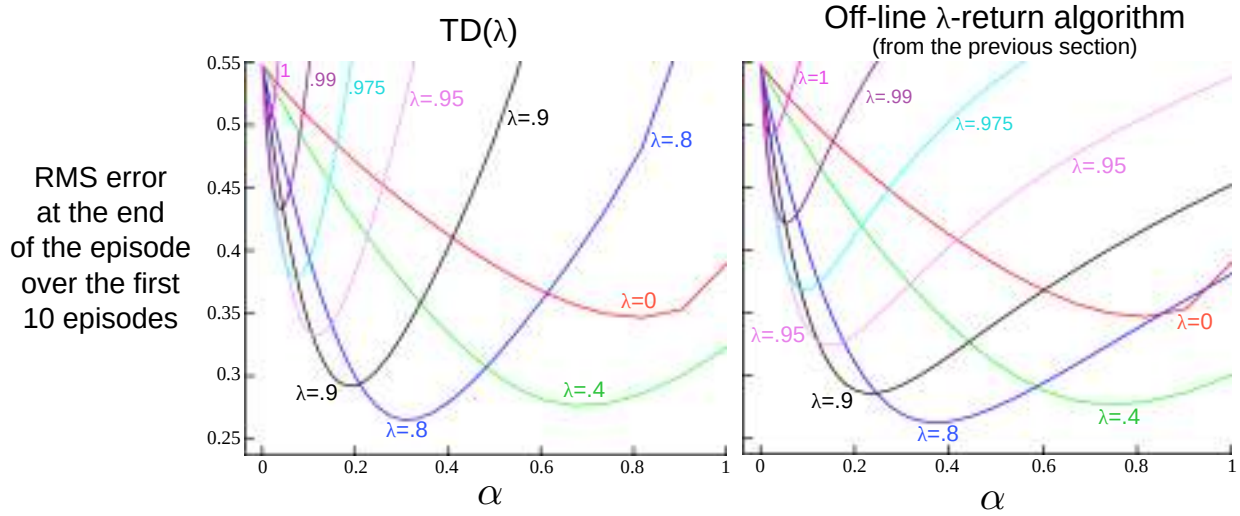


Figure 12.6: 19-state Random walk results (Example 7.1): Performance of $\text{TD}(\lambda)$ alongside that of the offline λ -return algorithm. The two algorithms performed virtually identically at low (less than optimal) α values, but $\text{TD}(\lambda)$ was worse at high α values.

Linear $\text{TD}(\lambda)$ has been proved to converge in the on-policy case if the step-size parameter is reduced over time according to the usual conditions (2.7). Just as discussed in Section 9.4, convergence is not to the minimum-error weight vector, but to a nearby weight vector that depends on λ . The bound on solution quality presented in that section (9.14) can now be generalized to apply for any λ . For the continuing discounted case,

$$\overline{\text{VE}}(\mathbf{w}_\infty) \leq \frac{1 - \gamma\lambda}{1 - \gamma} \min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w}). \quad (12.8)$$

That is, the asymptotic error is no more than $\frac{1 - \gamma\lambda}{1 - \gamma}$ times the smallest possible error. As λ approaches 1, the bound approaches the minimum error (and it is loosest at $\lambda = 0$). In practice, however, $\lambda = 1$ is often the poorest choice, as will be illustrated later in Figure 12.14.

Exercise 12.3 Some insight into how $\text{TD}(\lambda)$ can closely approximate the offline λ -return algorithm can be gained by seeing that the latter's error term (in brackets in (12.4)) can be written as the sum of TD errors (12.6) for a single fixed $\mathbf{w}$. Show this, following the pattern of (6.6), and using the recursive relationship for the λ -return you obtained in Exercise 12.1. $\square$

Exercise 12.4 Use your result from the preceding exercise to show that, if the weight updates over an episode were computed on each step but not actually used to change the weights ($\mathbf{w}$ remained fixed), then the sum of $\text{TD}(\lambda)$'s weight updates would be the same as the sum of the offline λ -return algorithm's updates. $\square$

12.3 *n*-step Truncated λ -return Methods

The offline λ -return algorithm is an important ideal, but it is of limited utility because it uses the λ -return (12.2), which is not known until the end of the episode. In the

continuing case, the λ -return is technically never known, as it depends on n -step returns for arbitrarily large n , and thus on rewards arbitrarily far in the future. However, the dependence becomes weaker for longer-delayed rewards, falling by $\gamma\lambda$ for each step of delay. A natural approximation, then, would be to truncate the sequence after some number of steps. Our existing notion of n -step returns provides a natural way to do this in which the missing rewards are replaced with estimated values.

In general, we define the *truncated λ -return* for time t , given data only up to some later horizon, h , as

$$G_{t:h}^\lambda \doteq (1 - \lambda) \sum_{n=1}^{h-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{h-t-1} G_{t:h}, \quad 0 \leq t < h \leq T. \quad (12.9)$$

If you compare this equation with the λ -return (12.3), it is clear that the horizon h is playing the same role as was previously played by T , the time of termination. Whereas in the λ -return there is a residual weight given to the conventional return G_t , here it is given to the longest available n -step return, $G_{t:h}$ (Figure 12.2).

The truncated λ -return immediately gives rise to a family of n -step λ -return algorithms similar to the n -step methods of Chapter 7. In all of these algorithms, updates are delayed by n steps and only take into account the first n rewards, but now all the k -step returns are included for $1 \leq k \leq n$ (whereas the earlier n -step algorithms used only the n -step return), weighted geometrically as in Figure 12.2. In the state-value case, this family of algorithms is known as truncated TD(λ), or TTD(λ). The compound backup diagram, shown in Figure 12.7, is similar to that for TD(λ) (Figure 12.1) except that the longest component update is at most n steps rather than always going all the way to the

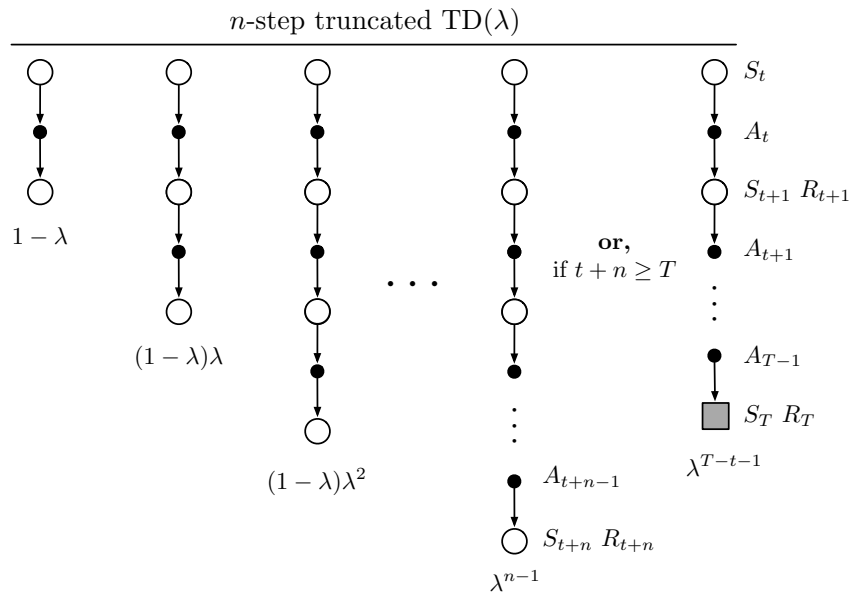


Figure 12.7: The backup diagram for truncated TD(λ).

end of the episode. $\text{TTD}(\lambda)$ is defined by (cf. (9.15)):

$$\mathbf{w}_{t+n} \doteq \mathbf{w}_{t+n-1} + \alpha [G_{t:t+n}^\lambda - \hat{v}(S_t, \mathbf{w}_{t+n-1})] \nabla \hat{v}(S_t, \mathbf{w}_{t+n-1}), \quad 0 \leq t < T.$$

This algorithm can be implemented efficiently so that per-step computation does not scale with n (though of course memory must). Much as in n -step TD methods, no updates are made on the first $n - 1$ time steps of each episode, and $n - 1$ additional updates are made upon termination. Efficient implementation relies on the fact that the k -step λ -return can be written exactly as

$$G_{t:t+k}^\lambda = \hat{v}(S_t, \mathbf{w}_{t-1}) + \sum_{i=t}^{t+k-1} (\gamma\lambda)^{i-t} \delta'_i, \quad (12.10)$$

where

$$\delta'_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_{t-1}).$$

Exercise 12.5 Several times in this book (often in exercises) we have established that returns can be written as sums of TD errors if the value function is held constant. Why is (12.10) another instance of this? Prove (12.10). $\square$

12.4 Redoing Updates: Online λ -return Algorithm

Choosing the truncation parameter n in truncated $\text{TD}(\lambda)$ involves a tradeoff. n should be large so that the method closely approximates the offline λ -return algorithm, but it should also be small so that the updates can be made sooner and can influence behavior sooner. Can we get the best of both? Well, yes, in principle we can, albeit at the cost of computational complexity.

The idea is that, on each time step as you gather a new increment of data, you go back and redo all the updates since the beginning of the current episode. The new updates will be better than the ones you previously made because now they can take into account the time step's new data. That is, the updates are always towards an n -step truncated λ -return target, but they always use the latest horizon. In each pass over that episode you can use a slightly longer horizon and obtain slightly better results. Recall that the truncated λ -return is defined in (12.9) as

$$G_{t:h}^\lambda \doteq (1 - \lambda) \sum_{n=1}^{h-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{h-t-1} G_{t:h}.$$

Let us step through how this target could ideally be used if computational complexity was not an issue. The episode begins with an estimate at time 0 using the weights $\mathbf{w}_0$ from the end of the previous episode. Learning begins when the data horizon is extended to time step 1. The target for the estimate at step 0, given the data up to horizon 1, could only be the one-step return $G_{0:1}$, which includes R_1 and bootstraps from the estimate

$\hat{v}(S_1, \mathbf{w}_0)$. Note that this is exactly what $G_{0:1}^\lambda$ is, with the sum in the first term of the equation degenerating to zero. Using this update target, we construct $\mathbf{w}_1$. Then, after advancing the data horizon to step 2, what do we do? We have new data in the form of R_2 and S_2 , as well as the new $\mathbf{w}_1$, so now we can construct a better update target $G_{0:2}^\lambda$ for the first update from S_0 as well as a better update target $G_{1:2}^\lambda$ for the second update from S_1 . Using these improved targets, we redo the updates at S_1 and S_2 , starting again from $\mathbf{w}_0$, to produce $\mathbf{w}_2$. Now we advance the horizon to step 3 and repeat, going all the way back to produce three new targets, redoing all updates starting from the original $\mathbf{w}_0$ to produce $\mathbf{w}_3$, and so on. Each time the horizon is advanced, all the updates are redone starting from $\mathbf{w}_0$ using the weight vector from the preceding horizon.

This conceptual algorithm involves multiple passes over the episode, one at each horizon, each generating a different sequence of weight vectors. To describe it clearly we have to distinguish between the weight vectors computed at the different horizons. Let us use $\mathbf{w}_t^h$ to denote the weights used to generate the value at time t in the sequence up to horizon h . The first weight vector $\mathbf{w}_0^h$ in each sequence is that inherited from the previous episode (so they are the same for all h), and the last weight vector $\mathbf{w}_h^h$ in each sequence defines the ultimate weight-vector sequence of the algorithm. At the final horizon $h = T$ we obtain the final weights $\mathbf{w}_T^T$ which will be passed on to form the initial weights of the next episode. With these conventions, the three first sequences described in the previous paragraph can be given explicitly:

$$h = 1 : \quad \mathbf{w}_1^1 \doteq \mathbf{w}_0^1 + \alpha [G_{0:1}^\lambda - \hat{v}(S_0, \mathbf{w}_0^1)] \nabla \hat{v}(S_0, \mathbf{w}_0^1),$$

$$\begin{aligned} h = 2 : \quad \mathbf{w}_1^2 &\doteq \mathbf{w}_0^2 + \alpha [G_{0:2}^\lambda - \hat{v}(S_0, \mathbf{w}_0^2)] \nabla \hat{v}(S_0, \mathbf{w}_0^2), \\ \mathbf{w}_2^2 &\doteq \mathbf{w}_1^2 + \alpha [G_{1:2}^\lambda - \hat{v}(S_1, \mathbf{w}_1^2)] \nabla \hat{v}(S_1, \mathbf{w}_1^2), \end{aligned}$$

$$\begin{aligned} h = 3 : \quad \mathbf{w}_1^3 &\doteq \mathbf{w}_0^3 + \alpha [G_{0:3}^\lambda - \hat{v}(S_0, \mathbf{w}_0^3)] \nabla \hat{v}(S_0, \mathbf{w}_0^3), \\ \mathbf{w}_2^3 &\doteq \mathbf{w}_1^3 + \alpha [G_{1:3}^\lambda - \hat{v}(S_1, \mathbf{w}_1^3)] \nabla \hat{v}(S_1, \mathbf{w}_1^3), \\ \mathbf{w}_3^3 &\doteq \mathbf{w}_2^3 + \alpha [G_{2:3}^\lambda - \hat{v}(S_2, \mathbf{w}_2^3)] \nabla \hat{v}(S_2, \mathbf{w}_2^3). \end{aligned}$$

The general form for the update is

$$\mathbf{w}_{t+1}^h \doteq \mathbf{w}_t^h + \alpha [G_{t:h}^\lambda - \hat{v}(S_t, \mathbf{w}_t^h)] \nabla \hat{v}(S_t, \mathbf{w}_t^h), \quad 0 \leq t < h \leq T.$$

This update, together with $\mathbf{w}_t \doteq \mathbf{w}_t^t$ defines the *online λ -return algorithm*.

The online λ -return algorithm is fully online, determining a new weight vector $\mathbf{w}_t$ at each step t during an episode, using only information available at time t . Its main drawback is that it is computationally complex, passing over the portion of the episode experienced so far on every step. Note that it is strictly more complex than the offline λ -return algorithm, which passes through all the steps at the time of termination but does not make any updates during the episode. In return, the online algorithm can be expected to perform better than the offline one, not only during the episode when it makes an update while the offline algorithm makes none, but also at the end of the episode because

the weight vector used in bootstrapping (in $G_{t:h}^\lambda$) has had a larger number of informative updates. This effect can be seen if one looks carefully at Figure 12.8, which compares the two algorithms on the 19-state random walk task.

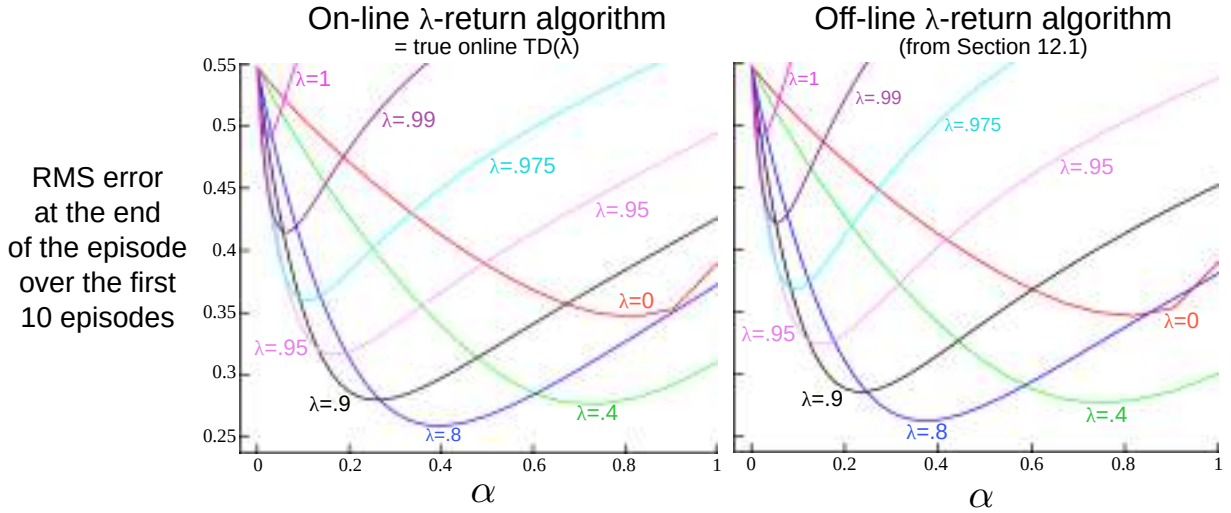


Figure 12.8: 19-state Random walk results (Example 7.1): Performance of online and offline λ -return algorithms. The performance measure here is the $\overline{VE}$ at the end of the episode, which should be the best case for the offline algorithm. Nevertheless, the online algorithm performs subtly better. For comparison, the $\lambda=0$ line is the same for both methods.

12.5 True Online TD(λ)

The online λ -return algorithm just presented is currently the best performing temporal-difference algorithm. It is an ideal which online TD(λ) only approximates. As presented, however, the online λ -return algorithm is very complex. Is there a way to invert this forward-view algorithm to produce an efficient backward-view algorithm using eligibility traces? It turns out that there is indeed an exact computationally congenial implementation of the online λ -return algorithm for the case of linear function approximation. This implementation is known as the true online TD(λ) algorithm because it is “truer” to the ideal of the online λ -return algorithm than the TD(λ) algorithm is.

The derivation of true online TD(λ) is a little too complex to present here (see the next section and the appendix to the paper by van Seijen et al., 2016) but its strategy is simple. The sequence of weight vectors produced by the online λ -return algorithm can be arranged in a triangle:

$$\begin{array}{ccccccc}
 \mathbf{w}_0^0 & & & & & & \\
 \mathbf{w}_0^1 & \mathbf{w}_1^1 & & & & & \\
 \mathbf{w}_0^2 & \mathbf{w}_1^2 & \mathbf{w}_2^2 & & & & \\
 \mathbf{w}_0^3 & \mathbf{w}_1^3 & \mathbf{w}_2^3 & \mathbf{w}_3^3 & & & \\
 \vdots & \vdots & \vdots & \vdots & \ddots & & \\
 \mathbf{w}_0^T & \mathbf{w}_1^T & \mathbf{w}_2^T & \mathbf{w}_3^T & \cdots & \mathbf{w}_T^T &
 \end{array}$$

One row of this triangle is produced on each time step. It turns out that the weight vectors on the diagonal, the $\mathbf{w}_t^t$, are the only ones really needed. The first, $\mathbf{w}_0^0$, is the initial weight vector of the episode, the last, $\mathbf{w}_T^T$, is the final weight vector, and each weight vector along the way, $\mathbf{w}_t^t$, plays a role in bootstrapping in the n -step returns of the updates. In the final algorithm the diagonal weight vectors are renamed without a superscript, $\mathbf{w}_t \doteq \mathbf{w}_t^t$. The strategy then is to find a compact, efficient way of computing each $\mathbf{w}_t^t$ from the one before. If this is done, for the linear case in which $\hat{v}(s, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s)$, then we arrive at the true online TD(λ) algorithm:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t + \alpha (\mathbf{w}_t^\top \mathbf{x}_t - \mathbf{w}_{t-1}^\top \mathbf{x}_t) (\mathbf{z}_t - \mathbf{x}_t),$$

where we have used the shorthand $\mathbf{x}_t \doteq \mathbf{x}(S_t)$, δ_t is defined as in TD(λ) (12.6), and $\mathbf{z}_t$ is defined by

$$\mathbf{z}_t \doteq \gamma \lambda \mathbf{z}_{t-1} + (1 - \alpha \gamma \lambda \mathbf{z}_{t-1}^\top \mathbf{x}_t) \mathbf{x}_t. \quad (12.11)$$

This algorithm has been proven to produce exactly the same sequence of weight vectors, $\mathbf{w}_t, 0 \leq t \leq T$, as the online λ -return algorithm (van Seijen et al. 2016). Thus the results on the random walk task on the left of Figure 12.8 are also its results on that task. Now, however, the algorithm is much less expensive. The memory requirements of true online TD(λ) are identical to those of conventional TD(λ), while the per-step computation is increased by about 50% (there is one more inner product in the eligibility-trace update). Overall, the per-step computational complexity remains of $O(d)$, the same as TD(λ). Pseudocode for the complete algorithm is given in the box.

True online TD(λ) for estimating $\mathbf{w}^\top \mathbf{x} \approx v_\pi$

Input: the policy π to be evaluated

Input: a feature function $\mathbf{x} : \mathcal{S}^+ \rightarrow \mathbb{R}^d$ such that $\mathbf{x}(\text{terminal}, \cdot) = \mathbf{0}$

Algorithm parameters: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize state and obtain initial feature vector $\mathbf{x}$

$\mathbf{z} \leftarrow \mathbf{0}$ (a d -dimensional vector)

$V_{old} \leftarrow 0$ (a temporary scalar variable)

 Loop for each step of episode:

 | Choose $A \sim \pi$

 | Take action A , observe $R, \mathbf{x}'$ (feature vector of the next state)

 | $V \leftarrow \mathbf{w}^\top \mathbf{x}$

 | $V' \leftarrow \mathbf{w}^\top \mathbf{x}'$

 | $\delta \leftarrow R + \gamma V' - V$

 | $\mathbf{z} \leftarrow \gamma \lambda \mathbf{z} + (1 - \alpha \gamma \lambda \mathbf{z}^\top \mathbf{x}) \mathbf{x}$

 | $\mathbf{w} \leftarrow \mathbf{w} + \alpha (\delta + V - V_{old}) \mathbf{z} - \alpha (V - V_{old}) \mathbf{x}$

 | $V_{old} \leftarrow V'$

 | $\mathbf{x} \leftarrow \mathbf{x}'$

 until $\mathbf{x}' = \mathbf{0}$ (signaling arrival at a terminal state)

The eligibility trace (12.11) used in true online TD(λ) is called a *dutch trace* to distinguish it from the trace (12.5) used in TD(λ), which is called an *accumulating trace*. Earlier work often used a third kind of trace called the *replacing trace*, defined only for the tabular case or for binary feature vectors such as those produced by tile coding. The replacing trace is defined on a component-by-component basis depending on whether the component of the feature vector was 1 or 0:

$$z_{i,t} \doteq \begin{cases} 1 & \text{if } x_{i,t} = 1 \\ \gamma \lambda z_{i,t-1} & \text{otherwise.} \end{cases} \quad (12.12)$$

Nowadays, we see replacing traces as crude approximations to dutch traces, which largely supersede them. Dutch traces usually perform better than replacing traces and have a clearer theoretical basis. Accumulating traces remain of interest for nonlinear function approximations where dutch traces are not available.

12.6 *Dutch Traces in Monte Carlo Learning

Although eligibility traces are closely associated historically with TD learning, in fact they have nothing to do with it. In fact, eligibility traces arise even in Monte Carlo learning, as we show in this section. We show that the linear MC algorithm (Chapter 9), taken as a forward view, can be used to derive an equivalent yet computationally cheaper backward-view algorithm using dutch traces. This is the only equivalence of forward- and backward-views that we explicitly demonstrate in this book. It gives some of the flavor of the proof of equivalence of true online TD(λ) and the online λ -return algorithm, but is much simpler.

The linear version of the gradient Monte Carlo prediction algorithm (page 202) makes the following sequence of updates, one for each time step of the episode:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha [G - \mathbf{w}_t^\top \mathbf{x}_t] \mathbf{x}_t, \quad 0 \leq t < T. \quad (12.13)$$

To simplify the example, we assume here that the return G is a single reward received at the end of the episode (this is why G is not subscripted by time) and that there is no discounting. In this case the update is also known as the Least Mean Square (LMS) rule. As a Monte Carlo algorithm, all the updates depend on the final reward/return, so none can be made until the end of the episode. The MC algorithm is an offline algorithm and we do not seek to improve this aspect of it. Rather we seek merely an implementation of this algorithm with computational advantages. We will still update the weight vector only at the end of the episode, but we will do some computation during each step of the episode and less at its end. This will give a more equal distribution of computation— $O(d)$ per step—and also remove the need to store the feature vectors at each step for use later at the end of each episode. Instead, we will introduce an additional vector memory, the eligibility trace, keeping in it a summary of all the feature vectors seen so far. This will be sufficient to efficiently recreate exactly the same overall update as the sequence of MC

updates (12.13), by the end of the episode:

$$\begin{aligned}
\mathbf{w}_T &= \mathbf{w}_{T-1} + \alpha (G - \mathbf{w}_{T-1}^\top \mathbf{x}_{T-1}) \mathbf{x}_{T-1} \\
&= \mathbf{w}_{T-1} + \alpha \mathbf{x}_{T-1} (-\mathbf{x}_{T-1}^\top \mathbf{w}_{T-1}) + \alpha G \mathbf{x}_{T-1} \\
&= (\mathbf{I} - \alpha \mathbf{x}_{T-1} \mathbf{x}_{T-1}^\top) \mathbf{w}_{T-1} + \alpha G \mathbf{x}_{T-1} \\
&= \mathbf{F}_{T-1} \mathbf{w}_{T-1} + \alpha G \mathbf{x}_{T-1}
\end{aligned}$$

where $\mathbf{F}_t \doteq \mathbf{I} - \alpha \mathbf{x}_t \mathbf{x}_t^\top$ is a *forgetting*, or *fading*, matrix. Now, recursing,

$$\begin{aligned}
&= \mathbf{F}_{T-1} (\mathbf{F}_{T-2} \mathbf{w}_{T-2} + \alpha G \mathbf{x}_{T-2}) + \alpha G \mathbf{x}_{T-1} \\
&= \mathbf{F}_{T-1} \mathbf{F}_{T-2} \mathbf{w}_{T-2} + \alpha G (\mathbf{F}_{T-1} \mathbf{x}_{T-2} + \mathbf{x}_{T-1}) \\
&= \mathbf{F}_{T-1} \mathbf{F}_{T-2} (\mathbf{F}_{T-3} \mathbf{w}_{T-3} + \alpha G \mathbf{x}_{T-3}) + \alpha G (\mathbf{F}_{T-1} \mathbf{x}_{T-2} + \mathbf{x}_{T-1}) \\
&= \mathbf{F}_{T-1} \mathbf{F}_{T-2} \mathbf{F}_{T-3} \mathbf{w}_{T-3} + \alpha G (\mathbf{F}_{T-1} \mathbf{F}_{T-2} \mathbf{x}_{T-3} + \mathbf{F}_{T-1} \mathbf{x}_{T-2} + \mathbf{x}_{T-1}) \\
&\vdots \\
&= \underbrace{\mathbf{F}_{T-1} \mathbf{F}_{T-2} \cdots \mathbf{F}_0 \mathbf{w}_0}_{\mathbf{a}_{T-1}} + \underbrace{\alpha G \sum_{k=0}^{T-1} \mathbf{F}_{T-1} \mathbf{F}_{T-2} \cdots \mathbf{F}_{k+1} \mathbf{x}_k}_{\mathbf{z}_{T-1}} \\
&= \mathbf{a}_{T-1} + \alpha G \mathbf{z}_{T-1}, \tag{12.14}
\end{aligned}$$

where $\mathbf{a}_{T-1}$ and $\mathbf{z}_{T-1}$ are the values at time $T-1$ of two auxiliary memory vectors that can be updated incrementally without knowledge of G and with $O(d)$ complexity per time step. The $\mathbf{z}_t$ vector is in fact a dutch-style eligibility trace. It is initialized to $\mathbf{z}_0 = \mathbf{x}_0$ and then updated according to

$$\begin{aligned}
\mathbf{z}_t &\doteq \sum_{k=0}^t \mathbf{F}_t \mathbf{F}_{t-1} \cdots \mathbf{F}_{k+1} \mathbf{x}_k, \quad 1 \leq t < T \\
&= \sum_{k=0}^{t-1} \mathbf{F}_t \mathbf{F}_{t-1} \cdots \mathbf{F}_{k+1} \mathbf{x}_k + \mathbf{x}_t \\
&= \mathbf{F}_t \sum_{k=0}^{t-1} \mathbf{F}_{t-1} \mathbf{F}_{t-2} \cdots \mathbf{F}_{k+1} \mathbf{x}_k + \mathbf{x}_t \\
&= \mathbf{F}_t \mathbf{z}_{t-1} + \mathbf{x}_t \\
&= (\mathbf{I} - \alpha \mathbf{x}_t \mathbf{x}_t^\top) \mathbf{z}_{t-1} + \mathbf{x}_t \\
&= \mathbf{z}_{t-1} - \alpha \mathbf{x}_t \mathbf{x}_t^\top \mathbf{z}_{t-1} + \mathbf{x}_t \\
&= \mathbf{z}_{t-1} - \alpha (\mathbf{z}_{t-1}^\top \mathbf{x}_t) \mathbf{x}_t + \mathbf{x}_t \\
&= \mathbf{z}_{t-1} + (1 - \alpha \mathbf{z}_{t-1}^\top \mathbf{x}_t) \mathbf{x}_t,
\end{aligned}$$

which is the dutch trace for the case of $\gamma\lambda=1$ (cf. Eq. 12.11). The $\mathbf{a}_t$ auxiliary vector is initialized to $\mathbf{a}_0 = \mathbf{w}_0$ and then updated according to

$$\mathbf{a}_t \doteq \mathbf{F}_t \mathbf{F}_{t-1} \cdots \mathbf{F}_0 \mathbf{w}_0 = \mathbf{F}_t \mathbf{a}_{t-1} = \mathbf{a}_{t-1} - \alpha \mathbf{x}_t \mathbf{x}_t^\top \mathbf{a}_{t-1}, \quad 1 \leq t < T.$$

The auxiliary vectors, $\mathbf{a}_t$ and $\mathbf{z}_t$, are updated on each time step $t < T$ and then, at time T when G is observed, they are used in (12.14) to compute $\mathbf{w}_T$. In this way we achieve exactly the same final result as the MC/LMS algorithm that has poor computational properties (12.13), but now with an incremental algorithm whose time and memory complexity per step is $O(d)$. This is surprising and intriguing because the notion of an eligibility trace (and the dutch trace in particular) has arisen in a setting without temporal-difference (TD) learning (in contrast to van Seijen and Sutton, 2014). It seems eligibility traces are not specific to TD learning at all; they are more fundamental than that. The need for eligibility traces seems to arise whenever one tries to learn long-term predictions in an efficient manner.

12.7 Sarsa(λ)

Very few changes in the ideas already presented in this chapter are required in order to extend eligibility-traces to action-value methods. To learn approximate action values, $\hat{q}(s, a, \mathbf{w})$, rather than approximate state values, $\hat{v}(s, \mathbf{w})$, we need to use the action-value form of the n -step return, from Chapter 10:

$$G_{t:t+n} \doteq R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{q}(S_{t+n}, A_{t+n}, \mathbf{w}_{t+n-1}), \quad t+n < T,$$

with $G_{t:t+n} \doteq G_t$ if $t+n \geq T$. Using this, we can form the action-value form of the truncated λ -return, which is otherwise identical to the state-value form (12.9). The action-value form of the offline λ -return algorithm (12.4) simply uses $\hat{q}$ rather than $\hat{v}$:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[G_t^\lambda - \hat{q}(S_t, A_t, \mathbf{w}_t) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t), \quad t = 0, \dots, T-1, \quad (12.15)$$

where $G_t^\lambda \doteq G_{t:\infty}^\lambda$. The compound backup diagram for this forward view is shown in Figure 12.9. Notice the similarity to the diagram of the TD(λ) algorithm (Figure 12.1). The first update looks ahead one full step, to the next state–action pair, the second looks ahead two steps, to the second state–action pair, and so on. A final update is based on the complete return. The weighting of each n -step update in the λ -return is just as in TD(λ) and the λ -return algorithm (12.3).

The temporal-difference method for action values, known as *Sarsa*(λ), approximates this forward view. It has the same update rule as given earlier for TD(λ):

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t,$$

except, naturally, using the action-value form of the TD error:

$$\delta_t \doteq R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t), \quad (12.16)$$

and the action-value form of the eligibility trace:

$$\begin{aligned} \mathbf{z}_{-1} &\doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \gamma \lambda \mathbf{z}_{t-1} + \nabla \hat{q}(S_t, A_t, \mathbf{w}_t), \quad 0 \leq t \leq T. \end{aligned}$$

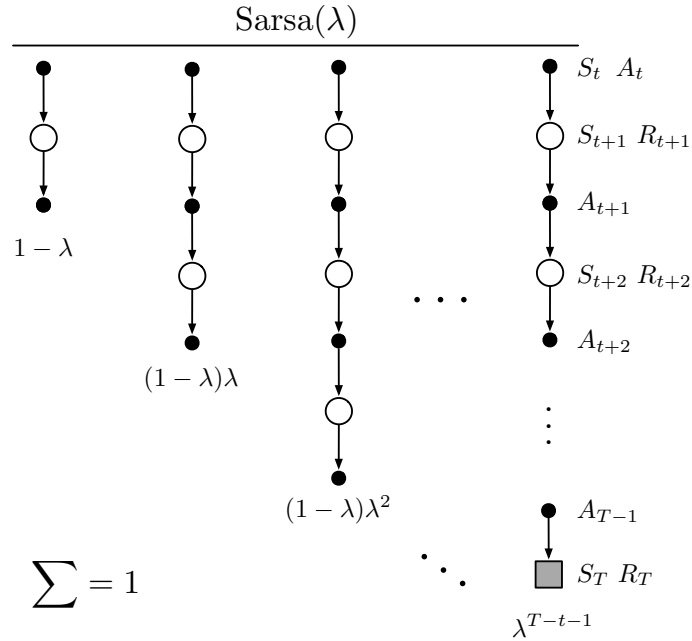
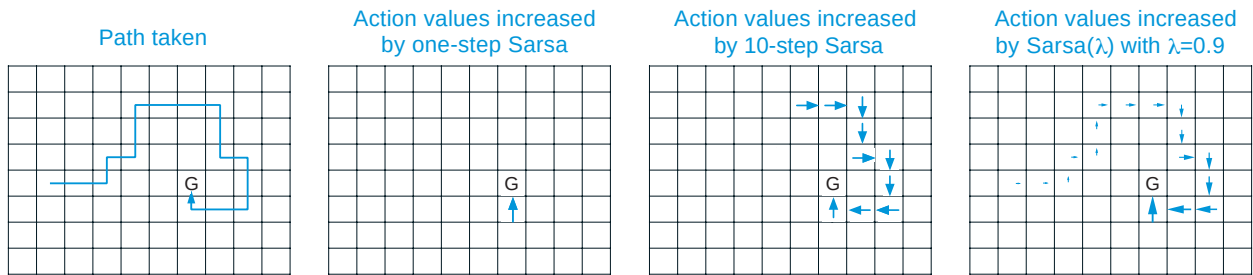


Figure 12.9: Sarsa(λ)'s backup diagram. Compare with Figure 12.1.

Complete pseudocode for Sarsa(λ) with linear function approximation, binary features, and either accumulating or replacing traces is given in the box on the next page. This pseudocode highlights a few optimizations possible in the special case of binary features (features are either active (=1) or inactive (=0)).

Example 12.1: Traces in Gridworld The use of eligibility traces can substantially increase the efficiency of control algorithms over one-step methods and even over n -step methods. The reason for this is illustrated by the gridworld example below.



The first panel shows the path taken by an agent in a single episode. The initial estimated values were zero, and all rewards were zero except for a positive reward at the goal location marked by G. The arrows in the other panels show, for various algorithms, which action-values would be increased, and by how much, upon reaching the goal. A one-step method would increment only the last action value, whereas an n -step method would equally increment the last n actions' values, and an eligibility trace method would update all the action values up to the beginning of the episode, to different degrees, fading with recency. The fading strategy is often the best. ■

Sarsa(λ) with binary features and linear function approximation for estimating $\mathbf{w}^\top \mathbf{x} \approx q_\pi$ or q_*

Input: a function $\mathcal{F}(s, a)$ returning the set of (indices of) active features for s, a
Input: a policy π (if estimating q_π)
Algorithm parameters: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$
Initialize: $\mathbf{w} = (w_1, \dots, w_d)^\top \in \mathbb{R}^d$ (e.g., $\mathbf{w} = \mathbf{0}$), $\mathbf{z} = (z_1, \dots, z_d)^\top \in \mathbb{R}^d$

Loop for each episode:
 Initialize S
 Choose $A \sim \pi(\cdot|S)$ or ε -greedy according to $\hat{q}(S, \cdot, \mathbf{w})$
 $\mathbf{z} \leftarrow \mathbf{0}$
 Loop for each step of episode:
 Take action A , observe R, S'
 $\delta \leftarrow R$
 Loop for i in $\mathcal{F}(S, A)$:
 $\delta \leftarrow \delta - w_i$
 $z_i \leftarrow z_i + 1$ (accumulating traces)
 or $z_i \leftarrow 1$ (replacing traces)
 If S' is terminal then:
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \mathbf{z}$
 Go to next episode
 Choose $A' \sim \pi(\cdot|S')$ or near greedily $\sim \hat{q}(S', \cdot, \mathbf{w})$
 Loop for i in $\mathcal{F}(S', A')$: $\delta \leftarrow \delta + \gamma w_i$
 $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \mathbf{z}$
 $\mathbf{z} \leftarrow \gamma \lambda \mathbf{z}$
 $S \leftarrow S'; A \leftarrow A'$

Exercise 12.6 Modify the pseudocode for Sarsa(λ) to use dutch traces (12.11) without the other distinctive features of a true online algorithm. Assume linear function approximation and binary features. $\square$

Example 12.2: Sarsa(λ) on Mountain Car Figure 12.10 (left) on the next page shows results with Sarsa(λ) on the Mountain Car task introduced in Example 10.1. The function approximation, action selection, and environmental details were exactly as in Chapter 10, and thus it is appropriate to numerically compare these results with the Chapter 10 results for n -step Sarsa (right side of the figure). The earlier results varied the update length n whereas here for Sarsa(λ) we vary the trace parameter λ , which plays a similar role. The fading-trace bootstrapping strategy of Sarsa(λ) appears to result in more efficient learning on this problem. $\blacksquare$

There is also an action-value version of our ideal TD method, the online λ -return algorithm (Section 12.4) and its efficient implementation as true online TD(λ) (Section 12.5). Everything in Section 12.4 goes through without change other than to use the action-value form of the n -step return given at the beginning of the current section. The analyses in Sections 12.5 and 12.6 also carry through for action values, the only change being the use

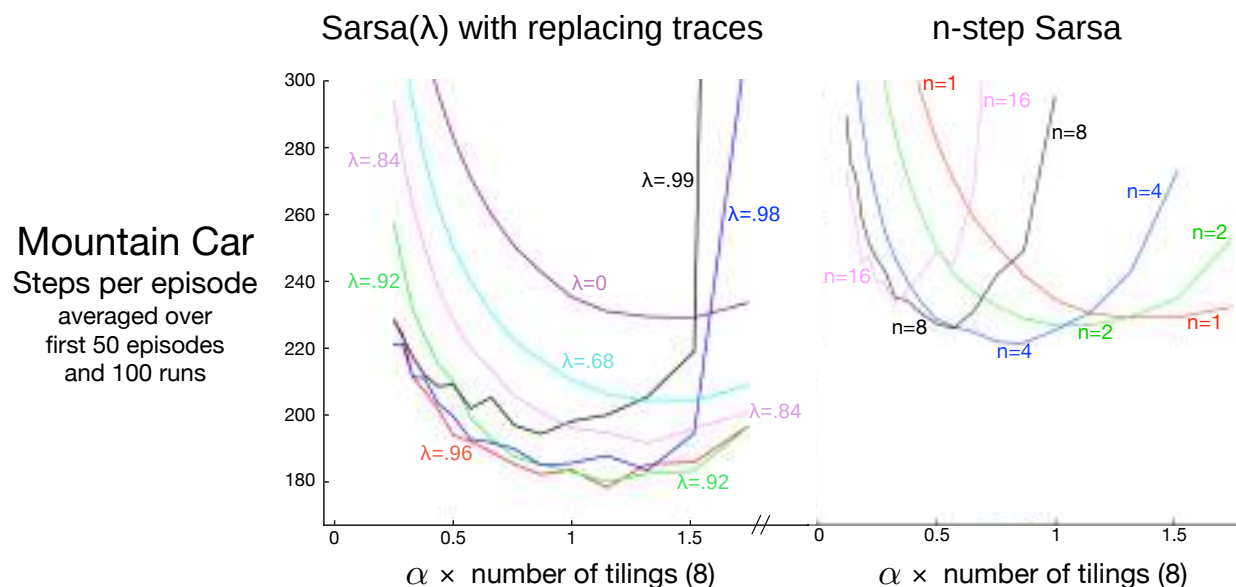


Figure 12.10: Early performance on the Mountain Car task of Sarsa(λ) with replacing traces and n -step Sarsa (copied from Figure 10.4) as a function of the step size, α .

of state-action feature vectors $\mathbf{x}_t = \mathbf{x}(S_t, A_t)$ instead of state feature vectors $\mathbf{x}_t = \mathbf{x}(S_t)$. Pseudocode for the resulting efficient algorithm, called *true online Sarsa(λ)* is given in the box on the next page. The figure below compares the performance of various versions of Sarsa(λ) on the Mountain Car example.

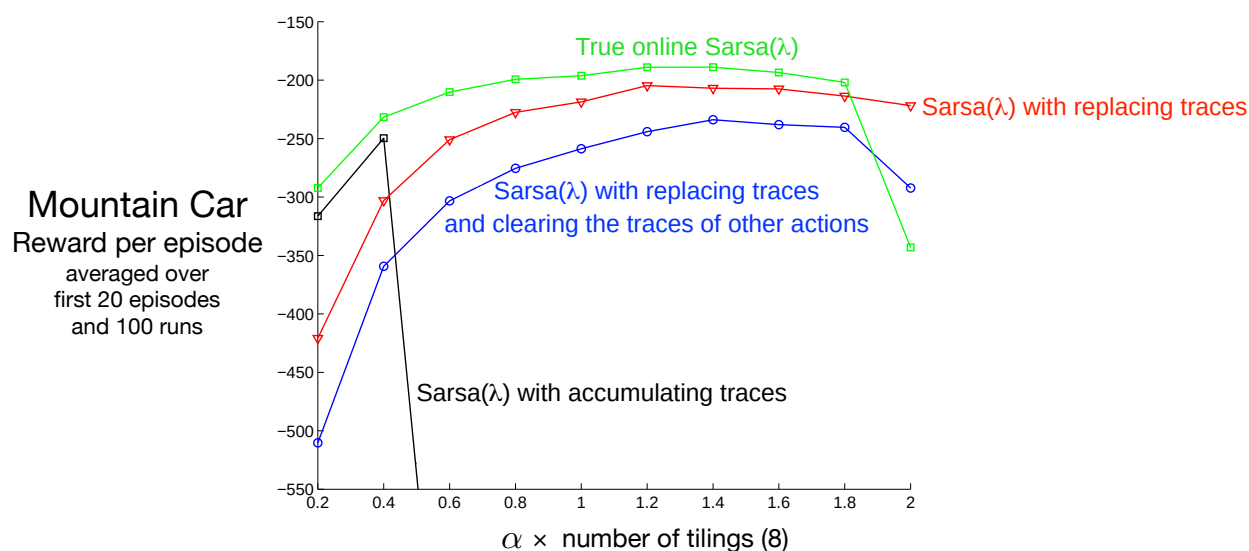


Figure 12.11: Summary comparison of Sarsa(λ) algorithms on the Mountain Car task. True online Sarsa(λ) performed better than regular Sarsa(λ) with both accumulating and replacing traces. Also included is a version of Sarsa(λ) with replacing traces in which, on each time step, the traces for the state and the actions not selected were set to zero.

True online Sarsa(λ) for estimating $\mathbf{w}^\top \mathbf{x} \approx q_\pi$ or q_*

Input: a feature function $\mathbf{x} : \mathcal{S}^+ \times \mathcal{A} \rightarrow \mathbb{R}^d$ such that $\mathbf{x}(\text{terminal}, \cdot) = \mathbf{0}$

Input: a policy π (if estimating q_π)

Algorithm parameters: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$

Initialize: $\mathbf{w} \in \mathbb{R}^d$ (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

 Choose $A \sim \pi(\cdot|S)$ or near greedily from S using $\mathbf{w}$

$\mathbf{x} \leftarrow \mathbf{x}(S, A)$

$\mathbf{z} \leftarrow \mathbf{0}$

$Q_{old} \leftarrow 0$

 Loop for each step of episode:

 | Take action A , observe R, S'

 | Choose $A' \sim \pi(\cdot|S')$ or near greedily from S' using $\mathbf{w}$

 | $\mathbf{x}' \leftarrow \mathbf{x}(S', A')$

 | $Q \leftarrow \mathbf{w}^\top \mathbf{x}$

 | $Q' \leftarrow \mathbf{w}^\top \mathbf{x}'$

 | $\delta \leftarrow R + \gamma Q' - Q$

 | $\mathbf{z} \leftarrow \gamma \lambda \mathbf{z} + (1 - \alpha \gamma \lambda \mathbf{z}^\top \mathbf{x}) \mathbf{x}$

 | $\mathbf{w} \leftarrow \mathbf{w} + \alpha(\delta + Q - Q_{old})\mathbf{z} - \alpha(Q - Q_{old})\mathbf{x}$

 | $Q_{old} \leftarrow Q'$

 | $\mathbf{x} \leftarrow \mathbf{x}'$

 | $A \leftarrow A'$

 until S' is terminal

Finally, there is also a truncated version of Sarsa(λ), called *forward Sarsa(λ)* (van Seijen, 2016), which appears to be a particularly effective model-free control method for use in conjunction with multi-layer artificial neural networks.

12.8 Variable λ and γ

We are starting now to reach the end of our development of fundamental TD learning algorithms. To present the final algorithms in their most general forms, it is useful to generalize the degree of bootstrapping and discounting beyond constant parameters to functions potentially dependent on the state and action. That is, each time step will have a different λ and γ , denoted λ_t and γ_t . We change notation now so that $\lambda : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is now a function from states and actions to the unit interval such that $\lambda_t \doteq \lambda(S_t, A_t)$, and similarly, $\gamma : \mathcal{S} \rightarrow [0, 1]$ is a function from states to the unit interval such that $\gamma_t \doteq \gamma(S_t)$.

Introducing the function γ , the *termination function*, is particularly significant because it changes the return, the fundamental random variable whose expectation we seek to

estimate. Now the return is defined more generally as

$$\begin{aligned}
G_t &\doteq R_{t+1} + \gamma_{t+1}G_{t+1} \\
&= R_{t+1} + \gamma_{t+1}R_{t+2} + \gamma_{t+1}\gamma_{t+2}R_{t+3} + \gamma_{t+1}\gamma_{t+2}\gamma_{t+3}R_{t+4} + \cdots \\
&= \sum_{k=t}^{\infty} \left(\prod_{i=t+1}^k \gamma_i \right) R_{k+1},
\end{aligned} \tag{12.17}$$

where, to assure the sums are finite, we require that $\prod_{k=t}^{\infty} \gamma_k = 0$ with probability one for all t . One convenient aspect of this definition is that it enables the episodic setting and its algorithms to be presented in terms of a single stream of experience, without special terminal states, start distributions, or termination times. An erstwhile terminal state becomes a state at which $\gamma(s)=0$ and which transitions to the start distribution. In that way (and by choosing $\gamma(\cdot)$ as a constant in all other states) we can recover the classical episodic setting as a special case. State dependent termination includes other prediction cases such as *pseudo termination*, in which we seek to predict a quantity without altering the flow of the Markov process. Discounted returns can be thought of as such a quantity, in which case state dependent termination unifies the episodic and discounted-continuing cases. (The undiscounted-continuing case still needs some special treatment.)

The generalization to variable bootstrapping is not a change in the problem, like discounting, but a change in the solution strategy. The generalization affects the λ -returns for states and actions. The new state-based λ -return can be written recursively as

$$G_t^{\lambda s} \doteq R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \hat{v}(S_{t+1}, \mathbf{w}_t) + \lambda_{t+1} G_{t+1}^{\lambda s} \right), \tag{12.18}$$

where now we have added the “s” to the superscript λ to remind us that this is a return that bootstraps from state values, distinguishing it from returns that bootstrap from action values, which we present below with “a” in the superscript. This equation says that the λ -return is the first reward, undiscounted and unaffected by bootstrapping, plus possibly a second term to the extent that we are not discounting at the next state (that is, according to γ_{t+1} ; recall that this is zero if the next state is terminal). To the extent that we aren’t terminating at the next state, we have a second term which is itself divided into two cases depending on the degree of bootstrapping in the state. To the extent we are bootstrapping, this term is the estimated value at the state, whereas, to the extent that we not bootstrapping, the term is the λ -return for the next time step. The action-based λ -return is either the Sarsa form

$$G_t^{\lambda a} \doteq R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) + \lambda_{t+1} G_{t+1}^{\lambda a} \right), \tag{12.19}$$

or the Expected Sarsa form,

$$G_t^{\lambda a} \doteq R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \bar{V}_t(S_{t+1}) + \lambda_{t+1} G_{t+1}^{\lambda a} \right), \tag{12.20}$$

where (7.8) is generalized to function approximation as

$$\bar{V}_t(s) \doteq \sum_a \pi(a|s) \hat{q}(s, a, \mathbf{w}_t). \tag{12.21}$$

Exercise 12.7 Generalize the three recursive equations above to their truncated versions, defining $G_{t:h}^{\lambda s}$ and $G_{t:h}^{\lambda a}$. $\square$

12.9 *Off-policy Traces with Control Variates

The final step is to incorporate importance sampling. Unlike in the case of n -step methods, for full non-truncated λ -returns one does not have a practical option in which the importance sampling is done outside the target return. Instead, we move directly to the bootstrapping generalization of per-decision importance sampling with control variates (Section 7.4). In the state case, our final definition of the λ -return generalizes (12.18), after the model of (7.13), to

$$G_t^{\lambda s} \doteq \rho_t \left(R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \hat{v}(S_{t+1}, \mathbf{w}_t) + \lambda_{t+1} G_{t+1}^{\lambda s} \right) \right) + (1 - \rho_t) \hat{v}(S_t, \mathbf{w}_t) \quad (12.22)$$

where $\rho_t = \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$ is the usual single-step importance sampling ratio. Much like the other returns we have seen in this book, the truncated version of this return can be approximated simply in terms of sums of the state-based TD error,

$$\delta_t^s \doteq R_{t+1} + \gamma_{t+1} \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t), \quad (12.23)$$

as

$$G_t^{\lambda s} \approx \hat{v}(S_t, \mathbf{w}_t) + \rho_t \sum_{k=t}^{\infty} \delta_k^s \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i \quad (12.24)$$

with the approximation becoming exact if the approximate value function does not change.

Exercise 12.8 Prove that (12.24) becomes exact if the value function does not change. To save writing, consider the case of $t = 0$, and use the notation $V_k \doteq \hat{v}(S_k, \mathbf{w})$. $\square$

Exercise 12.9 The truncated version of the general off-policy return is denoted $G_{t:h}^{\lambda s}$. Guess the correct equation, based on (12.24). $\square$

The above form of the λ -return (12.24) is convenient to use in a forward-view update,

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha \left(G_t^{\lambda s} - \hat{v}(S_t, \mathbf{w}_t) \right) \nabla \hat{v}(S_t, \mathbf{w}_t) \\ &\approx \mathbf{w}_t + \alpha \rho_t \left(\sum_{k=t}^{\infty} \delta_k^s \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i \right) \nabla \hat{v}(S_t, \mathbf{w}_t), \end{aligned}$$

which to the experienced eye looks like an eligibility-based TD update—the product is like an eligibility trace and it is multiplied by TD errors. But this is just one time step of a forward view. The relationship that we are looking for is that the forward-view update, summed over time, is approximately equal to a backward-view update, summed over time (this relationship is only approximate because again we ignore changes in the value

function). The sum of the forward-view update over time is

$$\begin{aligned}
\sum_{t=1}^{\infty} (\mathbf{w}_{t+1} - \mathbf{w}_t) &\approx \sum_{t=1}^{\infty} \sum_{k=t}^{\infty} \alpha \rho_t \delta_k^s \nabla \hat{v}(S_t, \mathbf{w}_t) \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i \\
&= \sum_{k=1}^{\infty} \sum_{t=1}^k \alpha \rho_t \nabla \hat{v}(S_t, \mathbf{w}_t) \delta_k^s \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i \\
&\quad \text{(using the summation rule: } \sum_{t=x}^y \sum_{k=t}^y = \sum_{k=x}^y \sum_{t=x}^k \text{)} \\
&= \sum_{k=1}^{\infty} \alpha \delta_k^s \sum_{t=1}^k \rho_t \nabla \hat{v}(S_t, \mathbf{w}_t) \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i,
\end{aligned}$$

which would be in the form of the sum of a backward-view TD update if the entire expression from the second sum left could be written and updated incrementally as an eligibility trace, which we now show can be done. That is, we show that if this expression was the trace at time k , then we could update it from its value at time $k-1$ by:

$$\begin{aligned}
\mathbf{z}_k &= \sum_{t=1}^k \rho_t \nabla \hat{v}(S_t, \mathbf{w}_t) \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i \\
&= \sum_{t=1}^{k-1} \rho_t \nabla \hat{v}(S_t, \mathbf{w}_t) \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i + \rho_k \nabla \hat{v}(S_k, \mathbf{w}_k) \\
&= \underbrace{\gamma_k \lambda_k \rho_k \sum_{t=1}^{k-1} \rho_t \nabla \hat{v}(S_t, \mathbf{w}_t) \prod_{i=t+1}^{k-1} \gamma_i \lambda_i \rho_i}_{\mathbf{z}_{k-1}} + \rho_k \nabla \hat{v}(S_k, \mathbf{w}_k) \\
&= \rho_k (\gamma_k \lambda_k \mathbf{z}_{k-1} + \nabla \hat{v}(S_k, \mathbf{w}_k)),
\end{aligned}$$

which, changing the index from k to t , is the general accumulating trace update for state values:

$$\mathbf{z}_t \doteq \rho_t (\gamma_t \lambda_t \mathbf{z}_{t-1} + \nabla \hat{v}(S_t, \mathbf{w}_t)), \quad (12.25)$$

This eligibility trace, together with the usual semi-gradient parameter-update rule for TD(λ) (12.7), forms a general TD(λ) algorithm that can be applied to either on-policy or off-policy data. In the on-policy case, the algorithm is exactly TD(λ) because ρ_t is always 1 and (12.25) becomes the usual accumulating trace (12.5) (extended to variable λ and γ). In the off-policy case, the algorithm often works well but, as a semi-gradient method, is not guaranteed to be stable. In the next few sections we will consider extensions of it that do guarantee stability.

A very similar series of steps can be followed to derive the off-policy eligibility traces for *action*-value methods and corresponding general Sarsa(λ) algorithms. One could start with either recursive form for the general action-based λ -return, (12.19) or (12.20), but the latter (the Expected Sarsa form) works out to be simpler. We extend (12.20) to the

off-policy case after the model of (7.14) to produce

$$\begin{aligned} G_t^{\lambda a} &\doteq R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \bar{V}_t(S_{t+1}) + \lambda_{t+1} [\rho_{t+1} G_{t+1}^{\lambda a} + \bar{V}_t(S_{t+1}) - \rho_{t+1} \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)] \right) \\ &= R_{t+1} + \gamma_{t+1} \left(\bar{V}_t(S_{t+1}) + \lambda_{t+1} \rho_{t+1} [G_{t+1}^{\lambda a} - \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)] \right) \end{aligned} \quad (12.26)$$

where $\bar{V}_t(S_{t+1})$ is as given by (12.21). Again the λ -return can be written approximately as the sum of TD errors,

$$G_t^{\lambda a} \approx \hat{q}(S_t, A_t, \mathbf{w}_t) + \sum_{k=t}^{\infty} \delta_k^a \prod_{i=t+1}^k \gamma_i \lambda_i \rho_i, \quad (12.27)$$

using the expectation form of the action-based TD error:

$$\delta_t^a = R_{t+1} + \gamma_{t+1} \bar{V}_t(S_{t+1}) - \hat{q}(S_t, A_t, \mathbf{w}_t). \quad (12.28)$$

As before, the approximation becomes exact if the approximate value function does not change.

Exercise 12.10 Prove that (12.27) becomes exact if the value function does not change. To save writing, consider the case of $t = 0$, and use the notation $Q_k = \hat{q}(S_k, A_k, \mathbf{w})$. Hint: Start by writing out δ_0^a and $G_0^{\lambda a}$, then $G_0^{\lambda a} - Q_0$. $\square$

Exercise 12.11 The truncated version of the general off-policy return is denoted $G_{t:h}^{\lambda a}$. Guess the correct equation for it, based on (12.27). $\square$

Using steps entirely analogous to those for the state case, one can write a forward-view update based on (12.27), transform the sum of the updates using the summation rule, and finally derive the following form for the eligibility trace for action values:

$$\mathbf{z}_t \doteq \gamma_t \lambda_t \rho_t \mathbf{z}_{t-1} + \nabla \hat{q}(S_t, A_t, \mathbf{w}_t). \quad (12.29)$$

This eligibility trace, together with the expectation-based TD error (12.28) and the usual semi-gradient parameter-update rule (12.7), forms an elegant, efficient Expected Sarsa(λ) algorithm that can be applied to either on-policy or off-policy data. It is probably the best algorithm of this type at the current time (though of course it is not guaranteed to be stable until combined in some way with one of the methods presented in the following sections). In the on-policy case with constant λ and γ , and the usual state-action TD error (12.16), the algorithm would be identical to the Sarsa(λ) algorithm presented in Section 12.7.

Exercise 12.12 Show in detail the steps outlined above for deriving (12.29) from (12.27). Start with the update (12.15), substitute $G_t^{\lambda a}$ from (12.26) for G_t^λ , then follow similar steps as led to (12.25). $\square$

At $\lambda = 1$, these algorithms become closely related to corresponding Monte Carlo algorithms. One might expect that an exact equivalence would hold for episodic problems and offline updating, but in fact the relationship is subtler and slightly weaker than that. Under these most favorable conditions still there is not an episode by episode equivalence of updates, only of their expectations. This should not be surprising as these methods

make irrevocable updates as a trajectory unfolds, whereas true Monte Carlo methods would make no update for a trajectory if any action within it has zero probability under the target policy. In particular, all of these methods, even at $\lambda = 1$, still bootstrap in the sense that their targets depend on the current value estimates—it’s just that the dependence cancels out in expected value. Whether this is a good or bad property in practice is another question. Recently, methods have been proposed that do achieve an exact equivalence (Sutton, Mahmood, Precup and van Hasselt, 2014). These methods require an additional vector of “provisional weights” that keep track of updates which have been made but may need to be retracted (or emphasized) depending on the actions taken later. The state and state–action versions of these methods are called $\text{PTD}(\lambda)$ and $\text{PQ}(\lambda)$ respectively, where the ‘P’ stands for Provisional.

The practical consequences of all these new off-policy methods have not yet been established. Undoubtedly, issues of high variance will arise as they do in all off-policy methods using importance sampling (Section 11.9).

If $\lambda < 1$, then all these off-policy algorithms involve bootstrapping and the deadly triad applies (Section 11.3), meaning that they can be guaranteed stable only for the tabular case, for state aggregation, and for other limited forms of function approximation. For linear and more-general forms of function approximation the parameter vector may diverge to infinity as in the examples in Chapter 11. As we discussed there, the challenge of off-policy learning has two parts. Off-policy eligibility traces deal effectively with the first part of the challenge, correcting for the expected value of the targets, but not at all with the second part of the challenge, having to do with the distribution of updates. Algorithmic strategies for meeting the second part of the challenge of off-policy learning with eligibility traces are summarized in Section 12.11.

Exercise 12.13 What are the dutch-trace and replacing-trace versions of off-policy eligibility traces for state-value and action-value methods? $\square$

12.10 Watkins’s $Q(\lambda)$ to Tree-Backup(λ)

Several methods have been proposed over the years to extend Q-learning to eligibility traces. The original is *Watkins’s $Q(\lambda)$* , which decays its eligibility traces in the usual way as long as a greedy action was taken, then cuts the traces to zero after the first non-greedy action. The backup diagram for Watkins’s $Q(\lambda)$ is shown in Figure 12.12. In Chapter 6, we unified Q-learning and Expected Sarsa in the off-policy version of the latter, which includes Q-learning as a special case, and generalizes it to arbitrary target policies, and in the previous section of this chapter we completed our treatment of Expected Sarsa by generalizing it to off-policy eligibility traces. In Chapter 7, however, we distinguished n -step Expected Sarsa from n -step Tree Backup, where the latter retained the property of not using importance sampling. It remains then to present the eligibility trace version of Tree Backup, which we will call *Tree-Backup(λ)*, or *TB(λ)* for short. This is arguably the true successor to Q-learning because it retains its appealing absence of importance sampling even though it can be applied to off-policy data.

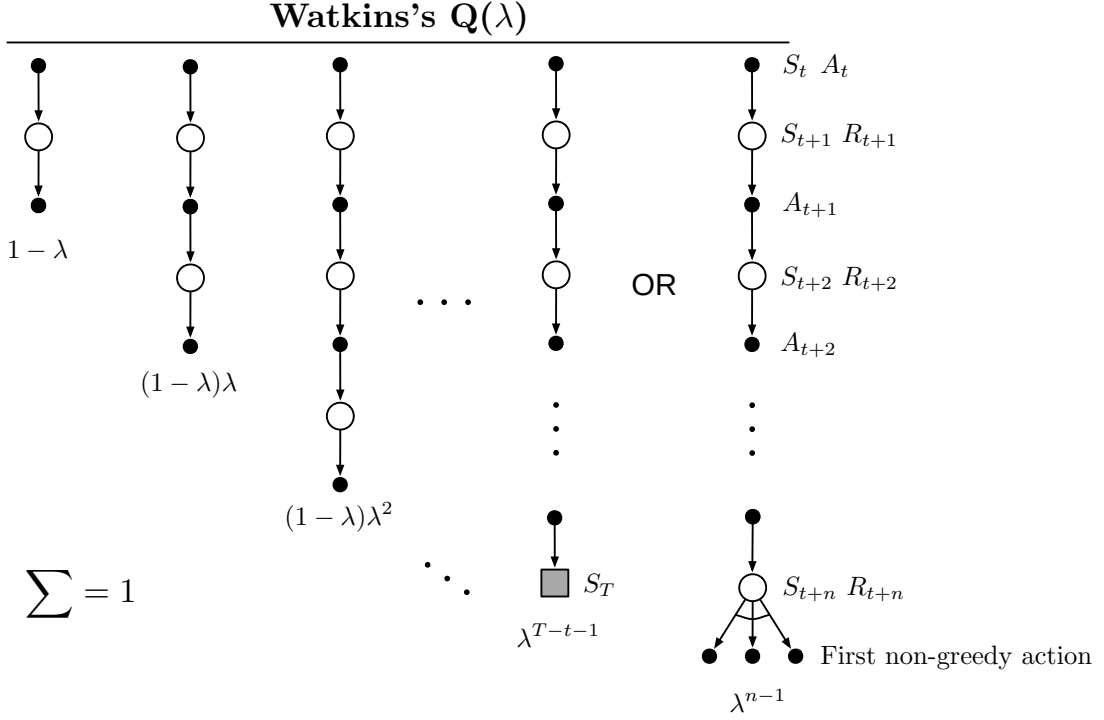


Figure 12.12: The backup diagram for Watkins's $Q(\lambda)$. The series of component updates ends either with the end of the episode or with the first nongreedy action, whichever comes first.

The concept of $TB(\lambda)$ is straightforward. As shown in its backup diagram in Figure 12.13, the tree-backup updates of each length (from Section 7.5) are weighted in the usual way dependent on the bootstrapping parameter λ . To get the detailed equations, with the right indices on the general bootstrapping and discounting parameters, it is best to start with a recursive form (12.20) for the λ -return using action values, and then expand the bootstrapping case of the target after the model of (7.16):

$$\begin{aligned}
 G_t^{\lambda a} &\doteq R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \bar{V}_t(S_{t+1}) + \lambda_{t+1} \left[\sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) \hat{q}(S_{t+1}, a, \mathbf{w}_t) + \pi(A_{t+1}|S_{t+1}) G_{t+1}^{\lambda a} \right] \right) \\
 &= R_{t+1} + \gamma_{t+1} \left(\bar{V}_t(S_{t+1}) + \lambda_{t+1} \pi(A_{t+1}|S_{t+1}) \left(G_{t+1}^{\lambda a} - \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) \right) \right)
 \end{aligned}$$

As per the usual pattern, it can also be written approximately (ignoring changes in the approximate value function) as a sum of TD errors,

$$G_t^{\lambda a} \approx \hat{q}(S_t, A_t, \mathbf{w}_t) + \sum_{k=t}^{\infty} \delta_k^a \prod_{i=t+1}^k \gamma_i \lambda_i \pi(A_i|S_i),$$

using the expectation form of the action-based TD error (12.28).

Following the same steps as in the previous section, we arrive at a special eligibility trace update involving the target-policy probabilities of the selected actions,

$$\mathbf{z}_t \doteq \gamma_t \lambda_t \pi(A_t|S_t) \mathbf{z}_{t-1} + \nabla \hat{q}(S_t, A_t, \mathbf{w}_t).$$

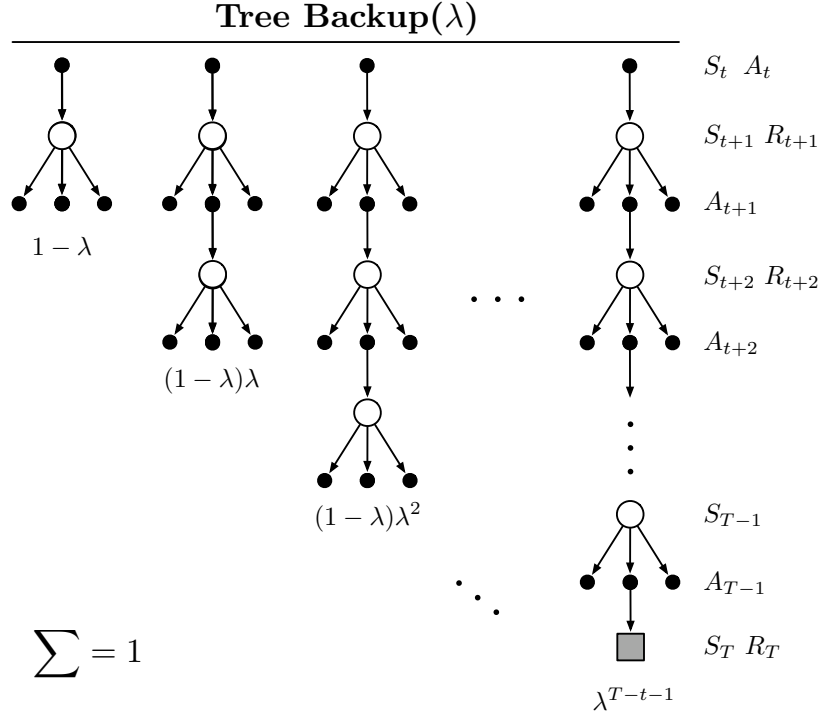


Figure 12.13: The backup diagram for the λ version of the Tree Backup algorithm.

This, together with the usual parameter-update rule (12.7), defines the TB(λ) algorithm. Like all semi-gradient algorithms, TB(λ) is not guaranteed to be stable when used with off-policy data and with a powerful function approximator. To obtain those assurances, TB(λ) would have to be combined with one of the methods presented in the next section.

**Exercise 12.14* How might Double Expected Sarsa be extended to eligibility traces? $\square$

12.11 Stable Off-policy Methods with Traces

Several methods using eligibility traces have been proposed that achieve guarantees of stability under off-policy training, and here we present four of the most important using this book's standard notation, including general bootstrapping and discounting functions. All are based on either the Gradient-TD or the Emphatic-TD ideas presented in Sections 11.7 and 11.8. All the algorithms assume linear function approximation, though extensions to nonlinear function approximation can also be found in the literature.

$GTD(\lambda)$ is the eligibility-trace algorithm analogous to TDC, the better of the two state-value Gradient-TD prediction algorithms discussed in Section 11.7. Its goal is to learn a parameter $\mathbf{w}_t$ such that $\hat{v}(s, \mathbf{w}) \doteq \mathbf{w}_t^\top \mathbf{x}(s) \approx v_\pi(s)$, even from data that is due to following another policy b . Its update is

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t^s \mathbf{z}_t - \alpha \gamma_{t+1} (1 - \lambda_{t+1}) (\mathbf{z}_t^\top \mathbf{v}_t) \mathbf{x}_{t+1},$$

with δ_t^s , $\mathbf{z}_t$, and ρ_t defined in the usual ways for state values (12.23) (12.25) (11.1), and

$$\mathbf{v}_{t+1} \doteq \mathbf{v}_t + \beta \delta_t^s \mathbf{z}_t - \beta (\mathbf{v}_t^\top \mathbf{x}_t) \mathbf{x}_t, \quad (12.30)$$

where, as in Section 11.7, $\mathbf{v} \in \mathbb{R}^d$ is a vector of the same dimension as $\mathbf{w}$, initialized to $\mathbf{v}_0 = \mathbf{0}$, and $\beta > 0$ is a second step-size parameter.

$GQ(\lambda)$ is the Gradient-TD algorithm for action values with eligibility traces. Its goal is to learn a parameter $\mathbf{w}_t$ such that $\hat{q}(s, a, \mathbf{w}_t) \doteq \mathbf{w}_t^\top \mathbf{x}(s, a) \approx q_\pi(s, a)$ from off-policy data. If the target policy is ε -greedy, or otherwise biased toward the greedy policy for $\hat{q}$, then $GQ(\lambda)$ can be used as a control algorithm. Its update is

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t^a \mathbf{z}_t - \alpha \gamma_{t+1} (1 - \lambda_{t+1}) (\mathbf{z}_t^\top \mathbf{v}_t) \bar{\mathbf{x}}_{t+1},$$

where $\bar{\mathbf{x}}_t$ is the average feature vector for S_t under the target policy,

$$\bar{\mathbf{x}}_t \doteq \sum_a \pi(a|S_t) \mathbf{x}(S_t, a),$$

δ_t^a is the expectation form of the TD error, which can be written

$$\delta_t^a \doteq R_{t+1} + \gamma_{t+1} \mathbf{w}_t^\top \bar{\mathbf{x}}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t,$$

$\mathbf{z}_t$ is defined in the usual way for action values (12.29), and the rest is as in $GTD(\lambda)$, including the update for $\mathbf{v}_t$ (12.30).

$HTD(\lambda)$ is a hybrid state-value algorithm combining aspects of $GTD(\lambda)$ and $TD(\lambda)$. Its most appealing feature is that it is a strict generalization of $TD(\lambda)$ to off-policy learning, meaning that if the behavior policy happens to be the same as the target policy, then $HTD(\lambda)$ becomes the same as $TD(\lambda)$, which is not true for $GTD(\lambda)$. This is appealing because $TD(\lambda)$ is often faster than $GTD(\lambda)$ when both algorithms converge, and $TD(\lambda)$ requires setting only a single step size. $HTD(\lambda)$ is defined by

$$\begin{aligned} \mathbf{w}_{t+1} &\doteq \mathbf{w}_t + \alpha \delta_t^s \mathbf{z}_t + \alpha ((\mathbf{z}_t - \mathbf{z}_t^b)^\top \mathbf{v}_t) (\mathbf{x}_t - \gamma_{t+1} \mathbf{x}_{t+1}), \\ \mathbf{v}_{t+1} &\doteq \mathbf{v}_t + \beta \delta_t^s \mathbf{z}_t - \beta (\mathbf{z}_t^{b\top} \mathbf{v}_t) (\mathbf{x}_t - \gamma_{t+1} \mathbf{x}_{t+1}), \quad \text{with } \mathbf{v}_0 \doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \rho_t (\gamma_t \lambda_t \mathbf{z}_{t-1} + \mathbf{x}_t), \quad \text{with } \mathbf{z}_{-1} \doteq \mathbf{0}, \\ \mathbf{z}_t^b &\doteq \gamma_t \lambda_t \mathbf{z}_{t-1}^b + \mathbf{x}_t, \quad \text{with } \mathbf{z}_{-1}^b \doteq \mathbf{0}, \end{aligned}$$

where $\beta > 0$ again is a second step-size parameter. In addition to the second set of weights, $\mathbf{v}_t$, $HTD(\lambda)$ also has a second set of eligibility traces, $\mathbf{z}_t^b$. These are conventional accumulating eligibility traces for the behavior policy and become equal to $\mathbf{z}_t$ if all the ρ_t are 1, which causes the last term in the $\mathbf{w}_t$ update to be zero and the overall update to reduce to $TD(\lambda)$.

Emphatic TD(λ) is the extension of the one-step Emphatic-TD algorithm (Sections 9.11 and 11.8) to eligibility traces. The resultant algorithm retains strong off-policy convergence guarantees while enabling any degree of bootstrapping, albeit at the cost of

high variance and potentially slow convergence. Emphatic TD(λ) is defined by

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t \\ \delta_t &\doteq R_{t+1} + \gamma_{t+1} \mathbf{w}_t^\top \mathbf{x}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t \\ \mathbf{z}_t &\doteq \rho_t (\gamma_t \lambda_t \mathbf{z}_{t-1} + M_t \mathbf{x}_t), \quad \text{with } \mathbf{z}_{-1} \doteq \mathbf{0}, \\ M_t &\doteq \lambda_t I_t + (1 - \lambda_t) F_t \\ F_t &\doteq \rho_{t-1} \gamma_t F_{t-1} + I_t, \quad \text{with } F_0 \doteq i(S_0),\end{aligned}$$

where $M_t \geq 0$ is the general form of *emphasis*, $F_t \geq 0$ is termed the *follow-on trace*, and $I_t \geq 0$ is the *interest*, as described in Section 11.8. Note that M_t , like δ_t , is not really an additional memory variable. It can be removed from the algorithm by substituting its definition into the eligibility-trace equation. Pseudocode and software for the true online version of Emphatic-TD(λ) are available on the web (Sutton, 2015b).

In the on-policy case ($\rho_t = 1$, for all t), Emphatic-TD(λ) is similar to conventional TD(λ), but still significantly different. In fact, whereas Emphatic-TD(λ) is guaranteed to converge for all state-dependent λ functions, TD(λ) is not. TD(λ) is guaranteed convergent only for all constant λ . See Yu's counterexample (Ghiassian, Rafiee, and Sutton, 2016).

12.12 Implementation Issues

It might at first appear that tabular methods using eligibility traces are much more complex than one-step methods. A naive implementation would require every state (or state-action pair) to update both its value estimate and its eligibility trace on every time step. This would not be a problem for implementations on single-instruction, multiple-data, parallel computers or in plausible artificial neural network (ANN) implementations, but it is a problem for implementations on conventional serial computers. Fortunately, for typical values of λ and γ the eligibility traces of almost all states are almost always nearly zero; only those states that have recently been visited will have traces significantly greater than zero and only these few states need to be updated to closely approximate these algorithms.

In practice, then, implementations on conventional computers may keep track of and update only the few traces that are significantly greater than zero. Using this trick, the computational expense of using traces in tabular methods is typically just a few times that of a one-step method. The exact multiple of course depends on λ and γ and on the expense of the other computations. Note that the tabular case is in some sense the worst case for the computational complexity of eligibility traces. When function approximation is used, the computational advantages of not using traces generally decrease. For example, if ANNs and backpropagation are used, then eligibility traces generally cause only a doubling of the required memory and computation per step. Truncated λ -return methods (Section 12.3) can be computationally efficient on conventional computers though they always require some additional memory.

12.13 Conclusions

Eligibility traces in conjunction with TD errors provide an efficient, incremental way of shifting and choosing between Monte Carlo and TD methods. The n -step methods of Chapter 7 also enabled this, but eligibility trace methods are more general, often faster to learn, and offer different computational complexity tradeoffs. This chapter has offered an introduction to the elegant, emerging theoretical understanding of eligibility traces for on- and off-policy learning and for variable bootstrapping and discounting. One aspect of this elegant theory is true online methods, which exactly reproduce the behavior of expensive ideal methods while retaining the computational congeniality of conventional TD methods. Another aspect is the possibility of derivations that automatically convert from intuitive forward-view methods to more efficient incremental backward-view algorithms. We illustrated this general idea in a derivation that started with a classical, expensive Monte Carlo algorithm and ended with a cheap incremental non-TD implementation using the same novel eligibility trace used in true online TD methods.

As we mentioned in Chapter 5, Monte Carlo methods may have advantages in non-Markov tasks because they do not bootstrap. Because eligibility traces make TD methods more like Monte Carlo methods, they also can have advantages in these cases. If one wants to use TD methods because of their other advantages, but the task is at least partially non-Markov, then the use of an eligibility trace method is indicated. Eligibility traces are the first line of defense against both long-delayed rewards and non-Markov tasks.

By adjusting λ , we can place eligibility trace methods anywhere along a continuum from Monte Carlo to one-step TD methods. Where shall we place them? We do not yet have a good theoretical answer to this question, but a clear empirical answer appears to be emerging. On tasks with many steps per episode, or many steps within the half-life of discounting, it appears significantly better to use eligibility traces than not to (e.g., see Figure 12.14). On the other hand, if the traces are so long as to produce a pure Monte Carlo method, or nearly so, then performance degrades sharply. An intermediate mixture appears to be the best choice. Eligibility traces should be used to bring us toward Monte Carlo methods, but not all the way there. In the future it may be possible to more finely vary the trade-off between TD and Monte Carlo methods by using variable λ , but at present it is not clear how this can be done reliably and usefully.

Methods using eligibility traces require more computation than one-step methods, but in return they offer significantly faster learning, particularly when rewards are delayed by many steps. Thus it often makes sense to use eligibility traces when data are scarce and cannot be repeatedly processed, as is often the case in online applications. On the other hand, in offline applications in which data can be generated cheaply, perhaps from an inexpensive simulation, then it often does not pay to use eligibility traces. In these cases the objective is not to get more out of a limited amount of data, but simply to process as much data as possible as quickly as possible. In these cases the speedup per datum due to traces is typically not worth their computational cost, and one-step methods are favored.

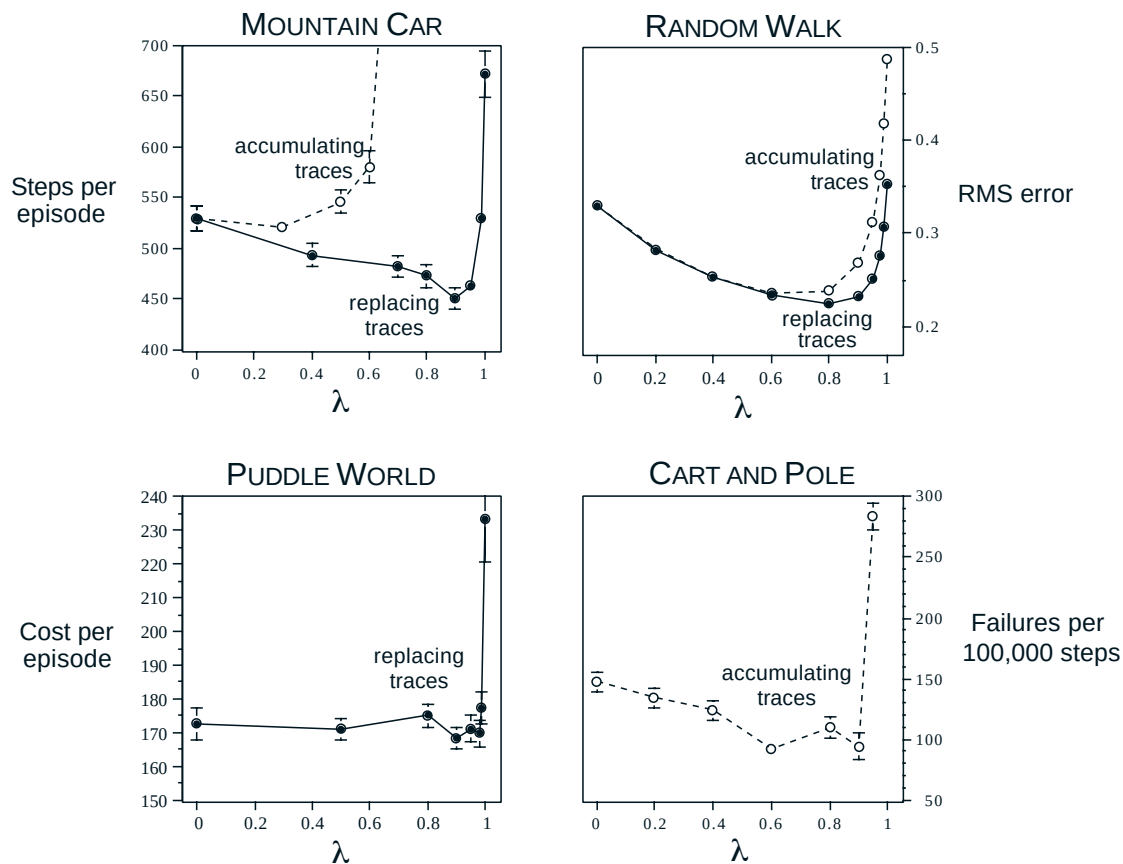


Figure 12.14: The effect of λ on reinforcement learning performance in four different test problems. In all cases, performance is generally best (a *lower* number in the graph) at an intermediate value of λ . The two left panels are applications to simple continuous-state control tasks using the Sarsa(λ) algorithm and tile coding, with either replacing or accumulating traces (Sutton, 1996). The upper-right panel is for policy evaluation on a random walk task using TD(λ) (Singh and Sutton, 1996). The lower right panel is unpublished data for the pole-balancing task (Example 3.4) from an earlier study (Sutton, 1984).



A (Long) Peek into Reinforcement Learning

Date: February 19, 2018 | Estimated Reading Time: 31 min | Author: Lilian Weng

► Table of Contents

[Updated on 2020-09-03: Updated the algorithm of [SARSA](#) and [Q-learning](#) so that the difference is more pronounced.

[Updated on 2021-09-19: Thanks to [38 38 38 的 11 11](#), we have this post in [Chinese](#)].

A couple of exciting news in Artificial Intelligence (AI) has just happened in recent years. AlphaGo defeated the best professional human player in the game of Go. Very soon the extended algorithm AlphaGo Zero beat AlphaGo by 100-0 without supervised learning on human knowledge. Top professional game players lost to the bot developed by OpenAI on DOTA2 1v1 competition. After knowing these, it is pretty hard not to be curious about the magic behind these algorithms — Reinforcement Learning (RL). I'm writing this post to briefly go over the field. We will first introduce several fundamental concepts and then dive into classic approaches to solving RL problems. Hopefully, this post could be a good starting point for newbies, bridging the future study on the cutting-edge research.

What is Reinforcement Learning?

Say, we have an agent in an unknown environment and this agent can obtain some rewards by interacting with the environment. The agent ought to take actions so as to maximize cumulative rewards. In reality, the scenario could be a bot playing a game to achieve high scores, or a robot trying to complete physical tasks with physical items; and not just limited to these.

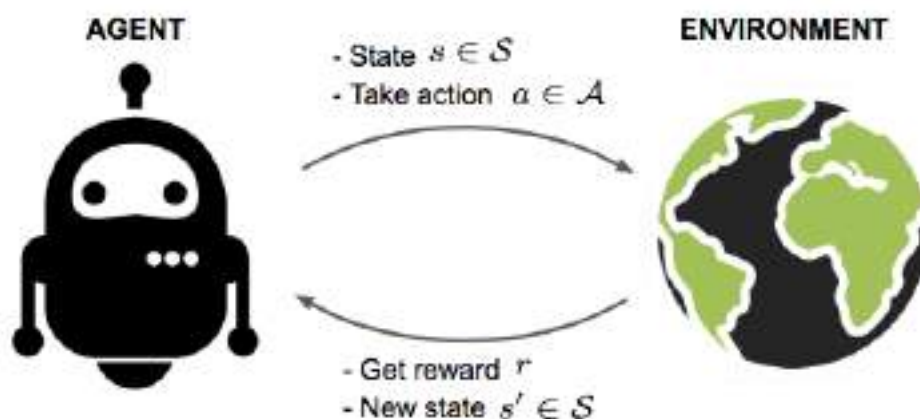




Fig. 1. An agent interacts with the environment, trying to take smart actions to maximize cumulative rewards.

The goal of Reinforcement Learning (RL) is to learn a good strategy for the agent from experimental trials and relative simple feedback received. With the optimal strategy, the agent is capable to actively adapt to the environment to maximize future rewards.

Key Concepts

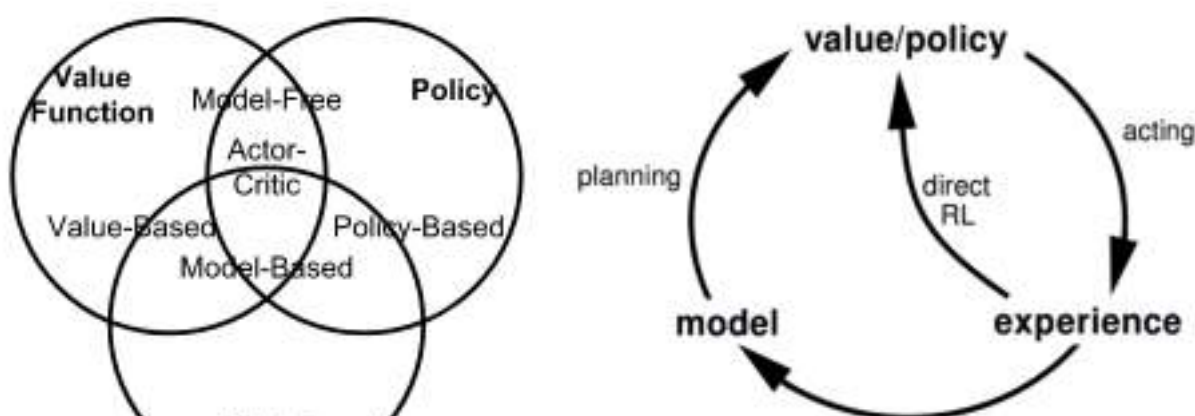
Now Let's formally define a set of key concepts in RL.

The agent is acting in an **environment**. How the environment reacts to certain actions is defined by a **model** which we may or may not know. The agent can stay in one of many **states** ($s \in \mathcal{S}$) of the environment, and choose to take one of many **actions** ($a \in \mathcal{A}$) to switch from one state to another. Which state the agent will arrive in is decided by transition probabilities between states (P). Once an action is taken, the environment delivers a **reward** ($r \in \mathcal{R}$) as feedback.

The model defines the reward function and transition probabilities. We may or may not know how the model works and this differentiate two circumstances:

- **Know the model:** planning with perfect information; do model-based RL. When we fully know the environment, we can find the optimal solution by Dynamic Programming (DP). Do you still remember "longest increasing subsequence" or "traveling salesmen problem" from your Algorithms 101 class? LOL. This is not the focus of this post though.
- **Does not know the model:** learning with incomplete information; do model-free RL or try to learn the model explicitly as part of the algorithm. Most of the following content serves the scenarios when the model is unknown.

The agent's **policy** $\pi(s)$ provides the guideline on what is the optimal action to take in a certain state with **the goal to maximize the total rewards**. Each state is associated with a **value** function $V(s)$ predicting the expected amount of future rewards we are able to receive in this state by acting the corresponding policy. In other words, the value function quantifies how good a state is. Both policy and value functions are what we try to learn in reinforcement learning.



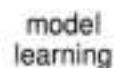


Fig. 2. Summary of approaches in RL based on whether we want to model the value, policy, or the environment. (Image source: reproduced from David Silver's RL course [lecture 1](#).)

The interaction between the agent and the environment involves a sequence of actions and observed rewards in time, $t = 1, 2, \dots, T$. During the process, the agent accumulates the knowledge about the environment, learns the optimal policy, and makes decisions on which action to take next so as to efficiently learn the best policy. Let's label the state, action, and reward at time step t as S_t , A_t , and R_t , respectively. Thus the interaction sequence is fully described by one **episode** (also known as "trial" or "trajectory") and the sequence ends at the terminal state S_T :

$$S_1, A_1, R_1, S_2, A_2, \dots, S_T$$

Terms you will encounter a lot when diving into different categories of RL algorithms:

- **Model-based**: Rely on the model of the environment; either the model is known or the algorithm learns it explicitly.
- **Model-free**: No dependency on the model during learning.
- **On-policy**: Use the deterministic outcomes or samples from the target policy to train the algorithm.
- **Off-policy**: Training on a distribution of transitions or episodes produced by a different behavior policy rather than that produced by the target policy.

Model: Transition and Reward

The model is a descriptor of the environment. With the model, we can learn or infer how the environment would interact with and provide feedback to the agent. The model has two major parts, transition probability function P and reward function R .

Let's say when we are in state s , we decide to take action a to arrive in the next state s' and obtain reward r . This is known as one **transition** step, represented by a tuple (s, a, s', r) .

The transition function P records the probability of transitioning from state s to s' after taking action a while obtaining reward r . We use $\mathbb{P}$ as a symbol of "probability".

$$P(s', r | s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a]$$

Thus the state-transition function can be defined as a function of $P(s', r | s, a)$:

$$P_{ss'}^a = P(s' | s, a) = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} P(s', r | s, a)$$



The reward function R predicts the next reward triggered by one action:

$$R(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} P(s', r | s, a)$$

Policy

Policy, as the agent's behavior function π , tells us which action to take in state s . It is a mapping from state s to action a and can be either deterministic or stochastic:

- Deterministic: $\pi(s) = a$.
- Stochastic: $\pi(a|s) = \mathbb{P}_\pi[A = a | S = s]$.

Value Function

Value function measures the goodness of a state or how rewarding a state or an action is by a prediction of future reward. The future reward, also known as **return**, is a total sum of discounted rewards going forward. Let's compute the return G_t starting from time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

The discounting factor $\gamma \in [0, 1]$ penalize the rewards in the future, because:

- The future rewards may have higher uncertainty; i.e. stock market.
- The future rewards do not provide immediate benefits; i.e. As human beings, we might prefer to have fun today rather than 5 years later ;).
- Discounting provides mathematical convenience; i.e., we don't need to track future steps forever to compute return.
- We don't need to worry about the infinite loops in the state transition graph.

The **state-value** of a state s is the expected return if we are in this state at time t , $S_t = s$:

$$V_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

Similarly, we define the **action-value** ("Q-value"; Q as "Quality" I believe?) of a state-action pair as:

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Additionally, since we follow the target policy π , we can make use of the probability distribution over possible actions and the Q-values to recover the state-value:

$$V_\pi(s) = \sum_{a \in \mathcal{A}} Q_\pi(s, a) \pi(a|s)$$

The difference between action-value and state-value is the action **advantage** function ("A-value"):



$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

Optimal Value and Policy

The optimal value function produces the maximum return:

$$V_*(s) = \max_{\pi} V_{\pi}(s), Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

The optimal policy achieves optimal value functions:

$$\pi_* = \arg \max_{\pi} V_{\pi}(s), \pi_* = \arg \max_{\pi} Q_{\pi}(s, a)$$

And of course, we have $V_{\pi_*}(s) = V_*(s)$ and $Q_{\pi_*}(s, a) = Q_*(s, a)$.

Markov Decision Processes

In more formal terms, almost all the RL problems can be framed as **Markov Decision Processes** (MDPs). All states in MDP has "Markov" property, referring to the fact that the future only depends on the current state, not the history:

$$\mathbb{P}[S_{t+1}|S_t] = \mathbb{P}[S_{t+1}|S_1, \dots, S_t]$$

Or in other words, the future and the past are **conditionally independent** given the present, as the current state encapsulates all the statistics we need to decide the future.

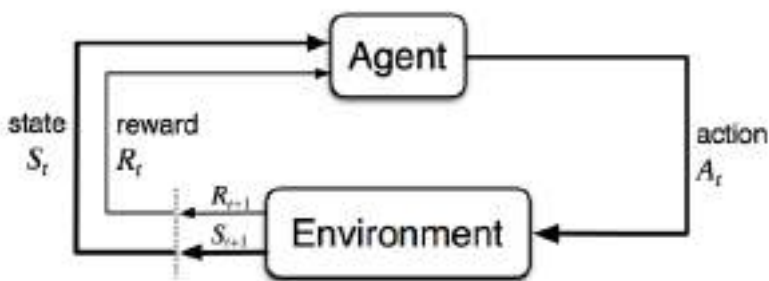


Fig. 3. The agent-environment interaction in a Markov decision process. (Image source: Sec. 3.1 Sutton & Barto (2017).)

A Markov decision process consists of five elements $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where the symbols carry the same meanings as key concepts in the previous section, well aligned with RL problem settings:

- $\mathcal{S}$ - a set of states;
- $\mathcal{A}$ - a set of actions;
- P - transition probability function;
- R - reward function;
- γ - discounting factor for future rewards. In an unknown environment, we do not have perfect knowledge about P and R .

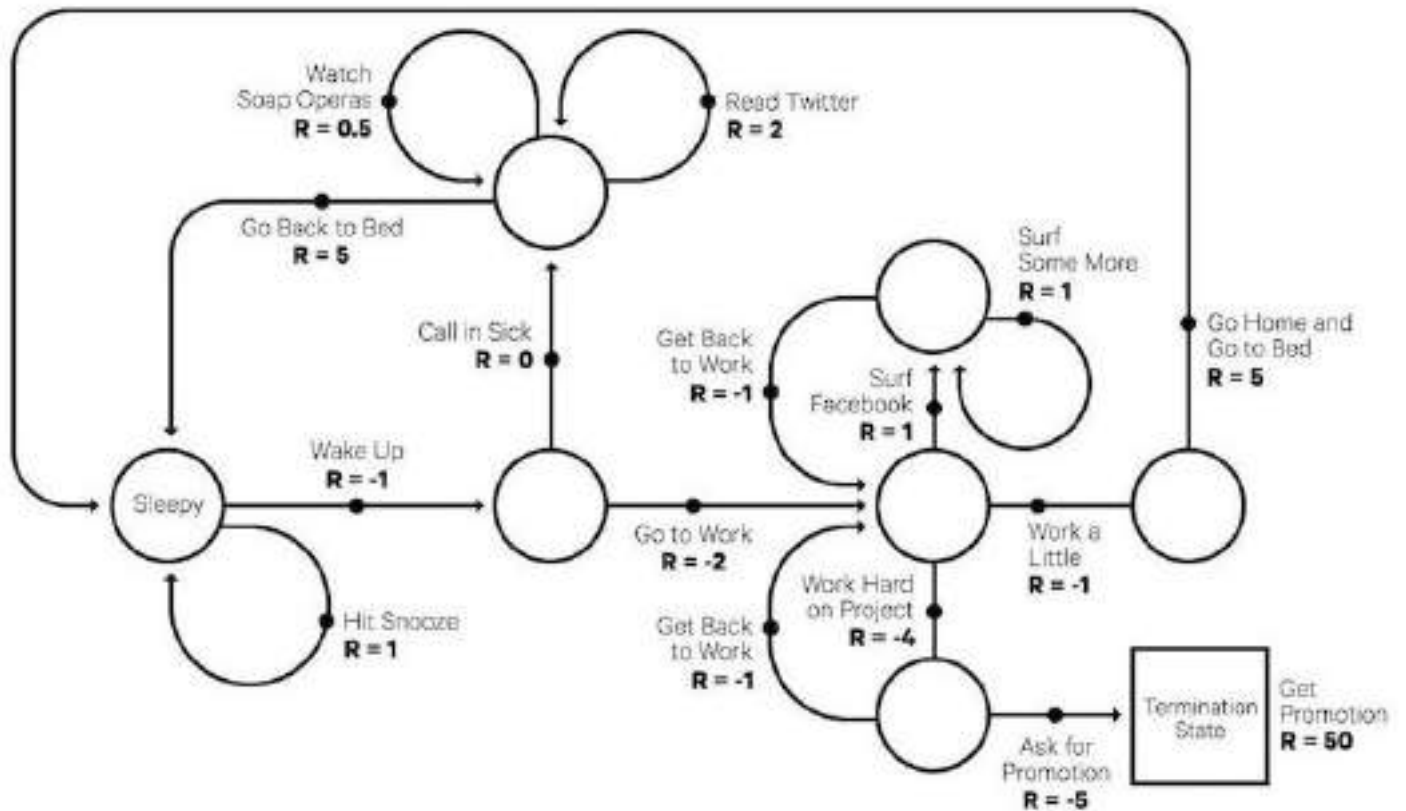


Fig. 4. A fun example of Markov decision process: a typical work day. (Image source: randomant.net/reinforcement-learning-concepts)

Bellman Equations

Bellman equations refer to a set of equations that decompose the value function into the immediate reward plus the discounted future values.

$$\begin{aligned}
 V(s) &= \mathbb{E}[G_t | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s] \\
 &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = s]
 \end{aligned}$$

Similarly for Q-value,

$$\begin{aligned}
 Q(s, a) &= \mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t = s, A_t = a] \\
 &= \mathbb{E}[R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi} Q(S_{t+1}, a') | S_t = s, A_t = a]
 \end{aligned}$$

Bellman Expectation Equations

The recursive update process can be further decomposed to be equations built on both state-value and action-value functions. As we go further in future action steps, we extend V and Q alternatively by following the policy π .

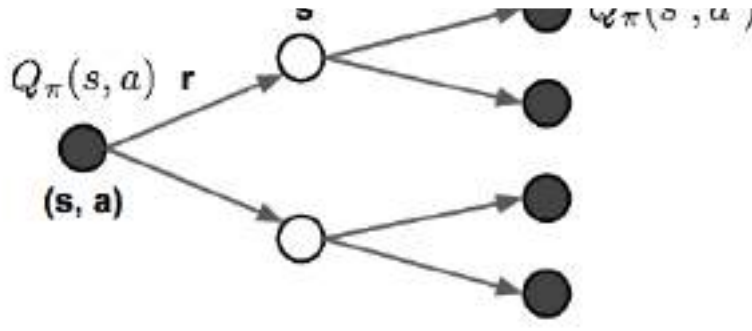


Fig. 5. Illustration of how Bellman expectation equations update state-value and action-value functions.

$$\begin{aligned}
 V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) Q_{\pi}(s, a) \\
 Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \\
 V_{\pi}(s) &= \sum_{a \in \mathcal{A}} \pi(a|s) \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_{\pi}(s') \right) \\
 Q_{\pi}(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') Q_{\pi}(s', a')
 \end{aligned}$$

Bellman Optimality Equations

If we are only interested in the optimal values, rather than computing the expectation following a policy, we could jump right into the maximum returns during the alternative updates without using a policy. RECAP: the optimal values V_* and Q_* are the best returns we can obtain, defined [here](#).

$$\begin{aligned}
 V_*(s) &= \max_{a \in \mathcal{A}} Q_*(s, a) \\
 Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \\
 V_*(s) &= \max_{a \in \mathcal{A}} \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a V_*(s') \right) \\
 Q_*(s, a) &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} Q_*(s', a')
 \end{aligned}$$

Unsurprisingly they look very similar to Bellman expectation equations.

If we have complete information of the environment, this turns into a planning problem, solvable by DP. Unfortunately, in most scenarios, we do not know $P_{ss'}^a$ or $R(s, a)$, so we cannot solve MDPs by directly applying Bellmen equations, but it lays the theoretical foundation for many RL algorithms.

Common Approaches

Now it is the time to go through the major approaches and classic algorithms for solving RL problems. In future posts, I plan to dive into each approach further.

Dynamic Programming

When the model is fully known, following Bellman equations, we can use Dynamic Programming (DP) to iteratively evaluate value functions and improve policy.

Policy Evaluation

Policy Evaluation is to compute the state-value V_π for a given policy π :

$$V_{t+1}(s) = \mathbb{E}_\pi[r + \gamma V_t(s') | S_t = s] = \sum_a \pi(a|s) \sum_{s', r} P(s', r | s, a) (r + \gamma V_t(s'))$$

Policy Improvement

Based on the value functions, Policy Improvement generates a better policy $\pi' \geq \pi$ by acting greedily.

$$Q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) | S_t = s, A_t = a] = \sum_{s', r} P(s', r | s, a) (r + \gamma V_\pi(s'))$$

Policy Iteration

The *Generalized Policy Iteration (GPI)* algorithm refers to an iterative procedure to improve the policy when combining policy evaluation and improvement.

$$\pi_0 \xrightarrow{\text{evaluation}} V_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{evaluation}} V_{\pi_1} \xrightarrow{\text{improve}} \pi_2 \xrightarrow{\text{evaluation}} \dots \xrightarrow{\text{improve}} \pi_* \xrightarrow{\text{evaluation}} V_*$$

In GPI, the value function is approximated repeatedly to be closer to the true value of the current policy and in the meantime, the policy is improved repeatedly to approach optimality. This policy iteration process works and always converges to the optimality, but why this is the case?

Say, we have a policy π and then generate an improved version π' by greedily taking actions, $\pi'(s) = \arg \max_{a \in \mathcal{A}} Q_\pi(s, a)$. The value of this improved π' is guaranteed to be better because:

$$\begin{aligned} Q_\pi(s, \pi'(s)) &= Q_\pi(s, \arg \max_{a \in \mathcal{A}} Q_\pi(s, a)) \\ &= \max_{a \in \mathcal{A}} Q_\pi(s, a) \geq Q_\pi(s, \pi(s)) = V_\pi(s) \end{aligned}$$

Monte-Carlo Methods

First, let's recall that $V(s) = \mathbb{E}[G_t | S_t = s]$. Monte-Carlo (MC) methods uses a simple idea: It learns from episodes of raw experience without modeling the environmental dynamics and computes the observed mean return as an approximation of the expected return. To compute the empirical return G_t , MC methods need to learn from **complete** episodes $S_1, A_1, R_2, \dots, S_T$ to compute $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ and all the episodes must eventually terminate.

The empirical mean return for state s is:

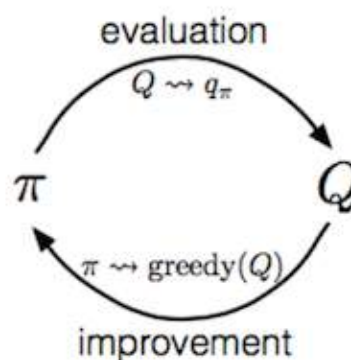


$$V(s) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s]}$$

where $\mathbb{1}[S_t = s]$ is a binary indicator function. We may count the visit of state s every time so that there could exist multiple visits of one state in one episode ("every-visit"), or only count it the first time we encounter a state in one episode ("first-visit"). This way of approximation can be easily extended to action-value functions by counting (s, a) pair.

$$Q(s, a) = \frac{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a] G_t}{\sum_{t=1}^T \mathbb{1}[S_t = s, A_t = a]}$$

To learn the optimal policy by MC, we iterate it by following a similar idea to [GPI](#).



1. Improve the policy greedily with respect to the current value function: $\pi(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$.
2. Generate a new episode with the new policy π (i.e. using algorithms like [ε-greedy](#) helps us balance between exploitation and exploration.)
3. Estimate Q using the new episode: $q_\pi(s, a) = \frac{\sum_{t=1}^T (\mathbb{1}[S_t=s, A_t=a] \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1})}{\sum_{t=1}^T \mathbb{1}[S_t=s, A_t=a]}$

Temporal-Difference Learning

Similar to Monte-Carlo methods, Temporal-Difference (TD) Learning is model-free and learns from episodes of experience. However, TD learning can learn from **incomplete** episodes and hence we don't need to track the episode up to termination. TD learning is so important that Sutton & Barto (2017) in their RL book describes it as "one idea ... central and novel to reinforcement learning".

Bootstrapping

TD learning methods update targets with regard to existing estimates rather than exclusively relying on actual rewards and complete returns as in MC methods. This approach is known as **bootstrapping**.

Value Estimation

The key idea in TD learning is to update the value function $V(S_t)$ towards an estimated return $R_{t+1} + \gamma V(S_{t+1})$ (known as "**TD target**"). To what extent we want to update the value function is

controlled by the learning rate hyperparameter α :

$$\begin{aligned}V(S_t) &\leftarrow (1 - \alpha)V(S_t) + \alpha G_t \\V(S_t) &\leftarrow V(S_t) + \alpha(G_t - V(S_t)) \\V(S_t) &\leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))\end{aligned}$$

Similarly, for action-value estimation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

Next, let's dig into the fun part on how to learn optimal policy in TD learning (aka "TD control"). Be prepared, you are gonna see many famous names of classic algorithms in this section.

SARSA: On-Policy TD control

"SARSA" refers to the procedure of updating Q-value by following a sequence of $\dots, S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, \dots$. The idea follows the same route of [GPI](#). Within one episode, it works as follows:

1. Initialize $t = 0$.
2. Start with S_0 and choose action $A_0 = \arg \max_{a \in \mathcal{A}} Q(S_0, a)$, where ϵ -greedy is commonly applied.
3. At time t , after applying action A_t , we observe reward R_{t+1} and get into the next state S_{t+1} .
4. Then pick the next action in the same way as in step 2: $A_{t+1} = \arg \max_{a \in \mathcal{A}} Q(S_{t+1}, a)$.
5. Update the Q-value function: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$.
6. Set $t = t + 1$ and repeat from step 3.

In each step of SARSA, we need to choose the *next* action according to the *current* policy.

Q-Learning: Off-policy TD control

The development of Q-learning (Watkins & Dayan, 1992) is a big breakout in the early days of Reinforcement Learning. Within one episode, it works as follows:

1. Initialize $t = 0$.
2. Starts with S_0 .
3. At time step t , we pick the action according to Q values, $A_t = \arg \max_{a \in \mathcal{A}} Q(S_t, a)$ and ϵ -greedy is commonly applied.
4. After applying action A_t , we observe reward R_{t+1} and get into the next state S_{t+1} .
5. Update the Q-value function:
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t)).$$
6. $t = t + 1$ and repeat from step 3.

The key difference from SARSA is that Q-learning does not follow the current policy to pick the second

action A_{t+1} . It estimates Q^* out of the best Q values, but which action (denoted as a^*) leads to this maximal Q does not matter and in the next step Q-learning may not follow a^* .



Fig. 6. The backup diagrams for Q-learning and SARSA. (Image source: Replotted based on Figure 6.5 in Sutton & Barto (2017))

Deep Q-Network

Theoretically, we can memorize $Q_*(.)$ for all state-action pairs in Q-learning, like in a gigantic table. However, it quickly becomes computationally infeasible when the state and action space are large. Thus people use functions (i.e. a machine learning model) to approximate Q values and this is called **function approximation**. For example, if we use a function with parameter θ to calculate Q values, we can label Q value function as $Q(s, a; \theta)$.

Unfortunately Q-learning may suffer from instability and divergence when combined with an nonlinear Q-value function approximation and bootstrapping (See Problems #2).

Deep Q-Network ("DQN"; Mnih et al. 2015) aims to greatly improve and stabilize the training procedure of Q-learning by two innovative mechanisms:

- **Experience Replay:** All the episode steps $e_t = (S_t, A_t, R_t, S_{t+1})$ are stored in one replay memory $D_t = \{e_1, \dots, e_t\}$. D_t has experience tuples over many episodes. During Q-learning updates, samples are drawn at random from the replay memory and thus one sample could be used multiple times. Experience replay improves data efficiency, removes correlations in the observation sequences, and smooths over changes in the data distribution.
- **Periodically Updated Target:** Q is optimized towards target values that are only periodically updated. The Q network is cloned and kept frozen as the optimization target every C steps (C is a hyperparameter). This modification makes the training more stable as it overcomes the short-term oscillations.

The loss function looks like this:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

where $U(D)$ is a uniform distribution over the replay memory D; θ^- is the parameters of the frozen target Q-network.

In addition, it is also found to be helpful to clip the error term to be between $[-1, 1]$. (I always get mixed feeling with parameter clipping, as many studies have shown that it works empirically but it makes the math much less pretty. :/)

```

Initialize replay memory  $D$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights  $\theta$ 
Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
For episode = 1,  $M$  do
    Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
    For  $t = 1, T$  do
        With probability  $\epsilon$  select a random action  $a_t$ 
        otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$ 
        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
        Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
        Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
        Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
        Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$ 
        Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$ 
        Every  $C$  steps reset  $\hat{Q} = Q$ 
    End For
End For

```

Fig. 7. Algorithm for DQN with experience replay and occasionally frozen optimization target. The preprocessed sequence is the output of some processes running on the input images of Atari games. Don't worry too much about it; just consider them as input feature vectors. (Image source: Mnih et al. 2015)

There are many extensions of DQN to improve the original design, such as DQN with dueling architecture (Wang et al. 2016) which estimates state-value function $V(s)$ and advantage function $A(s, a)$ with shared network parameters.

Combining TD and MC Learning

In the previous [section](#) on value estimation in TD learning, we only trace one step further down the action chain when calculating the TD target. One can easily extend it to take multiple steps to estimate the return.

Let's label the estimated return following n steps as $G_t^{(n)}$, $n = 1, \dots, \infty$, then:

n	G_t	Notes
$n = 1$	$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$	TD learning
$n = 2$	$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$	

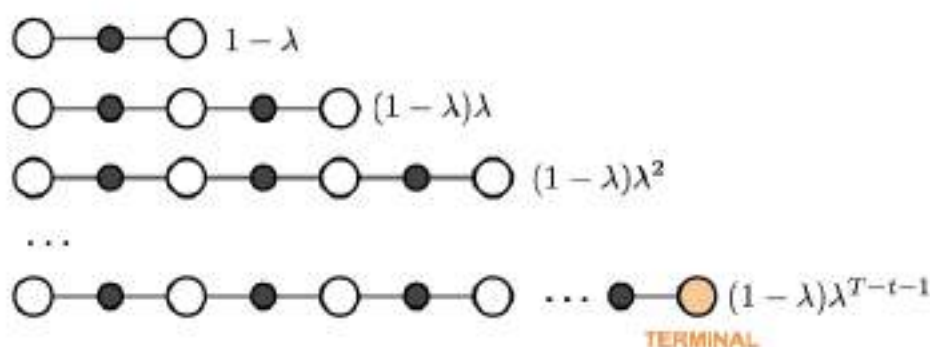


n	G_t	Notes
...		
$n = n$	$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$	
...		
$n = \infty$	$G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T + \gamma^{T-t} V(S_T)$	MC estimation

The generalized n-step TD learning still has the same form for updating the value function:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$$

At time step t , $TD(\lambda)$



We are free to pick any n in TD learning as we like. Now the question becomes what is the best n ? Which $G_t^{(n)}$ gives us the best return approximation? A common yet smart solution is to apply a weighted sum of all possible n -step TD targets rather than to pick a single best n . The weights decay by a factor λ with n , λ^{n-1} ; the intuition is similar to why we want to discount future rewards when computing the return: the more future we look into the less confident we would be. To make all the weight ($n \rightarrow \infty$) sum up to 1, we multiply every weight by $(1-\lambda)$, because:

$$\begin{aligned} \text{let } S &= 1 + \lambda + \lambda^2 + \dots \\ S &= 1 + \lambda(1 + \lambda + \lambda^2 + \dots) \\ S &= 1 + \lambda S \\ S &= 1/(1 - \lambda) \end{aligned}$$

This weighted sum of many n -step returns is called λ -return $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$. TD learning that adopts λ -return for value updating is labeled as **TD(λ)**. The original version we introduced above is equivalent to **TD(0)**.

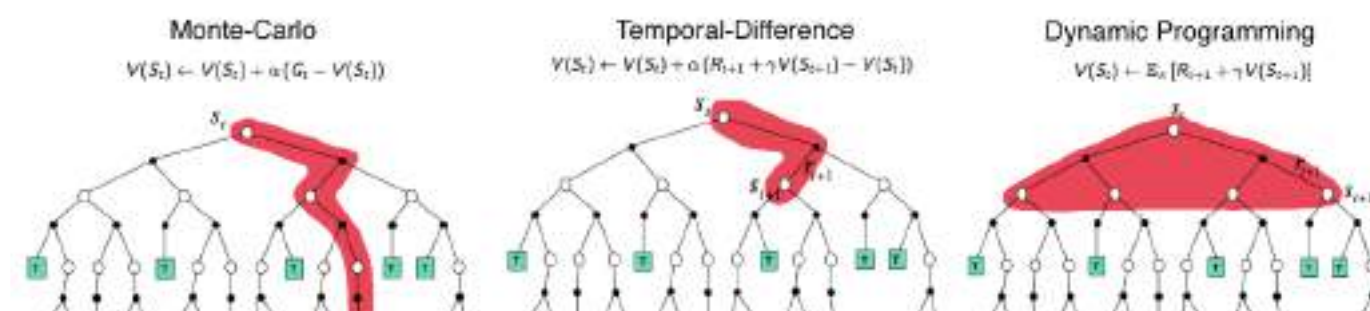




Fig. 8. Comparison of the backup diagrams of Monte-Carlo, Temporal-Difference learning, and Dynamic Programming for state value functions. (Image source: David Silver's RL course [lecture 4](#): "Model-Free Prediction")

Policy Gradient

All the methods we have introduced above aim to learn the state/action value function and then to select actions accordingly. Policy Gradient methods instead learn the policy directly with a parameterized function respect to θ , $\pi(a|s; \theta)$. Let's define the reward function (opposite of loss function) as *the expected return* and train the algorithm with the goal to maximize the reward function. My [next post](#) described why the policy gradient theorem works (proof) and introduced a number of policy gradient algorithms.

In discrete space:

$$\mathcal{J}(\theta) = V_{\pi_{\theta}}(S_1) = \mathbb{E}_{\pi_{\theta}}[V_1]$$

where S_1 is the initial starting state.

Or in continuous space:

$$\mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d_{\pi_{\theta}}(s) V_{\pi_{\theta}}(s) = \sum_{s \in \mathcal{S}} \left(d_{\pi_{\theta}}(s) \sum_{a \in \mathcal{A}} \pi(a|s, \theta) Q_{\pi}(s, a) \right)$$

where $d_{\pi_{\theta}}(s)$ is stationary distribution of Markov chain for π_{θ} . If you are unfamiliar with the definition of a "stationary distribution," please check this [reference](#).

Using *gradient ascent* we can find the best θ that produces the highest return. It is natural to expect policy-based methods are more useful in continuous space, because there is an infinite number of actions and/or states to estimate the values for in continuous space and hence value-based approaches are computationally much more expensive.

Policy Gradient Theorem

Computing the gradient *numerically* can be done by perturbing θ by a small amount ϵ in the k -th dimension. It works even when $J(\theta)$ is not differentiable (nice!), but unsurprisingly very slow.

$$\frac{\partial \mathcal{J}(\theta)}{\partial \theta_k} \approx \frac{\mathcal{J}(\theta + \epsilon u_k) - \mathcal{J}(\theta)}{\epsilon}$$

Or *analytically*,

$$\mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}}[r] = \sum_{s \in \mathcal{S}} d_{\pi_{\theta}}(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) R(s, a)$$

Actually we have nice theoretical support for (replacing $d(\cdot)$ with $d_\pi(\cdot)$):

$$\mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d_{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a) \propto \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a)$$

Check Sec 13.1 in Sutton & Barto (2017) for why this is the case.

Then,

$$\begin{aligned} \mathcal{J}(\theta) &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) Q_\pi(s, a) \\ \nabla \mathcal{J}(\theta) &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \nabla \pi(a|s; \theta) Q_\pi(s, a) \\ &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) \frac{\nabla \pi(a|s; \theta)}{\pi(a|s; \theta)} Q_\pi(s, a) \\ &= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s; \theta) \nabla \ln \pi(a|s; \theta) Q_\pi(s, a) \\ &= \mathbb{E}_{\pi_\theta} [\nabla \ln \pi(a|s; \theta) Q_\pi(s, a)] \end{aligned}$$

This result is named “Policy Gradient Theorem” which lays the theoretical foundation for various policy gradient algorithms:

$$\nabla \mathcal{J}(\theta) = \mathbb{E}_{\pi_\theta} [\nabla \ln \pi(a|s, \theta) Q_\pi(s, a)]$$

REINFORCE

REINFORCE, also known as Monte-Carlo policy gradient, relies on $Q_\pi(s, a)$, an estimated return by MC methods using episode samples, to update the policy parameter θ .

A commonly used variation of REINFORCE is to subtract a baseline value from the return G_t to reduce the variance of gradient estimation while keeping the bias unchanged. For example, a common baseline is state-value, and if applied, we would use $A(s, a) = Q(s, a) - V(s)$ in the gradient ascent update.

1. Initialize θ at random
2. Generate one episode $S_1, A_1, R_2, S_2, A_2, \dots, S_T$
3. For $t=1, 2, \dots, T$:
 1. Estimate the the return G_t since the time step t .
 2. $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla \ln \pi(A_t|S_t, \theta)$.

Actor-Critic

If the value function is learned in addition to the policy, we would get Actor-Critic algorithm.

- **Critic:** updates value function parameters w and depending on the algorithm it could be action-

value $Q(a|s; w)$ or state-value $V(s; w)$.

- **Actor:** updates policy parameters θ , in the direction suggested by the critic, $\pi(a|s; \theta)$.

Let's see how it works in an action-value actor-critic algorithm.

1. Initialize s, θ, w at random; sample $a \sim \pi(a|s; \theta)$.
2. For $t = 1 \dots T$:
 1. Sample reward $r_t \sim R(s, a)$ and next state $s' \sim P(s'|s, a)$.
 2. Then sample the next action $a' \sim \pi(s', a'; \theta)$.
 3. Update policy parameters: $\theta \leftarrow \theta + \alpha_\theta Q(s, a; w) \nabla_\theta \ln \pi(a|s; \theta)$.
 4. Compute the correction for action-value at time t :
$$G_{t:t+1} = r_t + \gamma Q(s', a'; w) - Q(s, a; w)$$
and use it to update value function parameters:
$$w \leftarrow w + \alpha_w G_{t:t+1} \nabla_w Q(s, a; w).$$
 5. Update $a \leftarrow a'$ and $s \leftarrow s'$.

α_θ and α_w are two learning rates for policy and value function parameter updates, respectively.

A3C

Asynchronous Advantage Actor-Critic (Mnih et al., 2016), short for A3C, is a classic policy gradient method with the special focus on parallel training.

In A3C, the critics learn the state-value function, $V(s; w)$, while multiple actors are trained in parallel and get synced with global parameters from time to time. Hence, A3C is good for parallel training by default, i.e. on one machine with multi-core CPU.

The loss function for state-value is to minimize the mean squared error, $\mathcal{J}_v(w) = (G_t - V(s; w))^2$ and we use gradient descent to find the optimal w . This state-value function is used as the baseline in the policy gradient update.

Here is the algorithm outline:

1. We have global parameters, θ and w ; similar thread-specific parameters, θ' and w' .
2. Initialize the time step $t = 1$
3. While $T \leq T_MAX$:
 1. Reset gradient: $d\theta = 0$ and $dw = 0$.
 2. Synchronize thread-specific parameters with global ones: $\theta' = \theta$ and $w' = w$.
 3. $t_{start} = t$ and get s_t .
 4. While $(s_t \neq \text{TERMINAL})$ and $(t - t_{start} \leq t_{max})$:



1. Pick the action $a_t \sim \pi(a_t|s_t; \theta')$ and receive a new reward r_t and a new state s_{t+1} .
2. Update $t = t + 1$ and $T = T + 1$.

5. Initialize the variable that holds the return estimation

$$R = \begin{cases} 0 & \text{if } s_t \text{ is TERMINAL} \\ V(s_t; w') & \text{otherwise} \end{cases}$$

.

6. For $i = t - 1, \dots, t_{\text{start}}$:

1. $R \leftarrow r_i + \gamma R$; here R is a MC measure of G_i .
2. Accumulate gradients w.r.t. θ' : $d\theta \leftarrow d\theta + \nabla_{\theta'} \log \pi(a_i|s_i; \theta')(R - V(s_i; w'))$;
Accumulate gradients w.r.t. w' : $dw \leftarrow dw + \nabla_{w'}(R - V(s_i; w'))^2$.

7. Update synchronously θ using $d\theta$, and w using dw .

A3C enables the parallelism in multiple agent training. The gradient accumulation step (6.2) can be considered as a reformation of minibatch-based stochastic gradient update: the values of w or θ get corrected by a little bit in the direction of each training thread independently.

Evolution Strategies

Evolution Strategies (ES) is a type of model-agnostic optimization approach. It learns the optimal solution by imitating Darwin's theory of the evolution of species by natural selection. Two prerequisites for applying ES: (1) our solutions can freely interact with the environment and see whether they can solve the problem; (2) we are able to compute a **fitness** score of how good each solution is. We don't have to know the environment configuration to solve the problem.

Say, we start with a population of random solutions. All of them are capable of interacting with the environment and only candidates with high fitness scores can survive (*only the fittest can survive in a competition for limited resources*). A new generation is then created by recombining the settings (*gene mutation*) of high-fitness survivors. This process is repeated until the new solutions are good enough.

Very different from the popular MDP-based approaches as what we have introduced above, ES aims to learn the policy parameter θ without value approximation. Let's assume the distribution over the parameter θ is an isotropic multivariate Gaussian with mean μ and fixed covariance $\sigma^2 I$. The gradient of $F(\theta)$ is calculated:



$$\begin{aligned}
& \nabla_{\theta} \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} F(\theta) \\
&= \nabla_{\theta} \int_{\theta} F(\theta) \Pr(\theta) \\
&= \int_{\theta} F(\theta) \Pr(\theta) \frac{\nabla_{\theta} \Pr(\theta)}{\Pr(\theta)} \\
&= \int_{\theta} F(\theta) \Pr(\theta) \nabla_{\theta} \log \Pr(\theta) \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} [F(\theta) \nabla_{\theta} \log \Pr(\theta)] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \nabla_{\theta} \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\theta-\mu)^2}{2\sigma^2}} \right) \right] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \nabla_{\theta} \left(-\log \sqrt{2\pi\sigma^2} - \frac{(\theta-\mu)^2}{2\sigma^2} \right) \right] \\
&= \mathbb{E}_{\theta \sim N(\mu, \sigma^2)} \left[F(\theta) \frac{\theta - \mu}{\sigma^2} \right]
\end{aligned}$$

$\Pr(\cdot)$ is the Gaussian density function.

Similar to how we do policy gradient update.

We can rewrite this formula in terms of a “mean” parameter θ (different from the θ above; this θ is the base gene for further mutation), $\epsilon \sim N(0, I)$ and therefore $\theta + \sigma\epsilon \sim N(\theta, \sigma^2)$. ϵ controls how much Gaussian noises should be added to create mutation:

$$\nabla_{\theta} \mathbb{E}_{\epsilon \sim N(0, I)} F(\theta + \sigma\epsilon) = \frac{1}{\sigma} \mathbb{E}_{\epsilon \sim N(0, I)} [F(\theta + \sigma\epsilon)\epsilon]$$

Algorithm 2 Parallelized Evolution Strategies

```

1: Input: Learning rate  $\alpha$ , noise standard deviation  $\sigma$ , initial policy parameters  $\theta_0$ 
2: Initialize:  $n$  workers with known random seeds, and initial parameters  $\theta_0$ 
3: for  $t = 0, 1, 2, \dots$  do
4:   for each worker  $i = 1, \dots, n$  do
5:     Sample  $\epsilon_i \sim \mathcal{N}(0, I)$ 
6:     Compute returns  $F_i = F(\theta_t + \sigma\epsilon_i)$ 
7:   end for
8:   Send all scalar returns  $F_i$  from each worker to every other worker
9:   for each worker  $i = 1, \dots, n$  do
10:    Reconstruct all perturbations  $\epsilon_j$  for  $j = 1, \dots, n$  using known random seeds
11:    Set  $\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{n\sigma} \sum_{j=1}^n F_j \epsilon_j$ 
12:   end for
13: end for

```

Fig. 9. A simple parallel evolution-strategies-based RL algorithm. Parallel workers share the random seeds so that they can reconstruct the Gaussian noises with tiny communication bandwidth. (Image source: Salimans et al. 2017.)

ES, as a black-box optimization algorithm, is another approach to RL problems (*In my original writing, I used the phrase “a nice alternative”; Seita pointed me to this discussion and thus I updated my wording.*). It has a couple of good characteristics (Salimans et al., 2017) keeping it fast and easy to train:

- ES does not need value function approximation;



- ES does not perform gradient back-propagation;
- ES is invariant to delayed or long-term rewards;
- ES is highly parallelizable with very little data communication.

Known Problems

Exploration-Exploitation Dilemma

The problem of exploration vs exploitation dilemma has been discussed in my previous [post](#). When the RL problem faces an unknown environment, this issue is especially a key to finding a good solution: without enough exploration, we cannot learn the environment well enough; without enough exploitation, we cannot complete our reward optimization task.

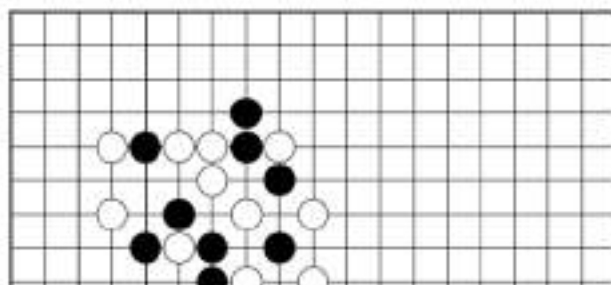
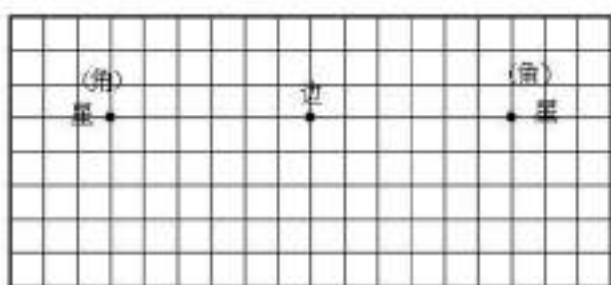
Different RL algorithms balance between exploration and exploitation in different ways. In [MC](#) methods, [Q-learning](#) or many on-policy algorithms, the exploration is commonly implemented by ϵ -greedy; In [ES](#), the exploration is captured by the policy parameter perturbation. Please keep this into consideration when developing a new RL algorithm.

Deadly Triad Issue

We do seek the efficiency and flexibility of TD methods that involve bootstrapping. However, when off-policy, nonlinear function approximation, and bootstrapping are combined in one RL algorithm, the training could be unstable and hard to converge. This issue is known as the **deadly triad** (Sutton & Barto, 2017). Many architectures using deep learning models were proposed to resolve the problem, including DQN to stabilize the training with experience replay and occasionally frozen target network.

Case Study: AlphaGo Zero

The game of [Go](#) has been an extremely hard problem in the field of Artificial Intelligence for decades until recent years. AlphaGo and AlphaGo Zero are two programs developed by a team at DeepMind. Both involve deep Convolutional Neural Networks ([CNN](#)) and Monte Carlo Tree Search (MCTS) and both have been approved to achieve the level of professional human Go players. Different from AlphaGo that relied on supervised learning from expert human moves, AlphaGo Zero used only reinforcement learning and self-play without human knowledge beyond the basic rules.



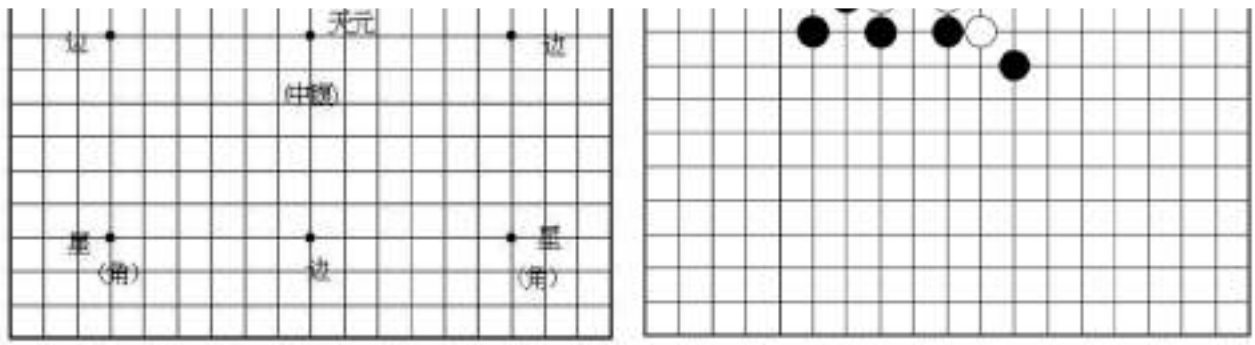


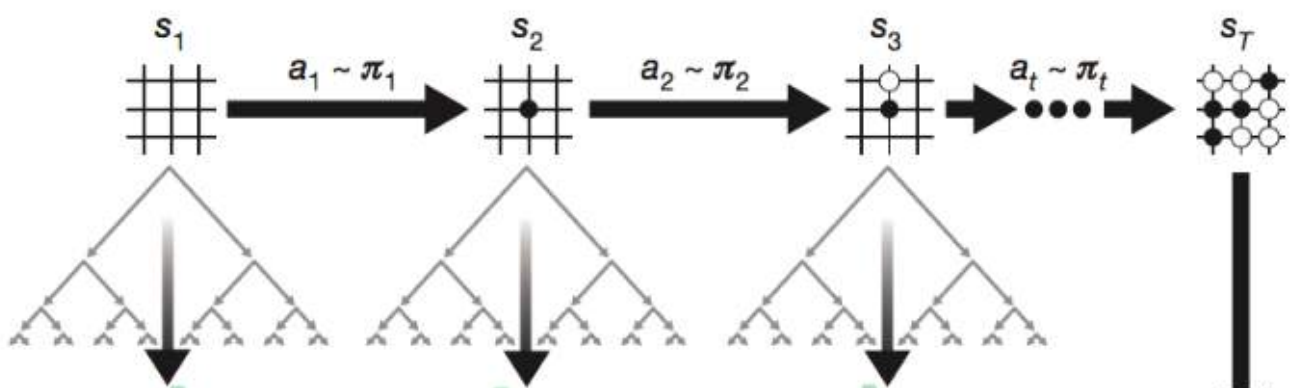
Fig. 10. The board of Go. Two players play black and white stones alternatively on the vacant intersections of a board with 19 x 19 lines. A group of stones must have at least one open point (an intersection, called a "liberty") to remain on the board and must have at least two or more enclosed liberties (called "eyes") to stay "alive". No stone shall repeat a previous position.

With all the knowledge of RL above, let's take a look at how AlphaGo Zero works. The main component is a deep CNN over the game board configuration (precisely, a ResNet with batch normalization and ReLU). This network outputs two values:

$$(p, v) = f_{\theta}(s)$$

- s : the game board configuration, 19 x 19 x 17 stacked feature planes; 17 features for each position, 8 past configurations (including current) for the current player + 8 past configurations for the opponent + 1 feature indicating the color (1=black, 0=white). We need to code the color specifically because the network is playing with itself and the colors of current player and opponents are switching between steps.
- p : the probability of selecting a move over $19^2 + 1$ candidates (19^2 positions on the board, in addition to passing).
- v : the winning probability given the current setting.

During self-play, MCTS further improves the action probability distribution $\pi \sim p(\cdot)$ and then the action a_t is sampled from this improved policy. The reward z_t is a binary value indicating whether the current player *eventually* wins the game. Each move generates an episode tuple (s_t, π_t, z_t) and it is saved into the replay memory. The details on MCTS are skipped for the sake of space in this post; please read the original paper if you are interested.



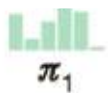


Fig. 11. AlphaGo Zero is trained by self-play while MCTS improves the output policy further in every step. (Image source: Figure 1a in Silver et al., 2017).

The network is trained with the samples in the replay memory to minimize the loss:

$$\mathcal{L} = (z - v)^2 - \pi^\top \log p + c \|\theta\|^2$$

where c is a hyperparameter controlling the intensity of L2 penalty to avoid overfitting.

AlphaGo Zero simplified AlphaGo by removing supervised learning and merging separated policy and value networks into one. It turns out that AlphaGo Zero achieved largely improved performance with a much shorter training time! I strongly recommend reading these [two papers](#) side by side and compare the difference, super fun.

I know this is a long read, but hopefully worth it. *If you notice mistakes and errors in this post, don't hesitate to contact me at [lilian dot wengweng at gmail dot com].* See you in the next post! :)

Cited as:

```
@article{weng2018bandit,  
  title   = "A (Long) Peek into Reinforcement Learning",  
  author  = "Weng, Lilian",  
  journal = "lilianweng.github.io",  
  year    = "2018",  
  url     = "https://lilianweng.github.io/posts/2018-02-19-rl-overview/"  
}
```

References

- [1] Yuxi Li. [Deep reinforcement learning: An overview](#). arXiv preprint arXiv:1701.07274. 2017.
- [2] Richard S. Sutton and Andrew G. Barto. [Reinforcement Learning: An Introduction; 2nd Edition](#). 2017.
- [3] Volodymyr Mnih, et al. [Asynchronous methods for deep reinforcement learning](#). ICML. 2016.
- [4] Tim Salimans, et al. [Evolution strategies as a scalable alternative to reinforcement learning](#). arXiv preprint arXiv:1703.03864 (2017).
- [5] David Silver, et al. [Mastering the game of go without human knowledge](#). Nature 550.7676 (2017):

DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills

XUE BIN PENG, University of California, Berkeley
 PIETER ABBEEL, University of California, Berkeley
 SERGEY LEVINE, University of California, Berkeley
 MICHEL VAN DE PANNE, University of British Columbia



Fig. 1. Highly dynamic skills learned by imitating reference motion capture clips using our method, executed by physically simulated characters. **Left:** Humanoid character performing a cartwheel. **Right:** Simulated Atlas robot performing a spinkick.

A longstanding goal in character animation is to combine data-driven specification of behavior with a system that can execute a similar behavior in a physical simulation, thus enabling realistic responses to perturbations and environmental variation. We show that well-known reinforcement learning (RL) methods can be adapted to learn robust control policies capable of imitating a broad range of example motion clips, while also learning complex recoveries, adapting to changes in morphology, and accomplishing user-specified goals. Our method handles keyframed motions, highly-dynamic actions such as motion-captured flips and spins, and retargeted motions. By combining a motion-imitation objective with a task objective, we can train characters that react intelligently in interactive settings, e.g., by walking in a desired direction or throwing a ball at a user-specified target. This approach thus combines the convenience and motion quality of using motion clips to define the desired style and appearance, with the flexibility and generality afforded by RL methods and physics-based animation. We further explore a number of methods for integrating multiple clips into the learning process to develop multi-skilled agents capable of performing a rich repertoire of diverse skills. We demonstrate results using multiple characters (human, Atlas robot, bipedal dinosaur, dragon) and a large variety of skills, including locomotion, acrobatics, and martial arts.

CCS Concepts: • **Computing methodologies** → **Animation**; *Physical simulation*; *Control methods*; *Reinforcement learning*;

Additional Key Words and Phrases: physics-based character animation, motion control, reinforcement learning

ACM Reference Format:

Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Trans. Graph.* 37, 4, Article 143 (August 2018), 18 pages. <https://doi.org/10.1145/3197517.3201311>

Authors' addresses: Xue Bin Peng, University of California, Berkeley; Pieter Abbeel, University of California, Berkeley; Sergey Levine, University of California, Berkeley; Michiel van de Panne, University of British Columbia.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
 0730-0301/2018/8-ART143 \$15.00
<https://doi.org/10.1145/3197517.3201311>

1 INTRODUCTION

Physics-based simulation of passive phenomena, such as cloth and fluids, has become nearly ubiquitous in industry. However, the adoption of physically simulated characters has been more modest. Modeling the motion of humans and animals remains a challenging problem, and currently, few methods exist that can simulate the diversity of behaviors exhibited in the real world. Among the enduring challenges in this domain are generalization and directability. Methods that rely on manually designed controllers have produced compelling results, but their ability to generalize to new skills and new situations is limited by the availability of human insight. Though humans are adept at performing a wide range of skills themselves, it can be difficult to articulate the internal strategies that underly such proficiency, and more challenging still to encode them into a controller. Directability is another obstacle that has impeded the adoption of simulated characters. Authoring motions for simulated characters remains notoriously difficult, and current interfaces still cannot provide users with an effective means of eliciting the desired behaviours from simulated characters.

Reinforcement learning (RL) provides a promising approach for motion synthesis, whereby an agent learns to perform various skills through trial-and-error, thus reducing the need for human insight. While deep reinforcement learning has been demonstrated to produce a range of complex behaviors in prior work [Duan et al. 2016; Heess et al. 2016; Schulman et al. 2015b], the quality of the generated motions has thus far lagged well behind state-of-the-art kinematic methods or manually designed controllers. In particular, controllers trained with deep RL exhibit severe (and sometimes humorous) artifacts, such as extraneous upper body motion, peculiar gaits, and unrealistic posture [Heess et al. 2017].¹ A natural direction to improve the quality of learned controllers is to incorporate motion capture or hand-authored animation data. In prior work, such systems have typically been designed by layering a physics-based tracking controller on top of a kinematic animation system [Da Silva et al. 2008; Lee et al. 2010a]. This type of approach is challenging because the kinematic animation system must produce reference motions that

¹See, for example, https://youtu.be/hx_bgoTF7bs

are feasible to track, and the resulting physics-based controller is limited in its ability to modify the motion to achieve plausible recoveries or accomplish task goals in ways that deviate substantially from the kinematic motion. Furthermore, such methods tend to be quite complex to implement.

An ideal learning-based animation system should allow an artist or motion capture actor to supply a set of reference motions for style, and then generate goal-directed and physically realistic behavior from those reference motions. In this work, we take a simple approach to this problem by directly rewarding the learned controller for producing motions that resemble reference animation data, while also achieving additional task objectives. We also demonstrate three methods for constructing controllers from multiple clips: training with a multi-clip reward based on a max operator; training a policy to perform multiple diverse skills that can be triggered by the user; and sequencing multiple single-clip policies by using their value functions to estimate the feasibility of transitions.

The central contribution of our paper is a framework for physics-based character animation that combines goal-directed reinforcement learning with data, which may be provided in the form of motion capture clips or keyframed animations. Although our framework consists of individual components that have been known for some time, the particular combination of these components in the context of data-driven and physics-based character animation is novel and, as we demonstrate in our experiments, produces a wide range of skills with motion quality and robustness that substantially exceed prior work. By incorporating motion capture data into a phase-aware policy, our system can produce physics-based behaviors that are nearly indistinguishable in appearance from the reference motion in the absence of perturbations, avoiding many of the artifacts exhibited by previous deep reinforcement learning algorithms, e.g., [Duan et al. 2016]. In the presence of perturbations or modifications, the motions remain natural, and the recovery strategies exhibit a high degree of robustness without the need for human engineering. To the best of our knowledge, we demonstrate some of the most capable physically simulated characters produced by learning-based methods. In our ablation studies, we identify two specific components of our method, reference state initialization and early termination, that are critical for achieving highly dynamic skills. We also demonstrate several methods for integrating multiple clips into a single policy.

2 RELATED WORK

Modeling the skilled movement of articulated figures has a long history in fields ranging from biomechanics to robotics and animation. In recent years, as machine learning algorithms for control have matured, there has also been an increase in interest in these problems from the machine learning community. Here we focus on the most closely related work in animation and RL.

Kinematic Models: Kinematic methods have been an enduring avenue of work in character animation that can be effective when large amounts of data are available. Given a dataset of motion clips, controllers can be built to select the appropriate clip to play in a given situation, e.g., [Agrawal and van de Panne 2016; Lee et al. 2010b; Safonova and Hodgins 2007]. Gaussian processes have been used to

learn latent representations which can then synthesize motions at runtime [Levine et al. 2012; Ye and Liu 2010b]. Extending this line of work, deep learning models, such as autoencoders and phase-functioned networks, have also been applied to develop generative models of human motion in a kinematic setting [Holden et al. 2017, 2016]. Given high quality data, data-driven kinematic methods will often produce higher quality motions than most simulation-based approaches. However, their ability to synthesize behaviors for novel situations can be limited. As tasks and environments become complex, collecting enough motion data to provide sufficient coverage of the possible behaviors quickly becomes untenable. Incorporating physics as a source of prior knowledge about how motions should change in the presence of perturbations and environmental variation provides one solution to this problem, as discussed below.

Physics-based Models: Design of controllers for simulated characters remains a challenging problem, and has often relied on human insight to implement task-specific strategies. Locomotion in particular has been the subject of considerable work, with robust controllers being developed for both human and nonhuman characters, e.g., [Coros et al. 2010; Ye and Liu 2010a; Yin et al. 2007]. Many such controllers are the products of an underlying simplified model and an optimization process, where a compact set of parameters are tuned in order to achieve the desired behaviors [Agrawal et al. 2013; Ha and Liu 2014; Wang et al. 2012]. Dynamics-aware optimization methods based on quadratic programming have also been applied to develop locomotion controllers [Da Silva et al. 2008; Lee et al. 2010a, 2014]. While model-based methods have been shown to be effective for a variety of skills, they tend to struggle with more dynamics motions that require long-term planning, as well as contact-rich motions. Trajectory optimization has been explored for synthesizing physically-plausible motions for a variety of tasks and characters [Mordatch et al. 2012; Wampler et al. 2014]. These methods synthesize motions over an extended time horizon using an offline optimization process, where the equations of motion are enforced as constraints. Recent work has extended such techniques into online model-predictive control methods [Hämäläinen et al. 2015; Tassa et al. 2012], although they remain limited in both motion quality and capacity for long-term planning. The principal advantage of our method over the above approaches is that of generality. We demonstrate that a single model-free framework is capable of a wider range of motion skills (from walks to highly dynamic kicks and flips) and an ability to sequence these; the ability to combine motion imitation and task-related demands; compact and fast-to-compute control policies; and the ability to leverage rich high-dimensional state and environment descriptions.

Reinforcement Learning: Many of the optimization techniques used to develop controllers for simulated characters are based on reinforcement learning. Value iteration methods have been used to develop kinematic controllers to sequence motion clips in the context of a given task [Lee et al. 2010b; Levine et al. 2012]. Similar approaches have been explored for simulated characters [Coros et al. 2009; Peng et al. 2015]. More recently, the introduction of deep neural network models for RL has given rise to simulated agents that can perform a diverse array of challenging tasks [Brockman et al. 2016a; Duan et al. 2016; Liu and Hodgins 2017; Peng et al. 2016; Rajeswaran

et al. 2017; Teh et al. 2017]. Policy gradient methods have emerged as the algorithms of choice for many continuous control problems [Schulman et al. 2015a, 2017; Sutton and Barto 1998]. Although RL algorithms have been capable of synthesizing controllers using minimal task-specific control structures, the resulting behaviors generally appear less natural than their more manually engineered counterparts [Merel et al. 2017; Schulman et al. 2015b]. Part of the challenge stems from the difficulty in specifying reward functions for natural movement, particularly in the absence of biomechanical models and objectives that can be used to achieve natural simulated locomotion [Lee et al. 2014; Wang et al. 2012]. Naïve objectives for torque-actuated locomotion, such as forward progress or maintaining a desired velocity, often produce gaits that exhibit extraneous motion of the limbs, asymmetric gaits, and other objectionable artifacts. To mitigate these artifacts, additional objectives such as effort or impact penalties have been used to discourage these undesirable behaviors. Crafting such objective functions requires a substantial degree of human insight, and often yields only modest improvements. Alternatively, recent RL methods based on the imitation of motion capture, such as GAIL [Ho and Ermon 2016], address the challenge of designing a reward function by using data to induce an objective. While this has been shown to improve the quality of the generated motions, current results still do not compare favorably to standard methods in computer animation [Merel et al. 2017]. The DeepLoco system [Peng et al. 2017a] takes an approach similar to the one we use here, namely adding an imitation term to the reward function, although with significant limitations. It uses fixed initial states and is thus not capable of highly dynamic motions; it is demonstrated only on locomotion tasks defined by foot-placement goals computed by a high-level controller; and it is applied to a single armless biped model. Lastly, the multi-clip demonstration involves a hand-crafted procedure for selecting suitable target clips for turning motions.

Motion Imitation: Imitation of reference motions has a long history in computer animation. An early instantiation of this idea was in bipedal locomotion with planar characters [Sharon and van de Panne 2005; Sok et al. 2007], using controllers tuned through policy search. Model-based methods for tracking reference motions have also been demonstrated for locomotion with 3D humanoid characters [Lee et al. 2010a; Muico et al. 2009; Yin et al. 2007]. Reference motions have also been used to shape the reward function for deep RL to produce more natural locomotion gaits [Peng et al. 2017a,b] and for flapping flight [Won et al. 2017]. In our work, we demonstrate the capability to perform a significantly broader range of difficult motions: highly dynamic spins, kicks, and flips with intermittent ground contact, and we show that reference-state initialization and early termination are critical to their success. We also explore several options for multi-clip integration and skill sequencing.

The work most reminiscent of ours in terms of capabilities is the Sampling-Based Controller (SAMCON) [Liu et al. 2016, 2010]. An impressive array of skills has been reproduced by SAMCON, and to the best of our knowledge, SAMCON has been the only system to demonstrate such a diverse corpus of highly dynamic and acrobatic motions with simulated characters. However, the system is complex, having many components and iterative steps, and requires defining

a low dimensional state representation for the synthesized linear feedback structures. The resulting controllers excel at mimicking the original reference motions, but it is not clear how to extend the method for task objectives, particularly if they involve significant sensory input. A more recent variation introduces deep Q-learning to train a high-level policy that selects from a precomputed collection of SAMCON control fragments [Liu and Hodgins 2017]. This provides flexibility in the order of execution of the control fragments, and is demonstrated to be capable of challenging non-terminating tasks, such as balancing on a bongo-board and walking on a ball. In this work, we propose an alternative framework using deep RL, that is conceptually much simpler than SAMCON, but is nonetheless able to learn highly dynamic and acrobatic skills, including those having task objectives and multiple clips.

3 OVERVIEW

Our system receives as input a character model, a corresponding set of kinematic reference motions, and a task defined by a reward function. It then synthesizes a controller that enables the character to imitate the reference motions, while also satisfying task objectives, such as striking a target or running in a desired direction over irregular terrain. Each reference motion is represented as a sequence of target poses $\{\hat{q}_t\}$. A control policy $\pi(a_t|s_t, g_t)$ maps the state of the character s_t , a task-specific goal g_t to an action a_t , which is then used to compute torques to be applied to each of the character's joints. Each action specifies target angles for proportional-derivative (PD) controllers that then produce the final torques applied at the joints. The reference motions are used to define an imitation reward $r^I(s_t, a_t)$, and the goal defines a task-specific reward $r^G(s_t, a_t, g_t)$. The final result of our system is a policy that enables a simulated character to imitate the behaviours from the reference motions while also fulfilling the specified task objectives. The policies are modeled using neural networks and trained using the proximal policy optimization algorithm [Schulman et al. 2017].

4 BACKGROUND

Our tasks will be structured as standard reinforcement learning problems, where an agent interacts with an environment according to a policy in order to maximize a reward. In the interest of brevity, we will exclude the goal g , from the notation, but the following discussion readily generalizes to include this. A policy $\pi(a|s)$ models the conditional distribution over action $a \in A$ given a state $s \in S$. At each control timestep, the agent observes the current state s_t and samples an action a_t from π . The environment then responds with a new state $s' = s_{t+1}$, sampled from the dynamics $p(s'|s, a)$, and a scalar reward r_t that reflects the desirability of the transition. For a parametric policy $\pi_\theta(a|s)$, the goal of the agent is to learn the optimal parameters θ^* that maximizes its expected return

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

where $p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} p(s_{t+1}|s_t, a_t) \pi_\theta(a_t|s_t)$ is the distribution over all possible trajectories $\tau = (s_0, a_0, s_1, \dots, a_{T-1}, s_T)$ induced by the policy π_θ , with $p(s_0)$ being the initial state distribution. $\sum_{t=0}^T \gamma^t r_t$ represents the total return of a trajectory, with a horizon

of T steps. T may or may not be infinite, and $\gamma \in [0, 1]$ is a discount factor that can be used to ensure the return is finite. A popular class of algorithms for optimizing a parametric policy is policy gradient methods [Sutton et al. 2001], where the gradient of the expected return $\nabla_{\theta} J(\theta)$ is estimated with trajectories sampled by following the policy. The policy gradient can be estimated according to

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d_{\theta}(s_t), a_t \sim \pi_{\theta}(a_t|s_t)} [\nabla_{\theta} \log(\pi_{\theta}(a_t|s_t)) \mathcal{A}_t],$$

where $d_{\theta}(s_t)$ is the state distribution under the policy π_{θ} . $\mathcal{A}_t$ represents the advantage of taking an action a_t at a given state s_t

$$\mathcal{A}_t = R_t - V(s_t).$$

$R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ denotes the return received by a particular trajectory starting from state s_t at time t . $V(s_t)$ is a value function that estimates the average return of starting in s_t and following the policy for all subsequent steps

$$V(s_t) = \mathbb{E}[R_t | \pi_{\theta}, s_t].$$

The policy gradient can therefore be interpreted as increasing the likelihood of actions that lead to higher than expected returns, while decreasing the likelihood of actions that lead to lower than expected returns. A classic policy gradient algorithm for learning a policy using this empirical gradient estimator to perform gradient ascent on $J(\theta)$ is REINFORCE [Williams 1992].

Our policies will be trained using the proximal policy optimization algorithm [Schulman et al. 2017], which has demonstrated state-of-the-art results on a number of challenging control problems. The value function will be trained using multi-step returns with TD(λ). The advantages for the policy gradient will be computed using the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]. A more in-depth review of these methods can be found in the supplementary material.

5 POLICY REPRESENTATION

Given a reference motion clip, represented by a sequence of target poses $\{\hat{q}_t\}$, the goal of the policy is to reproduce the desired motion in a physically simulated environment, while also satisfying additional task objectives. Since a reference motion only provides kinematic information in the form of target poses, the policy is responsible for determining which actions should be applied at each timestep in order to realize the desired trajectory.

5.1 States and Actions

The state s describes the configuration of the character's body, with features consisting of the relative positions of each link with respect to the root (designated to be the pelvis), their rotations expressed in quaternions, and their linear and angular velocities. All features are computed in the character's local coordinate frame, with the root at the origin and the x-axis along the root link's facing direction. Since the target poses from the reference motions vary with time, a phase variable $\phi \in [0, 1]$ is also included among the state features. $\phi = 0$ denotes the start of a motion, and $\phi = 1$ denotes the end. For cyclic motions, ϕ is reset to 0 after the end of each cycle. Policies trained to achieve additional task objectives, such as walking in a particular direction or hitting a target, are also provided with a goal g , which can be treated in a similarly fashion as the state. Specific

goals used in the experiments are discussed in section 9. The action a from the policy specifies target orientations for PD controllers at each joint. The policy is queried at 30Hz, and target orientations for spherical joints are represented in axis-angle form, while targets for revolute joints are represented by scalar rotation angles. Unlike the standard benchmarks, which often operate directly on torques, our use of PD controllers abstracts away low-level control details such as local damping and local feedback. Compared to torques, PD controllers have been shown to improve performance and learning speed for certain motion control tasks [Peng et al. 2017b].

5.2 Network

Each policy π is represented by a neural network that maps a given state s and goal g to a distribution over action $\pi(a|s, g)$. The action distribution is modeled as a Gaussian, with a state dependent mean $\mu(s)$ specified by the network, and a fixed diagonal covariance matrix Σ that is treated as a hyperparameter of the algorithm:

$$\pi(a|s) = \mathcal{N}(\mu(s), \Sigma).$$

The inputs are processed by two fully-connected layers with 1024, and 512 units each, followed by a linear output layer. ReLU activations are used for all hidden units. The value function is modeled by a similar network, with exception of the output layer, which consists of a single linear unit.

For vision-based tasks, discussed in section 9, the inputs are augmented with a heightmap H of the surrounding terrain, sampled on a uniform grid around the character. The policy and value networks are augmented accordingly with convolutional layers to process the heightmap. A schematic illustration of this visuomotor policy network is shown in Figure 2. The heightmap is first processed by a series of convolutional layers, followed by a fully-connected layer. The resulting features are then concatenated with the input state s and goal g , and processed by a similar fully-connected network as the one used for tasks that do not require vision.

5.3 Reward

The reward r_t at each step t consists of two terms that encourage the character to match the reference motion while also satisfying

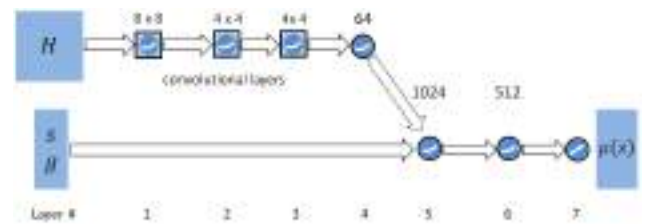


Fig. 2. Schematic illustration of the visuomotor policy network. The heightmap H is processed by 3 convolutional layers with 16 8x8 filters, 32 4x4 filters, and 32 4x4 filters. The feature maps are then processed by 64 fully-connected units. The resulting features are concatenated with the input state s and goal g and processed by two fully-connected layer with 1024 and 512 units. The output $\mu(s)$ is produced by a layer of linear units. ReLU activations are used for all hidden layers. For tasks that do not require a heightmap, the networks consist only of layers 5-7.

additional task objectives:

$$r_t = \omega^I r_t^I + \omega^G r_t^G.$$

Here, r_t^I and r_t^G represent the imitation and task objectives, with ω^I and ω^G being their respective weights. The task objective r_t^G incentivizes the character to fulfill task-specific objectives, the details of which will be discussed in the following section. The imitation objective r_t^I encourages the character to follow a given reference motion $\{\hat{q}_t\}$. It is further decomposed into terms that reward the character for matching certain characteristics of the reference motion, such as joint orientations and velocities, as follows:

$$r_t^I = w^p r_t^p + w^v r_t^v + w^e r_t^e + w^c r_t^c$$

$$w^p = 0.65, w^v = 0.1, w^e = 0.15, w^c = 0.1$$

The pose reward r_t^p encourages the character to match the joint orientations of the reference motion at each step, and is computed as the difference between the joint orientation quaternions of the simulated character and those of the reference motion. In the equation below, q_t^j and $\hat{q}_t^j$ represent the orientations of the j th joint from the simulated character and reference motion respectively, $q_1 \ominus q_2$ denotes the quaternion difference, and $\|q\|$ computes the scalar rotation of a quaternion about its axis in radians:

$$r_t^p = \exp \left[-2 \left(\sum_j \|\hat{q}_t^j \ominus q_t^j\|^2 \right) \right].$$

The velocity reward r_t^v is computed from the difference of local joint velocities, with $\dot{q}_t^j$ being the angular velocity of the j th joint. The target velocity $\hat{\dot{q}}_t^j$ is computed from the data via finite difference.

$$r_t^v = \exp \left[-0.1 \left(\sum_j \|\hat{\dot{q}}_t^j - \dot{q}_t^j\|^2 \right) \right].$$

The end-effector reward r_t^e encourages the character's hands and feet to match the positions from the reference motion. Here, p_t^e denotes the 3D world position in meters of end-effector $e \in [\text{left foot, right foot, left hand, right hand}]$:

$$r_t^e = \exp \left[-40 \left(\sum_e \|\hat{p}_t^e - p_t^e\|^2 \right) \right].$$

Finally, r_t^c penalizes deviations in the character's center-of-mass p_t^c from that of the reference motion $\hat{p}_t^c$:

$$r_t^c = \exp \left[-10 \left(\|\hat{p}_t^c - p_t^c\|^2 \right) \right].$$

6 TRAINING

Our policies are trained with PPO using the clipped surrogate objective [Schulman et al. 2017]. We maintain two networks, one for the policy $\pi_\theta(a|s, g)$ and another for the value function $V_\psi(s, g)$, with parameters θ and ψ respectively. Training proceeds episodically, where at the start of each episode, an initial state s_0 is sampled uniformly from the reference motion (section 6.1), and rollouts are generated by sampling actions from the policy at every step. Each episode is simulated to a fixed time horizon or until a termination condition has been triggered (section 6.2). Once a batch of data

has been collected, minibatches are sampled from the dataset and used to update the policy and value function. The value function is updated using target values computed with TD(λ) [Sutton and Barto 1998]. The policy is updated using gradients computed from the surrogate objective, with advantages $\mathcal{A}_t$ computed using GAE(λ) [Schulman et al. 2015b]. Please refer to the supplementary material for a more detailed summary of the learning algorithm.

One of the persistent challenges in RL is the problem of exploration. Since most formulations assume an unknown MDP, the agent is required to use its interactions with the environment to infer the structure of the MDP and discover high value states that it should endeavor to reach. A number of algorithmic improvements have been proposed to improve exploration, such as using metrics for novelty or information gain [Bellemare et al. 2016; Fu et al. 2017; Houthoofd et al. 2016]. However, less attention has been placed on the structure of the episodes during training and their potential as a mechanism to guide exploration. In the following sections, we consider two design decisions, the initial state distribution and the termination condition, which have often been treated as fixed properties of a given RL problem. We will show that appropriate choices are crucial for allowing our method to learn challenging skills such as highly-dynamic kicks, spins, and flips. With common default choices, such as a fixed initial state and fixed-length episodes, we find that imitation of these difficult motions is often unsuccessful.

6.1 Initial State Distribution

The initial state distribution $p(s_0)$ determines the states in which an agent begins each episode. A common choice for $p(s_0)$ is to always place the agent in a fixed state. However, consider the task of imitating a desired motion. A simple strategy is to initialize the character to the starting state of the motion, and allow it to proceed towards the end of the motion over the course of an episode. With this design, the policy must learn the motion in a sequential manner, by first learning the early phases of the motion, and then incrementally progressing towards the later phases. Before mastering the earlier phases, little progress can be made on the later phases. This can be problematic for motions such as backflips, where learning the landing is a prerequisite for the character to receive a high return from the jump itself. If the policy cannot land successfully, jumping will actually result in *worse* returns. Another disadvantage of a fixed initial state is the resulting exploration challenge. The policy only receives reward retrospectively, once it has visited a state. Therefore, until a high-reward state has been visited, the policy has no way of learning that this state is favorable. Both disadvantages can be mitigated by modifying the initial state distribution.

For many RL tasks, a fixed initial state can be more convenient, since it can be challenging to initialize the agent in other states (e.g., physical robots) or obtain a richer initial state distribution. For motion imitation tasks, however, the reference motion provides a rich and informative state distribution, that can be leveraged to guide the agent during training. At the start of each episode, a state can be sampled from the reference motion, and used to initialize the state of the agent. We will refer to this strategy as *reference state initialization* (RSI). Similar strategies have been previously used for planar bipedal walking [Sharon and van de Panne 2005] and manipulation [Nair et al. 2017; Rajeswaran et al. 2017]. By sampling initial



Fig. 3. 3D simulated characters. Our framework is able to train policies for a wide range of character morphologies.

states from the reference motion, the agent encounters desirable states along the motion, even before the policy has acquired the proficiency needed to reach those states. For example, consider the challenge of learning to perform a backflip. With a fixed initial state, in order for the character to discover that performing a full rotation mid-air will result in high returns, it must first learn to perform a carefully coordinated jump. However, for the character to be motivated to perform such a jump, it must be aware that the jump will lead to states that yield higher rewards. Since the motion is highly sensitive to the initial conditions at takeoff, many strategies will result in failure. Thus the agent is unlikely to encounter states from a successful flip, and never discover such high reward states. With RSI, the agent immediately encounters such promising states during the early stages of training. Instead of accessing information from the reference motion only through the reward function, RSI can be interpreted as an additional channel through which the agent can access information from the reference motion in the form of a more informative initial state distribution.

6.2 Early Termination

For cyclic skills, the task can be modeled as an infinite horizon MDP. But during training, each episode is simulated for a finite horizon. An episode terminates either after a fixed period of time, or when certain termination conditions have been triggered. For locomotion, a common condition for early termination (ET) is the detection of a fall, characterized by the character's torso making contact with the ground [Peng et al. 2016] or certain links falling below a height threshold [Heess et al. 2016]. While these strategies are prevalent, they are often mentioned in passing and their impact on performance has not been well evaluated. In this work, we will use a similar termination condition as [Peng et al. 2016], where an episode is terminated whenever certain links, such as the torso or head, makes contact with the ground. Once early termination has been triggered, the character is left with zero reward for the remainder of the episode. This instantiation of early termination provides another means of shaping the reward function to discourage undesirable behaviors. Another advantages of early termination is that it can function as a curating mechanism that biases the data distribution in favor of samples that may be more relevant for a task. In the case of skills such as walking and flips, once the character has fallen, it can be challenging for it to recover and return to its nominal trajectory.

Table 1. Properties of the characters.

Property	Humanoid	Atlas	T-Rex	Dragon
Links	13	12	20	32
Total Mass (kg)	45	169.8	54.5	72.5
Height (m)	1.62	1.82	1.66	1.83
Degrees of Freedom	34	31	55	79
State Features	197	184	262	418
Action Parameters	36	32	64	94

Without early termination, data collected during the early stages of training will be dominated by samples of the character struggling on the ground in vain, and much of the capacity of the network will be devoted to modeling such futile states. This phenomena is analogous to the class imbalance problem encountered by other methodologies such as supervised learning. By terminating the episode whenever such failure states are encountered, this imbalance can be mitigated.

7 MULTI-SKILL INTEGRATION

The discussion thus far has been focused on imitating individual motion clips. But the ability to compose and sequence multiple clips is vital for performing more complex tasks. In this section, we propose several methods for doing this, which are each suited for different applications. First, we need not be restricted to a single reference clip to define a desired motion style, and can instead choose to use a richer and more flexible *multi-clip* reward. Second, we can further provide the user with control over which behavior to trigger, by training a *skill selector* policy that takes in a user-specified one-hot clip-selection input. Third, we can avoid training new policies for every clip combination by instead constructing a *composite policy* out of existing single-clip policies. In this setup, multiple policies are learned independently and, at runtime, their value functions are used to determine which policy should be activated.

Multi-Clip Reward: To utilize multiple reference motion clips during training, we define a composite imitation objective calculated simply as the max over the previously introduced imitation objective applied to each of the k motion clips:

$$r_t^I = \max_{j=1, \dots, k} r_t^j,$$

where r_t^j is the imitation objective with respect to the j th clip. We will show that this simple composite objective is sufficient to integrate multiple clips into the learning process. Unlike [Peng et al. 2017a], which required a manually crafted kinematic planner to select a clip for each walking step, our objective provides the policy with the flexibility to select the most appropriately clip for a given situation, and the ability to switch between clips whenever appropriate, without the need to design a kinematic planner.

Skill Selector: Besides simply providing the policy with multiple clips to use as needed to accomplish a goal, we can also provide the user with control over which clip to use at any given time. In this approach, we can train a policy that simultaneously learns to imitate a set of diverse skills and, once trained, is able to execute arbitrary sequences of skills on demand. The policy is provided with a goal g_t represented by a one-hot vector, where each entry $g_{t,i}$ corresponds to the motion that should be executed. The character's goal then is to perform the motion corresponding to the nonzero entry of g_t . There is no additional task objective r_t^G , and the character is trained only to optimize the imitation objective r_t^I , which is computed based on the currently selected motion $r_t^I = r_t^i$, where $g_{t,i} = 1$ and $g_{t,j} = 0$ for $j \neq i$. During training, a random g_t is sampled at the start of each cycle. The policy is therefore required to learn to transition between all skills within the set of clips.

Composite Policy: The previously described methods both learn a single policy for a collection of clips. But requiring a network to learn multiple skills jointly can be challenging as the number of skills increases, and can result in the policy failing to learn any of the skills adequately. An alternative is to adopt a divide-and-conquer strategy, where separate policies are trained to perform different skills, and then integrated together into a composite policy. Since the value function provides an estimate of a policy's expected performance in a particular state, the value functions can be leveraged to determine the most appropriate skill to execute in a given state. Given a set of policies and their value functions $\{V^i(s), \pi^i(a|s)\}_{i=1}^k$, a composite policy $\Pi(a|s)$ can be constructed using a Boltzmann distribution

$$\Pi(a|s) = \sum_{i=1}^k p^i(s) \pi^i(a|s), \quad p^i(s) = \frac{\exp[V^i(s)/\mathcal{T}]}{\sum_{j=1}^k \exp[V^j(s)/\mathcal{T}]},$$

where $\mathcal{T}$ is a temperature parameter. Policies with larger expected values at a given state will therefore be more likely to be selected. By repeatedly sampling from the composite policy, the character is then able to perform sequences of skills from a library of diverse motions without requiring any additional training. The composite policy resembles the mixture of actor-critics experts model (MACE) proposed by [Peng et al. 2016], although it is even simpler as each sub-policy is trained independently for a specific skill.

8 CHARACTERS

Our characters include a 3D humanoid, an Atlas robot model, a T-Rex, and a dragon. Illustrations of the characters are available in Figure 3, and Table 1 details the properties of each character. All characters are modeled as articulated rigid bodies, with each link attached to its parent link via a 3 degree-of-freedom spherical joint, except for the knees and elbows, which are attached via 1

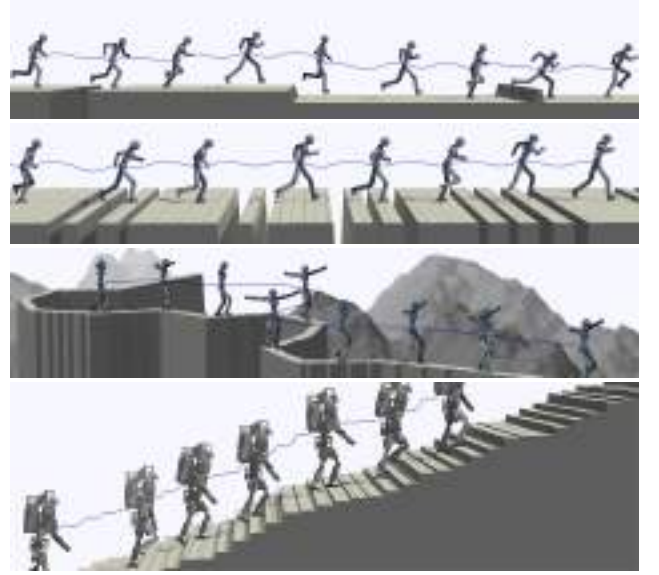


Fig. 4. Characters traversing randomly generated terrains. **Top-to-bottom:** mixed obstacles, dense gaps, winding balance beam, stairs. The blue line traces the trajectory of the character's center-of-mass.

degree-of-freedom revolute joints. PD controllers are positioned at each joint, with manually specified gains that are kept constant across all tasks. Both the humanoid and Atlas share similar body structures, but their morphology (e.g., mass distribution) and actuators (e.g., PD gains and torque limits) differ significantly, with the Atlas being almost four times the mass of the humanoid. The T-Rex and dragon provide examples of learning behaviors for characters from keyframed animation when no mocap data is available, and illustrate that our method can be readily applied to non-bipedal characters. The humanoid character has a 197D state space and a 36D action space. Our most complex character, the dragon, has a 418D state space and 94D action space. Compared to standard continuous control benchmarks for RL [Brockman et al. 2016b], which typically have action spaces varying between 3D to 17D, our characters have significantly higher-dimensional action spaces.

9 TASKS

In addition to imitating a set of motion clips, the policies can be trained to perform a variety of tasks while preserving the style prescribed by the reference motions. The task-specific behaviors are encoded into the task objective r_t^G . We describe the tasks evaluated in our experiments below.

Target Heading: Steerable controllers can be trained by introducing an objective that encourages the character to travel in a target direction d_t^* , represented as a 2D unit vector in the horizontal plane. The reward for this task is given by

$$r_t^G = \exp \left[-2.5 \max(0, v^* - v_t^T d_t^*)^2 \right],$$

where v^* specifies the desired speed along the target direction d_t^* , and v_t represents the center-of-mass velocity of the simulated character. The objective therefore penalizes the character for traveling

Table 2. Performance statistics of imitating various skills. All skills are performed by the humanoid unless stated otherwise. Policies are trained only to imitate a reference motion without additional task objectives. T_{cycle} is the length of the clip. $N_{samples}$ specifies the number of samples collected to train the final policy. NR represents the normalized return of the final policy averaged over 32 episodes, with 0 being the minimum possible return per episode, and 1 the maximum return. For cyclic skills, each episode has a horizon of 20s. For acyclic skills, the horizon is specified by T_{cycle} .

Skill	T_{cycle} (s)	$N_{samples}$ (10^6)	NR
Backflip	1.75	72	0.729
Balance Beam	0.73	96	0.783
Baseball Pitch	2.47	57	0.785
Cartwheel	2.72	51	0.804
Crawl	2.93	68	0.932
Dance A	1.62	67	0.863
Dance B	2.53	79	0.822
Frontflip	1.65	81	0.485
Getup Face-Down	3.28	49	0.885
Getup Face-Up	4.02	66	0.838
Headspin	1.92	112	0.640
Jog	0.80	51	0.951
Jump	1.77	86	0.947
Kick	1.53	50	0.854
Landing	2.83	66	0.590
Punch	2.13	60	0.812
Roll	2.02	81	0.735
Run	0.80	53	0.951
Sideflip	2.44	64	0.805
Spin	4.42	191	0.664
Spinkick	1.28	67	0.748
Vault 1-Handed	1.53	41	0.695
Vault 2-Handed	1.90	87	0.757
Walk	1.26	61	0.985
Atlas: Backflip	1.75	63	0.630
Atlas: Run	0.80	48	0.846
Atlas: Spinkick	1.28	66	0.477
Atlas: Walk	1.26	44	0.988
T-Rex: Walk	2.00	140	0.979
Dragon: Walk	1.50	139	0.990

slower than the desired speed along the target direction, but does not penalize it for exceeding the requested speed. The target direction is provided as the input goal $g_t = d_t^*$ to the policy. During training, the target direction is randomly varied throughout an episode. At runtime, d_t^* can be manually specified to steer the character.

Strike: In this task, the character's goal is to strike a randomly placed spherical target using specific links, such as the feet. The reward is given by

$$r_t^G = \begin{cases} 1, & \text{target has been hit} \\ \exp[-4||p_t^{tar} - p_t^e||^2], & \text{otherwise} \end{cases}$$

p_t^{tar} specifies the location of the target, and p_t^e represents the position of the link used to hit the target. The target is marked as being hit if the center of the link is within 0.2m of the target location. The goal $g_t = (p_t^{tar}, h)$ consists of the target location p_t^{tar}

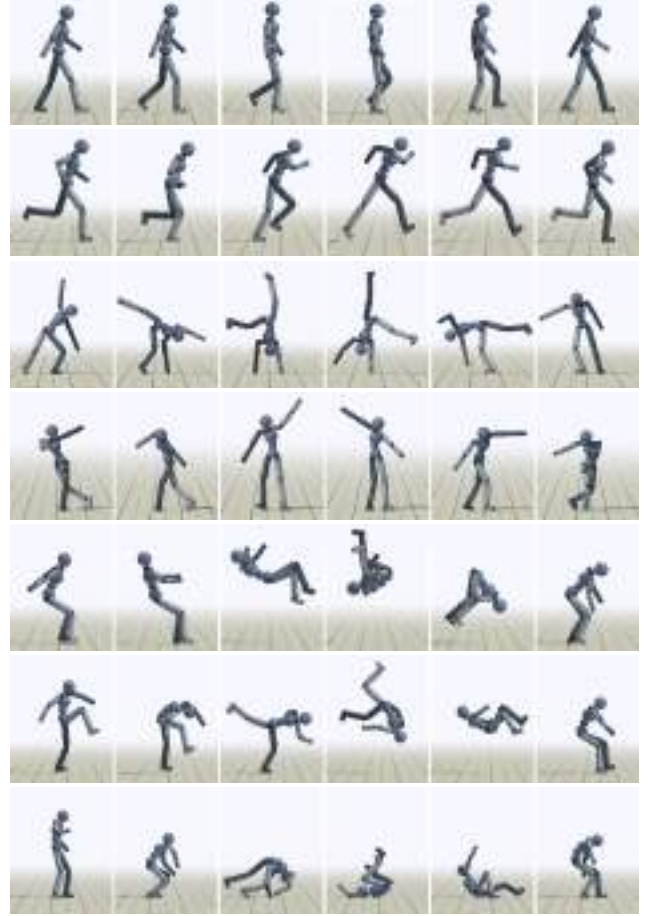


Fig. 5. Snapshots of motions from the trained policies. **Top-to-bottom:** walk, run, cartwheel, dance A, backflip, frontflip, roll.

and a binary variable h that indicates if the target has been hit in a previous timestep. As we are using feedforward networks for all policies, h acts as a memory for the state of the target. The target is randomly placed within a distance of $[0.6, 0.8]m$ from the character, the height is sampled randomly between between $[0.8, 1.25]m$, and the initial direction from the character to the target varies by $2rad$. The target location and h are reset at the start of each cycle. The memory state h can be removed by training a recurrent policy, but our simple solution avoids the complexities of training recurrent networks while still attaining good performance.

Throw: This task is a variant of the strike task, where instead of hitting a target with one of the character's links, the character is tasked with throwing a ball to the target. At the start of an episode, the ball is attached to the character's hand via a spherical joint. The joint is released at a fixed point in time during the episode. The goal g_t and reward r_t^G is the same as the strike task, but the character state s_t is augmented with the position, rotation, linear and angular velocity of the ball. The distance of the target varies between $[2.5, 3.5]m$, with height between $[1, 1.25]m$, and direction between $[0.7, 0.9]rad$.

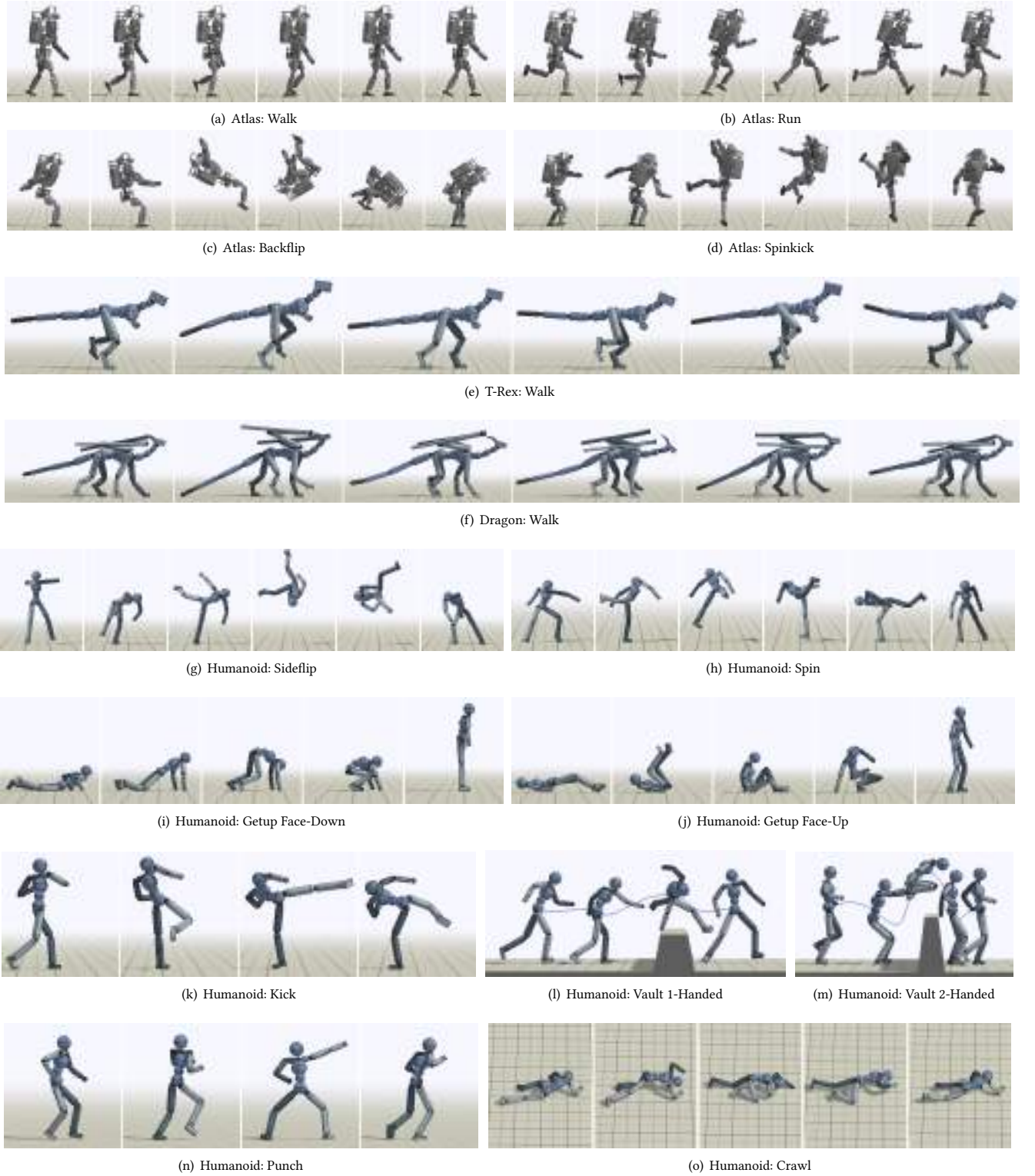


Fig. 6. Simulated characters performing various skills. Our framework is able to train policies for a broad range of characters, skills, and environments.



Fig. 7. **Top:** Spinkick policy trained to strike a target with the character's right foot. **Bottom:** Baseball pitch policy trained to throw a ball to a target.

Terrain Traversal: In this task, the character is trained to traverse obstacle-filled environments. The goal g_t and task-objective r_t^G are similar to those of the target heading task, except the target heading is fixed along the direction of forward progress.

We consider four environments consisting of mixed obstacles, dense gaps, a winding balance beam, and stairs. Figure 4 illustrates examples of the environments. The mixed obstacles environment is composed of gap, step, and wall obstacles similar to those presented in [Peng et al. 2016]. Each gap has a width between $[0.2, 1]m$, the height of each wall varies between $[0.25, 0.4]m$, and each step has a height between $[0.35, -0.35]m$. The obstacles are interleaved with flat stretches of terrain between $[5, 8]m$ in length. The next environment consists of sequences of densely packed gaps, where each sequence consists of 1 to 4 gaps. The gaps are $[0.1, 0.3]m$ in width, with $[0.2, 0.4]m$ of separation between adjacent gaps. Sequences of gaps are separated by $[1, 2]m$ of flat terrain. The winding balance beam environment sports a narrow winding path carved into irregular terrain. The width of the path is approximately $0.4m$. Finally, we constructed a stairs environment, where the character is to climb up irregular steps with height varying between $[0.01, 0.2]m$ and a depth of $0.28m$.

To facilitate faster training, we adopt a progressive learning approach, where the standard fully-connected networks (i.e., without the input heightmap and convolutional layers) are first trained to imitate their respective motions on flat terrain. Next, the networks are augmented with an input heightmap and corresponding convolutional layers, then trained in the irregular environments. Since the mixed obstacles and dense gaps environments follow a linear layout, the heightmaps are represented by a 1D heightfield with 100 samples spanning $10m$. In the winding balance beam environment, a 32×32 heightmap is used, covering a $3.5 \times 3.5m$ area.

10 RESULTS

The motions from the trained policies are best seen in the supplemental videos. Snapshots of the skills performed by the simulated characters are available in Figure 5, 6, and 7. The policies are executed at 30Hz. Physics simulation is performed at 1.2kHz using the Bullet physics engine [Bullet 2015]. All neural networks are built and trained using TensorFlow. The characters' motions are driven by torques computed using stable PD controllers [Tan et al. 2011]. Results for the humanoid are demonstrated for a large collection of locomotion, acrobatic, and martial arts skills, while the results for the dragon and T-Rex are demonstrated for locomotion. Each skill

Table 3. Performance statistics of imitating motion clips while also fulfilling additional task objectives.

Environment	$N_{samples} (10^6)$	NR
Humanoid: Walk - Target Heading	85	0.911
Humanoid: Jog - Target Heading	108	0.876
Humanoid: Run - Target Heading	40	0.637
Humanoid: Spinkick - Strike	85	0.601
Humanoid: Baseball Pitch - Throw	221	0.675
Humanoid: Run - Mixed Obstacles	466	0.285
Humanoid: Run - Dense Gaps	265	0.650
Humanoid: Winding Balance Beam	124	0.439
Atlas: Walk - Stairs	174	0.808

is learned from approximately 0.5-5s of mocap data collected from <http://mocap.cs.cmu.edu> and <http://mocap.cs.sfu.ca>. For characters such as the T-Rex and dragon, where mocap data is not available, we demonstrate that our framework is also capable of learning skills from artist-authored keyframes. Before being used for training, the clips are manually processed and retargeted to their respective characters. A comprehensive list of the learned skills and performance statistics is available in Table 2. Learning curves for all policies are available in the supplemental material. Each environment is denoted by "Character: Skill - Task". The task is left unspecified for policies that are trained solely to imitate a reference motion without additional task objectives. Performance is measured by the average return normalized by the minimum and maximum possible return per episode. Note that the maximum return may not be achievable. For example, for the throwing task, the maximum return requires moving the ball instantaneously to the target. When evaluating the performance of the policies, early termination is applied and the state of the character at the start of each episode is initialized via RSI. The weights for the imitation and task objectives are set to $\omega^I = 0.7$ and $\omega^G = 0.3$ for all tasks. More details regarding hyperparameter settings are available in the supplemental material.

By encouraging the policies to imitate motion capture data from human subjects, our system is able to learn policies for a rich repertoire of skills. For locomotion skills such as walking and running, our policies produce natural gaits that avoid many of the artifacts exhibited by previous deep RL methods [Merel et al. 2017; Schulman et al. 2015b]. The humanoid is able to learn a variety of acrobatic skills with long flight phases, such as backflips and spinkicks, which are particularly challenging since the character needs to learn to coordinate its motion in mid-air. The system is also able to reproduce contact-rich motions, such as crawling and rolling, as well as motions that require coordinated interaction with the environment, such as the vaulting skills shown in Figure 6. The learned policies are robust to significant external perturbation and generate plausible recovery behaviors. The policies trained for the T-Rex and dragon demonstrate that the system can also learn from artist generated keyframes when mocap is not available and scale to much more complex characters than those that have been demonstrated by previous work using deep RL.

10.1 Tasks

In addition to imitating reference motions, the policies can also adapt the motions as needed to satisfy additional task objectives,



Fig. 8. Policy trained for the throw task without a reference motion. Instead of throwing the ball, the character learns to run towards the target.

Table 4. Success rate of policies trained with the imitation or task objectives disabled. Each policy is evaluated over 100 trials. Simply imitating the reference motions proves insufficient for fulfilling the task objectives. Training without a reference motion produces policies that develop awkward, but functional, strategies for satisfying the task objectives.

Environment	$r^I + r^G$	r^I	r^G
Humanoid: Strike - Spinkick	99%	19%	55%
Humanoid: Baseball Pitch - Throw	75%	5%	93%

such as following a target heading and throwing a ball to a randomly placed target. Performance statistics for each task are available in Table 3. To investigate the extent to which the motions are adapted for a particular task, we compared the performance of policies trained to optimize both the imitation objective r^I and the task objective r^G to policies trained only with the imitation objective. Table 4 summarizes the success rates of the different policies. For the throwing task, the policy trained with both objectives is able to hit the target with a success rate of 75%, while the policy trained only to imitate the baseball pitch motion is successful only in 5% of the trials. Similarly, for the strike task, the policy trained with both objectives successfully hits 99% of the targets, while the policy trained only to imitate the reference motion has a success rate of 19%. These results suggest that simply imitating the reference motions is not sufficient to successfully perform the tasks. The policies trained with the task objective are able to deviate from the original reference motion and developing additional strategies to satisfy their respective goals. We further trained policies to optimize only the task objective, without imitating a reference motion. The resulting policies are able to fulfill the task objectives, but without a reference motion, the policies develop unnatural behaviors, similar to those produced by prior deep RL methods. For the throw task, instead of throwing the ball, the policy adopts an awkward but functional strategy of running towards the target with the ball. Figure 8 illustrates this behavior.

10.2 Multi-Skill Integration

Our method can construct policies for reproducing a wide range of individual motion clips. However, many more complex skills require choosing from among a set of potential behaviors. In this section, we discuss several methods by which our approach can be extended to combine multiple clips into a single compound skill.

Multi-Clip Reward: To evaluate the multi-clip reward, we constructed an imitation objective from 5 different walking and turning clips. Using this reward, a humanoid policy is then trained to walk while following a desired heading. The resulting policy learns to utilize a variety of agile stepping behaviours in order to follow the target heading. To determine if the policy is indeed learning to imitate multiple clips, we recorded the ID of the clip that best matches the motion of the simulated character at each timestep as it follows

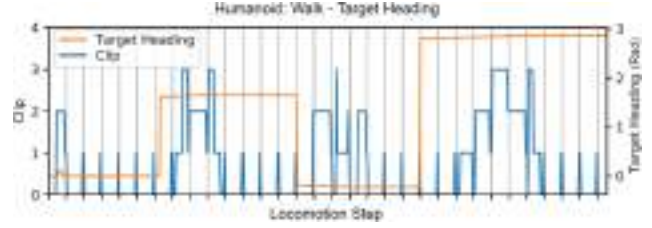


Fig. 9. Visualization of the walking reference motion clip that most closely matches the simulated character's motion as the character follows a changing target heading. Clip 0 is a forward walking motion, and clip 1-4 are turning motions. The policy is trained with the multi-clip reward.

a changing target heading. The best matching clip is designated as the clip that yields the maximum reward at the current timestep. Figure 9 records the best matching clips and the target heading over a 20s episode. Clip 0 corresponds to a forward walking motion, and clips 1-4 are different turning motions. When the target heading is constant, the character's motion primarily matches the forward walk. When the target heading changes, the character's motion becomes more closely matched with the turning motions. After the character has realigned with the target heading, the forward walking clip once again becomes the best match. This suggests that the multi-clip reward function does indeed enable the character to learn from multiple clips of different walking motions. When using the multi-clip reward, we found that the clips should generally be from a similar type of motion (e.g. different kinds of walks and turns). Mixing very different motions, like a sidelfip and a frontflip together, can result in the policy imitating only a subset of the clips. For more diverse clips, we found the composite policy to be a more effective integration method.

Skill Selector: Using the one-hot vector representation, we trained a policy to perform various flips, and another to jump in different directions. The flip policy is trained to perform a frontflip, backflip, left sidelfip, and right sidelfip. The jump policy is trained to perform a forward, backward, left, and right jump. The first and last frame of each clip are duplicated to ensure that all clips within each set have the same cycle period. Once trained, the policies are able to perform arbitrary sequences of skills from their respective repertoires. The one-hot vector also provides a convenient interface for users to direct the character in real-time. Footage of the humanoid executing sequences of skills specified by a user is available in the supplemental video.

Composite Policy: While the multi-clip objective can be effective for integrating multiple clips that correspond to the same category (e.g., walking), we found it to be less effective for integrating clips of more diverse skills. To integrate a more diverse corpus of skills, we constructed a composite policy from the backflip, frontflip, sidelfip, cartwheel, spinkick, roll, getup face-down, and getup face-up policies. The output of the value functions are normalized to be between $[-1, 1]$ and the temperature is set to $\mathcal{T} = 0.3$. A new skill is sampled from the composite policy at the start of each cycle, and the selected skill is executed for a full cycle before selecting a new skill. To prevent the character from repeatedly executing the same skill, the policy is restricted to never sample the same skill in consecutive



Fig. 10. **Left:** Original landing motion on flat terrain. **Right:** Policy trained to imitating landing motion while jumping down from a 2m ledge. Despite being provided only with a motion recorded on flat terrain, the policy is able to adapt the skill to jump down from a tall ledge.

cycles. Unlike the skill selector policies, the individual policies in the composite policy are never explicitly trained to transition between different skills. By using the value functions to guide the selection of skills, the character is nonetheless able to successfully transition between the various skills. When the character falls, the composite policy activates the appropriate getup policy without requiring any manual scripting, as shown in the supplemental video.

10.3 Retargeting

Due to modeling discrepancies between simulation and the real world, the dynamics under which a motion capture clip was recorded can differ dramatically from the dynamics of the simulated environments. Furthermore, keyframed motions may not be physically correct at all. To evaluate our framework's robustness to these discrepancies, we trained policies to perform similar skills with different character models, environments, and physics.

Character Retargeting: To demonstrate the system's capabilities in retargeting motions to different characters, we trained policies for walking, running, backflips and spinkicks on a simulated model of the Atlas robot from <http://www.mujoco.org/forum/index.php?resources/atlas-v5.16/>. The Atlas has a total mass of 169.8kg, almost four times the mass of the humanoid, as well as a different mass distribution. The serial 1D revolute joints in the original Atlas model are aggregated into 3D spherical joints to facilitate simpler retargeting of the reference motions. To retarget the motion clips, we simply copied the local joint rotations from the humanoid to the Atlas, without any further modification. New policies are then trained for the Atlas to imitate the retargeted clips. Despite the starkly different character morphologies, our system is able to train policies that successfully reproduce the various skills with the Atlas model. Performance statistics of the Atlas policies are available in Table 2. The performance achieved by the Atlas policies are comparable to those achieved by the humanoid. Qualitatively, the motions generated by the Atlas character closely resemble the reference clips. To further highlight the differences in the dynamics of the characters, we evaluated the performance of directly applying policies trained for the humanoid to the Atlas. The humanoid policies, when applied to the Atlas, fail to reproduce any of the skills, achieving a normalized return of 0.013 and 0.014 for the run and backflip, compared to 0.846

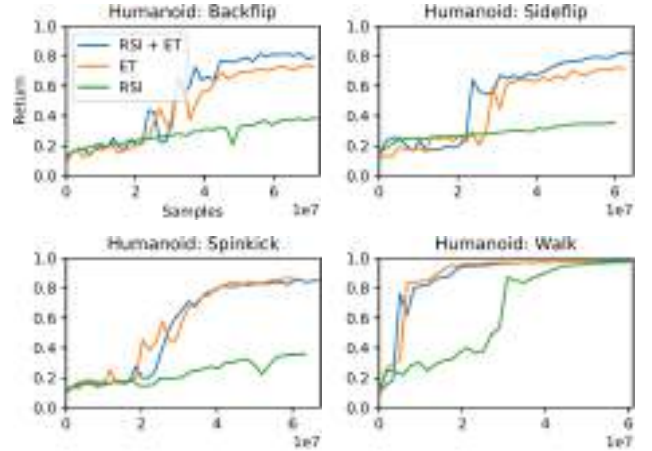


Fig. 11. Learning curves for policies trained with and without reference state initialization (RSI) and early termination (ET).

Table 5. Normalized average returns of policies trained with and without reference state initialization (RSI) and early termination (ET).

Skill	RSI + ET	ET	RSI
Backflip	0.791	0.730	0.379
Sideflip	0.823	0.717	0.355
Spinkick	0.848	0.858	0.358
Walk	0.980	0.981	0.974

and 0.630 achieved by policies that were trained specifically for the Atlas but using the same reference clips.

Environment Retargeting: While most of the reference motions were recorded on flat terrain, we show that the policies can be trained to adapt the motions to irregular environments. First, we consider the case of retargeting a landing motion, where the original reference motion is of a human subject jumping and landing on flat ground. From this reference motion, we trained a character to reproduce the motion while jumping down from a 2m ledge. Figure 10 illustrates the motion from the final policy. The system was able to adapt the reference motion to a new environment that is significantly different from that of the original clip.

Next, we explore vision-based locomotion in more complex procedurally generated environments. By augmenting the networks with a heightmap input, we are able to train the humanoid to run across terrains consisting of random obstacles. Examples of the environments are available in Figure 4. Over the course of training, the policies are able to adapt a single clip of a forward running motion into a variety of strategies for traversing across the different classes of obstacles. Furthermore, by training a policy to imitate a balance beam walk, the character learns to follow a narrow winding path using only a heightmap for pathfinding. The balance beam policy was trained with only a forward walking clip, but is nonetheless able to develop turning motions to follow the winding path. In addition to the humanoid, we also trained a policy for the Atlas to climb up stairs with irregular step heights. The policy was able to adapt the original walking clip on flat terrain to climb the steps, although the resulting motion still exhibits an awkward gait. We suspect that

the problem is partly related to the walking reference motion being ill-suited for the stairs environment.

Physics Retargeting: To further evaluate the framework’s robustness to discrepancies between the dynamics of the motion capture data and simulation, we trained policies to perform a spinkick and cartwheel under moon gravity ($1.622m/s^2$). Despite the difference in gravity, the policies were able to adapt the motions to the new dynamics, achieving a return of 0.792 for the spinkick and 0.688 for the cartwheel.

10.4 Ablations

To evaluate the impact of our design decisions, we compare our full method against alternative training schemes that disable some of the components. We found that the reference state initialization and early termination are two of the most important components of our training procedure. The comparisons include training with reference state initialization and with a fixed initial state, as well as training with early termination and without early termination. Without early termination, each episode is simulated for the full 20s. Figure 11 compares the learning curves with the different configurations and Table 5 summarizes the performance of the final policies. During evaluation, early termination is disabled and the character is initialized to a fixed state at the start of each episode. Due to the time needed to train each policy, the majority of performance statistics are collected from one run of the training process. However, we have observed that the behaviors are consistent across multiple runs. Early termination proves to be crucial for reproducing many of the skills. By heavily penalizing the character for making undesirable contacts with the ground, early termination helps to eliminate local optima, such as those where the character falls and mimes the motions as it lies on the ground. RSI also appears vital for more dynamic skills that have significant flight phases, such as the backflip. While the policies trained without RSI appear to achieve a similar return to those trained with RSI, an inspection of the resulting motions show that, without RSI, the character often fails to reproduce the desired behaviours. For the backflip, without RSI, the policy never learns to perform a full mid-air flip. Instead, it performs a small backwards hop while remaining upright.

10.5 Robustness

To determine the policies’ robustness to external perturbations, we subjected the trained policies to perturbation forces and recorded the maximum force the character can tolerate before falling. The perturbation forces are applied halfway through a motion cycle to the character’s pelvis for 0.2s. The magnitude of the force is increased by 10N until the character falls. This procedure is repeated for forces applied along the forward direction (x-axis) and sideways direction (z-axis). Table 6 summarizes the results from the experiments on the humanoid. The learned policies show comparable-or-better robustness than figures reported for SAMCON [Liu et al. 2016]. The run policy is able to recover from $720N \times 0.2s$ forward pushes, while the spin-kick policy is able to survive $600N \times 0.2s$ perturbations in both directions. Note that no external perturbations are applied during the training process; we suspect that the policies’ robustness is in large part due to the exploration noise applied by the stochastic policy used during training.

Table 6. Maximum forwards and sideways push each policy can tolerate before falling. Each push is applied to the character’s pelvis for 0.2s.

Skill	Forward (N)	Sideway (N)
Backflip	440	100
Cartwheel	200	470
Run	720	300
Spinkick	690	600
Walk	240	300

11 DISCUSSION AND LIMITATIONS

We presented a data-driven deep reinforcement learning framework for training control policies for simulated characters. We show that our method can produce a broad range of challenging skills. The resulting policies are highly robust and produce natural motions that are nearly indistinguishable from the original motion capture data in the absence of perturbations. Our framework is able to re-target skills to a variety of characters, environments, and tasks, and multiple policies can be combined into composite policies capable of executing multiple skills.

Although our experiments illustrate the flexibility of this approach, there are still numerous limitations to be addressed in future work. First, our policies require a phase variable to be synchronized with the reference motion, which advances linearly with time. This limits the ability of the policy to adjust the timing of the motion, and lifting this limitation could produce more natural and flexible perturbation recoveries. Our multi-clip integration approach works well for small numbers of clips, but has not yet been demonstrated on large motion libraries. The PD controllers used as the low-level servos for the characters still require some insight to set properly for each individual character morphology. The learning process itself is also quite time consuming, often requiring several days per skill, and is performed independently for each policy. Although we use the same imitation reward across all motions, this is still currently based on a manually defined state-similarity metric. The weighting of the reward terms also needs to be defined with some care.

We believe this work nevertheless opens many exciting directions for exploration. In future work, we wish to understand how the policies might be deployed on robotic systems, as applied to locomotion, dexterous manipulation, and other tasks. It would be interesting to understand the learned control strategies and compare them to the equivalent human strategies. We wish to integrate diverse skills that would enable a character to perform more challenging tasks and more complex interactions with their environments. Incorporating hierarchical structure is likely to be beneficial towards this goal.

ACKNOWLEDGEMENTS

We thank KangKang Yin, Libin Liu, and Ziva Dynamics for providing reference motion data for this project, the anonymous reviewers for their helpful feedback, and AWS for providing computational resources. This research was funded by an NSERC Postgraduate Scholarship, a Berkeley Fellowship for Graduate Study, and ONR PECASE N000141612723.

REFERENCES

Shailen Agrawal, Shuo Shen, and Michiel van de Panne. 2013. Diverse Motion Variations for Physics-based Character Animation. *Symposium on Computer Animation* (2013).

SUPPLEMENTARY MATERIAL

A MULTI-STEP RETURNS

We will refer to the return $R_t = \sum_{l=0}^{T-t} \gamma^l r_{t+l}$ as the Monte-Carlo return. R_t provides an unbiased sample of the expected return at a given state, but due to stochasticity from the dynamics of the environment and policy, each reward r_t can be a random variable, the sum of which can result in a high variance estimator of the expected return. Alternatively, an n -step return can be used to provide a lower-variance estimator at the cost of introducing some bias. The n -step return can be computed by truncating the sum of returns after n steps, and approximating the return from the remaining steps using a value function $V(s)$:

$$R_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V(s_{t+n}).$$

$R_t^{(1)} = r_t + \gamma V(s_{t+1})$ results in the 1-step return commonly used in Q -learning [Mnih et al. 2015] and $R_t^{(\infty)} = \sum_{l=0}^{T-t} \gamma^l r_{t+l} = R_t$ recovers the original Monte-Carlo return, given that all rewards after the horizon T are 0. While $R_t^{(\infty)}$ provides an unbiased but high variance estimator, $R_t^{(1)}$ provides a biased but low variance estimator. Therefore, n acts as a trade-off between bias and variance for the value estimator.

Another method to trade off between the bias and variance of the estimator is to use a λ -return [Sutton and Barto 1998], calculated as an exponentially-weighted average of n -step returns with decay parameter λ :

$$R_t(\lambda) = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} R_t^n.$$

Assuming all rewards after step T are 0, such that $R_t^n = R_t^{T-t}$ for all $n \geq T - t$, the infinite sum can be calculated according to

$$R_t(\lambda) = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} R_t^n + \lambda^{T-t-1} R_t^{T-t}$$

$\lambda = 0$ reduces to the single-step return $R_t^{(1)}$, and $\lambda = 1$ recovers the Monte-Carlo return R_t . Intermediate values of $\lambda \in (0, 1)$ produces interpolants that can be used to balance the bias and variance of the value estimator.

Updating the value function using the temporal difference computed with the λ -return results in the TD(λ) algorithm [Sutton and Barto 1998]. Similarly, estimating the advantage with the λ -return, yields the generalized advantage estimator GAE(λ) [Schulman et al. 2015b]:

$$\hat{\mathcal{A}}_t = R_t(\lambda) - V(s_t).$$

Empirically, the λ -return has been shown to produce better performance than simply using an n -step return [Schulman et al. 2015b; Sutton and Barto 1998].

B OFF-POLICY LEARNING

In its previously stated form, the gradient of the expected return $\nabla_{\theta} J(\theta)$ is estimated with respect to the current policy parameters θ . As such, data collected from the current policy can be justified for use only in performing a single gradient step, after which a new

batch of data is required to estimate the gradient with respect to the updated parameters. Data efficiency can be improved by introducing importance sampling, which provides an unbiased estimate of the policy gradient using only off-policy samples from an older policy $\pi_{\theta_{old}}$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t | s_t)} [w_t(\theta) \nabla_{\theta} \log(\pi_{\theta}(a_t | s_t)) \mathcal{A}_t]$$

$$w_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$$

The importance-sampled policy gradient can be interpreted as optimizing the surrogate objective

$$L^{IS}(\theta) = \mathbb{E}_{s_t \sim d_{\theta_{old}}(s_t), a_t \sim \pi_{\theta_{old}}(a_t | s_t)} [w_t(\theta) \mathcal{A}_t].$$

With L^{IS} , the same batch of data can be used to perform multiple update steps for θ . While in theory importance sampling allows for the policy gradient to be estimated with data collected from any policy with sufficient support, we will consider only the case where the data is collected from a previous set of policy parameters.

C PROXIMAL POLICY OPTIMIZATION

In practice, the policy gradient estimator discussed thus far suffers from high variance, often leading to instability during learning, where the policy's performance fluctuates drastically between iterations. While problems due to noisy gradients can be mitigated by using large batches of data per update, policy gradient algorithms can still be extremely unstable. Trust region methods have been proposed as a technique for improving the stability of policy gradient algorithms [Schulman et al. 2015a]. Trust Region Policy Optimization (TRPO) optimizes the same objective $J(\theta)$ but includes an additional KL-divergence constraint to prevent the behaviour of the current policy π_{θ} from deviating too far from the previous policy $\pi_{\theta_{old}}$.

$$\begin{aligned} \max_{\theta} \quad & J(\theta) \\ \text{s.t.} \quad & \mathbb{E}_{s_t \sim d_{\theta}(s_t)} \left[KL \left(\pi_{\theta_{old}}(\cdot | s_t) \middle| \pi_{\theta}(\cdot | s_t) \right) \right] \leq \delta_{KL} \end{aligned}$$

δ_{KL} is a hyper parameters that defines the trust region. TRPO has been successfully applied to solve a wide variety of challenging RL problems [Duan et al. 2016; Ho and Ermon 2016; Rajeswaran et al. 2016]. However, ensuring that the constraint is satisfied can be difficult, and it is often instead enforced approximately using the conjugate gradient algorithm with an adaptive stepsize selected by a line search to ensure the approximate constraint is satisfied [Schulman et al. 2015a].

Proximal Policy Optimization (PPO) is a variant of TRPO, where the hard constraint is replaced by optimizing a surrogate loss [Schulman et al. 2017]. In this work, we will be concerned mainly with the clipped surrogate loss $L^{CLIP}(\theta)$ defined according to

$$L^{CLIP}(\theta) = \mathbb{E}_{s_t, a_t} [\min(w_t(\theta) \mathcal{A}_t, \text{clip}(w_t(\theta), 1 - \epsilon, 1 + \epsilon) \mathcal{A}_t)].$$

When $\theta = \theta_{old}$, $w_t(\theta) = 1$, but as θ is changed, the likelihood ratio

moves away from 1. Therefore, the likelihood ratio can be interpreted as a measure of the similarity between two policies. To discourage the policies from deviating too far apart, $\text{clip}(w_t(\theta), 1 - \epsilon, 1 + \epsilon)$ sets the gradient to zero whenever the ratio is more than ϵ away from 1. This term therefore serves a similar function to the KL-divergence constraint in TRPO. The minimum is then taken between the clipped and unclipped advantages to create a lower bound of $L(\theta)$.

D LEARNING ALGORITHM

Algorithm 1 summarizes the common learning procedure used to train all policies. Policy updates are performed after a batch of $m = 4096$ samples has been collected. Minibatches of size $n = 256$ are then sampled from the data for each gradient step. A discount factor $\gamma = 0.95$ is used for all motions. $\lambda = 0.95$ is used for both TD(λ) and GAE(λ). The likelihood ratio clipping threshold is set to $\epsilon = 0.2$. A stepsize of $\alpha_v = 10^{-2}$ is used for the value function. A policy step size of $\alpha_\pi = 5 \times 10^{-5}$ is used for the humanoid and Atlas, and $\alpha_\pi = 2 \times 10^{-5}$ for the dragon and T-Rex. Once gradients have been computed, the network parameters are updated using stochastic gradient descent with momentum 0.9. The same hyperparameter settings are used for all characters and skills, with the exception of the step size. Humanoid policies for imitating individual skills typically require about 60 million samples to train, requiring about 2 days on an 8-core machine. All simulation and network updates are performed on the CPU and no GPU acceleration is used.

ALGORITHM 1: Proximal Policy Optimization

```

1:  $\theta \leftarrow$  random weights
2:  $\psi \leftarrow$  random weights
3: while not done do
4:    $s_0 \leftarrow$  sample initial state from reference motion
5:   Initialize character to state  $s_0$ 
6:   for step = 1, ...,  $m$  do
7:      $s \leftarrow$  start state
8:      $a \sim \pi_\theta(a|s)$ 
9:     Apply  $a$  and simulate forward one step
10:     $s' \leftarrow$  end state
11:     $r \leftarrow$  reward
12:    record  $(s, a, r, s')$  into memory  $D$ 
13:   end for
14:    $\theta_{old} \leftarrow \theta$ 
15:   for each update step do
16:     Sample minibatch of  $n$  samples  $\{(s_i, a_i, r_i, s'_i)\}$  from  $D$ 
17:     Update value function:
18:     for each  $(s_i, a_i, r_i, s'_i)$  do
19:        $y_i \leftarrow$  compute target values using TD( $\lambda$ )
20:     end for
21:      $\psi \leftarrow \psi + \alpha_v \left( \frac{1}{n} \sum_i \nabla_\psi V_\psi(s_i)(y_i - V(s_i)) \right)$ 
22:     Update policy:
23:     for each  $(s_i, a_i, r_i, s'_i)$  do
24:        $\mathcal{A}_i \leftarrow$  compute advantage using  $V_\psi$  and GAE
25:        $w_i(\theta) \leftarrow \frac{\pi_\theta(a_i|s_i)}{\pi_{\theta_{old}}(a_i|s_i)}$ 
26:     end for
27:      $\theta \leftarrow$ 
28:        $\theta + \alpha_\pi \frac{1}{n} \sum_i \nabla_\theta \min(w_i(\theta)\mathcal{A}_i, \text{clip}(w_i(\theta), 1 - \epsilon, 1 + \epsilon)\mathcal{A}_i)$ 
29:   end for
30: end while

```

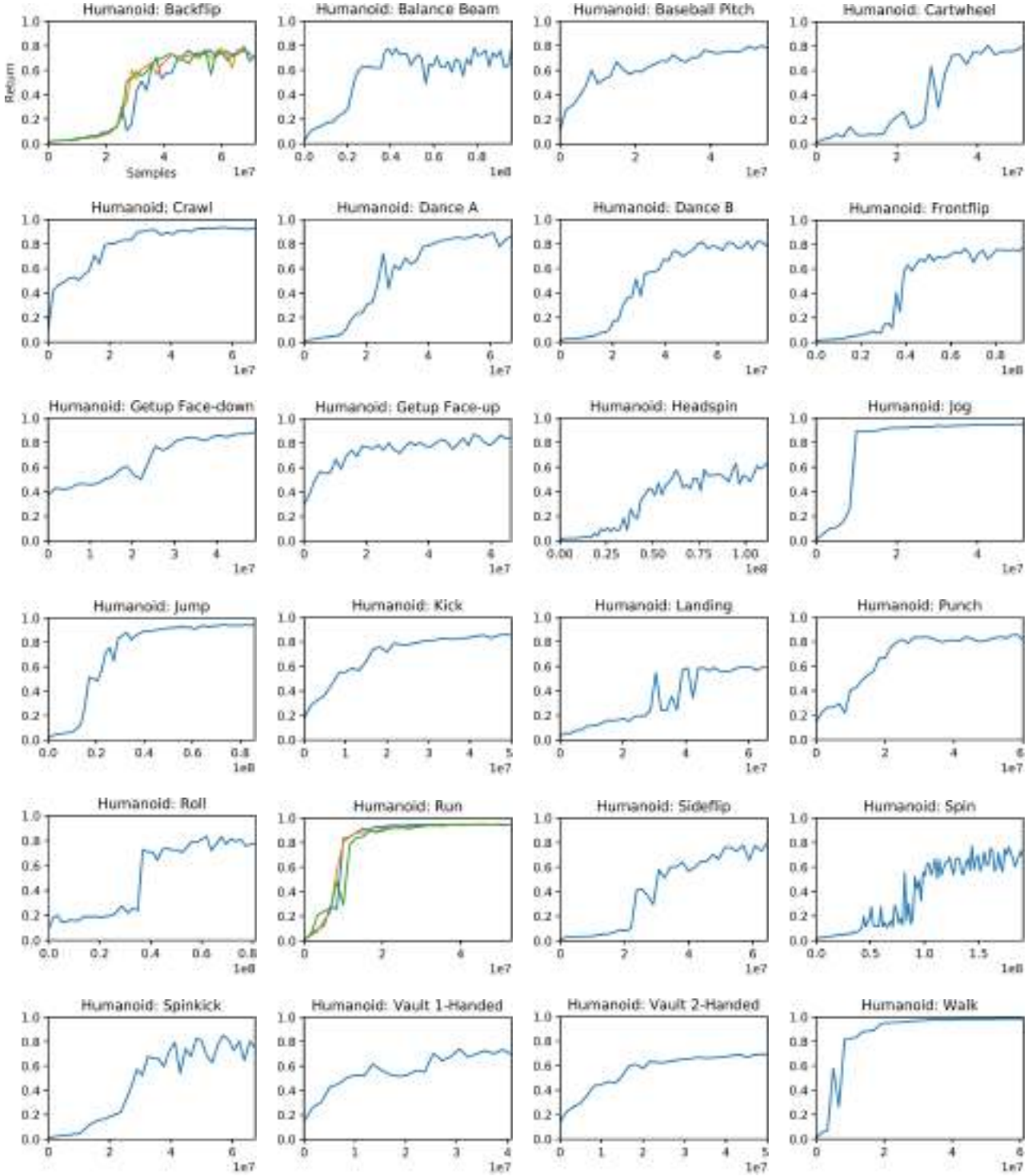


Fig. 12. Learning curves of the humanoid imitating individual motion clips. Performance is calculated as the mean return over 32 episodes. The returns are normalized by the minimum and maximum possible return per episode. Due to the time needed to train each policy, performance statistics are collected only from one run of the training process for the majority of skills. For the backflip and run policies, we collected statistics from 3 training runs using different random seeds. Performance appears consistent across multiple runs, and we have observed similar behaviours for many of the other skills.

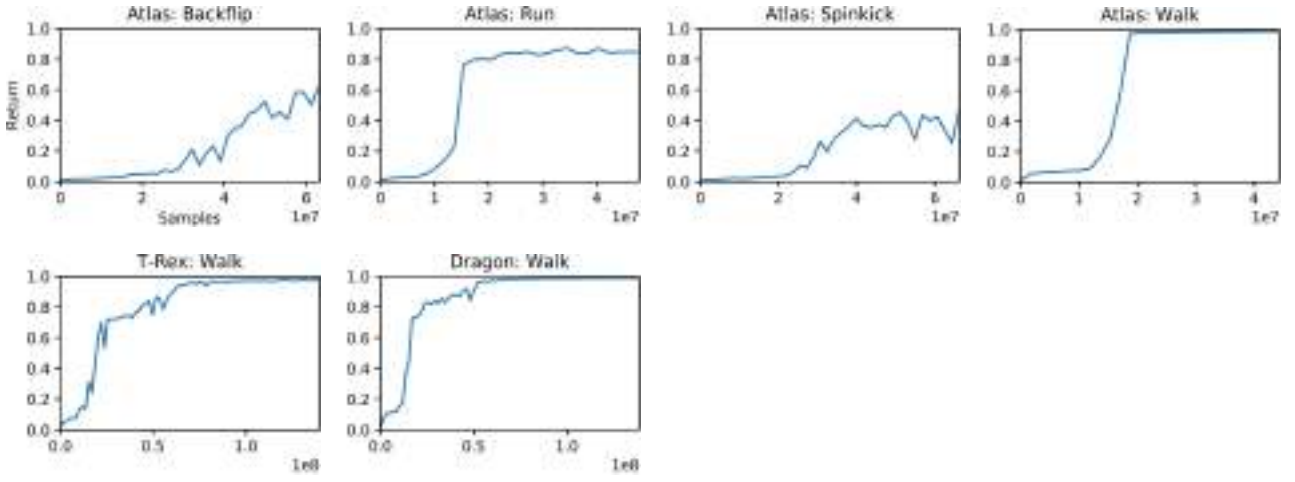


Fig. 13. Learning curves of the Atlas, T-Rex, and dragon imitating individual motion clips.

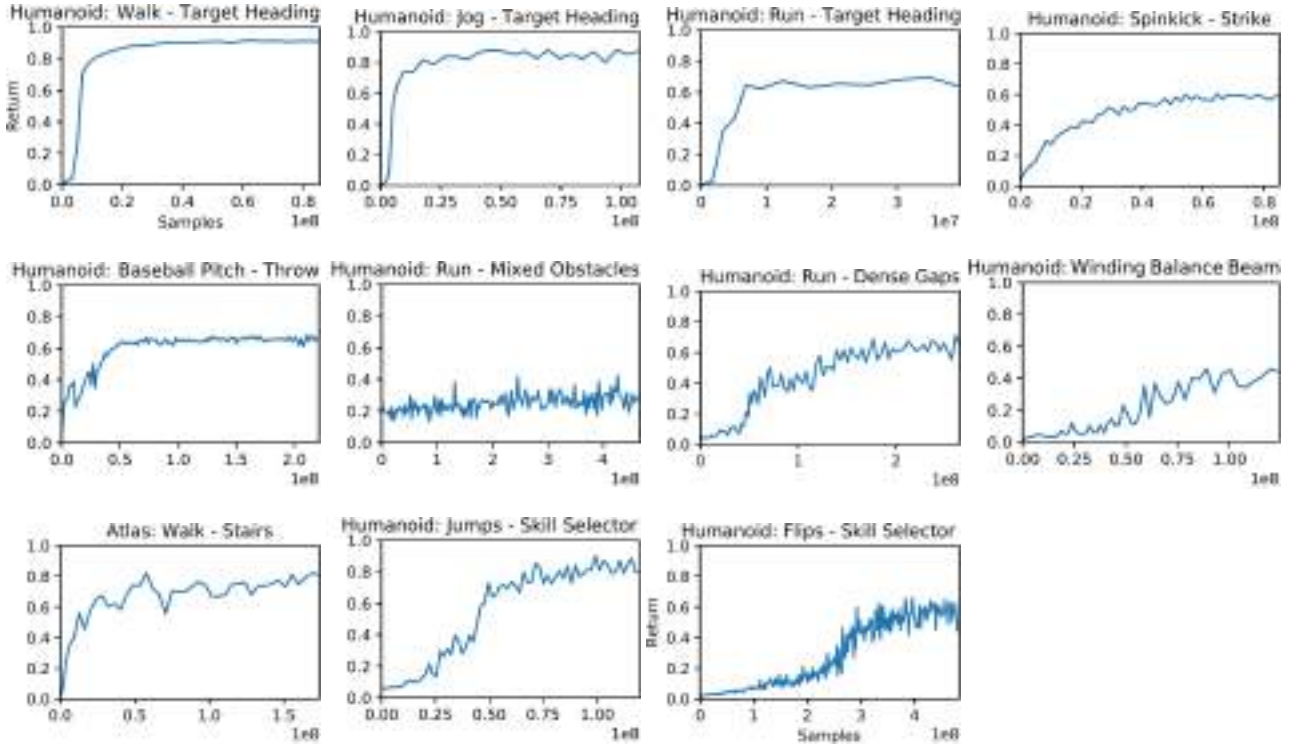


Fig. 14. Learning curves of policies trained to imitate motion clips while fulfilling additional task objectives.